Institut Saint-Laurent – Cours du soir

Le Langage C#
Avec Visual Studio

Marc Simoens
V0.5

Table des Matières

TABLE DES MATIÈRES **2**
TABLE DES TABLEAUX **7**
1. CAPACITÉS TERMINALES ET DEGRÉS DE MAÎTRISE – ÉVALUATION **8**
1.1. GRILLE D'ÉVALUATION 8
2. QUELQUES EXPLICATIONS **10**
3. LES CAPACITÉS À ACQUÉRIR **10**
4. VUE GÉNÉRALE DE MICROSOFT VISUAL STUDIO 2008 ..... **11**
4.1. CRÉATION D'UNE SOLUTION 11
5. ORGANISATION D'UNE APPLICATION **12**
5.1. LES SOLUTIONS 12
5.1.1. Présentation *12*
5.1.1.A. Création d'une solution 12
5.1.2. Modification d'une solution *13*
5.1.2.A. Ajouter un projet 13
5.1.2.B. Supprimer un projet 14
5.1.2.C. Renommer un projet 14
5.1.2.D. Décharger un projet 14
5.1.3. Organisation d'une solution *14*
5.1.4. Le dossier éléments de solution *15*
5.1.5. Le dossier Fichiers divers *15*
5.1.6. Configuration d'une solution *16*
5.1.6.A. Configuration du Projet de démarrage 17
5.1.6.B. Dépendances de projets 17
5.1.6.C. Fichiers sources pour le débogage 18
5.1.6.D. Configurations 19
5.2. LES PROJETS 20
5.2.1. Création d'un Projet *20*
5.2.1.A. Les modèles de Projets 20
5.2.1.B. Création de modèle de projet 22
5.2.2. Modification d'un projet *23*
5.2.3. Propriétés des projets *24*
5.2.3.A. Application 24
5.2.3.B. Générer 26
5.2.3.C. Événements de génération 29
5.2.3.D. Propriétés de débogage 30
5.2.3.E. Ressources d'un projet 30
5.2.3.F. Paramètres d'application 31
6. LES BASES DU LANGAGE **34**
6.1. LES VARIABLES, CONSTANTES ET ÉNUMÉRATION 34
6.1.1. Les variables *34*
6.1.1.A. Le nom des variables 34
6.1.1.B. Le type des variables 34
6.1.1.C. Conversions de type 38
6.1.1.D. Déclaration des variables 41
6.1.1.E. Inférence de type 41
6.1.1.F. Portée des variables 42
6.1.1.G. Les niveaux d'accès des variables 42
6.1.1.H. Durée de vie des variables 43
6.1.2. Les constantes *43*
6.1.3. Les énumérations *43*
6.1.4. Les Tableaux *44*
6.1.4.A. Les tableaux à une dimension 45
6.1.4.B. Les Tableaux à plusieurs dimensions 45
6.1.4.C. Manipulations courantes des tableaux 46

6.1.5. Les chaînes de caractères ... *46*
6.1.5.A. Codage d'une chaîne de caractères sur plusieurs lignes ... 47
6.1.5.B. Affectation d'une valeur à une chaîne ... 47
6.1.5.C. Extraction d'un caractère particulier ... 47
6.1.5.D. Obtention de la longueur d'une chaîne ... 47
6.1.5.E. Découpage de chaîne ... 47
6.1.5.F. Comparaison de chaînes ... 47
6.1.5.G. Insertion dans une chaîne ... 48
6.1.5.H. Suppression des espaces ... 48
6.1.5.I. Changement de la casse ... 48
6.1.5.J. Recherche dans une chaîne ... 48
6.1.5.K. Remplacement dans une chaîne ... 48
6.1.6. Les structures ... *48*
6.1.6.A. Déclaration d'une structure ... 49
6.1.6.B. Utilisation des structures ... 50
6.2. LES OPERATEURS ... 50
6.2.1. Les Opérateurs d'assignation ... *50*
6.2.2. Les Opérateurs arithmétiques ... *51*
6.2.3. Les Opérateurs binaires ... *51*
6.2.4. Les Opérateurs de comparaison ... *51*
6.2.5. L'Opérateur de concaténation ... *51*
6.2.6. Les Opérateurs logiques ... *52*
6.2.7. Ordre d'évaluation des opérateurs ... *53*
6.3. LES STRUCTURES DE CONTRÔLES ... 53
6.3.1. Les structures de décision ... *53*
6.3.1.A. La structure if ... 53
6.3.1.B. La structure Switch ... 54
6.3.2. Les structures de boucle ... *54*
6.3.2.A. La structure while ... 54
6.3.2.B. La structure do…while ... 55
6.3.2.C. La structure for ... 55
6.3.2.D. La structure foreach ... 55
6.3.2.E. La structure using (using statement) ... 56
6.4. LES PROCÉDURES ET LES FONCTIONS ... 56
6.4.1. Les Procédures ... *57*
6.4.2. Les Fonctions ... *57*
6.4.3. Procédures de propriétés ... *58*
6.4.4. Les procédures Opérateur ... *59*
6.4.5. Les Arguments des procédures et des fonctions ... *60*
6.5. ASSEMBLIES, NAMESPACE ET ATTRIBUTS ... 62
6.5.1. Les Assemblies ... *62*
6.5.2. Les Namespaces ... *63*
6.6. LES ATTRIBUTS ... 66
6.6.1. Les attributs les plus courants en Visual C# ... *66*
6.6.1.A. Serializable, NonSerialized Attribute ... 66
6.6.1.B. Static ... 67
6.6.2. Exemple ... *68*

7. POO AVEC VISUAL C# ... 70

7.1. CRÉATION D'UNE CLASSE ... 70
7.1.1. Déclaration de la classe ... *70*
7.1.2. Classe partielle ... *71*
7.1.3. Création de propriétés ... *72*
7.1.4. Création de Méthodes ... *76*
7.1.4.A. Définition ... 76
7.1.4.B. Surcharge de Méthodes ... 77
7.1.4.C. Substitution de Méthodes ... 78
7.1.4.D. Masquage de Méthode ... 79
7.1.4.E. Méthode partielle ... 80
7.1.4.F. Méthode d'extension ... 80
7.1.4.G. Les constructeurs et destructeurs ... 83
7.1.4.H. Les membres statiques ... 84
7.2. UTILISATION D'UNE CLASSE ... 85

7.2.1. Création d'un objet (instance de classe) ... *85*
7.2.2. Destruction d'un objet ... *85*
7.2.3. Liaison tardive et liaison précoce ... *86*
7.2.4. Clonage d'objets ... *86*
7.2.4.A. Interface ICloneable ... 87
7.2.4.B. Type-Safe Clone ... 87
7.3. L'HÉRITAGE ... 88
7.3.1. Introduction ... *88*
7.3.2. L'emploi du mot réservé Base ... *89*
7.3.3. Les classes abstraites ... *90*
7.3.4. Classes finales ... *90*
7.3.5. Les classes anonymes ... *91*
7.4. LES INTERFACES ... 93
7.5. LES EVENEMENTS ... 95
7.5.1. Déclaration et déclenchement d'événements ... *95*
7.5.2. Gérer les événements ... *97*
7.5.2.A. Ajoute et suppression de gestionnaire d'événement ... 97
7.5.2.B. Événement et héritage ... 99
7.6. LES DELEGUES ... 99
7.6.1. Déclaration et création d'un délégué ... *99*
7.6.2. Utilisation des délégués ... *100*
7.7. LES TYPES GENERIQUES ... 101
7.7.1. Introduction ... *101*
7.7.2. Les classes génériques ... *101*
7.7.2.A. Définition d'une classe générique ... 102
7.7.2.B. Utilisation d'une classe générique ... 104
7.7.3. Les Procédures et les Fonctions génériques ... *105*
7.7.3.A. Création d'une procédure ou d'une fonction générique ... 106
7.7.3.B. Utilisation d'une procédure ou d'une fonction générique ... 106
7.8. LES COLLECTIONS ... 107
7.8.1. Les Collections prédéfinies ... *107*
7.8.1.A. La classe Array ... 107
7.8.1.B. Les classes ArrayList et List ... 107
7.8.1.C. Hashtable et Dictionary ... 109
7.8.1.D. Queue (File) ... 110
7.8.1.E. Stack (Pile) ... 110

8. GESTION DES ERREURS ET DEBOGAGE DU CODE ... 111

8.1. TRAITEMENT DES EXCEPTIONS ... 111
8.1.1. Récupération d'exceptions ... *111*
8.1.1.A. Création et déclenchement d'exceptions ... 113
8.2. LES OUTILS DE DEBOGAGE ... 114
8.2.1. Contrôle de l'exécution ... *115*
8.2.1.A. Démarrage de la solution ... 115
8.2.1.B. Arrêter la solution ... 115
8.2.1.C. Interrompre la solution ... 115
8.2.1.D. Poursuivre l'exécution ... 116
8.2.2. Les points d'arrêt et les TracePoint ... *117*
8.2.2.A. Placer un point d'arrêt ... 117
8.2.2.B. Activer, désactiver, supprimer un point d'arrêt ... 121
8.2.3. Examen du contenu des variables ... *122*
8.2.3.A. Data Tips ... 122
8.2.3.B. Fenêtre automatique ... 122
8.2.3.C. Fenêtre variables locales ... 122
8.2.3.D. Les fenêtres Espion ... 122
8.2.3.E. La fenêtre Espion Express ... 123
8.2.4. Les autres fenêtres de débogage ... *123*
8.2.5. Autres techniques de débogage : la compilation conditionnelle ... *123*

9. LES APPLICATIONS WINDOWS ... 124

9.1. LES DIFFERENTS TYPES D'APPLICATION ... 124
9.1.1. Modes de présentation des fenêtres ... *124*
9.1.1.A. Interface mono document (SDI) ... 124

9.1.1.B. Interface multi documents (MDI) 124
9.1.1.C. Interface de type explorateur 124
9.2. LES FENETRES 124
9.2.1. Dimension et position des fenêtres 127
9.2.2. Couleurs et Polices utilisées sur les fenêtres 131
9.2.3. Les fenêtres MDI 132
9.2.4. Les fenêtres de type explorateur 135
9.2.5. Quelques trucs 139
9.2.5.A. Comment empêcher la fermeture d'un formulaire? 139
9.2.5.B. Ne pas afficher l'icône de fermeture d'un formulaire? 139
9.3. LES ÉVÉNEMENTS CLAVIER ET SOURIS 140
9.3.1. Les événements clavier 140
9.3.2. Les événements souris 144
9.4. LE DRAG AND DROP 147
9.4.1. Démarrage du Drag and Drop 147
9.4.2. Configuration des contrôles pour la réception 148
9.4.3. Récupération de l'élément accroché 149
9.5. LES BOITES DE DIALOGUE 151
9.5.1. La boîte de message 151
9.5.2. Les boîtes de dialogue de Windows 153
9.5.2.A. Dialogue pour l'ouverture d'un fichier 153
9.5.2.B. Dialogue d'enregistrement de fichier 155
9.5.2.C. Dialogue de choix de répertoire 155
9.5.2.D. Dialogue de choix d'une couleur 155
9.5.2.E. Dialogue de choix d'une police 156
9.5.2.F. Dialogue de mise en page 157
9.5.2.G. Dialogue de configuration d'impression 158
9.5.3. Les boîtes de dialogue personnalisées 159
9.6. UTILISATION DES CONTROLES 160
9.6.1. Ajoute des contrôles 161
9.6.2. Position et dimension des contrôles 162
9.6.3. Passage du focus entre contrôles 166
9.6.4. Les raccourcis clavier 167
9.7. LES CONTROLES 168
9.7.1. La classe Control 168
9.7.1.A. Dimensions et position 168
9.7.1.B. Apparence des contrôles 170
9.7.1.C. Comportement des contrôles 171
9.7.2. Les contrôles d'affichage d'informations 174
9.7.2.A. Le contrôle Label 174
9.7.2.B. Le contrôle LinkLabel 175
9.7.2.C. Le contrôle StatusStrip 176
9.7.2.D. Le contrôle ToolTip 176
9.7.2.E. Le contrôle ErrorProvider 177
9.7.2.F. Le contrôle NotifyIcon 177
9.7.2.G. Le contrôle HelpProvider 178
9.7.2.H. Le contrôle ProgressBar 179
9.7.3. Les contrôles d'édition de texte 181
9.7.3.A. Le contrôle TextBox 181
9.7.3.B. Le contrôle MaskedTextBox 182
9.7.3.C. Le contrôle RichTextBox 183
9.7.4. Les contrôles de déclenchement d'actions 185
9.7.4.A. Le contrôle Button 185
9.7.4.B. Le contrôle MenuStrip 185
9.7.4.C. Le menu ContextMenuStrip 187
9.7.4.D. Le contrôle ToolStrip 188
9.7.4.E. Le contrôle ToolStripContainer 189
9.7.5. Les contrôles de sélection 190
9.7.5.A. Le contrôle CheckBox 190
9.7.5.B. Le contrôle RadioButton 191
9.7.5.C. Le contrôle ListBox 193
9.7.5.D. Le contrôle NumericUpDown 195
9.7.5.E. Le contrôle TrackBar 196
9.7.5.F. Le contrôle DomainUpDown 196

9.7.5.G. Le contrôle CheckedListBox 196
9.7.5.H. Le contrôle ComboBox 197
9.7.5.I. Le contrôle TreeView 198
9.7.5.J. Le contrôle ListView 200
9.7.6. Les contrôles de regroupement 206
9.7.6.A. Le contrôle GroupBox 206
9.7.6.B. Le contrôle Panel 206
9.7.6.C. Le contrôle TabControl 206
9.7.6.D. Le contrôle SplitContainer 207
9.7.6.E. Le contrôle FlowLayoutPanel 208
9.7.6.F. Le contrôle TableLayoutPanel 208
9.7.7. Les contrôles graphiques 210
9.7.7.A. Le contrôle PictureBox 210
9.7.7.B. Le contrôle ImageList 211
9.7.8. Les contrôles de gestion du temps 213
9.7.8.A. Le contrôle DateTimePicker 213
9.7.8.B. Le contrôle MonthCalendar 213
9.7.8.C. Le contrôle Timer 214
9.7.8.D. Le composant BackGroundWorker 215
9.7.9. Comment ajouter un control à la ToolBox 216
9.8. AUTRES CLASSES 218
9.8.1. Directory ou DirectoryInfo 218

10. GESTION DES IMPRESSIONS 219

11. L'ACCES AUX BASES DE DONNEES 222

11.1. PRESENTATION D'ADO.NET 222
11.1.1. Le mode connecté 222
11.1.2. Le mode non connecté 222
11.1.3. Quelques définitions 223
11.2. UTILISATION DU MODE CONNECTE 223
11.2.1. Connexion à une base de données 223
11.2.1.A. La chaîne de connexion 224
11.2.1.B. Le Pool de connexion 225
11.2.1.C. Événements de connexion 226
11.2.2. Exécution d'une commande 227
11.2.2.A. Création d'une commande 227
11.2.2.B. Lecture d'informations 227
11.2.2.C. Modification des informations 229
11.2.2.D. Utilisation de paramètres 229
11.2.3. Exécution de procédures stockées 231
11.3. UTILISATION DU MODE NON CONNECTE 232

12. DEPLOIEMENT D'APPLICATIONS ET DE COMPOSANTS 233

12.1. DEPLOIEMENT AVEC WINDOWS INSTALLER 233
12.1.1. Création d'un projet d'installation 233
12.1.2. Configuration du programme d'installation 235
12.1.2.A. L'éditeur du système de fichiers 236
12.1.2.B. L'éditeur de registre 238
12.1.2.C. L'éditeur des types de fichiers 238
12.1.2.D. L'éditeur de l'interface utilisateur 239
12.1.2.E. L'éditeur des actions personalisées 240
12.1.2.F. L'éditeur des conditions de lancement 241
12.2. DEPLOIEMENT AVEC CLICKONCE 243
12.2.1. Principe de fonctionnement de ClickOnce 244
12.2.2. Les différentes méthodes de déploiement 245
12.2.2.A. Installation à partir du Web ou d'un partage réseau 245
12.2.2.B. Installation à partir d'un CD 245
12.2.2.C. Démarrage de l'application à partir du Web ou d'un partage réseau 245
12.2.3. Les mises à jour de l'application 245
12.2.4. Mise en œuvre de la publication ClickOnce 246

13. ANNEXES 253

13.1. EXPLICATION DE QUELQUES FONCTIONS 253

14. BIBLIOGRAPHIE 254

Table des Tableaux

TABLE 1: LES CAPACITÉS TERMINALES 9
TABLE 2: LES DEGRÉS DE MAÎTRISE 9
TABLE 3: LES TYPES ENTIERS SIGNÉS ET NON SIGNÉS 34
TABLE 4: LES TYPES DÉCIMAUX 35
TABLE 5: LES ESCAPE CODES 35
TABLE 6: COMBINAISONS POSSIBLES ENTRE VARIABLES BOOL NULLABLES 37
TABLE 7: OPTIONS DE FORMATAGE DE LA FONCTION FORMAT() 39
TABLE 8: OPTIONS DE FORMATAGE DE LA FONCTION FORMAT() SPÉCIFIQUES AUX DATES ET HEURES 40
TABLE 9: LES DIFFÉRENTES PORTÉES DISPONIBLES EN C# 42
TABLE 10: LISTE DES NIVEAUX D'ACCÈS 43
TABLE 11: LISTE DES OPÉRATEURS ARITHMÉTIQUES 51
TABLE 12: LISTE DES OPÉRATEURS BINAIRES 51
TABLE 13: LISTE DES OPÉRATEURS DE COMPARAISON 51
TABLE 14: LISTE DES OPÉRATEURS LOGIQUES 53
TABLE 15: LES SYNTAXES POSSIBLES DE L'INSTRUCTION IF 54
TABLE 16: DIFFÉRENCES ENTRE VARIABLE ET PROPRIÉTÉ 58
TABLE 17: VALEURS POSSIBLES DE LA PROPRIÉTÉ STARTPOSITION 128
TABLE 18: VALEURS POSSIBLES DE LA PROPRIÉTÉ WINDOWSSTATE 128
TABLE 19: LISTE DES PRÉFIXES POUR LES CONTRÔLES 162
TABLE 20: CARACTÈRES DE FORMATAGE UTILISABLES DANS LE CONTRÔLE DE TYPE MASKEDTEXTBOX 183
TABLE 21: LISTE DES MOTS CLÉS D'UNE CHAÎNE DE CONNEXION 225
TABLE 22: LISTE DES CLASSES UTILISABLES DANS LE MODE NON CONNECTÉ 232
TABLE 23: EXPLICATION DE CERTAINES FONCTIONS 253

1. Capacités terminales et degrés de maîtrise – évaluation

1.1. Grille d'évaluation

Pour atteindre le seuil de réussite, l'étudiant devra prouver qu'il est capable :

- de réaliser, sous interface graphique, un projet partiel s'intégrant dans un projet global mené en équipe et faisant l'objet d'un cahier des charges ;
- de justifier dans un rapport écrit argumenté :
  - les choix stratégiques et méthodologiques ;
  - la prise de décision de ces choix (justification) ;
  - l'optimisation des ressources logicielles, matérielles et humaines ;
  - les ajustements apportés en cours de réalisation ;
  - la stratégie d'évaluation continue mise en œuvre ;
- de présenter les sous-ensembles de programmation écrits.

Pour la détermination du degré de maîtrise, il sera tenu compte:

- de la rigueur et du respect des spécificités du logiciel de développement utilisé ;
- de la convivialité de l'interface homme-machine réalisée ;
- de la prise en compte du cahier des charges ;
- de sa collaboration et de son apport au sein de l'équipe de développement ;
- de la clarté et de la précision dans l'utilisation du vocabulaire technique et du langage utilisateur ;
- de la prise de conscience de la valeur du travail en équipe ;
- du degré d'autonomie atteint.

Les étudiants du cours devront travailler en groupe en se répartissant les tâches parmi les trois niveaux suivants :

- Gestion de la couche présentation,
- Gestion applicative des données,
- Gestion de la base de données.

Selon le temps disponible, les étudiants devront si possible change de niveau en cours de réalisation afin de toucher à un maximum d'aspects du développement.

Avant de commencer, les étudiants devront :

- Bâtir l'architecture globale du developpement.
- Établir avec précision comment chaque niveau va s'interfacer avec les autres.
- Établir un dictionnaire des données manipulées dans l'application.

Pour réaliser cette application réelle, le groupe recevra un cahier très précis des spécifications à réaliser. Le respect scrupuleux de celles-ci sera un point très important.

Au terme du cours, chaque étudiant se devra de produire un document écrit dans lequel seront décrits les capacités terminales telles que décrites dans le tableau ci-dessous :

CAPACITÉS TERMINALES À montrer suite au développement de l'application Vétérinaire réalisée en groupe		
Énoncé de la capacité	**Critères**	**Atteinte ou non**
De réaliser, sous interface graphique, un projet partiel s'intégrant dans un projet global mené en équipe et faisant l'objet d'un cahier des charges ;		OUI/NON
Justifier les choix stratégiques et méthodologiques;		OUI/NON
Justifier la prise de décision de ces choix;		OUI/NON
Justifier l'optimisation des ressources logicielles, matérielles et humaines;		OUI/NON
Justifier les ajustements apportés en cours de réalisation;		OUI/NON
Justifier la stratégie d'évaluation continue mise en œuvre;		OUI/NON
Les capacités terminales sont-elles atteintes ?		**OUI/NON**

Table 1: Les capacités terminales

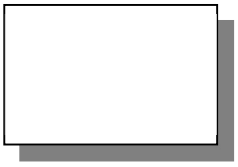
Chaque étudiant devra également fournir avec ce dossier écrit, la liste de tous les programmes qu'il aura écrits.

LES DEGRÉS DE MAÎTRISE À montrer suite au développement de l'application Vétérinaire réalisée en groupe		
Énoncé du degré de ma^trise	**Critères**	**Atteint ou non**
De la rigueur et du respect des spécificités du logiciel de développement utilisé ;		OUI/NON
De la convivialité de l'interface homme-machine réalisée ;		OUI/NON
De la prise en compte du cahier des charges ;		OUI/NON
De sa collaboration et de son apport au sein de l'équipe de développement ;		OUI/NON
De la clarté et de la précision dans l'utilisation du vocabulaire technique et du langage utilisateur ;		OUI/NON
De la prise de conscience de la valeur du travail en équipe;		OUI/NON
Du degré d'autonomie atteint;		OUI/NON
Pourcentage obtenu avec les degrés de maîtrise		**……%**
Pourcentage total obtenu sur le cours		**……%**

Table 2: Les degrés de maîtrise

2. Quelques explications

Afin d'enrichir le syllabus, je me suis permis à diverses reprises d'inclure du texte venant de sources extérieures. Afin de signaler clairement cette copie, le texte sera toujours placé dans un encadré tel que celui-ci :



Les différents exercices que l'on réalisera au cours sont introduits par le sigle :





Enfin, beaucoup de difficultés peuvent être supprimées si l'on connaît un certain nombre de principes importants du langage. Ces principes seront tous introduits par le sigle suivant :

De nombreux exemples se trouvent dans ce syllabus. Dans chacun d'eux, les éléments importants ou nouveaux seront mis en évidence par l'usage de caractères gras et soulignés.

3. Les capacités à acquérir

Pour atteindre le seuil de réussite, l'étudiant devra prouver qu'il est capable :

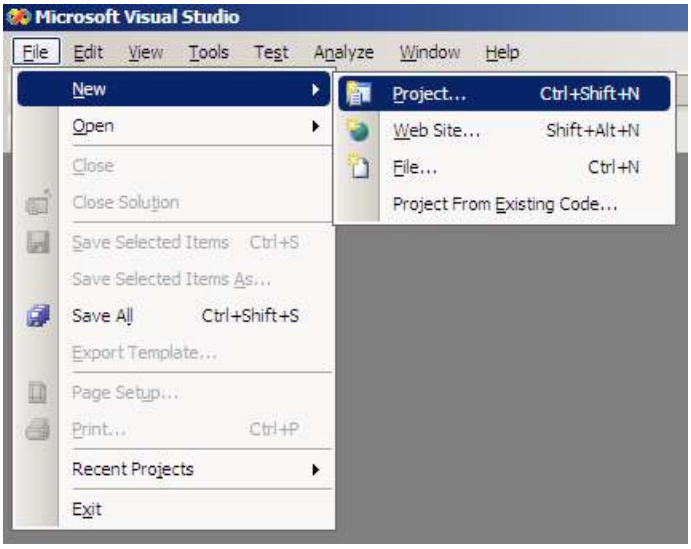
- De réaliser, sous interface graphique, un projet partiel s'intégrant dans un projet global mené en équipe et faisant l'objet d'un cahier des charges.
- De justifier dans un rapport écrit argumenté :
    - les choix stratégiques et méthodologiques,
    - la prise de décision de ces choix (justification),
    - l'optimisation des ressources logicielles, matérielles et humaines,
    - les ajustements apportés en cours de réalisation,
    - la stratégie d'évaluation continue mise en œuvre,
- De présenter les sous-ensembles de programmation écrits.

Pour la détermination du degré de maîtrise, il sera tenu compte :

- De la rigueur et du respect des spécificités du logiciel de dévelopement utilisé,
- De la convivialité de l'interface homme-machine réalisée,
- De la prise en compte du cahier des charges,
- De sa collaboration et de son apport au sein de l'équipe de développement,
- De la clarté et de la précision dans l'utilisation du vocabulaire technique et du langage de l'utilisateur ;
- De la prise de conscience de la valeur du travail en équipe,
- Du degré d'autonomie atteint.

4. Vue générale de Microsoft Visual Studio 2008

4.1. Création d'une solution



5. Organisation d'une application

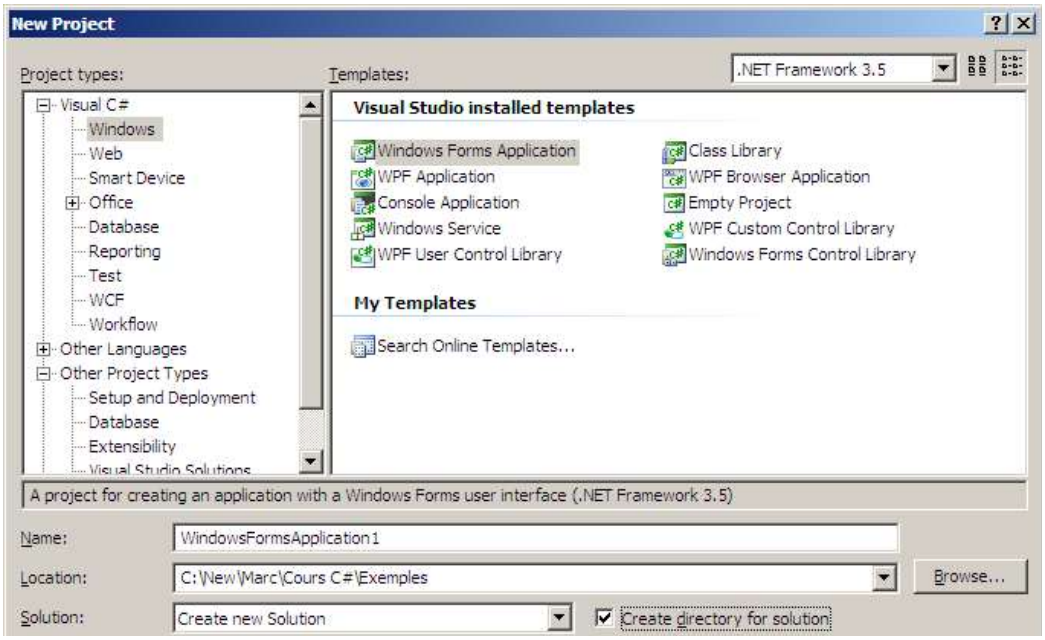
5.1. Les Solutions

5.1.1. Présentation

Pour aider à la création d'applications, Visual Studio propose plusieurs éléments servant à regrouper les composants d'une application. Le conteneur de plus haut niveau est la solution dans laquelle on peut placer un ou plusieurs projets. Ces projets contiendront à leur tour, tous les éléments pour que le compilateur soit capable de générer le fichier exécutable ou dll du projet. L'explorateur de solutions va permettre de manipuler tous ces éléments.

5.1.1.A. Création d'une solution

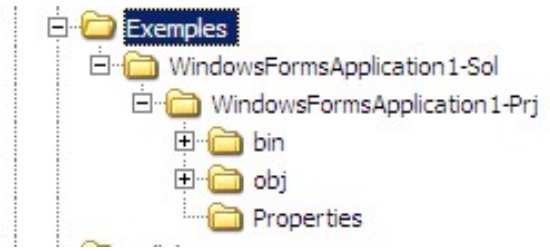
La création d'une solution est automatique lorsque l'on démarre un nouveau projet dans Visual Studio. Lors de la création d'un nouveau projet, il sera demandé plusieurs informations le concernant.



Par l'intermédiaire de cette boîte de dialogue, on va fournir les informations suivantes :

- La version du Framework nécessaire pour utiliser l'application,
- Le langage utilisé pour développer le projet,
- Le type de projet à créer,
- Le nom du projet,
- Le répertoire de base où seront stockés les fichiers,
- Le nom de la solution,
- La création d'un répertoire popur la solution.

Après validation de cette boîte de dialogue, l'explorateur de solutions va présenter la nouvelle solution sur laquelle on va pouvoir travailler. Tous les fichiers de la solution sont déjà créés et sauvegardés sur disque à l'emplacement qui a été spécifié :



Une solution contiendra au moins les fichiers suivants :

- Un fichier avec l'extension **.sln** qui est le fichier de configuration de la solution. Ce fichier contient entre autres la liste de tous les projets composants la solution. Il est complété au fur et à mesure que l'on ajoute de nouveaux projets à la solution.

- Un fichier avec l'extension **.suo** enregistrant les options associées à la solution. Ce fichier permet de retrouver ces options.

- Un fichier pour le projet, portant l'extension **.csproj**. Ce fichier contient toutes les informations de configuration du projet avec notamment la liste des fichiers constituant le projet, la liste de références utilisées pour ce projet, les options à utiliser pour la compilation du projet, …

- De nombreux fichiers ayant l'extension **.cs** qui vont contenir le code source de toutes les classes, feuilles et modules constituant le projet.

- Un fichier **.resx** associé à chaque feuille ou formulaire de l'application. Ce fichier au format XML contient entre autres la liste des ressources utilisées sur cette feuille.

- Au final, une solution contient de nombreux fichiers en fonction des éléments utilisés dans le projet.

5.1.2. Modification d'une solution

Les solutions étant des conteneurs, il est bien sûr possible de gérer leurs éléments. On peut ajouter, supprimer, renommer des éléments dans la solution.

5.1.2.A. Ajouter un projet

Plusieurs possibilités existent pour ajouter un projet :

- Si on souhaite créer un nouveau projet, on choisira l'option *Nouveau Projet* du menu *Fichier – Ajouter*. Une boîte de dialogue propose alors de configurer les caractéristiques du nouveau projet. Cette boîte de dialogue propose notamment un répertoire par défaut pour l'enregistrement du projet. Si ce répertoire ne correspond pas à l'emplacement où l'on désire enregistrer le projet, on peut sélectionner un autre emplacement.

  Cette opération sera à réaliser pour chaque nouveau projet que l'on ajoutera. Il peut être intéressant de modifier le chemin proposé par défaut pour l'enregistrement des projets. Pour cela, il faut ouvrir le menu *Outils – Options*, puis dans la boîte de dialogue, il faut choisir l'option *Projets et solutions* et y modifier la rubrique *Visual Studio Projects location*.

- Si on souhaite ajouter un projet déjà existant, on doit utiliser l'option *Projet existant* du menu *Fichier – Ajouter*. Une boîte de dialogue de sélection de fichiers permet alors de choisir le fichier .csproj du projet que l'on soouhaite ajouter à la solution. À noter que le projet reste à son emplacement d'origine sur le disque.

5.1.2.B. Supprimer un projet

Pour supprimer un projet, il faut utiliser le menu contextuel de l'explorateur de solutions en effectuant un clic droit sur le nom du projet que l'on souhaite supprimer de la solution. Le projet sera alors éliminé de la solution mais restera enregistré sur le disque. Pour le supprimer définitivement, il faut utiliser l'explorateur Windows pour supprimer les fichiers de ce projet. Si on ne supprime pas les fichiers, le projet peut, par la suite, être de nouveau ajouté à une solution.

5.1.2.C. Renommer un projet

Pour renommer un projet, il faut utiliser le menu contextuel de l'explorateur de solutions en effectuant un clic droit sur le nom du projet que l'on souhaite renommer.

Le nom du projet devient alors modifiable dans l'explorateur de solutions. Cette modification concerne uniquement le nom du fichier .csproj associé au projet. Elle ne modifie en aucun cas le nom du répertoire dans lequel se trouvent les fichiers du projet.

5.1.2.D. Décharger un projet

Lorsque l'on souhaite exclure temporairement un projet du processus de génération ou rendre l'édition de ses composants impossible, on peut décharger le projet de la solution grâce à l'option *Décharger le projet*. Un projet déchargé n'est pas retiré de la solution mais simplement marqué comme indisponible.

Le projet peut bien sûr, être réhabilité dans la solution en utilisant l'option *Recharger le projet* du menu contextuel.

5.1.3. Organisation d'une solution

Si on travaille avec une solution contenant de nombreux projets, on peut ajouter un niveau de hiérarchisation en créant des dossiers de solutions. Ceux-ci permettent le regroupement logique de projets au sein d'une solution. Pour cela, il faut créer dans un premier temps, les dossiers dans la solution puis, il faut organiser les projets dans ce dossier.

Les dossiers de solutions ne créent pas de dossiers physiques sur disque, ils sont juste des conteneurs logiques à l'intérieur de la solution.

Création d'un dossier de solution

Un dossier de solution peut être créé par trois méthodes différentes.

- Pour toutes ces méthodes, il faut sélectionner la solution dans l'explorateur de solution.
- Ensuite, il faut utiliser :
  - Soit, le bouton : 

  - Soit, le menu *Projet – Ajouter un nouveau dossier de solution*

- Ou encore le menu contextuel disponible par un clic droit sur le nom de la solution.

- Quelle que soit la méthode utilisée, on doit fournir un nom pour le dossier créé.

Créer un projet dans un dossier

La création d'un projet dans un dossier de solution est identique à la création d'un projet directement dans la solution. Il faut juste au préalable sélectionner le dossier dans lequel on souhaite créer le projet.

Déplacer un projet dans un dossier

Il arrive fréquemment que la nécessité d'organiser une solution avec des dossiers se fasse sentir alors que des projets existent déjà dans la solution. Il faut créer dans ce cas, les dossiers puis d'effectuer un glissé-déplacé des projets dans les dossiers correspondants.

5.1.4. Le dossier éléments de solution

Les solutions contiennent principalement des projets, cependant il est possible d'avoir, dans une solution, des fichiers gérés indépendemment d'un projet particulier mais associés à la solution. C'est le cas, par exemple, d'un fichier icône que l'on souhaite utiliser dans plusieurs projets de la solution. Ces fichiers sont appelés *éléments de solution* et sont placés dans un dossier spécifique de la solution.

Pour ajouter un nouvel élémen de solution, il faut utiliser le menu contextuel sur le nom de la solution et choisir l'option *Ajouter – Nouvel éléments* ou l'option *Ajouter – Elément existant*.

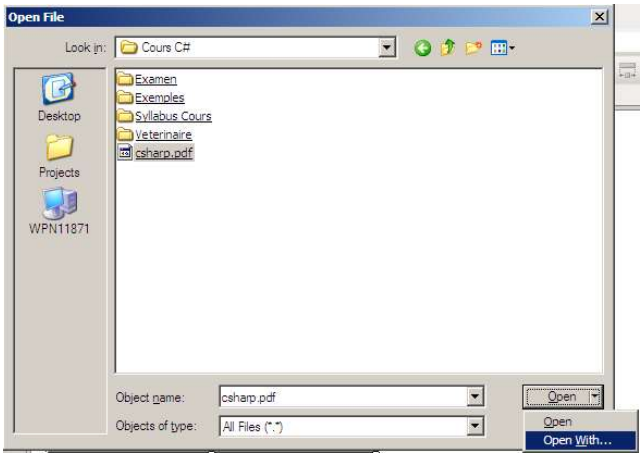
Le nouvel élément est alors ajouté dans le dossier *Eléments de solution*. Il est à noter que ce dossier n'existe pas, par défaut, dans une solution mais il est créé automatiquement lors de l'ajout du premier élément de solution. Les éléments de solution peuvent ensuite être modifiés avec un éditeur spécifique au type de fichier créé.

5.1.5. Le dossier Fichiers divers

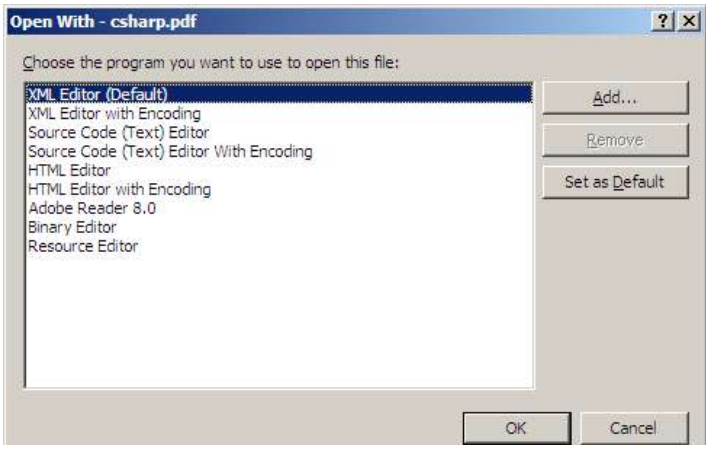
On peut parfois vouloir visualiser le contenu d'un fichier alors que l'on travaille sur une solution, comme par exemple le compte rendu d'une réunion. Ce fichier ne doit pas faire partie de manière permanente de la solution. On peut l'ouvrir avec un éditeur externe et basculer entre Visual Studio et cet éditeur externe mais il est plus pratique de visualiser le fichier directement dans l'environnement Visual Studio.

Il faut utiliser l'opion *Ouvrir un fichier* du menu *Fichier*.

La boîte de dialogue permet de choisir le fichier à ouvrir. En fonction du type de fichier, un éditeur par défaut lui sera automatiquement associé pour permettre sa modification. Il est parfois utile de pouvoir choisir l'éditeur associé à un fichier. Pour cela, le bouton *Ouvrir* de la boîte de dialogue dispose d'un menu proposant l'option *Ouvrir avec* permettant le choix de l'éditeur associé au fichier.



La boîte de dialogue suivante propose la liste des éditeurs disponibles :



Il suffit de choisir l'éditeur associé au fichier sur lequel on souhaite travailler. Le fichier est alors disponible dans le dossier *Fichiers divers* de la solution. Comme le dossier *Eléments de solution,* le dossier *Fichiers divers* n'existe pas, par défaut, dans la solution mais sera créé automatiquement lors de l'ouverture d'un fichier.

Il sera visible dans l'explorateur de solutions uniquement si l'option correspondante est activée dans l'environnement Visual Studio. Pour cela, il faut ouvrir le menu *Outils – Options* puis dans la boîte de dialogue, il faut choisir l'option *Environnement – Documents* et activer l'option *Afficher les fichiers divers dans l'explorateur de solutions*. Comme le dossier Eléments de solution, ce dossier est un dossier *logique* et ne correspond pas à un emplacement sur le disque.

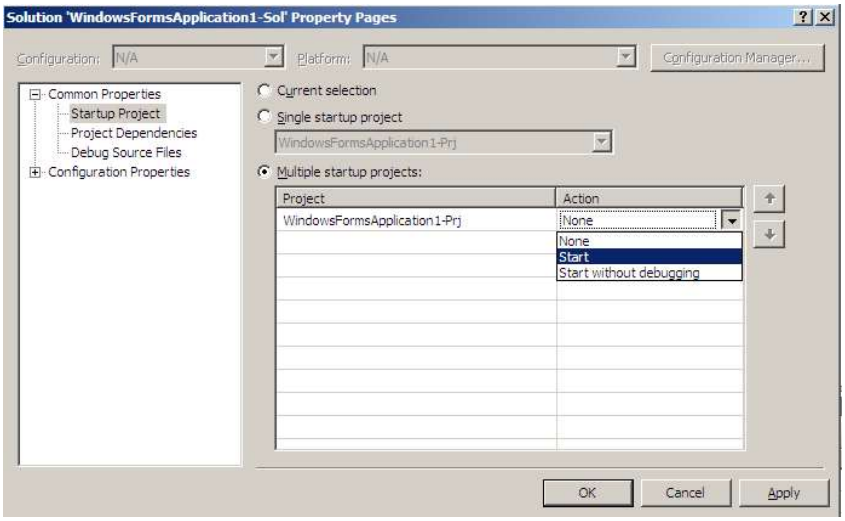
5.1.6. Configuration d'une solution

Les solutions disposent de propriétés permettant de configurer leur comportement lors de la génération ou de l'excution de l'application. Ces différentes propriétés sont regroupées dans une boîte de dialogue accessible par l'otpion *Propriétés* du menu contextuel d'une solution. Quatre catégories de propriétés sont disponibles :

- Projet de démarrage,
- Dépendances de Projets,
- Fichiers sources pour débogage,
- Configurations.

5.1.6.A. Configuration du Projet de démarrage

Cette page de propriétés de la solution détermine, parmi les projets disponibles, celui ou ceux lancés au démarrage de la solution.



Deux options sont disponibles :

- Projet de démarrage unique. Une ComboBox propose la liste des projets disponibles dans la solution parmi lesquels on doit choisir celui qui sera exécuté au démarrage de la solution. Ce projet sera par la suite signalé dans l'explorateur de solutions par son nom apparaissant en gras. Cette sélection peut également se faire par le menu contextuel de l'explorateur de solutions en choisissant l'option *Définier comme projet de démarrage*.

- Plusieurs projets de démarrage. Un tableau présente la liste de tous les projets disponibles dans la solution. Pour chacun d'eux, on doit indiquer l'action à exécuter lors du lancement de l'application. Les choix possibles sont :

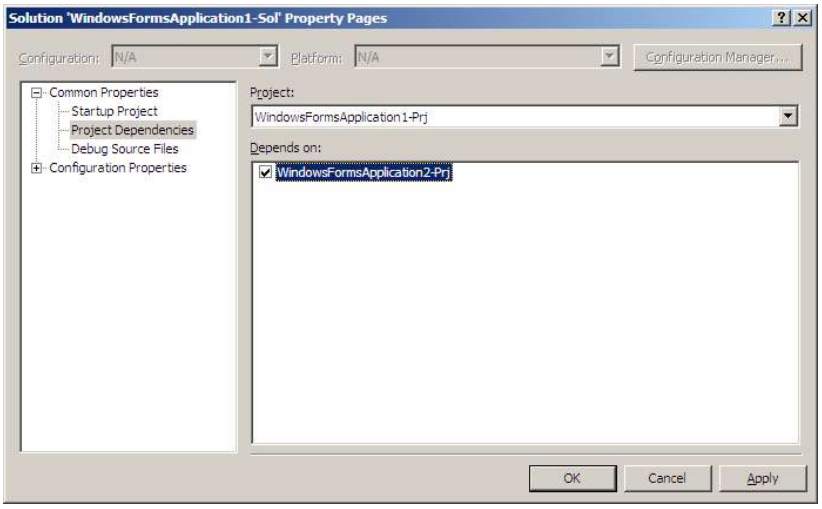
  - Aucune action,
  - Démarrer le projet,
  - Exécuter le projet dans débogage.

  Si on souhaite démarrer plusieurs projets au lancement de la solution, on doit également indiquer l'ordre dans lequel ces projets seront démarrés. Cet ordre correspond en fait à l'ordre des projets dans le tableau.

5.1.6.B. Dépendances de projets

La génération de certains projets nécessite au préalable la génération d'autres projets. C'est le cas, par exemple, si on demande la génération d'un projet qui utilise une référence vers un autre projet : celui-ci est alors une dépendance du projet initial.

La page de propriétés suivante permet de configurer les dépendances :



Dans la liste des projets, il faut sélectionner celui pour lequel on souhaite configurer les dépendances. Les autres projets de la solution sont alors listés, avec, pour chacun d'eux, une case à cocher. Lors de la génération du projet, tous les projets dont il dépend seront automatiquement regénérés, s'ils ont été modifiés depuis la dernière génération ou s'ils n'ont jamais été générés.

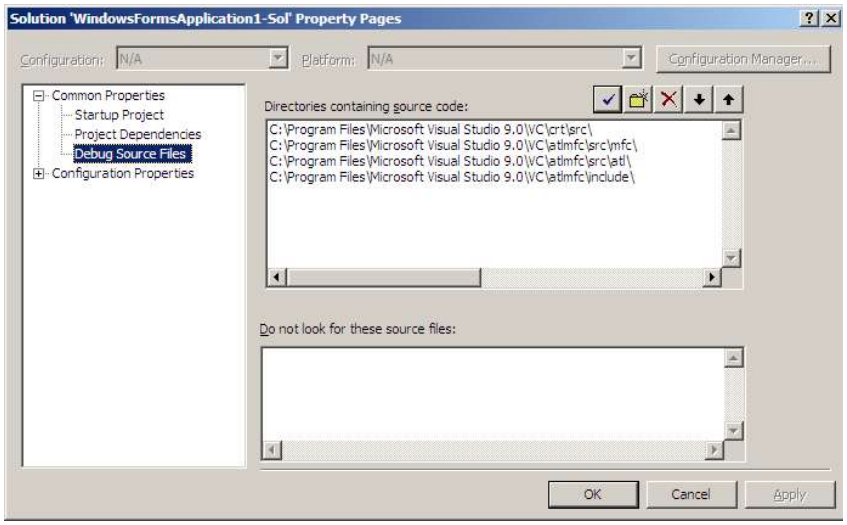
Certaines dépendances ne peuvent être modifiées : la case à cocher apparaît alors en gris.

C'est en général le cas, lorsqu'un projet possède une référence sur un autre projet ou que j'ajout d'une dépendance risque de créer une boucle.

Les dépendances de projet peuvent être également configurées par le menu contextuel de l'explorateur de solutions avec l'otpion *Dépendances du projet*.

5.1.6.C. Fichiers sources pour le débogage

Lors du débogage d'une application, l'environnement Visual Studio a besoin d'accéder au fichier source du code qu'il est en train de déboguer. Cette page de propriété permet de spécifier les répertoires qui seront analysés à la recherche du code source :



La liste *Répertoires contenant du code source* affiche le nom des répertoires qui seront scrutés à la recherche de code source, pendant le débogage d'une application. Cette liste peut être gérée par la barre d'outils dont les boutons permettent de :

 Vérifier l'existence du répertoire.

 Ajouter un nouveau répertoire.

 Supprimer le répertoire sélectionné dans la liste.

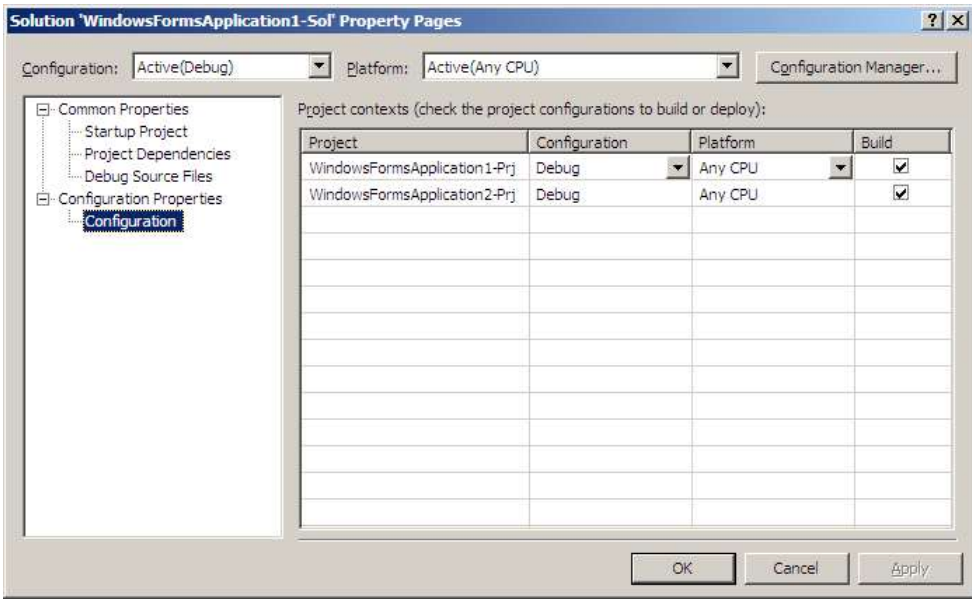
 Déplacer le répertoire vers le haut ou vers le bas de la liste.

La liste *Ne pas rechercher ces fichiers sources* exclut certains fichiers de la recherche.

5.1.6.D. Configurations

Les options de configuration permettent de définir comment différentes versions d'une solution et des projets qui la composent seront générés. Par défaut, deux configurations sont disponibles pour une solution dans Visual Studio : la configuration *Debug* et la configuration *Release*.

Pour chacun des projets présents dans la solution, les deux configurations permettent de définir des options de compilation. La configuration *Debug* est utilisée pendant le développement et les tests du projet. La configuration *Release* est utilisée pour la génération finale du projet.



On a en fait un système à trois niveaux : pour chaque configuration de solution, on indique quelle configuration utiliser pour chaque projet et, pour chaque configuration de projet, on spécifie des otpions de compilation. Les options de compilation sont modifiables au niveau des propriétés du projet.

5.2. Les Projets

Les projets sont les conteneurs de deuxième niveau dans une application. Ils sont utilisés pour organiser logiquement, gérer, générer et déboguer les composants d'une application. La génération d'un projet produit, en général, un fichier exécutable ou une bibliothèque *dll*. Un projet peut être très simple et ne contenir que deux éléments : un fichier source et le fichier de projet (.csproj).

Plus généralement, les projets contiennent des nombreux fichiers sources, des scripts de base de données, des références vers des services Web, des ressources graphiques, …

Visual Studio propose par défaut un ensemble de modèles de projets. Ces modèles fournissent un point de départ pour la majorité des besoins dans le développement d'une application. Pour des cas plus spécifiques, on peut créer ses propres modèles de projet.

5.2.1. Création d'un Projet

Pour la création d'un projet, il faut sélectionner l'option *Fichier – Nouveau Projet*. Une boîte de dialogue propose alors de choisir les caractéristiques du nouveau projet. Il faut choisir :

- La version du Framework pour laquelle on souhaite développer le projet. La version choisie influence les types de projet que l'on va pouvoir créer.
- Le langage dans lequel on souhaite développer le projet. Les choix disponibles dépendent des langages installés dans Visual Studio.
- Le type de projet que l'on souhaite développer. La boîte de dialogue propose alors les différents modèles de projet disponible en fonction du type de projet choisi.

Après avoir réalisé le choix, on doit indiquer le nom du projet, un emplacement pour les fichiers de projet et un nom pour la solution. Le modèle sélectionné est alors utilisé par l'assistant pour créer les éléments du projet.

5.2.1.A. Les modèles de Projets

De nombreux modèles de projets sont disponibles dans Visual Studio. Ces modèles fournissent les éléments de base nécessaires pour développer chaque type de projet. Ils contiennent toujours au moins le fichier de projet, plus un exemplaire de l'élément le plus utilisé pour le type de projet correspondant ; par exemple, pour un projet de bibliothèque classe, un fichier source contenant une ébauche de classe est créé. Les modèles fournissent également des références et des importantions par défaut pour les bibliothèques et les espaces de noms les plus utiles en fonction du type de projet.

Applications Windows

Ce modèle de projet est certainement le plus utilisé. Il permet le développement d'applications Windows standards. Le modèle ajoute au projet les éléments suivants :

- Un fichier AssemblyInfo.cs utilisé pour la description de l'application avec notamment les informations concernant la version.
- Un formulaire de base avec son fichier source *form1.cs*.

Bibliothèque de classe

Ce modèle de projet est utilisable pour créer des classes et des composants qui pourront par la suite être partagés avec d'autres projets. Les éléments suivants sont automatiquement ajoutés au projet :

- Un fichier *Assembly.cs* utilisé pour la description du projet avec notamment les informations concernant la version.

- Une classe de base avec son fichier source *class1.cs*.

Bibliothèque de contrôles Windows

Comme le modèle précédent, ce type de projet permet de créer une bibliothèque de classes utilisable dans d'autres projets. Cette bibliothèque est plus spécifique puisqu'elle est dédiée à la création de contrôles, utilisables par la suite dans une application Windows. Ces contrôles étendent la boîte à outils déjà disponible dans les applications Windows. Les éléments suivants sont automatiquement ajoutés au projet :

- Un fichier *Assembly.cs* utilisé pour la description du projet avec notamment les informations concernant la version.

- Une classe *UserControll* héritant de la classe *System.Windows.Forms.UserControl* fournissant les fonctionnalités de base pour un contrôle Windows, avec son fichier source *UserControl.cs*.

Application Console

Ce type d'application est destiné à être exécuté à partir de la ligne de commande d'une fenêtre de type DOS. Elle est bien sûr conçue sans interface graphique car les entrées/sorties se font à partir de la ligne de commande et vers la console.

Ce type d'application est très pratique ppour réaliser des tests avec Visual C# car elle permet de se concentrer sur un point particulier sans avoir à se soucier de l'aspect présentation de l'application. À part sa facilité d'utilisation, ce type d'application est devenu un peu obsolète.

Les éléments suivants sont jaoutés par défaut au projet :

- Un fichier *Assembly.cs* utilisé pour la description du projet avec notamment les informations concernant la version.

- Une classe de base avec son fichier source *Program.cs*.

Service Windows

Ce modèle de projet est conçu pour la création d'applications s'exécutant en tâche de fond sur le système. Le lancement de ce type peut être effectué automatiquement au démarrage du système et ne nécessite pas qu'une session utilisateur soit ouverte pour pouvoir s'exécuter.

Ce type d'application est dépourvu d'interface utilisateur. Si des informations doivent être communiquées à l'utilisateur, elles devront transiter par les journaux système, consultables par l'observateur d'évènements. Les éléments suivants seront ajoutés au projet :

- Un fichier *Assembly.cs* utilisé pour la description du projet avec notamment les informations concernant la version.

- Une classe de base avec le squelette des procédures *OnStart* et *OnStop* appelés automatiquement au démarrage et à l'arrêt du service.

Application WPF

Ce modèle de projet permet de bénéficier du nouveau système d'affichage graphique de Windows utilisé dans Windows Vista.

Les éléments suivants sont automatiquement ajoutés au projet :

- Un fichier *Assembly.cs* utilisé pour la description du projet avec notamment les informations concernant la version.
- Un fichier *App.xaml* et son fichier de code associé, *App.xaml.cs*, permettant la gestion des événements déclecnhés au niveau de l'application.
- Une fenêtre de base *Window1.xaml* et son fichier de code associé *Window1.xaml.vb.*

Bibliothèque de contrôles WPF

Comme la bibliothèque de contrôles Windows, ce type de projet permet d'étendre la boîte à outils déjà disponible dans les applications WPF. Les éléments suivants sont ajoutés au projet :

- Un fichier *Assembly.cs* utilisé pour la description du projet avec notamment les informations concernant la version.
- Un fichier *UserControl1.xaml* pour la définition de l'aspect graphique du contrôle.
- Un fichier *UserControl1.xaml.cs* pour le code associé à ce contrôle.

Bibliothèque de contrôles WPF personnalisés

Ce type de projet a également pour vocation d'étendre la boîte à outils disponible pour les applications WPF. Contrairement au type précédent, les contrôles ne sont pas créés de toute pièce mais sont basés sur des contrôles existants dont ils étendent les propriétés. Les références et importations sont identiques au type de projet précédent.

Projet vide

Ce modèle doit être utilisé lorsque l'on souhaite créér un projet propre. Seul le fichier est créé. Par contre, aucun autre élément n'est ajouté automatiquement et aucune référence n'est créée ou importée.

5.2.1.B. Création de modèle de projet

On peut créer son propre modèle de projet en fonction des habitudes de développement et faire en sorte qu'il apparaisse parmi les modèles prédéfinis.

On doit concevoir les éléments suivants :

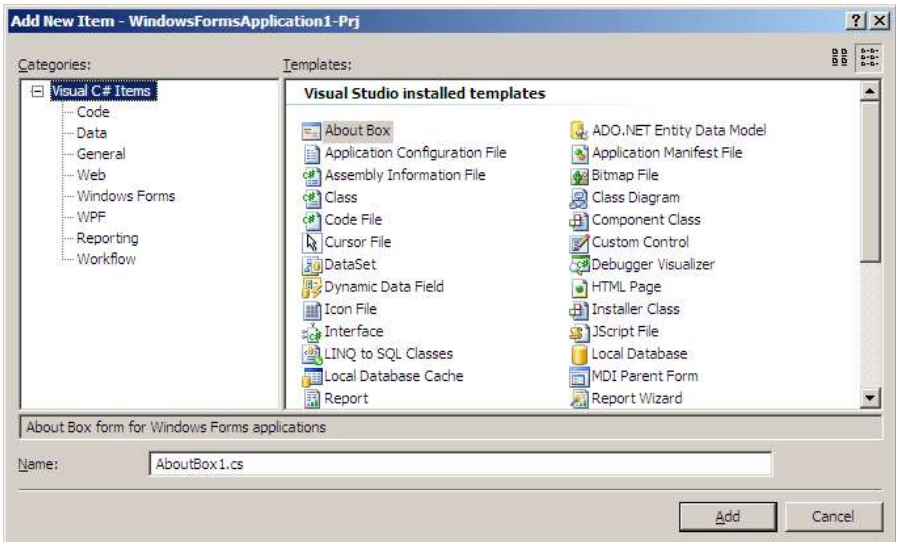
- Un fichier de définition contenant les métadonnées du modèle. Ce fichier est utilisé par Visual Studio pour l'affichage du projet dans l'environnement de développement et pour l'affectation de propriétés par défaut au projet. Ces informations sont contenues dans un fichier XML ayant l'extension .vstemplate.
- Un fichier pour le projet.
- Les fichiers sources et resosurces inclus par défaut lors de la création d'un projet à partir de ce modèle.

Ces fichiers doiovent être compressés dans un fichier .zip. Le fichier .zip doit contenir les fichiers individuellement et non le dossier dans lequel ils sont placés.

5.2.2. Modification d'un projet

Les modèles de projets sont très utiles pour créer rapidement les bases d'une application mais, très fréquemment, nécessiteront l'ajout de nouveaux éléments au projet. Ces ajouts se font par l'intermédiaire du menu contextuel de l'explorateur de projet.

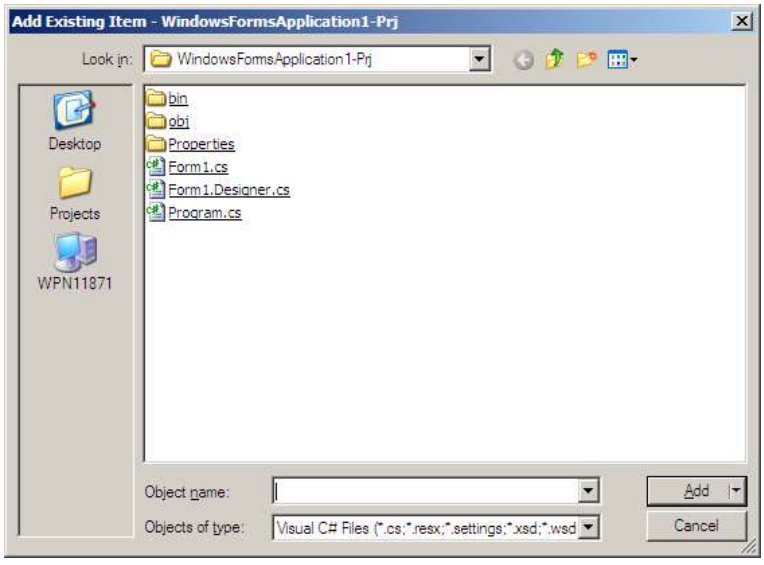
Il faut activer l'option *Ajouter – Nouvel élément* afin de choisir le type d'élément que l'on souhaite ajouter au projet. La boîte de dialogue propose un nombre impressionnant d'élements pouvant être ajoutés à un projet.



Il faut ensuite indiquer un nom pour le fichier contenant le nouvel élément.

En fonction des types de projet, des options supplémentaires permettant rapidement d'ajouter un nouvel élément sont disponibles dans le menu contextuel. Celles-ci affichent simplement la boîte de dialogue précédente avec le type d'élément correspondant déjà préselectionné.

Il est également possible de reprendre un élément existant dans un autre projet et de l'ajouter à un projet. Il faut utiliser dans ce cas, l'option *Ajouter – Elément existant* du menu contextuel de l'explorateur de projets. Une boîte de dialogue propose la sélection du fichier à inclure dans le projet.



Le bouton *AJouter* de cette boîte de dialogue comporte un menu permettant d'ajouter le fichier normalement (une copie locale du fichier est alors réalisée) ou de créer un lien sur le fichier (le fichier original est alors utilisé). Il faut être prudent avec cette possibilité car le fichier *n'appartient pas* réellement à l'application mais peut être partagé entre plusieurs applications. Si le fichier est supprimé du disque, toutes les applications l'utilisant ne pourront plus être compilées.

La gestion des fichiers dans l'explorateur de solutions est identique à la gestion des fichiers dans l'explorateur Windows. Les fichiers peuvent être copiés, collés ou déplacés par un cliqué-glissé d'un dossier à un autre. L'utilisation des touches [ctrl], [shift] et [ctrl][shift] pendant le cliqué-glissé modifie l'action réalisée.

Un cliqué-glissé au sein d'un même projet effectue un déplacement de fichier. S'il est réalisé entre deux projets, c'est alors une copie de fichier qui est effectuée.

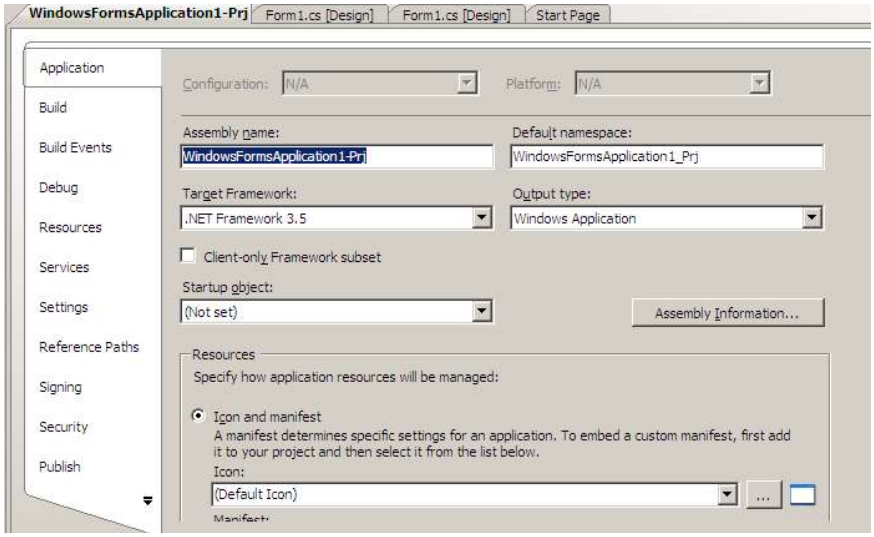
Afin de retirer un élément du projet, deux options sont accessibles par le menu contextuel de l'explorateur de solutions :

- L'option *Supprimer* supprime le fichier du projet mais également du disque.
- L'option *Exclure du projet* retire le fichier du projet mais ne le supprime pas du disque. Cette option est utile si d'autres projets utilisent ce projet par l'intermédiaire d'un lien.

5.2.3. Propriétés des projets

Les projets sont des éléments fondamentaux de la conception d'une application avec Visual C#. Ils possèdent de nombreuses propriétés permettant de modifier leurs comportements au moment de la conception ou de l'exécution de l'application. L'ensemble des propriétés sont accessibles par une boîte de dialogue présentant, par l'intermédiaire d'onglets, les différentes rubriques de configuration d'un projet.

L'activation de cette boîte de dialogue se fait par l'option *Propriété* du menu contextuel de l'explorateur de projet.



5.2.3.A. Application

Les propriétés présentées sur cet onglet vont permettre de configurer le comportement de l'application.

Nom de l'Assembly

Cette propriété détermine le nom utilisé pour le fichier résultant de la compilation de l'application. Par défaut, ce fichier porte le même nom que le projet mais ils peuvent être modifiés indépendemment l'un de l'autre. L'extension associée au fichier dépend du type du projet.

Type de sortie

Cette propriété détermine le type d'application générée par la compilation du projet. En règle générale, cette propriété est déterminée par le modèle choisi au moment de la création du projet. Cette propriété est très rarement modifiée par la suite car elle dépend énormément du code du projet (si on a conçu une application comme une application Windows et que l'on souhaite la considérer comme une application console, il risque d'y avoir beaucoup de code inutile !).

Objet de démarrage

Cette propriété détermine le point d'entrée dans l'application, lors de son exécution. Elle correspond au nom de la classe contenant la fonction *main*. Cette fonction est en général chargée de créer l'instance de la fenêtre principale de l'application et d'assurer son affichage. Cette propriété n'est disponible que pour les projets pouvant s'exécuter de manière autonome. Dans le cas d'une bibliothèque de classe, par exemple, cet attribut contient la valeur *Non défini*.

Espace de noms par défaut

Tous les éléments du projet, accessibles à partir d'un autre projet, appartiennent à l'espace de nom défini par cette propriété. Celle-ci vient s'ajouter aux éventuels espaces de noms définis au niveau du code lui-même. Par défaut, cette propriété correspond au nom du projet mais elle peut être modifiée indépendemment de clui-ci. Elle peut même être vide permettant ainsi de gérer les espaces de noms directement dans le code.

Option Icône et Manifeste

Cette option permet de rendre accessibles les options pour la configuration de l'icône et du manifeste de l'application.

Icône

Cette propriété configure l'icône associée au fichier compilé du projet, lorsqu'il est affiché dans l'explorateur Windows ou lorsque l'application apparaît sur la barre des tâches de Windows.

Manifeste

Le manifeste est utilisé lors de la création de l'application sous Windows Vista pour déterminer le niveau d'exécution requis pour l'application (UAC : *User Account Control*). Trois options sont possibles :

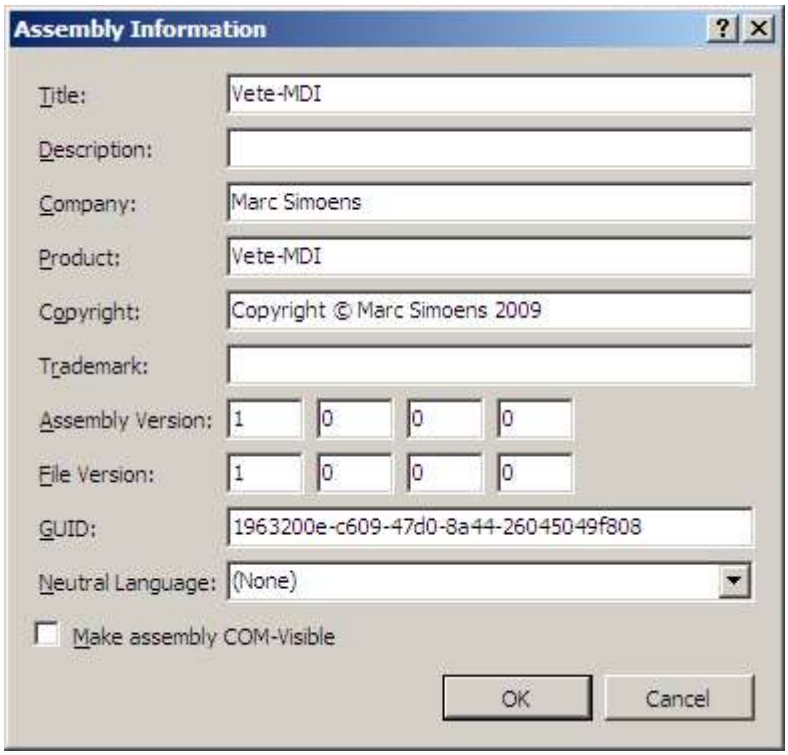
- Incorporer les paramètres par défaut dans le fichier manifeste : avec cette option, un fichier manifeste est généré automatiquement pendant la compilation. Ce fichier détermine que l'application doit s'exécuter avec l'identitté actuelle de l'utilisateur et ne demande pas d'augmentation de privilèges.
- Créer une application sans fichier manifeste : cette option active la virtualisation lors de l'exécution de l'application sous Windows Vista.
- Fournir un fichier manifeste dont le nom doit dans ce cas apparaître comme troisième option

Fichier de ressources

On doit sélectionner cette option lorsque l'on indique un fichier de ressources personalisé pour le projet. La sélection de cette option désactive les options icône et manifeste.

Informations de l'assembly

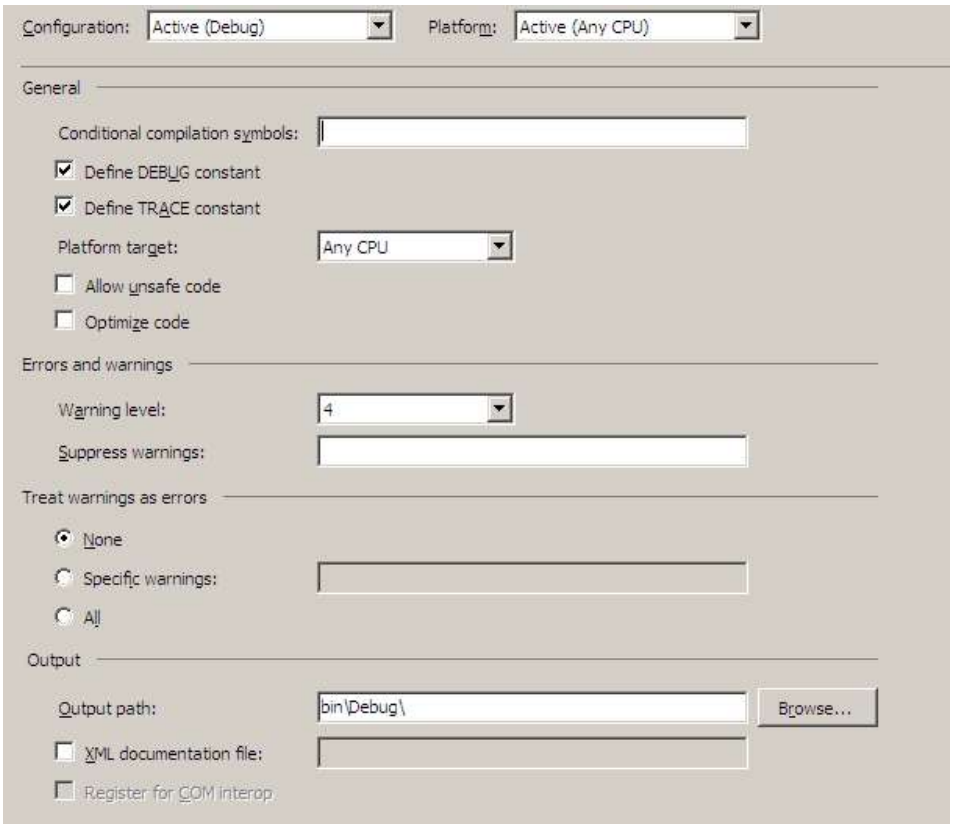
Cette option permet de fournir des informations sur le code généré par la compilation du projet. Une boîte de dialogue permet de remplir différentes rubriques concernant la description du projet.



L'utilisateur du code pourra consulter ses informations en affichant les propriétés du fichier compilé dans l'explorateur Windows.

5.2.3.B. Générer

Cette page de propriétés est utilisée pour configurer les différentes options génération.



Il faut d'abord choisir à quelle configuration (debug ou release) et à quelle plate-forme vont s'appliquer les paramètres.

Symboles de compilation conditionnelle

Cette zone de saisie est utilisée pour définir des constantes qui sont testées lors de la compilation. On peut par exemple définir la constante *DEMO* et l'utiliser comme dans l'exemple ci-dessous pour modifier le titre d'une fenêtre :

```
#if (DEMO)
     Text = "Version de démo";
#else
     Text = "Version complète";
#endif
```

Si plusieurs constantes doivent être définies, il faut les séparer par un espace.

Définir la constante DEBUG

Définit automatiquement une constante de compilation conditionnelle nommée *DEBUG*.

Définir la constante TRACE

Définit automatiquement une constante de compilation conditionnelle nommée *TRACE*.

Plateforme cible

Cette option spécifie le processeur pour lequel le code doit être généré. Quatre options sont disponibles :

- **X86** : pour les processeurs 32 bits compatibles Intel.
- **Itanium** : pour les processeurs Intel Itanium 64 bits.
- **X64** : pour les autres processeurs 64 bits.
- **Any** : CPU pour tous les processeurs.

Autoriser du code unsafe

Autorise la compilation du code utilisant le mot clé *unsafe*. Ce mot clé est utilisé lorsque le code doit manipuler directement des pointeurs.

Optimiser le code

Active ou désactive les optimisations effectuées par le compilateur pour générer du code plus efficace.

Niveau d'avertissement

Lors de son travail, le compilateur peut rencontrer des situations lui apparaissant anormales. Il génère dans ce cas un avertissement. Cette option permet de configurer les types d'avertissements générés :

- 0 : désactive l'émission de tous les messages d'avertissement.
- 1 : affiche les messages d'avertissement graves.

- 2 : Affiche les avertissements de niveau 1 ainsi que quelques avertissements moins graves.
- 3 : Affiche les avertissements de niveau 2 ainsi que quelques avertissements moins graves, par exemple pour signaler des expressions prenant toujours la valeur *true* ou *false*.
- 4 : affiche tous les avertissements de niveau 3 plus les avertissements d'information.

Supprimer les avertissements

Cette option permet la génération de certains avertissements par le compilateur. Les avertissements doivent être indiqués par leur numéro en les séparant par des virgules ou des points-virgules.

Considérer les avertissements comme des erreurs

Détermine quels sont les avertissements du compilateur qui seront traités comme des erreurs et bloqueront la compilation. Les valeurs suivantes sont proposées :

- **Aucun** : ne considère aucun avertissement comme une erreur.
- **Avertissements spécifiques** : considère les avertissements spécifiés comme des erreurs. Comme pour la rubrique *Supprimer les avertissements,* les numéros des avertissements doivent être séparés par une virgule ou un point virgule.
- **Tous** : considère tous les avertissements comme des erreurs.

Chemin de sortie

Cette option indique le répertooire où seront copiés les fichiers générés par le compilateur.

Fichiers de documentation XML

Indique le nom du fichier dans lequel sera copiée la documentation générée à partir des commentaires placés dans le code.

Inscrire pour COM Interop

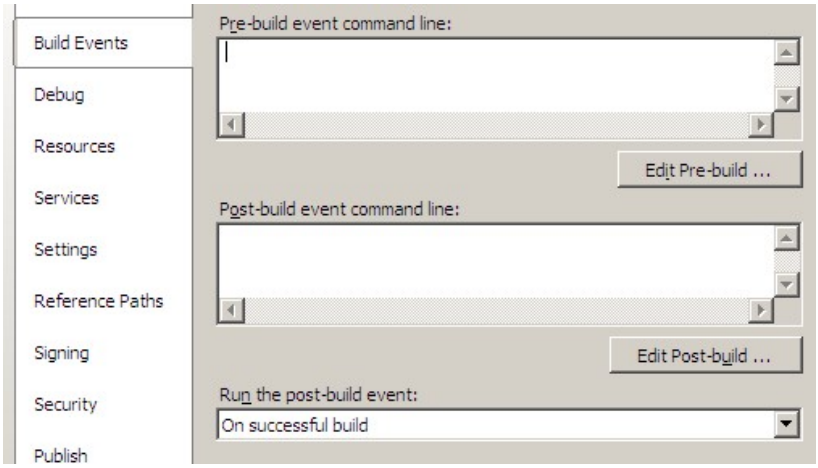
Cette option indique au compilateur qu'il doit générer du code compatible avec l'environnement COM. Cette option n'est disponible que pour les projets de type bibliothèque de classes.

Générer un assembly de sérialisation

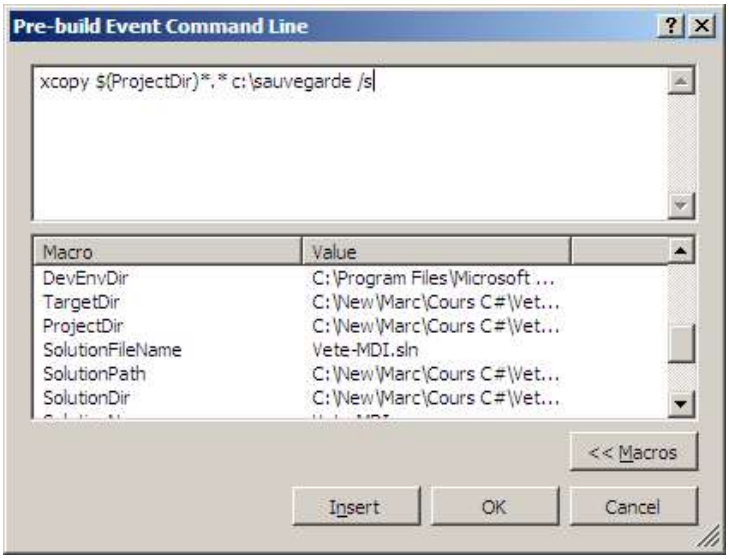
Demande au compilateur qu'il doit optimiser le code pour les opérations de sérialisation et désérialisations des instances de classes du projet.

5.2.3.C. Événements de génération

Cette boîte de dialogue permet de configurer une commande pouvant être lancée automatiquement avant ou après la génération du projet.



Chacune des commandes peut être saisie dans la zone de texte correspondante. Les boutons *Modifier avant génération* et *Modifier après génération* ouvrent une fenêtre d'édition facilitant la saisie de la commande.



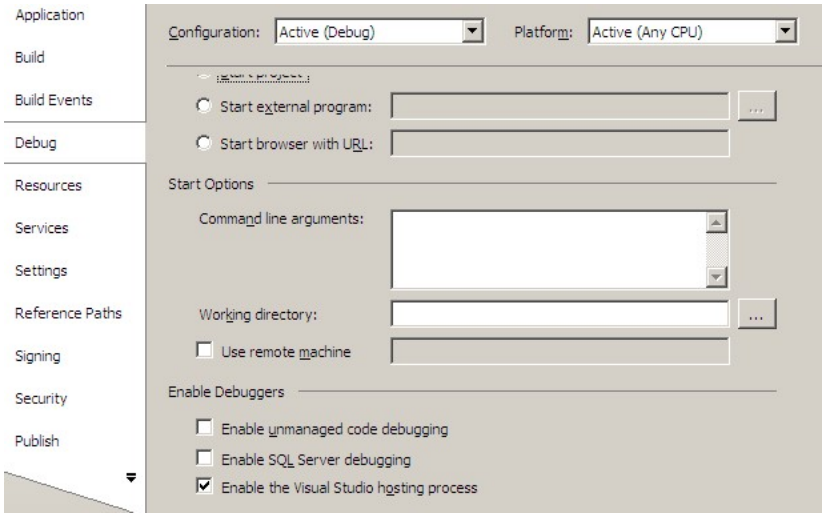
Cette boîte de dialogue propose également une liste de macros permettant la récupération et l'utilisation par la commande de certains paramètres du projet. L'exemple présenté sur la figure précédente effectue une copie complète du répertoire de l'application dans le répertooire de sauvegarde avant chaque génération.

L'exécution de la commande après génération peut être conditionnelle et avoir lieu uniquement en cas de génération réussie ou si la génération a mis à jour la sortie du projet.

Si une commande d'après génération doit exécuter un fichier .bat, l'appel de celui-ci doit être précédé du mot réservé *call*.

5.2.3.D. Propriétés de débogage

Les propriétés présentes dans cette partie déterminent le comportement du projet lors de on débogage.



Action de démarrage

Cette propriété détermine le comportement du projet lors du démarrage du déboggage. Trois options sont possibles :

- Démarrer le projet : indique que le projet lui-même doit être exécuté. Cette option n'est à utiliser que pour les projets d'application Windows ou les projets d'application *console*.
- Démarrer le programme externe : permet de provoquer l'exécution d'une application externe qui va se charger de faire des appels au code du projet. Cette option est utilisée pour le débogage de débogage de classes.
- Démarrer le navigateur avec l'URL : est identique à l'option précédente, mis à part que l'application démarrée est une application Web.

Options de démarrage

Arguments de la ligne de commande précise les arguments passés à l'application lors de son exécution par Visual Studio. Ces arguments peuvent être utilisés par le code pour déterminer l'action à entreprendre : par exemple, démarrer l'application en mode maintenance.

Répertoire de travail permet de spécifier le répertoire actif pendant l'exécution de l'application.

Utiliser l'ordinateur distant autorise le débogage d'une application s'exécutant sur une autre machine. Dans ce cas, le nom de la machine distante sur laquelle le code va s'exécuter est à indiquer.

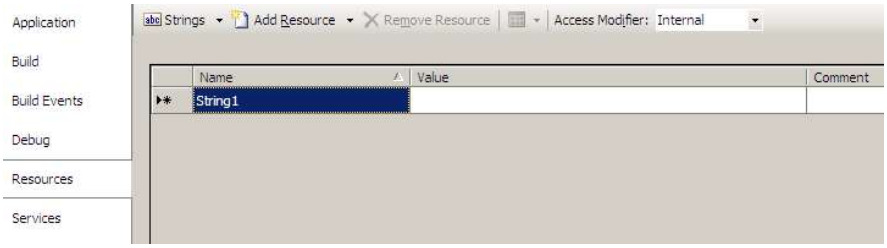
Activer les débogueurs

Ces options déterminent les différents types de débogueur actifs, en complément du débogueur de code managé de Visual Studio.

5.2.3.E. Ressources d'un projet

Les ressources sont utilisées pour externaliser certains éléments d'une application. Elles permettent de réaliser rapidement des modifications simples d'une application, sans avoir à rechercher dans des milliers de lignes de code.

L'utilisation la plus classique consiste à séparer, du code, les constantes chaînes de caractères. On peut également créer des ressources icônes, images, fichier texte ou audio. Toutes les ressources sont gérées par la boîte de dialogue suivante :



Pour chaque ressource, il faut indiquer un nom et une valeur. Le nom sera bien sûr utilisé dans le code pour pouvoir récupérer la valeur.

En fonctions du type de ressource, on a à sa disposition un éditeur adapté pour modifier la ressource. Les ressources peuvent être liées ou embarquées, en fonction de leur type. Une ressource liée est stockée dans son propre fichier et le fichier *resources.resx* contient simplement un lien vers le fichier original. Une ressource embarquée est stockée directement dans le fichier *Resources.resx* de l'application. Dans tous les cas, les ressources seront compilées dans l'exécutable de l'application.

Voyons maintenant comment accéder aux ressources à partir du code de l'application. Toutes les ressources sont accessibles par la propriété *Resources de l'objet My*. L'exemple suivant utilise :

- Une ressource chaîne de caractères (MessageBienvenueFr)
- Une ressource icône (IconAppli)
- Une ressource image bitmap (ImageFond)
- Un fichier son (Musique)

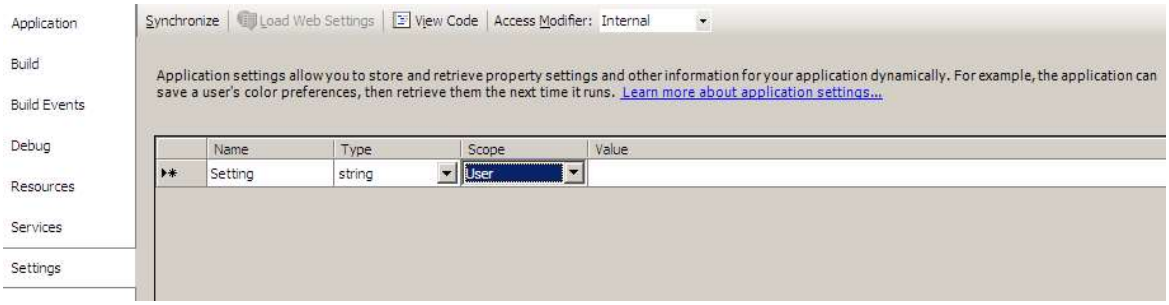
```
private void Form1_Load(object sender, EventArgs e)
{
      this.Icon = Vete_MDI.Properties.Resources.IconAppli;
      this.BackgroundImage = Vete_MDI.Properties.Resources.ImageFond;

      new SoundPlayer(Vete_MDI.Properties.Resources.Musique).PlayLooping();
      MessageBox.Show(Vete_MDI.Properties.Resources.MessageBienvenueFr);
}
```

5.2.3.F. Paramètres d'application

Les paramètres d'application sont, en général, utilisés pour stocker et charger dynamiquement les paramètres de configuration d'une application, comme par exemple, les préférences de l'utilisateur ou les derniers fichiers utilisés dans l'application.

Les paramètres doivent d'abors être créés dans la page de propriétés suivante :



Pour chaque paramètre, on doit fournir un nom utilisé pour manipuler le paramètre dans le code et un type, pour le paramètre. On doit également fournir une portée pour le paramètre. Deux choix sont possibles :

- Utilisateur : le paramètre peut être modifié pendant le fonctionnement de l'application.
- Application: le paramètre est en lecture seule pendant l'exécution et peut uniquement être modifié par l'intermédiaire de cette boîte de dialogue.

La dernière chose à faire est de spécifier une valeur pour le paramètre.

On va maintenant étudier comment manipuler les paramètres dans le code. On doit réaliser trois opérations :

- Au démarrage de l'application, on doit charger les paramètres. L'accès aux paramètres se fait par la propriété *Default* de l'objet *Settings* :

```
Vete_MDI.Properties.Settings.Default.Reload();
```

- Pendant l'exécution de l'application, on a accès aux parameters également par cette propriété *Default* de l'objet *Settings* à laquelle on ajouote le nom du paramètre. Ceci permet la lecture de la valeur du paramètre ou l'affectation d'une valeur au paramètre.

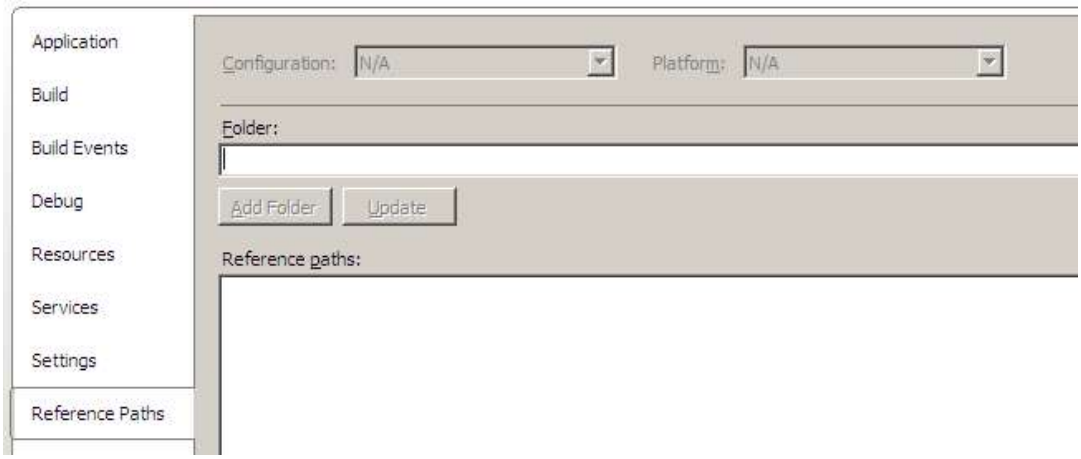
```
This.BackColor = Vete_MDI.Properties.Settings.Default. CouleurFond;
Vete_MDI.Properties.Settings.Default.DerniereUtilisation = DateTime.Now();
```

- À la fermeture de l'application, on doit enfin sauvegarder les parameters en utilisant la méthode *Save* :

```
Vete_MDI.Properties.Settings.Default.Save();
```

Chemin d'accès des references

Lorsque l'on reference un assembly dans le projet, Visual Studio commence par le rechercher dans le repertoire du projet. S'il ne le trouve pas dans ce répertoire, il va alors le rechercher dans le ou les répertoires que l'on a configuré dans la boîte de dialogue *chemin d'accès des références*.



Les répertoires sont parcourrus dans l'ordre de la liste. Cet ordre peut être modifié en sélectionnant un répertoire dans la liste et en le déplaçant vers le haut ou le bas à l'aide des flèches disponibles sur la droite de la boîte de dialogue.

Pour chaque utilisateur de l'application, une version distincte des paramètres est sauvegardée.

6. Les Bases du Langage

6.1. Les variables, constantes et énumération

6.1.1. Les variables

6.1.1.A. Le nom des variables

Règles à respecter pour nommer les variables :

- Le nom d'une variable commence obligatoirement par une lettre,
- Il peut être constitué de lettres, de chiffres et du caractère de soulignement '_',
- Il peut contenir un maximum de 1023 caractères,
- Il y a distinction entre majuscules et minuscules,
- Les mots réservés du langage ne peuvent pas être utilisés.

6.1.1.B. Le type des variables

Deux catégories de types de variables sont disponibles :

- **Les types valeur** : la variable contient réellement les informations.
- **Les types référence** : la variable contient l'adresse mémoire où se trouvent les informations. Les variables de type référence doivent être réellement créées par l'usage de l'opérateur *new*.

Les différents types de variables disponibles sont définis au niveau du Framework lui-même. On peut également utiliser les alias définis au niveau de Visual C# qui sont certainement plus explicites. Ainsi :

- Le type **System.Int32** défini au niveau du Framework peut être remplacé par le type **int** dans Visual C#.

Les différents types de variables peuvent être classés en 6 catégories :

- Les types numériques entiers,
- Les types décimaux,
- Les types caractères,
- Le type bool,
- Le type Object,
- Les types Nullables,

Les types numériques entiers

Les types entiers signés			
Sbyte	-128	127	8 bits
Short	-32768	32767	16 bits
Int	-2 147 483 648	2 147 483 647	32 bits
Long	-9223372036854775808	9223372036854775807	64 bits
Les types entiers non signés			
Byte	0	255	8 bits
Ushort	0	65535	16 bits
Uint	0	4294967295	32 bits
Ulong	0	18446744073709551615	64 bits

Table 3: Les types entiers signés et non signés

L'usage du type *int* permet d'optimaliser le temps d'exécution.

Les types décimaux

Les types décimaux			
float	-3.40282347^{E}+38	3.40282347^{E}+38	4 bytes
double	-1.7976931348623157^{E}+308	1.7976931348623157^{E}+308	8 bytes
decimal	-79228162514264337593543950335	79228162514264337593543950335	16 bytes

Table 4: Les types décimaux

L'usage du type *double* permet d'optimaliser le temps d'exécution.

Les types de caractères

Une variable de type *char* utilise deux bytes pour stocker le code Unicode du caractère. Dans ce code, les 128 premiers caractères sont identiques au jeu de caractère ASCII.

L'assignation d'une valeur à une variable de type *char* doit être effectuée par une constante de type *char*. Une constante de type *char* est encadrée par les caractères ' '.

Certaines valeurs spéciales de caractères sont identifiés par un *escape code* (code d'échappement). Un *escape code* est toujours introduit par le caractère \.

Les escape codes	
\'	' (simple quote)
\ "	" (double quote)
\\	\ (backslash)
\0	0 (zéro numérique)
\a	Alerte
\b	Backspace
\f	Saut de page
\n	Saut de ligne
\r	Carriage return
\t	tabulation horizontale
\v	Tabulation verticale

Table 5: Les escape codes

Pour pouvoir stocker des chaînes de caractères, il convient d'utiliser le type *string,* qui représente une suite de zéro à 2147483648 caractères. Les chaînes de caractères sont invariables car, lors de l'assignation d'une valeur à une chaîne de caractères, de l'espace est réservé en mémoire pour le stockage. Si, par la suite, cette variable reçoit une nouvelle valeur, le système lui assigne un nouvel emplacement en mémoire. Ce mécanisme est totalement transparent pour le programmeur et la variable fera toujours référence à la valeur qui lui a été assignée. Avec ce mécanisme, les chaînes de caractères peuvent avoir une taille variable. L'espace occupé en mémoire est automatiquement ajusté à la longueur de la chaîne de caractères.

Pour assigner une chaîne de caractères à une variable, le contenu de la chaîne doit être saisi entre " ":

```
strNom = "Dupont";
```

Si dans une chaîne de caractères, on doit utiliser des caractères spéciaux, on va utiliser les escapes codes. Cependant, il existe une autre méthode qui consiste à faire précéder la chaîne de caractères du symbole @. Le compilateur considère alors que l'ensemble des caractères se trouvant entre les " " doivent être utilisés tels quels. La seule limitation concerne le caractère " qui, s'il doit être utilisé, doit être doublé :

```
strChaine = "Que dit-il ? \rIl dit \"Bonjour\"";

strChaine = @"Que dit-il ?

Il dit ""Bonjour""";
```

Le résultat sera:

Dans la classe *string*, il existe de nombreuses fonctions qui permettent la manipulation des chaînes de caractères.

Le type bool

Le type boolean permet d'utiliser une variable qui peut prendre deux états: vrai/faux. L'assignation de valeurs se fait directement avec les valeurs *true* ou *false* comme dans l'exemple suivant :

```
bPresent = true;
bValide = false;
```

Il est toutefois possible d'assigner une valeur numérique à une variable de type *bool*. Dans ce cas, une valeur égale à zéro sera considérée comme une valeur *false* alors que toute autre valeur positive ou négative sera considérée comme *true*.

Le type Object

C'est peut-être le type le plus universel de Visual C#. Dans une variable de type Object, on peut stocker n'importe quoi. En fait, ce type de variable ne stocke rien. La variable va contenir non pas la valeur elle-même, mais l'adresse mémoire où l'on va trouver la valeur de la variable. Ce mécanisme est totalement transparent pour le programmeur qui n'aura jamais à manipuler lui-même les adresses.

Une variable de type Object pourra faire référence à n'importe quel autre type de valeur y compris des types numériques simples. Cependant, le code sera moins rapide du fait de l'utilisation d'une référence.

Les types Nullables

Il arrive parfois que dans certaines circonstances une variable n'ait pas de valeur bien définie. Comment représenter cette situation avec des variables en C# ?

Une solution consiste à utiliser une valeur n'ayant aucune signification pour l'application. Par exemple, pour une variable numérique représentant un code postal, on peut utiliser une valeur négative convenue pour indiquer l'absence de valeur pour cette variable. Le reste du programme doit évidemment tenir compte de cette standardisation.

Cependant, pour certaines variables, cette solution n'est pas envisageable. Prenons simplement le cas des variables de type *bool* pour lesquelles il n'y a que deux valeurs possibles. Ou encore l'exemple d'une variable numérique représentant un total : toutes les valeurs positives ou négatives sont envisageables.

Pour résoudre ce problèmes Visual C# propose les types Nullables. Ils permettent aux variables de type valeur de ne contenir aucune information. Pour activer cette fonctionnalité sur une variable, il faut simplement utiliser le caractère '?' à la suite de la variable comme dans l'exemple suivant :

```
int? dCodePostal;
```

Il faut être prudent lors de l'utilisation d'une variable de ce type et vérifier avant de l'utiliser si elle contient effectivement une valeur. Pour cela, il faut utiliser la propriété *HasValue* de la variable pour déterminer si elle contient effectivement une valeur. Si c'est le cas, cette valeur est disponible via la propriété *Value* de la variable. Cette propriété est en lecture seule car l'assignation d'une valeur se fait directement sur la variable.

```
dCodePostal = 1000;

if (dCodePostal.HasValue)
{
     Console.WriteLine(dCodePostal.Value);
}
else
{
     Console.WriteLine("Le code postal est vide");
}
```

Il est indispensable de tester la propriété *HasValue* avant l'utilisation de la propriété *Value* car si la variable ne contient pas de valeur, il y a déclenchement d'une exception. Contrairement à une variable classique, une variable nullable ne contient pas de valeur par défaut.

Une variable contenant une valeur peut revenir à l'état *non renseigné* si on lui affecte la valeur *null*.

L'utilisation des variables de type boolean nullable avec les opérateurs logiques peut parfois être problématique. Voici la table donnant les résultats pour chacune des possibilités :

B1	B2	B1 & B2	B1 | B2
Null	Null	Null	Null
Null	True	Null	True
Null	False	False	Null
True	Null	Null	True
True	True	True	True
True	False	False	True
False	Null	False	Null
False	True	False	True
False	False	False	False

Table 6: Combinaisons possibles entre variables bool nullables

Quelques propriétés :

- On ne peut pas créer un type *nullable* au départ d'un type référence : les types référence prennent déjà en charge les valeurs *null*.
- La syntaxe *T?* est le format abrégé pour *System.Nullable<T>*, où *T* est un type valeur. Les deux formats sont interchangeables.
- La propriété *Value* retourne une valeur si une valeur est assignée, sinon une *System.InvalidOperationException* est levée.
- Il est possible d'assigner une variable *nullable* à une autre variable de même type mais *non nullable*. Exemple :

```
DateTime? d = new DateTime(1900,01,01);
```

6.1.1.C. Conversions de type

Les conversions de type consistent à transformer une variable d'un type dans un autre type. Les conversions peuvent s'effectuer vers un type supérieur ou vers un type inférieur.

Si une conversion vers un type inférieur est utilisée, il risque d'y avoir une perte d'information. Par exemple, la conversion d'un type double vers un type long fera perdre la partie décimale de la valeur.

Pour limiter ce risque, le compilateur surveille les conversions réalisées dans le code et déclenche une erreur lorsqu'il rencontre une telle situation.

```
double lfX;
long lY;

lfX = 21.456;
lY = lfX;
```

Cannot implicitly convert type 'double' to 'long'. An explicit conversion exists (are you missing a cast?)

Ces conversions ne sont pourtant pas totalement interdites. On peut simplement avertir le compilateur de la volonté de réaliser une conversion explicite en utilisant une opération de transtypage. En fait, il n'y a pas d'opérateur spécifique pour le transtypage. C'est le type de données vers lequel on souhaite faire le transtypage qui doit être utilisé comme opérateur. Il suffit de préfixer la variable que l'on souhaite convertir avec le type cible en prenant la précaution de le placer entre parenthèses. L'exemple précédent devient donc :

```
double lfX;
long lY;

lfX = 21.456;
lY = (long) lfX;
```

L'usage de cette syntaxe ne provoque pas d'erreur de compilation car le compilateur considère que l'on réalise cette opération en toute connaissance de cause.

Conversion vers une chaîne de caractères

La fonction *format* de la classe *string* permet de choisir la forme du résultat de la conversion d'une valeur quelconque en chaîne de caractères. Cette fonction attend comme premier paramètre, une chaîne de caractères représentant le format dans lequel on souhaite obtenir le résultat. Le deuxième paramètre correspond à la valeur à convertir.

Certains formats standards sont prédéfinis mais il est également possible de personnaliser le résultat de la fonction *format*. Ci-après, les paramètres de cette fonction sont présentés.

Format	Description	Exemple
Currency (C ou c)	Format monétaire tel que défini dans les options régionales et linguistiques du panneau de configuration du système.	`string.Format("{0:c}", 12.35);` `12,35 €`
décimal (D ou d)		`string.Format("{0:d5}", 12);` `00012`

Fixed (F ou f)	Le séparateur décimal est celui défini dans les options régionales et linguistiques du panneau de configuration du système.	`string.Format("{0:f}", 2.5);`
		`2,50`
		`string.Format("{0:f2}", 2.5);`
		`2,50`

		`string.Format("{0:f0}", 2.5);` `2`
Général (G ou g)		`string.Format("{0:g}", 2.5);` `2,5`
Percent	Multiplie la valeur indiquée par 100 et ajoute le symbole '%' à la suite.	`string.Format("{0:p}", 0.2);` `20,00 %`
Standard (N ou n)	Format numérique tel que défini dans les options régionales et linguistiques du panneau de configuration du système.	`string.Format("{0:n}", 67565443.67);` `67.565.443,67`
Scientific (E ou e)	Notation scientifique.	`string.Format("{0:e}", 250000);` `2.500000E+005`
Hexadécimal (X ou x)	Format hexadécimal. Utilisable uniquement pour les valeurs entières.	`string.Format("{0:x}", 67565);` `107ed`
0	Réserve un emplacement pour un caractère numérique. Les zéros non significatifs sont affichés.	`string.Format("{0:00000000.0000}", 67565.6);` `00067565,6000`
#	Réserve un emplacement pour un caractère numérique. Les zéros non significatifs ne sont pas affichés.	`string.Format("{0:########.####}", 67565.6);` `67565,6`
.	Réserve un emplacement pour le séparateur décimal. Le caractère réellement utilisé dans le résultat dépend de la configuration des options régionales et linguistiques du panneau de configuration du système.	
,	Réserve un emplacement pour le séparateur de millier. Le caractère réellement utilisé dans le résultat dépend de la configuration des options régionales et linguistiques du panneau de configuration du système.	
G	Format Date court et format Heure tel que défini dans les options régionales et linguistiques du panneau de configuration du système.	`string.Format("{0:G}",DateTime.Now);` `18/05/2009 17:33:52`
D	Format Date longue tel que défini dans les options régionales et linguistiques du panneau de configuration du système.	`string.Format("{0:D}",DateTime.Now);` `lundi 18 mai 2009`
D	Format Date court tel que défini dans les options régionales et linguistiques du panneau de configuration du système.	`string.Format("{0:d}",DateTime.Now);` `18/09/2009`
T	Format Heure tel que défini dans les options régionales et linguistiques du panneau de configuration du système.	`string.Format("{0:T}",DateTime.Now);` `11:45:40`
s	Format triable	`string.Format("{0:s}",DateTime.Now);` `2009-05-18T17:54:42`

Table 7: Options de formatage de la fonction Format()

Option	Description

d	Jour du mois sans zéro non significatif
dd	Jour du mois avec zéro non significatif
ddd	Nom du jour de la semaine abrégé
dddd	Nom du jour de la semaine complet
M	Numéro du mois sans zéro non significatif
MM	Numéro du mois avec zéro non significatif
MMM	Nom du mois abrégé
MMMM	Nom du mois complet
h	Heure sans zéro non significatif (format 12H)
hh	Heure avec zéro non significatif (format 12H)
H	Heure sans zéro non significatif (format 24H)
HH	Heure avec zéro non significatif (format 24H)
m	Minute sans zéro non significatif
mm	Minute avec zéro non significatif
s	Seconde sans zéro non significatif
ss	Seconde avec zéro non significatif
y	Année sur un chiffre. Si, c'est le seul caractère de la chaîne de formatage, il faut dans ce cas utiliser %y
yy	Année sur deux chiffres
yyyy	Année sur quatre chiffres
zzz	Décalage par rapport au temps universel (GMT)

Table 8: Options de formatage de la fonction Format() spécifiques aux dates et heures

Pour convertir une date en chaîne de caractères, il est également possible de travailler au départ d'un objet de la classe *DateTime* comme suit :

```
DateTime oDateNaiss;
strDateNaiss = oDateNaiss.ToShortDateString();
```

Les fonctions disponibles sont :

- *ToLongDateString* : "jj Mois aaaa"
- *ToShortDateString* : "jj/mm/aaaa"
- *ToLongTimeString* : "hh:mm:ss"
- *ToShortTimeString* : "hh:mm"

Conversion depuis une chaîne de caractères

La conversion inverse, depuis une chaîne de caractères vers un type numérique, se fait à l'aide de la fonction Parse. Cette fonction est disponible dans les principales classes représentant les différents types numériques. Il faut donc utiliser la méthode Parse correspondant au type de données que l'on souhaite obtenir.

L'exemple suivant convertit une chaîne de caractères en type float :

```
float fTVA = float.Parse("18.6");
```

Lors de l'appel, on doit être certain que la conversion pourra être effectuée sans problèmes sinon, il y aura déclenchement d'une exception. Il est donc recommandé de gérer les exceptions lors de l'exécution de la fonction *Parse*.

Une alternative plus rapide consiste à utiliser la fonction *TryParse*. Cette fonction attend comme premier paramètre la chaîne de caractères à partir de laquelle on souhaite effectuer la conversion. Le deuxième paramètre correspond à la variable dans laquelle sera disponible le résultat de la conversion. À la différence de la fonction *Parse*, cette fonction ne génère pas d'exception si la conversion échoue. La fonction retourne simplement une valeur *false* et la

variable devant contenir le résultat de la conversion est initialisée à zéro. Si la conversion s'exécute correctement la fonction retourne une valeur *true* et la variable est initialisée avec le résultat de la conversion.

```
float fTVA;

if (float.TryParse("18A", out fTVA))
   Console.WriteLine("conversion OK");
else Console.WriteLine("problème pendant la conversion");
```

6.1.1.D. Déclaration des variables

Le compilateur considère que toute variable qui apparaît dans une application doit avoir été déclarée.

La syntaxe générale de déclaration d'une variable est la suivante :

```
Type prfNomVariable [= valeur initiale] [, prfNomVariable2]…;
```

Les valeurs entre crochets sont optionnelles.

Si la valeur initiale est omise, la variable sera automatiquement initialisée soit à zéro pour les variables numériques, à une chaîne de caractères vide pour les variables de type *string*, à la valeur *null* pour les objets de type *Object* et à *false* pour les variables de type *bool*. Malgré cette initialisation par défaut, il est recommandé d'effectuer une initialisation explicite de chaque variable avant de l'utiliser.

Ces règles ne s'appliquent pas aux variables déclarées à l'intérieur d'une fonction qui doivent être initialisées avant de pouvoir être utilisées. Cette initialisation peut avoir lieu au moment de la déclaration ou ultérieurement mais obligatoirement avant d'être utilisée.

```
int dA;

Console.WriteLine(dA);
```

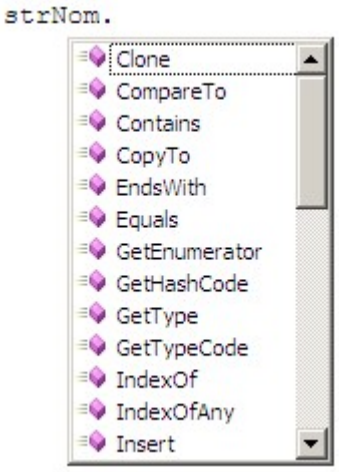
Use of unassigned local variable 'dA'

6.1.1.E. Inférence de type

Comme indiqué au paragraphe précédent, il est obligatoire de déclarer chaque variable avant de pouvoir l'utiliser. Cependant, dans certains cas, il est envisageable de laisser le compilateur faire une partie du travail. Grâce à l'*Inférence de Type,* le compilateur peut déterminer le type à utiliser pour une variable locale. Pour cela, il se base sur le type de l'expression utilisée pour initialiser la variable. Le nom de la variable doit, dans ce cas, être précédé du mot réservé *var*. Dans l'exemple ci-dessous, la variable est considérée comme une chaîne de caractères :

```
var strNom = "Dupont";
```

Pour s'assurer que cette variable est bien considérée comme une chaîne de caractères, il suffit de demander à IntelliSense ce qu'il propose comme fonction pour utiliser cette variable :



On obtient bien la liste des méthodes et des propriétés de la classe *String*.

Pour que l'inférence de type fonctionne correctement, il faut impérativement respecter quelques règles :

- L'inférence ne fonctionne que pour les variables locales,
- L'initialisation doit se faire sur la même ligne de code que la déclaration.

6.1.1.F. Portée des variables

La portée d'une variable est la portion de code à partir de laquelle l'on peut manipuler cette variable. Elle est fonction de l'emplacement où est située la déclaration.

Portée	Signification
Portée niveau bloc	Seul le code du bloc aura la possibilité de travailler avec la variable.
Portée niveau fonction	Seul le code de la fonction où est déclarée la variable pourra modifier son contenu.
Portée niveau classe	Une variable déclarée à l'intérieur d'une classe est utilisable par toutes les fonctions membres de cette classe sans restriction. En fonction du niveau d'accès, elle pourra être éventuellement utilisée à l'extérieur de la fonction. Cependant, dans tous les cas, un objet doit exister pour que les variables soient réellement existantes en mémoire et donc utilisées.

Table 9: Les différentes portées disponibles en C#

6.1.1.G. Les niveaux d'accès des variables

Le niveau d'accès d'une variable ne peut être utilisé que pour les données membres d'une classe. La table ci-dessous présente les valeurs disponibles.

Niveau d'accès	Signification
Public	Les éléments déclarés en *public* seront accessibles de n'importe quelle portion de code du projet dans lequel ils sont déclarés et de n'importe quel autre projet référençant celui dans lequel ils sont déclarés.
Protected	Dans ce cas, l'accès est restreint aux fonctions membres de la classe et à toutes les fonctions membres des classes dérivées.

Internal	Les éléments déclarés avec ce mot clé seront accessibles dans l'assemblage dans lequel ils sont déclarés.
Protected internal	Ce niveau d'accès est l'union des niveaux d'accès protected et internal. Il rend visible la variable à l'ensemble de l'assemblage dans lequel elle est déclarée et à toutes les classes dérivées.
Private	Ce mot clé restreint l'accès aux seules fonctions membres de la classe.

Table 10: Liste des niveaux d'accès

Si aucun mot clé n'est utilisé lors de la déclaration de la variable, elle est considérée comme *private*. Il est toutefois préférable de toujours spécifier un niveau d'accès explicite pour chaque variable.

6.1.1.H. Durée de vie des variables

La durée de vie d'une variable permet de spécifier pendant combien de temps une variable sera existante en mémoire. Pour une variable déclarée dans une fonction ou dans une procédure, la durée de vie correspond à la durée d'exécution de la procédure ou de la fonction. Dés la fin de l'exécution de la procédure ou de la fonction, la variable sera éliminée de la mémoire. Elle serre recrée lors du prochain appel de la procédure ou de la fonction. La durée de vie des variables déclarées dans les classes est associée à la durée de vie des objets de cette classe.

6.1.2. Les constantes

Dans une application, il arrive fréquemment que l'on utilise des valeurs numériques ou chaînes de caractères qui ne seront pas modifies pendant le fonctionnement de l'application. Il est conseillé, pour faciliter la lecture du code, de définir ces valeurs sous forme de constantes.

La définition d'une constante s'effectue par le mot réservé *const*.

```
const int dValeurMax = 100;
const string strMsg = "Trop grand";
```

La constante peut alors être utilisée dans le code à la place de la valeur qu'elle représente :

```
int dResult = 0;

 if (dResult > dValeurMax)
      Console.WriteLine(strMsg);
```

Les règles concernant la durée de vie et la portée des constantes sont identiques à celles concernant les variables.

La valeur d'une constante peut également être calculée à partir d'une autre constante :

```
const int dValeurMax = 100;
const int dDemi = dValeurMax / 2;
```

Dans ce cas de figure, il faut être prudent et ne pas créer de référence circulaire qui provoquerait une erreur de compilation.

6.1.3. Les énumérations

Une énumération va permettre de définir un ensemble de constantes qui sont liées entre elles. La déclaration s'effectue de la manière suivante :

```
enum etJours
{
```

```
        Lundi,
        Mardi,
        Mercredi,
        Jeudi,
        Vendredi,
        Samedi,
        Dimanche
}
```

Par défaut, la première valeur de l'énumération est initialisée à zéro. Les constantes suivantes sont ensuite initialisées avec un incrément de un. La déclaration précédente aurait pu s'écrire :

```
const int Lundi = 0;
const int Mardi = 1;
const int Mercredi = 2;
const int Jeudi = 3;
const int Vendredi = 4;
const int Samedi = 5;
const int Dimanche = 6;
```

La séquence d'incrémentation automatique dans une énumération peut être interrompue, voire ne pas être utilisée comme dans l'exemple suivant :

```
enum etColor {
     Rouge = 78,
     Bleu = 98,
     Vert = 14
}
```

Il faut toutefois que les valeurs utilisées dans une énumération soient des valeurs entières.

Une fois définie, une énumération peut ensuie être utilisée comme un type de variables spécifique :

```
etColor enColor;
```

Les seules valeurs que l'on peut assigner la variable *enColor* sont celles qui sont définies dans l'énumération.



Lorsque l'on souhaite faire référence à un élément de l'énumération, on doit le faire précéder du nom de l'énumération.



La déclaration d'une énumération ne peut pas se faire à l'intérieur d'une fonction !

La portée d'une énumération suit les mêmes règles que celle des variables (utilisation des mots réservés *public, private, internal, protected*).

6.1.4. Les Tableaux

Les tableaux vont permettre de faire référence à un ensemble de variables par un nom unique et d'utiliser un index pour accéder à chacune d'elle. Un tableau peut avoir une ou plusieurs dimensions. Le premier élément du tableau sera toujours accédé par un index de valeur zéro.

Le nombre d'éléments est spécifié lors de la création du tableau. La valeur maximale de l'index est égale à la taille de la dimension – 1.

6.1.4.A. Les tableaux à une dimension

La création d'un tableau s'effectue en deux étapes. La première consiste à déclarer une variable de type tableau. Cette déclaration se réalise comme pour une variable ordinaire sauf que l'on doit ajouter à la suite du type de données les caractères [et].

Voici par exemple, la déclaration d'un tableau de type *int* :

```
int [] tTabInt;
```

La deuxième étape va permettre de réserver une zone mémoire pour stocker le contenu du tableau. Cette opération nécessite l'usage de l'opérateur *new*. C'est lors de cette étape que l'on indique le nombre d'éléments que pourra contenir le tableau.

La ligne suivante crée un tableau de douze *int* :

```
tTabInt = new int [12];
```

Les éléments du tableau sont accessibles de la même manière qu'une variable classique. Il suffit d'ajouter au nom du tableau, entre les caractères [et], l'index de l'élément que l'on souhaite manipuler.

```
tTabInt[10] = 0;
```

Une autre solution existe pour la création du tableau. Elle permet simultanément la création du tableau et l'initialisation de son contenu. La syntaxe est la suivante :

```
float [] tTauxTVA = {0, 6, 21};
```

Dans ce cas, il n'est nul besoin de spécifier la taille du tableau. Le compilateur calcule automatiquement la taille en fonction du nombre de valeurs à charger dans le tableau.

6.1.4.B. Les Tableaux à plusieurs dimensions

La syntaxe de déclaration est similaire à celle d'un tableau à une dimension, mis à part que l'on doit définir le nombre de dimensions en ajoutant une ou plusieurs virgules lors de la déclaration du tableau en fonction du nombre de dimensions souhaité. À la création du tableau, il faudra spécifier la taille de chacune des dimensions.

```
int [,,] tTab;

tTab = new int [5, 10, 20];
```

L'accès à un élément du tableau s'effectue de manière identique, en indiquant les index permettant d'identifier l'élément du tableau souhaité :

```
tTab [1, 2, 3] = 0;
```

La syntaxe permettant l'initialisation d'un tableau à plusieurs dimensions au moment de sa déclaration suivra la règle suivante :

```
int [,] tGrille = {{1,2}, {3,4}, {5,6}};
```

Les tableaux ont toujours une taille fixe qui ne peut pas être modifiée sans création d'un nouveau tableau.

6.1.4.C. Manipulations courantes des tableaux

Lorsque l'on travaille avec les tableaux, certaines opérations doivent être fréquemment réalisées. Ce paragraphe décrit les opérations les plus courantes réalisées sur les tableaux.

Obtenir la taille d'un tableau

Il suffit d'utiliser la propriété *Length* du tableau pour connaître le nombre d'éléments qu'il peut contenir. Dans le cas d'un tableau multidimensionnel, le résultat correspond au nombre total de cases du tableau, soit le produit de la taille de chacune des dimensions. Il ne s'agit pas de l'espace mémoire occupé par le tableau mais bien le nombre total de cases du tableau.

```
Console.WriteLine(tGrille.Length);
```

Obtenir la taille d'une des dimensions du tableau

La méthode *GetLength* attend comme paramètre la dimension du tableau pour laquelle l'on souhaite obtenir la taille.

```
Console.WriteLine("Taille première dimension {0}", tGrille.GetLength(0));
Console.WriteLine("Taille deuxième dimension {0}", tGrille.GetLength(1));
```

Obtenir la dimension d'un tableau

La propriété *Rank* d'un tableau renvoi directement la dimension du tableau.

```
Console.WriteLine("Nombre de dimensions {0}", tGrille.Rank);
```

Rechercher un élément dans un tableau

La fonction *IndexOf* de la classe *Array* permet d'effectuer une recherche dans un tableau à une dimension. Elle accepte comme paramètres, le tableau dans lequel se fait la recherche et l'élément recherché dans le tableau. La valeur retournée correspond à l'index où l'élément a été trouvé dans le tableau ou -1 si l'élément ne se trouve pas dans le tableau.

```
string[] tAlim = { "pain", "beurre", "moutarde", "confiture" };

Console.WriteLine(Array.IndexOf(tAlim, "moutarde"));
```

Trier un tableau

La procédure *Sort* de la classe *Array* assure le tri du tableau à une dimension qu'elle reçoit en paramètre. Le tri s'effectue par ordre alphabétique pour les tableaux de chaîne de caractères et par ordre croissant pour les tableaux de valeurs numériques.

```
Array.Sort(tAlim);

foreach (string strAlim in tAlim)
{
    Console.WriteLine(strALim);
}
```

6.1.5. Les chaînes de caractères

Les variables de type *string* permettent la manipulation de chaînes de caractères par l'application. Voici les opérations les plus courantes.

6.1.5.A. Codage d'une chaîne de caractères sur plusieurs lignes

Pour encode une constante de type chaîne de caractères sur plus d'une ligne, il faut simplement utiliser l'opérateur + :

```
oCmd.CommandText = "Select * from RACE inner join ANIMAL on RACE.RaceId = "+
                   "ANIMAL.RaceId ";
```

6.1.5.B. Affectation d'une valeur à une chaîne

Pour assigner une valeur à une chaîne, il faut la spécifier entre les caractères " ".

```
string strExempl;

strExempl = "Il a dit : \"ça suffit\"";
```

6.1.5.C. Extraction d'un caractère particulier

Pour obtenir le caractère présent à une position donnée dans une chaîne de caractères, on va considérer la chaîne comme un tableau de caractères et ainsi utiliser un index pour accéder au caractère voulu. L'indexation commence à zéro comme pour les tableaux.

6.1.5.D. Obtention de la longueur d'une chaîne

Pour déterminer la longueur d'une chaîne, la propriété *Length* de la classe *string* est disponible;

```
Console.WriteLine("Longueur de la chaîne {0}", strExempl.Length);
```

6.1.5.E. Découpage de chaîne

La fonction *Substring* de la classe *string* retourne une portion de chaîne en fonction de la position de départ (le premier caractère ayant la position zéro) et du nombre de caractères à retourner.

```
Console.WriteLine("Sous-chaîne {0}", strExempl.Substring(6, 3));
```

6.1.5.F. Comparaison de chaînes

Plusieurs solutions sont possibles selon l'objectif à atteindre avec la comparaison des deux chaînes. Si le but est seulement de vérifier l'égalité de deux chaînes, on peut utiliser l'opérateur == ou la fonction *Equals* de la classe *string*.

```
string strCh1, strCh2;

if (strCh1 == strCh2)

if (strCh1.Equals(strCh2))
```

Pour réaliser un classement, on doit par contre utiliser la méthode *Compare* de la classe *string*. Les deux chaînes à comparer doivent être passées comme paramètres. Le résultat de la comparaison est retourné sous forme d'un entier inférieur à zéro si la première chaîne est inférieure à la deuxième, égal à zéro si les deux chaînes sont identiques et supérieur à zéro si la première chaîne est supérieure à la deuxième.

```
string.Compare(strChA, strChB);
```

6.1.5.G. Insertion dans une chaîne

La méthode *Insert* de la classe *string* permet l'insertion d'une chaîne dans une autre. Elle attend comme paramètre un entier et une chaîne et retourne la chaîne de départ dans laquelle se trouve insérée, à la position spécifiée, la chaîne passée en paramètre.

```
string strNew;

strNew = strExempl.Insert(24, "maintenant");
```

6.1.5.H. Suppression des espaces

- Au début de la chaîne : `strNew = strNew.TrimStart();`
- À la fin de la chaîne : `strNew = strNew.TrimEnd();`
- Au début et à la fin de la chaîne : `strNew = strNew.Trim();`

6.1.5.I. Changement de la casse

- Tout en majuscule : `strNew = strNew.ToUpper();`
- Tout en minuscule : `strNew = strNew.ToLower();`

6.1.5.J. Recherche dans une chaîne

La méthode *IndexOf* de la classe *string* permet la recherche d'une chaîne à l'intérieur d'une autre. Le paramètre correspond à la chaîne recherchée. La fonction retourne un entier indiquant la position à laquelle la chaîne a été trouvée ou -1 si la chaîne n'a pas été trouvée. Par défaut la recherche commence au début de la chaîne. La position est égale à zéro dans le cas où la chaîne recherchée commence à la première position.

Si on utilise une autre version de la méthode *IndexOf* qui, elle, attend deux paramètres : le premier paramètre étant la chaîne recherchée et le deuxième étant la position de départ de la recherche.

```
Console.WriteLine("Position de départ : {0}", strExempl.IndexOf("I"));
```

6.1.5.K. Remplacement dans une chaîne

Il est parfois souhaitable de pouvoir rechercher la présence d'une chaîne à l'intérieur d'une autre, comme dans l'exemple précédent, mais également de remplacer les portions de chaînes trouvées. La méthode *Replace* permet de spécifier une chaîne de substitution pour la chaîne recherchée. Elle attend deux paramètres :

- La chaîne recherchée,
- La chaîne de remplacement.

```
strExempl = strExempl.Replace("Il", "Elle");
```

6.1.6. Les structures

Les structures offrent la possibilité de combiner les données de différets types pour créer un nouveau type composite. Ce nouveau type pourra ensuite être utilisé dans la déclaration de variables comme n'importe quel autre type standard du C#.

Les structures sont très pratiques lorsque l'on souhaite manipuler des informations ayant un lien entre elles. Par exemple, les informations relatives à un client peuvent être facilement gérées sous forme d'une structure plutôt que par des variables séparées.

Différence entre une structure et une classe :



Une classe est un type référence. Lorsqu'un objet de la classe est créé, la variable à laquelle l'objet est assigné contient uniquement une référence à cette mémoire. Lorsque la référence d'objet est assignée à une nouvelle variable, la nouvelle variable fait référence à l'objet d'origine. Les modifications effectuées par le biais d'une variable sont répercutées dans l'autre variable car toutes deux font référence aux mêmes données.



Un struct est un type valeur. Lorsqu'un struct est créé, la variable à laquelle le struct est assigné contient les données réelles du struct. Lorsque le struct est assigné à une nouvelle variable, il est copié. La nouvelle variable et la variable d'origine contiennent par conséquent deux copies distinctes des mêmes données. Les modifications apportées à une copie n'affectent pas l'autre copie.

6.1.6.A. Déclaration d'une structure

La déclaration d'une structure se fait avec le mot réservé *struct* suivi par le nom de la structure. On doit ensuite placer un bloc à l'intérieur duquel sont déclarés les membres de la structure. Les membres de la structure sont, en fait, tout simplement des variables, des procédures ou des fonctions déclarées à l'intérieur de la structure.

Comme pour tout élément déclaré dans Visual C#, on a la possibilité de spécifier un niveau d'accès pour chaque membre de la structure. Sans information spécifique, le membre sera considéré comme étant *public*. Par contre, il est impossible d'initialiser les membres d'une structure au moment de la déclaration.

```
struct etClient
{
     public int dCode;
     public string strNom;
     public string strPrenom;
     public string strAdresse;
}
```

Les membres d'une structure peuvent eux-mêmes être des variables de type structure. Dans l'exemple précédent, l'adresse peut elle-même être décomposée comme suit :

```
struct etAdresse
{
     public string strRue;
     public int dNum;
     public string strLocalite;
     public int dCodPos;
}

struct etClient
{
     public int dCode;
     public string strNom;
     public string strPrenom;
     public etAdresse stAdresse;
}
```

Les structures acceptent également des fonctions comme membres. Elles sont généralement utilisées pour manipuler les variables membres de la structure.

```
struct etAdresse
```

```
{
      public string strRue;
      public int dNum;
      public string strLocalite;
      public int dCodPos;
      public string GetAdresse()
      {
            return dNum + " " + strRue + "\r\n" + dCodPos + "   " +
                        strLocalite;
      }
}
```

La définition d'une structure doit se placer à l'extérieur de toute fonction.

6.1.6.B. Utilisation des structures

Les structures sont utilisées comme des types de données classiques. Il convient, au préalable, de déclarer une variable de type structure :

```
etAdresse stAdresse;
```

Par la suite, cette variable permet l'accès aux membres de la structure grâce à l'opérateur ".", appelé *opérateur d'accès*.

```
stAdresse.dCodPos = 4000;
stAdresse.strLocalite = "Liège";
```

Si l'un des membres de la structure est lui-même de type structure, on doit également utiliser l'opérateur d'accès pour pouvoir manipuler les membres imbriqués :

```
stClient.stAdresse.dNum = 42;
```

Les types structure sont des types de données par valeur, c'est-à-dire que la variable contient les données de la structure (à mettre en opposition avec les variables de type référence où la variable ne contient que l'emplacement, en mémoire, où se trouvent les données).

Ceci permet d'assigner à une variable de type structure le contenu d'une autre variable de même type. Il y a dans ce cas, recopie des informations de chaque membre de la variable source dans le membre correspondant de la variable destination.

6.2. Les Opérateurs

Les opérateurs sont des mots réservés du langage permettant l'exécution d'opérations sur le contenu de certains éléments, en général, des variables, des constantes, des valeurs littérales ou des retours de fonctions.

La combinaison d'un ou plusieurs opérateurs avec les éléments sur lesquels les opérateurs vont s'appuyer se nomme une *expression*. Ces expressions sont évaluées au moment de leur exécution, en fonction des opérateurs et des valeurs qui sont associées.

Les opérateurs peuvent être répartis en 6 catégories.

6.2.1. Les Opérateurs d'assignation

Le seul opérateur disponible dans cette catégorie est l'opérateur =. Il permet d'assigner une valeur à une variable. Le même opérateur est utilisé quelque soit le type de la variable (numérique, chaîne de caractères, …).

6.2.2. Les Opérateurs arithmétiques

Les opérateurs arithmétiques permettent d'effectuer des calculs sur le contenu des variables :

Opérateur	Opération réalisée	Exemple	Résultat
+	Addition	6 + 4	10
-	Soustraction	12 – 6	6
*	Multiplication	3 * 4	12
/	Division	25 / 3	8.3333333333
%	Modulo	25 % 3	1

Table 11: Liste des opérateurs arithmétiques

6.2.3. Les Opérateurs binaires

Ces opérateurs effectuent des opérations sur des entiers uniquement (Byte, Short, Integer, Long). Ils travaillent au niveau du bit.

Opérateur	Opération réalisée	Exemple	Résultat
&	Et Binaire	45 & 255	45
|	Ou Binaire	99 ! 46	111
^	Ou exclusif	99 ^ 46	77
~	Négation	~23	-24

Table 12: Liste des opérateurs binaires

6.2.4. Les Opérateurs de comparaison

Les opérateurs relationnels sont utilisés dans les structures de contrôle d'une application (if, while, …). Ils renvoient une valeur de type boolean en fonction du résultat de la comparaison effectuée. Cette valeur sera ensuite utilisée par la structure de contrôle.

Opérateur	Opération réalisée	Exemple	Résultat
==	Égalité	3 == 6	False
!=	Différent de	7 != 8	True
<	Plus petit que	2 < 5	True
>	Plus grand que	2 > 5	False
<=	Plus petit ou égal	2 <= 5	True
>=	Plus grand ou égal	2 >= 5	False
Is	Comparaison du type de la variable avec un type donné	oClient is Client	True si la variable *oClient* est de type *Client*.

Table 13: liste des opérateurs de comparaison

6.2.5. L'Opérateur de concaténation

L'opérateur est utilisé pour la concaténation de chaînes de caractères. C'est le même opérateur qui est utilisé pour l'addition. Cependant, il n'y a pas de risque de confusion car Visual C# ne fait pas de conversion implicite des chaînes de caractères en numérique. Il détermine donc que si l'un des deux opérandes est une chaîne de caractères, c'est une concaténation qui doit être exécutée et cela même si une des opérandes représente une valeur numérique.

Le code suivant :

```
string strChaine = "1234";
Console.WriteLine(strChaine + 456);
```

Va afficher :

```
1234456
```

L'inconvénient de l'opérateur + est qu'il n'est pas très rapide pour la concaténation. Si on a de nombreuses concaténations à exécuter sur une chaîne, il est préférable d'utiliser la classe *StringBuilder*.

```
using System.Text;

long ldDuree;
string strLievre;
string strTortue = "";

DateTime oDebut, oFin;
oDebut = DateTime.Now;

for (int dI = 0; dI < 50000; dI++)
{
    strTortue += " " + dI;
}

oFin = DateTime.Now;

ldDuree = new TimeSpan(oFin.Ticks - oDebut.Ticks).Seconds;
Console.WriteLine("durée de la tortue : " + ldDuree + "s");

oDebut = DateTime.Now;
StringBuilder sbLievre = new StringBuilder();

for (int dI = 0; dI < 50000; dI++)
{
     sbLievre.Append(" ");
     sbLievre.Append(dI);
}
strLievre = sbLievre.ToString();
oFin = DateTime.Now;

ldDuree = new TimeSpan(oFin.Ticks - oDebut.Ticks).Seconds;
Console.WriteLine("durée pour le lièvre : " + ldDuree + "s");
```

```
durée de la tortue : 18s
durée pour le lièvre : 0s
```

6.2.6. Les Opérateurs logiques

Les opérateurs logiques permettent de combiner les expressions dans des structures conditionnelles ou de boucle.

Opérateur	Opération réalisée	Exemple	Résultat
&&	Et logique	If (expr1 && expr2)	Vrai si *expr1* et *expre2* sont vraies. L'*expr2* est évaluee uniquement si *expr1* est vraie.

||	Ou logique	If (expr1 || expr2)	Vrai si *expr1* ou *expre2* sont vraies. L'*expr2* est évaluee uniquement si *expr1* est fausse.
!	Not	If (!expr1)	Inverse le résultat de *expr1*.

Table 14: Liste des opérateurs logiques

6.2.7. Ordre d'évaluation des opérateurs

Lorsque plusieurs opérateurs sont combinés dans une expression, ils sont évalués dans un ordre bien précis. Les opérations aithmétiques sont exécutées en premier pluis les opérateurs de comparaison et enfin les opérateurs logiques.

Les opérateurs arithmétiques ont entre eux également un ordre d'évaluation. L'ordre d'évaluation est le suivant :

- Négation (-)
- Multiplication et division (*, /)
- Modulo (%)
- Addition et soustraction (+, -) et concaténation de chaînes (+)

Si la priorité de certains opérateurs doit être augmentée, alors il faut les placer entre parenthèses.

6.3. Les structures de contrôles

Les structures de contrôles permettent de modifier l'ordre d'exécution des instructions dans le code. Deux types de structures sont disponibles :

- **Les structures de décision** : elles aiguillent l'exécution du programme en fonction des valeurs que pourra prendre une expression de test.
- **Les structures de boucle** : elles feront exécuter une portion du code un certain nombre de fois, jusqu'à ce qu'une condition soit remplie ou tant qu'une condition est remplie.

6.3.1. Les structures de décision

6.3.1.A. La structure if

Quatre syntaxes sont utilisables pour l'instruction *if*.

Syntaxe	Signification
`if (expression)` `  Instruction;`	Si l'expression donne la valeur booleanne vraie alors l'instruction est exécutée.

`if (expression)`
`{`
`  Instruction 1;`
`  Instruction 2;`
`}`
	Si l'expression donne la valeur booleanne vraie alors le bloc d'instructions est exécuté.

`if (expression)`
`{`
`  Instruction 1;`
`  Instruction 2;`
	On peut spécifier une ou plusieurs instructions a être exécutées dans le cas où l'expression donne la valeur fausse.

}
else
{
Instruction 7;
Instruction 8;
}	

Table 15: Les syntaxes possibles de l'instruction if

6.3.1.B. La structure Switch

La structure *switch* permet un fonctionnement équivallent, mais offre une meilleure lisibilité du code. La syntaxe est la suivante :

```
switch (expression)
{
     case valeur1: séquence de code 1;break

     case valeur2: séquence de code 2;break;

     case valeur3: séquence de code 3;break;
     …
     case valeurn: séquence de code n;break;

     default: séquence de code
}
```

La valeur de l'expression est comparée avec chacune des valeurs. Lorsqu'une égalité est trouvée alors la séquence de code associée à la clause *case* sera exécutée. Si aucune égalité n'est trouvée alors si une clause *default* est définie, alors elle sera exécutée.

Chaque séquence de code doit se terminer par l'instruction *break*.

6.3.2. Les structures de boucle

Il existe quatre structures de boucle :

- while
- do… while
- for
- foreach

6.3.2.A. La structure while

La syntaxe de la boucle *while* est la suivante :

```
While (expression)
{
     Liste d'instructions;
}
```

Cette syntaxe permet d'exécuter la liste d'instructions tant que l'expression donne la valuer *vraie*. La condition est évaluée avant même le premier passage dans la boucle, donc le bloc de code pourra très bien ne jamais être exécuté si l'expression est fausse dés le départ.

Dans le cas où la condition est vraie au premier passage, le bloc de code sera exécuté et la condition sera à nouveau testée. Dans le cas contraire, la prochaine instruction exécutée sera celle qui suit le bloc de code.

Il est toutefois possible de provoquer une sortie prématurée de la boucle en utilisant l'instruction *break*. Dans ce cas, l'exécution continue avec l'instruction suivant immédiatement la boucle.

6.3.2.B. La structure do…while

La structure *do…while* utilise la syntaxe suivante :

```
do
{
     Liste d'instructions;
}
while (expression);
```

Cette syntaxe permet de garantir que le bloc de code sera exécuté au moins une fois puisque la condition sera testée à la fin du bloc de code.

6.3.2.C. La structure for

Lorsque l'on connaît le nombre d'itérations à réaliser dans une boucle, il est préférable d'utiliser la structure *for*. Pour pouvoir utiliser cette instruction, une variable compteur doit être déclarée. Cette variable peut être déclarée dans la structure *for* ou à l'extérieur. Elle doit dans ce cas être déclarée avant la structure *for*.

La syntaxe de la boucle *for* est la suivante :

```
for (expression initialisation; expression d'arrêt; expression
                                                    d'itération)
{
     Liste d'instructions;
}
```

L'expression d'initialisation est exécutée une seule fois lors de l'entrée dans la boucle. L'expression d'arrêt est évaluée lors de l'entrée dans la boucle et ensuite après chaque itération. Le résultat de cette expression détermine si ou non le corps de la boucle sera exécuté.

Après l'exécution du corps de la boucle, l'expression d'itération sera exécutée. Ensuite, on réexécute l'expression d'arrêt et ainsi de suite jusqu'à ce que cette dernière expression donne la valeur *false*.

L'instruction *break* peut être utilisée pour provoquer une sortie prématurée de la boucle. L'instruction *continue* permet quant à elle de revenir immédiatement à l'exécution de l'expression d'arrêt. Ces deux instructions doivent évidemment être exécutées de manière conditionnelle sinon les lignes situées après ne seront jamais plus exécutées.

6.3.2.D. La structure foreach

Une autre syntaxe de la boucle *for* permet d'exécuter un bloc de code pour chaque élément contenu dans un tableau ou dans une collection. La syntaxe générale de cette instruction est la suivante :

```
String[] tTabl = {"rouge", "bleu", "vert", "noir"};

for (int dI = 0; dI < tTabl.Length; dI++)
{
     Console.WriteLine(tTabl[dI]);
}
```

```
foreach (string strS in tTabl)
{
     Console.WriteLine(strS);
}
```

Attention : la variable utilisée pour parcourir le tableau doit obligatoirement être déclarée dans l'instruction *foreach* et non à l'extérieur.

6.3.2.E. La structure using (using statement)

Cette structure est destinée à accueillir un bloc de code utilisant une ressource externe, comme par exemple un fichier. Cette structure prend en charge automatiquement la libération de la resource à la fin du bloc de code. La ressource peut être créée dans la structure ou bien exister auparavant et être passée sous contrôle de la structure. À la fin de la structure, la ressource est libérée automatiquement par un appel à la méthode *dispose*.

```
using System.IO;

using (StreamWriter swFile = File.CreateText(Path))
{
       swFile.WriteLine("rouge");
       swFile.WriteLine("bleu");
       swFile.WriteLine("vert");
}
```

Autres exemples extraits du MSDN :

```
using System.Drawing;

using (Font MyFont = new Font("Arial", 10.0f),
       MyFont2 = new Font("Arial", 10.0f))
{
    // use MyFont and MyFont2
}   // compiler will call Dispose on MyFont and MyFont2

Font MyFont3 = new Font("Arial", 10.0f);

using (MyFont3)
{
    // use MyFont3
}
```

6.4. Les Procédures et les Fonctions

Dans une application Visual C#, toutes les instructions doivent obligatoirement être placées dans une procédure ou dans une fonction. Ces procédures ou fonctions permettent de créer des blocs de code qui pourront ensuite être utilisés à différents endroits de l'application.

Pour qu'elles soient plus facilement réutilisables, on a la possibilité d'utiliser des paramètres. Les valeurs de ces paramètres sont spécifiées lors de l'appel. La recommandation est d'en créer le plus possible.

Dans Visual C#, quatre types sont possibles :

- Les procédures *sub* qui exécutent simplement un code à la demande.
- Les fonctions qui exécutent un bloc de code et renvoient le résultat de leur calcul au code qui les a appelées.
- Les procédures de propriétés qui permettent de manipuler les données membres (propriétés) des objets créés dans l'application.

- Les procédures opérateur utilisées pour modifier le fonctionnement d'un opérateur lorsqu'il s'applique à une classe ou à une structure.

6.4.1. Les Procédures

Le code d'une procédure doit être placé dans un code délimité par des accolades. Pour pouvoir identifier ce bloc de code, il faut le faire précéder d'un nom qui sera par la suite utilisé pour faire appel à la procédure. Par défaut Visual C# ne sait utiliser que des fonctions, c'es-à-dire un bloc de code exécutant un traitement et renvoyant un résultat.

Pour pouvoir créer une procédure, il faut indiquer que le bloc de retournera aucun résultat en utilisant le type *void*. La syntaxe générale de déclaration d'une procédure est la suivante :

```
void AffichageResultat()
{
     Console.WriteLine("Procédure de test");
}
```

Les parenthèses après le nom sont utilisées pour spécifier les caractéristiques des paramètres qui seront passés lors de l'appel. Les parenthèses sont obligatoires dans la déclaration si aucun paramètre n'est requis pour la procédure.

De nombreux mots réservés sont utilisables dans la déclaration d'une procédure pour modifier les possibilités de réuilisation de cette procédure. La plupart d'entre eux sont liés a la programmation objet et seront étudiés ultérieurement. Par contre, pour modifier la visibilité de la procédure, on peut utiliser les mots réservés déjà utilisés pour la déclaration de variables : *private, public, internal*. Sans spécification particulière, une procédure sera considérée comme *public*.

Pour demander l'exécution d'une procédure, il suffit de spécifier son nom suivi entre parenthèses de la liste de ses arguments.

La déclaration d'une procédure doit toujours se faire à l'intérieur d'une classe.

6.4.2. Les Fonctions

Une fonction se déclare suivant le même principe qu'une procédure. Cependant, la fonction devant renvoyer un résultat à son code appelant, on doit indiquer le type du résultat retourné. Ce type de retour doit précéder le nom de la fonction (en remplacement du mot réservé *void* utilisé pour les procédures). N'importe quel type de données peut être utilisé comme type de retour d'une fonction.

La syntaxe de déclaration est la suivante :

```
type NonFonction(liste des paramètres)
{
     Instructions du corps de la fonction
}
```

Dans le code de la fonction, on va devoir spécifier quelle sera la valeur retorunée par la fonction. Pour cela, on devra utiliser l'instruction *return* en indiquant la valeur que l'on souhaite renvoyer. La valeur retournée sera le résultat de l'expression se trouvant indiquée juste après le *return*. L'exécution de l'instruction *return* provoque la sortie immédiate de la fonction et cela même si ce n'est pas la dernière instruction de la fonction;

Une fonction peut ensuite être utilisée dans une expression à la place d'une valeur du même type que celui renvoyé par la fonction. Le programme appelant n'est pas tenu d'utiliser le résultat de la fonction ; dans un tel cas, le résultat sera perdu et la fonctin sera utilisée comme une procédure.

6.4.3. Procédures de propriétés

Les procédures de propriétés vont permettre d'ajouter une propriété à une classe, un module ou une structure. Ces procédures sont appelées *accesseurs*. Elles seront utilisées lorsque l'on modifie (*set*) ou que l'on utilise (*get*) la propriété.

Leur utilisation semble similaire à l'utilisation d'une variable : on peut affecter une valeur à une propriété ou lire la valeur d'une propriété. Cependant, il existe de nombreuses différences importantes entre les variables et les propriétés :

VARIABLE	PROPRIETE
Les variables ne nécessitent qu'une seule ligne de code pour la déclaration.	Les propriétés nécessitent un bloc de code pour la déclaration.
L'accès à une variable s'effectue directement.	L'accès à une propriété implique l'exécution d'une portion de code.
Le contenu d'une variable est toujours récupéré tel quel.	Le contenu d'une propriété peut être modifié par le code, lors de l'accès à la propriété.

Table 16: Différences entre variable et propriété

La syntaxe générale de création d'une propriété est la suivante :

```
public type NomPropriété
{
     get
     {
          …
     }
     Set
     {
          …
     }
}
```

Get et *set* sont appelés *accesseurs.*

Dans cette déclaration :

- *NomPropriété* correspond au nom par lequel la donnée membre/propriété est manipulable dans le code.
- *Type* correspond au type de données associé à la propriété. On peut utiliser n'importe quel type de données pour une propriété (les types de base du langage ou un type personnalisé comme par exemple une classe).

La déclaration d'une propriété ressemble beaucoup à la déclaration d'une fonction. La petite différence réside dans les deux blocs de code *get* et *set* placés à l'intérieur :

- Le bloc *get* contient le code exécuté lors de la lecture de la propriété. Il doit obligatoirement contenir une instruction *return* pour fournir la valeur de la propriété.
- Le bloc *set* contient le code exécuté lors de l'assignation d'une valeur à la propriété. Dans ce bloc de code, une variable locale nommée *value* est automatiquement disponible et contient la valeur qui doit être assignée à la propriété.

Comme tout élément déclaré dans Visual C#, on peut spécifier un modificateur de niveau d'accès pour une propriété. Il s'applique automatiquement aux blocs *set* et *get*. On peut également spécifier un modificateur de niveau d'accès

pour chacun des blocs *get* et *set*. Dans ce cas, ils doivent être plus restrictifs que celui indiqué au niveau de la propriété. De plus, seul un des deux blocs pourra avoir un niveau d'accès plus restrictif :

```
int m_dVar;

public int dVar
{
     private get
     {
         return m_dVar;
     }
     set
     {
         m_dVar = value;
     }
}

dVar = 20 ;
```

Les propriétés peuvent également être en lecture seule ou en écriture seule. On doit dans ce cas, éliminer le bloc de code *set* dans le cas d'une propriété en lecture seule, et le bloc *get* dans le cas d'une propriété en écriture seule.

Les propriétés peuvent être implémentées automatiquement lorsqu'aucun traitement n'est nécessaire dans les blocs *get* et *set*. La propriété se déclare alors de la manière suivante :

```
int m_dTotal;

public int m_dTotal { get; set; }
```

Lorsque l'on déclare ainsi une propriété, le compilateur crée un espace de stockage privé et anonyme qui peut être accédé uniquement via les accesseurs *get* et *set* de la propriété.

6.4.4. Les procédures Opérateur

Ce type de procédure permet la redéfinition d'un opérateur standard du langage pour l'utiliser sur des types personalisés (classe ou structure). Imaginons la structure client déjà utilisée :

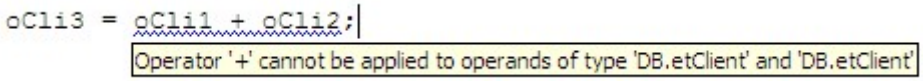
```
struct etClient
{
     public int dCode;
     public string strNom;
     public string strPrenom;
     public etAdresse stAdresse;
}
```

Essayons le code suivant :

```
etClient oCli1, oCli2, oCli3;

oCli3 = oCli1 + oCli2;
```

Le compilateur n'apprécie pas car il n'a aucune idée du comment exécuter l'opérateur d'addition sur deux clients :



Pour que ce code fonctionne, on doit lui indiquer la procédure à suivre pour réaliser cette opération d'addition entre deux objets client. On doit donc redéfinir l'opérateur '+' pour l'utiliser avec deux objets clients :

```
struct etClient
{
      public int dCode;
      public string strNom;
      public string strPrenom;
      public etAdresse stAdresse;
      public static etClient operator +(etClient stCli1, etClient stCli2)
      {
            etClient stTemp;

            stTemp.dCode = stCli1.dCode + stCli2.dCode;
            stTemp.strNom = stCli1.strNom + stCli2.strNom;
            stTemp.strPrenom = stCli1.strPrenom + stCli2.strPrenom;
            return stTemp;
      }
}
```

6.4.5. Les Arguments des procédures et des fonctions

Pour que le code soit plus facilement réutilisable, les valeurs qui sont manipulées par les procédures et les fonctions peuvent être passées comme paramètre au moment de l'appel de la procédure ou de la fonction. Lors de la déclaration de la procédure, on doit spécifier la liste des paramètres qui seront attendus.

Cette liste est située entre les parenthèses qui suivent le nom de la procédure. On doit indiquer pour chaque paramètre, son nom et son type. Si plusieurs paramètres sont attendus, il convient de les séparer par une virgule. Dans le code de la procédure, les paramètres sont considérés comme des variables déclarées localement.

```
double lfPrixHT = 100;
double lfPrixTTc;

lfPrixTTc = CalculTTC(lfPrixHT, 5.5);
Console.WriteLine(lfPrixTTc);

public static double CalculTTC(double lfPht, double lfTaux)
{
      return lfPht * (1 + (lfTaux / 100));
}
```

Lors de l'appel de la procédure, une valeur, pour chacun des paramètres attendus, devra être indiquée. Pour passer une variable comme paramètre à une procédure, il existe deux possibilités :

- **Le passage par valeur** : dans ce cas, l'information transmise à la procédure sera simplement le résultat de l'expression passée comme argument.

- **Le passage par référence** : dans ce cas, l'information transmise à la procédure n'est plus le contenu d'une variable mais l'emplacement où est stockée la variable (son adresse); le code de la procédure va donc chercher à cet emplacement la valeur dont elle a besoin. Le code de la procédure peut également modifier le contenu de la variable et, dans ce cas, les modifications seront visibles dans le code qui a appelé la procédure.

Par défaut, c'est le type du paramètre qui détermine la technique utilisée. Les types suivants : numériques entiers, numériques virgule flottante, décimal, bool et structures sont passées par valeur. Les autres types sont passés par référence.

Il est cependant possible de forcer le passage par référence d'un ou de plusieurs paramètres en utilisant le mot réservé *ref* ou *out* lors de la déclaration du paramètre dans la fonction. Cette solution est utilisée pour que toute modification apportée au paramètre dans la procédure soit reflétée dans le code appelant lorsqu'il récupère le contrôle.

```
public static double CalculTTC(double lfPht,
                                double lfTaux,
                                ref double lfTVA)
 {
     lfTVA = lfPht * (1 + (lfTaux / 100));
     return lfPht + lfTVA;
 }
```

L'utilisation du mot réservé *ref* dans la déclaration d'une fonction impose deux contraintes lors de l'appel de la fonction :

- Ce mot réservé doit également être utilisé lors de l'appel.
- La variable doit obligatoirement être initialisée avant l'appel.
- Dans la fonction, le paramètre *ref* doit obligatoirement être modifié.

```
double lfPrixHT = 100;
double lfPrixTTc;
double lfMontantTVA = 0;

lfPrixTTc = CalculTTC(lfPrixHT, 5.5, ref lfMontantTVA);
Console.WriteLine(lfMontantTVA);
Console.WriteLine(lfPrixTTc);
```

Le mot réservé *out* présente un fonctionnement similaire hormis la contrainte d'initialisation obligatoire qui ne s'applique pas. Mais, dans la fonction, il est obligatoire de modifier la valeur du paramètre associé à *out*.

Le passage par référence ne fonctionne pas si l'information passée à la fonction est une propriété qui n'est pas considérée comme une variable.

Une autre possibilité permet de créer une procédure qui pourra prendre un nombre quelconque de paramètres. Dans ce cas, il faut déclarer un tableau de paramètre au moyen du mot réservé *Params*. Seulement un seul mot réservé *Params* est autorisé par fonction et à la suite de ce tableau de paramètre, aucune autre déclaration de paramètre n'est autorisée : il doit être le dernier de la liste.

```
public static void UseParams(params int[] tList)
{
     for (int dI = 0; dI < tList.Length; dI++)
         Console.WriteLine(tList[dI]);
     Console.WriteLine();
}

public static void UseParams2(params object[] tList)
{
     for (int dI = 0; dI < tList.Length; dI++)
         Console.WriteLine(tList[dI]);
     Console.WriteLine();
}


UseParams(1, 2, 3);
UseParams2(1, 'a', "test");
```

```
int[] tMyArray = new int[3] { 10, 11, 12 };
UseParams(tMyArray);
```

6.5. Assemblies, Namespace et attributs

6.5.1. Les Assemblies

Visual C# est conçu autour du framework .NET, ce qui lui permet de bénéficier de nombreux avantages notamment en terme de sécurité, lors de l'exécution et de la gestion de la mémoire. Cette imbrication dans le framework permet également d'assurer la compatibilité entre du code écrit dans les différents langages disponibles. On peut ainsi utiliser dans Visual C# des éléments conçus avec d'autres langages et inversément. Cela de manière totalement transparente.

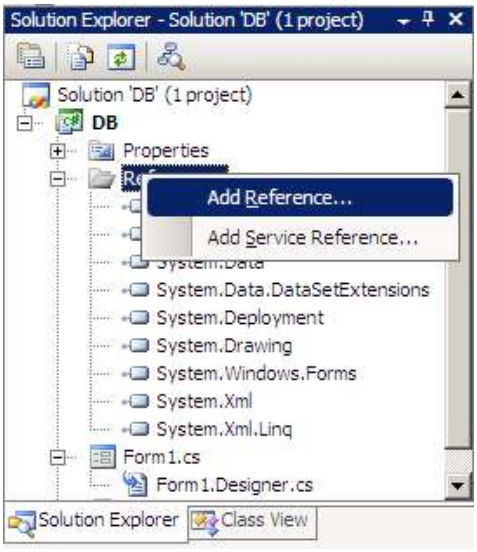
L'élément de base de cette réutilisation dans le framework .NET est *l'assembly*. Il peut être considéré comme le regroupement de types, de ressources et de fonctionnalités conçus pour fonctionner ensemble.

Les *assemblies* sont stockés dans des fichiers .exe ou .dll en fonction du type. Ils sont simplement générés par la compilation du projet correspondant.

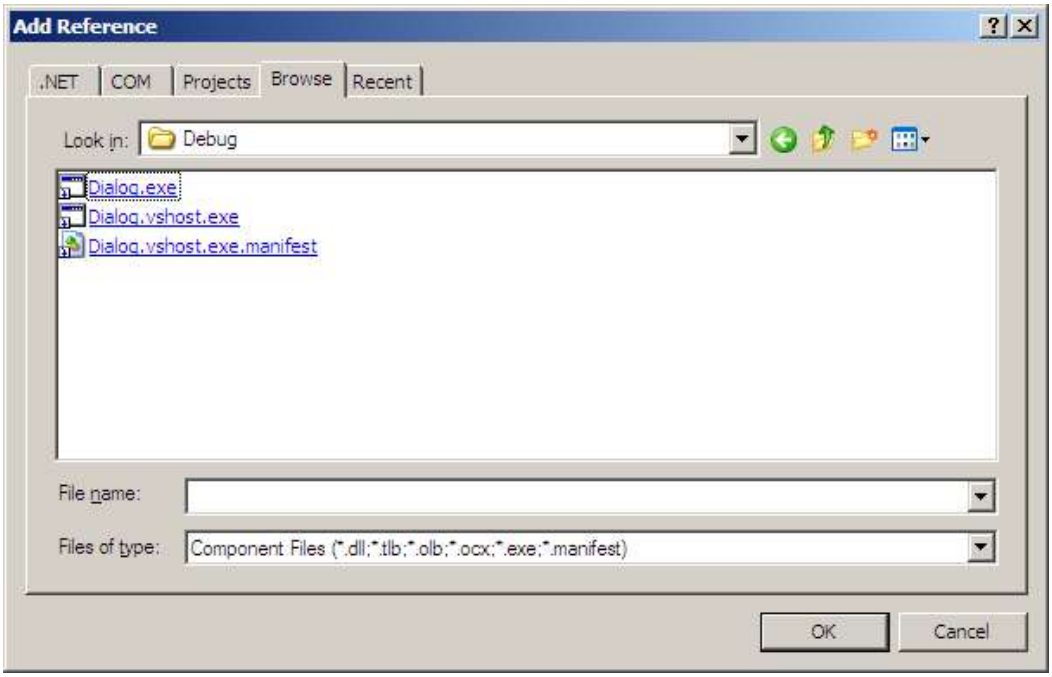
Ils sont autodescriptifs car ils contiennent les informations nécessaires pour leur utilisation dans un autre projet. Ces informations sont contenues dans le *manifest* de l'assembly. Ce *manifest* contient entre autres :

- L'identité du l'*assembly* : son nom et sa version.
- Une liste des fichiers utilisés par l'*assembly* : par exemple, les autres *assemblies* utilisés par celui-ci, les ressources bitmap, …

Pour pouvoir utiliser un assembly dans un projet, il faut simplement ajouter une référence vers cet *assembly*. Pour cela, il faut utiliser le menu contextuel du dossier référence du projet.



La boîte de dialogue suivante permet alors de choisir les références à ajouter au projet :



Les différents onglets permettent de choisir, par catégorie, le type de référence à ajouter au projet :

- **.NET**: l'ensemble des composants du Framework .NET disponibles.
- **COM** : Les composants COM et ActiveX enregistrés sur le système.
- **Projects** : les autres projets de la solution courante.
- **Browse** : recherche d'un fichier contenant les ressources.

Il est possible d'ajouter plusieurs références simultanément, en utilisant la touche [ctrl] lors de la sélection dans cette boîte de dialogue.

Après avoir réalisé ces deux opérations, les ressources présentes dans l'*assembly* sont directement accessibles dans le code du projet.

6.5.2. Les Namespaces

Les *namespaces* organisent logiquement les objets disponibles dans un assembly. Ils sont utilisés pour lever les ambiguïtés lorsque dans un projet, des références sont ajoutées sur des assemblies contenant des éléments ayant des noms identiques.

Par exemple, la classe *ListBox* existe dans les assemblies *System.Web* et *System.Windows.Forms*. Si des références sont ajoutées dans un projet vers ces deux assemblies, le compilateur risque de ne pas pouvoir déterminer laquelle de ces classes on souhaite réellement utiliser.

L'utilisation du nom pleinement qualifié, incluant le *namespace* dans lequel la classe est définie, permet de résoudre ce genre de problème.

On peut par exemple utiliser le code suivant :

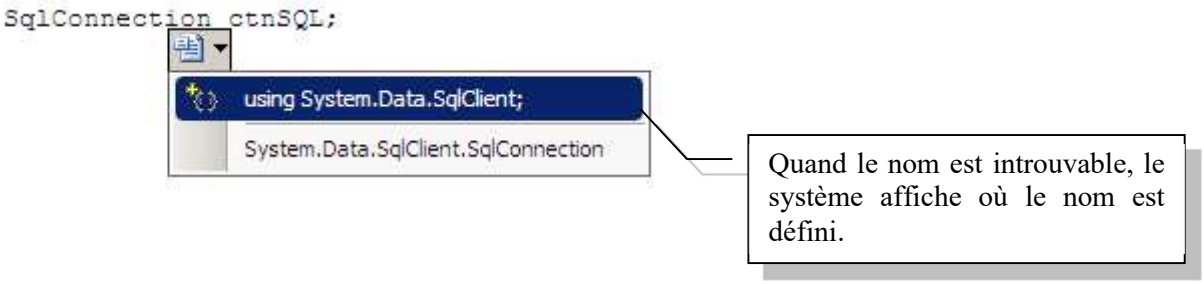
```
System.Windows.Forms.ListBox lstWindows;
```

Cependant, l'usage du nom pleinement qualifié peut devenir pesant dans l'écriture du code. Il est possible d'utiliser le mot réservé *using* pour alléger le code. Il indique au compilateur que certains *namespaces* sont sous-entendus.

Par exemple, l'instruction *using System.Data.SqlClient* autorise l'utilisation de la déclaration suivante :

```
SqlConnection ctnSQL;
```

Sans l'importation du *namespace* aurait provoqué une erreur de compilation :

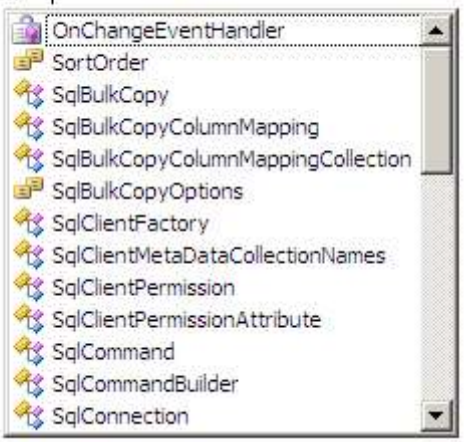


Les instructions *using* doivent être les premières lignes de code d'un fichier source Visual C#.

Quand un nom est défini dans deux *namespaces* différents et que tous deux sont inclus dans la source, alors on peut avoir un problème d'ambiguïté. Dans ce cas, il est possible d'ajouter un alias dans l'usage de la clause *using*.

```
using ctrlSQL=System.Data.SqlClient;

ctrlSQL.
```



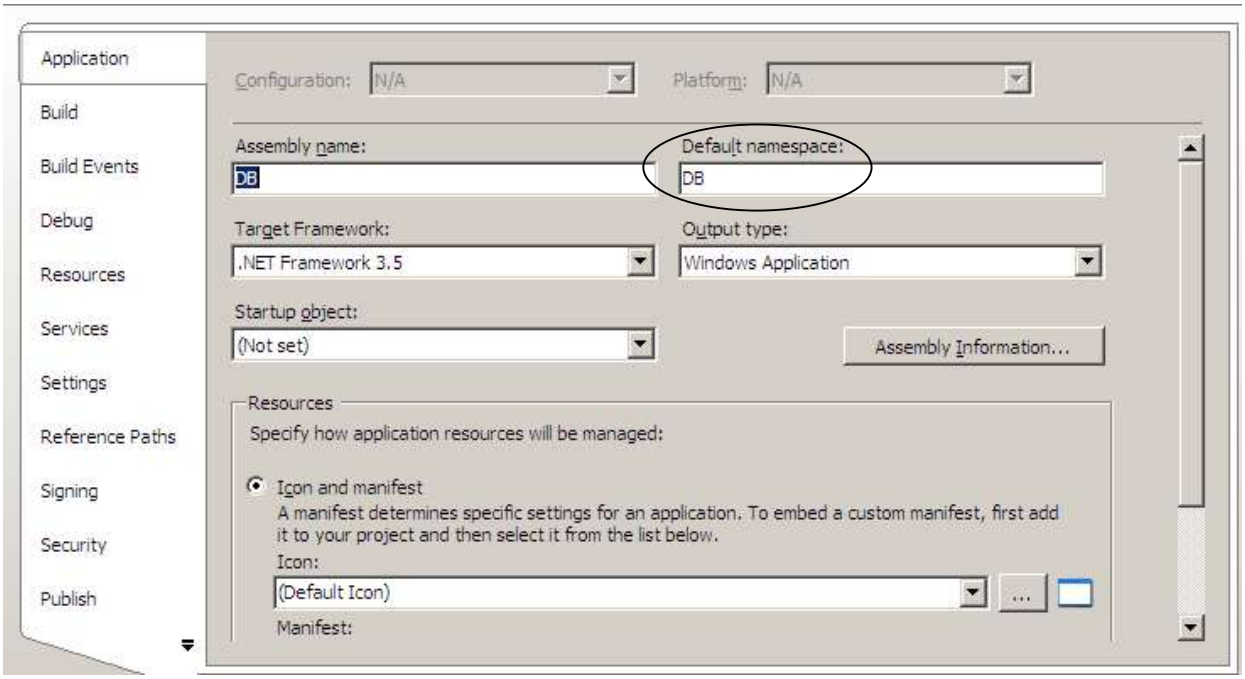
Cette solution autorise l'utilisation de noms d'une longueur raisonnable en évitant les conflits. Il est également à noter que, en fonction du type de projet sur lequel on travaille, des références et des importations sont réalisées par défaut.

Les *namespaces* seront déclarés dans le code à l'aide du mot réservé *namespace* suivi du nom du *namespace* et d'un bloc de code. Tous les éléments déclarés dans ce bloc de code seront accessibles en les préfixant avec le nom du *namespace*.

```
namespace Facturation
{
     class Tarification
     {
          public static double CalculTTC(double lfPht,
```

```
                                                double lfTaux,
                                                ref double lfTVA)
            {
                  lfTVA = lfPht * (1 + (lfTaux / 100));
                  return lfPht + lfTVA;
            }
      }
}
```

Dans l'exemple ci-dessus, la fonction *CalculTTC* définie dans la classe *Tarification* est accessible en la préfixant par le nom du namespace. Il faut également noter que Visual Studio ajoute automatiquement une instruction namespace dans le code de tous les éléments que l'on ajoute au projet. Il utilise comme nom les informations indiquées au niveau des propriétés du projet.



La fonction *CalculTTC* est accessible par le code suivant :

```
static void Main()
{

      double lfTaux = 21;
      double ltPrixTTC = Facturation.Tarification.CalculTTC(100,
                                                            5.5,
                                                            ref lfTaux);
```

On peut également avoir des namespaces imbriqués.

```
namespace Gestion
{
    namespace Paye
    {
        class Salaire
```

```
            {
            }
        }
        namespace Facturation
        {
            class Facture
            {
            }
        }
    }
```

6.6. Les Attributs

Les attributs sont des marques que l'on peut placer dans le code afin d'ajouter des informations supplémentaires aux éléments de l'application.

Ils sont sauvegardés dans les métadonnées de l'assembly pendant la compilation du projet. Les métadonnées sont utilisées par le runtime pour gérer le débogage, le suivi des versions, la compilation et d'autres informations sur l'utilisation du code. Les attributs peuvent s'appliquer à un assembly, un module ou une portion de code plus petite, telle qu'une procédure ou une fonction. Ils pourront parfois accepter des arguments pour modifier leur signification.

Les attributs sont placés dans le code entre les symboles "[" et "]". Si plusieurs attributs sont utilisés, ils doivent être séparés par des virgules. Les éventuels paramètres d'un attribut seront placés entre parenthèses.

La portée d'un attribut peut également être étendue par les mots réservés *Assembly:* ou *Module:* placés devant l'attribut. La syntaxe d'utilisation d'un attribut est donc :

```
Portée: [Attribut(paramètre1,…), Attribut2, …]
```

6.6.1. Les attributs les plus courants en Visual C#

Parmi les attributs disponibles, certains d'entre eux sont très fréquemment utilisés dans le développement avec Visual C#.

6.6.1.A. Serializable, NonSerialized Attribute

Ces deux attributs contrôlent la sérialisation d'une classe et de ses membres. La sérialisation permet l'enregistrement d'un objet dans un fichier. Le fichier généré peut être au format binaire ou XML. Dans ce dernier cas, il facilite l'échange d'informations entre applications.

Pour qu'une classe soit utilisable par le mécanisme de sérialisation, celle-ci doit être marquée avec l'attribut *Serializable*. Lors de l'opération de sérialisation, le contenu de chacun des membres de l'objet de la classe sera enregistré dans le fichier. Si certains d'entre eux ne doivent pas être sauvegardés dans le fichier, ils doivent être marqués avec l'attribut *NonSerializable*.

L'exemple ci-dessous définit la classe *Personne* avec deux membres, *Nom* et *Prenom* qui seront sérialisés et un membre *Age* qui ne le sera pas. Un objet de la classe est créé puis sauvegardé dans un fichier au format XML.

```
namespace Paye
{
     class Salaire
     {
     }
     [Serializable()] class Personne
     {
           public string m_strNom;
```

```
            public string m_strPrenom;
            [NonSerialized()] public int m_dAge;
            public Personne()
            {
            }
      }
}

using System.Runtime.Serialization;
using System.Runtime.Serialization.Formatters;
using System.Runtime.Serialization.Formatters.Soap;

Personne oPersonne;

oPersonne = new Personne();
oPersonne.m_strNom = "Dupont";
oPersonne.m_strPrenom = "Paul";
oPersonne.m_dAge = 20;

Stream oFlux;

oFlux = File.Open("c:\\data.xml", FileMode.Create);
SoapFormatter oFormateur;
oFormateur = new SoapFormatter();
oFormateur.Serialize(oFlux, oPersonne);
oFlux.Close();
```

6.6.1.B. Static

Le mot reserve *static* a exactement la même signification qu'en C++. Il permet de spécifier qu'un membre de la classe est applicable non pas à un objet de la classe mais bien à la classe elle-même.

Voici un exemple extrait du MSDN :

```
class Test
{
   int x;
   static int y;
   void F()
   {
      x = 1;          // Ok, same as this.x = 1
      y = 1;          // Ok, same as Test.y = 1
   }
   static void G()
   {
      x = 1;          // Error, cannot access this.x
      y = 1;          // Ok, same as Test.y = 1
   }
}

static void Main()
{
      Test t = new Test();
      t.x = 1;         // Ok
      t.y = 1;         // Error, cannot access static member through instance
      Test.x = 1;      // Error, cannot access instance member through type
      Test.y = 1;      // Ok
}
```

Les méthodes propriété peuvent également être utilisées comme *static* :

```
        private static string m_strLocation;

        public static string prLocation
        {
             get
             {
                 return m_strLocation;
             }
             set
             {
                 m_strLocation = value;
             }
        }
```

Dans l'exemple précédent, la propriété *prLocation* sera utilisée avec le nom de la classe :

```
        ManageDB.prLocation = strFileName;
```

6.6.2. Exemple

Prenons un un exemple pour illustrer les attributs en associant une aide à la classe *X*. Cette classe est développée et distribuée par la société *Cie* (d'où utilisation de l'espace de noms *Cie*).

```
namespace Cie
{
    [Aide("Cette classe fait cci ou encore cela")]
    public class X
    {
        [Aide("f ne fait pas grand chose")]
        public void f()
        {
        }
    }
}
```

Pour que *Aide* puisse être un attribut, il faut créer dans le même espace de noms, une classe dérivée de *System.Attribute*. La classe en question pourrait s'appeler *Aide* mais Microsoft recommande de suffixer ce nom par *Attribute*. Cette classe (Aide ou mieux AideAttribute) doit avoir un constructeur avec un argument de type *string* (pour spécifier le message d'aide) ainsi qu'une ou plusieurs propriétés. Nous ajoutons tout cela dans l'espace de nom précédent :

```
    public class AideAttribute : System.Attribute
    {
        private string m_strMsgAide;

        public msgAide(string strMsg)
        {
            m_strMsgAide = strMsg;
        }

        public string prMsgAide
        {
            get
            {
                return m_strMsgAide;
            }
        }
    }
```

La directive *AttributeUsage* indique à quel genre d'éléments peut s'appliquer l'attribut : Assembly, Module, Class, Struct, Enum, Constructor, Method, Property, Field, Event, Interface, Parameter, Return, Delegate, All ou ClassMembers. Un attribute pourrait également s'appliquer à plusieurs évènements :

```
[AttributeUsage(AttributeTargets.Method | AttributeTargets.Property)]
```

Quand il rencontre une directive telle que :

```
[Aide("Cette classe fait cci ou encore cela")]
```

le compilateur (lors de la compilation de la classe) recherche une classe dérivée de *System.Attribute* et qui s'appelle *Aide* ou *AideAttribute*. Le compilateur vérifie si la cible de l'attribut correspond bien à la directive *AttributeUsage*. Il recherche ensuite un constructeur pour cette classe qui accepte un argument de type *string*. Plusieurs arguments pourraient être utilisés, ce qui correspondrait à autant de manière d'utiliser l'attribut *Aide*.

Les valeurs associées aux attributs sont alors stockées dans le fichier DLL résultant de la compilation de la classe. Elles sont stockées dans la partie métadonnées de ce fichier, informations qui décrivent complètement cette DLL.

Le programme utilisateur doit ajouter une référence à *Cie.dll*. Le programme utilisateur de la classe retrouve les valeurs des attributs en appelant des méthodes de classe *Type* (*typeof* renvoie un objet de la classe *Type*) :

```
object[] tAttrib = typeof(Cie.X).GetCustomAttributes(true);

Cie.AideAttribute oAide = tAttrib[0] as Cie.AideAttribute;
string strMsg = oAide.prMsgAide;
```

GetCustomAttributes renvoie un tableau d'attributs pour la classe : dans notre cas, il n'y en aura qu'un seul mais il pourrait y en avoir plusieurs. L'argument indique s'il faut, éventuellement, remonter la hiérarchie de la classe pour trouver l'information.

Montrons maintenant comment retrouver l'aide relative à la fonction *f*.

```
if (tAttrib.Length != 0)
{
      MethodInfo mi = typeof(Cie.X).GetMethod("f");

      object[] tObject = mi.GetCustomAttribute(true);
      Cie.AideAttribute oAttr = tObject[0] as Cie.AideAttribute;

      string strMsg = oAttr.msgAide;
}
```

7. POO avec Visual C#

Tout au long de ce chapitre, nous allons travailler avec une classe d'exemple qui sera la classe *Personne*.

7.1. Création d'une classe

La création d'une classe passe par la déclaration de la classe elle-même et de tous les éléments la constituant.

7.1.1. Déclaration de la classe

La déclaration d'une classe se fait en utilisant le mot clé *class* suivi d'un bloc de code délimité par les caractères { et }. Dans ce bloc, on trouve des déclarations de variables qui seront les champs de la classe et des procédures qui seront les méthodes de la classe.

La syntaxe générale de définition d'une classe est la suivante :

```
[attributs][modificateurs][partial] class nomDeLaClasse
[: classe de base] [,interface1, interface 2, …]
{
    Code de la classe
}
```

De nombreux mots réservés sont disponibles pour la personnalisation d'une classe. Au moment de sa déclaration, on peut spécifier la visibilité de la classe. Les mots réservés suivants sont disponibles :

Mot réservé	Signification
`public`	La classe pourra être utilisée dans l'ensemble du projet mais aussi dans d'autres projets.
`internal`	L'accès à la classe est limité au projet dans lequel elle est définie.
`private`	La classe ne peut être utilisée que dans le module dans lequel elle est définie.
`protected`	La classe ne peut être utilisée qua dans une sous-classe de celle dans laquelle elle est définie. Ce mot réservé ne peut être utilisé que pour une classe déclarée dans une autre classe.
`protected internal`	Identique à l'union des portées *protected* et *internal*.

Il est aussi possible d'indiquer comment la classe va se comporter vis-à-vis de l'héritage. Deux options sont possibles :

- **abstract**: indique que la classe sert de classe de base dans une relation d'héritage. On ne pourra pas créer d'objets dans cette classe. Dans ce genre de classe, en général, on ne tolère que la définition de méthodes. Les classes dérivées devront contenir une substitution.
- **sealed**: cette classe sera la dernière de la hiérarchie. Il ne sera donc pas possible d'utiliser cette classe comme classe de base d'une autre classe.

Pour indiquer que la classe hérite des propriétés d'une autre classe, on doit utiliser le caractère : suivi du nom de la classe de base.

Voici le début de la définition de la classe *Personne*:

```
public class Personne
```

```
{
     private string m_strNom;
     private string m_strPrenom;
     private DateTime m_oDateNaiss;
}
```

7.1.2. Classe partielle

La définition d'une classe peut être répartie sur plusieurs déclarations en utilisant le mot réservé *partial*. Cette technique autorise la définition de la classe dans plusieurs fichiers sources. Elle est très utilisée dans Visual Studio pour permettre la personnalisation de classes générées automatiquement. Le code généré est en général placé dans un fichier nommé *.designer.cs* qui ne doit, en principe, pas être modifié directement. Lors de la compilation, le compilateur regroupe toutes les définitions partielles pour obtenir le code source de la classe.

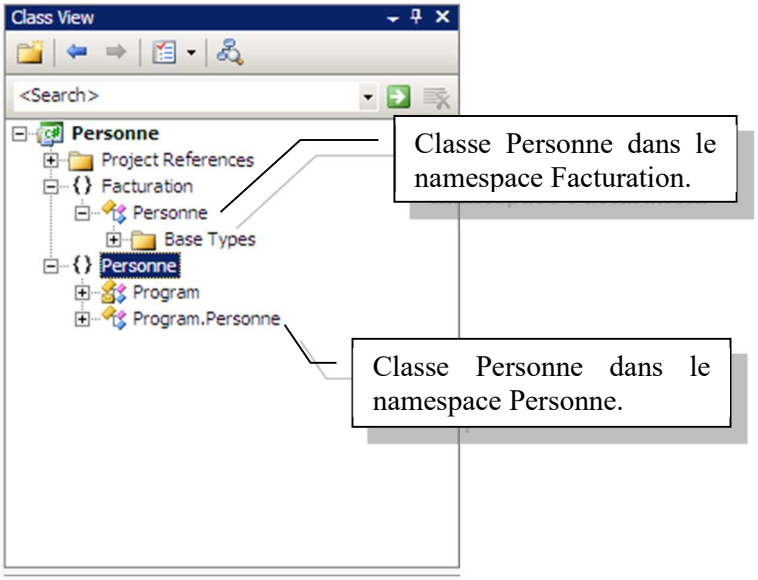
Les différentes parties de la définition d'une classe doivent par contre être dans le même projet et faire partie du même namespace. Par exemple, le code suivant:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Personne
{
     class Program
     {
       static void Main(string[] args)
       {
       }
     }
     public partial class Personne
     {
           private string m_strNom;
           private string m_strPrenom;
           private DateTime m_oDateNaiss;
           int CalculAge()
           {
               return DateTime.Now.Year - m_oDate_naiss.Year;
           }
     }
}

namespace Facturation
{
   public partial class Personne
   {
       int CalculAge()
       {
           return DateTime.Now.Year - DateTime.MinValue.Year;
       }
   }
}
```

Au premier abord rien d'illégal puisque le compilateur génère le code correctement. Par contre, il n'a pas la même vision des choses puisqu'il présente la configuration suivante :



Deux classes *Personne* sont disponibles. Le compilateur a en fait déterminé que les deux définitions de classe ne font pas partie du même namespace.

7.1.3. Création de propriétés

On peut créer des variables simples (données membres) pour stocker les informations de la classe et on essaiera toujours de les définir en *private*.

Les propriétés vont fournir une grande flexibilité et un grand contrôle sur le stockage des informations dans une classe. Elles vont permettre à la classe de protéger et de valider ses propres données.

Une propriété est semblable à une fonction avec à l'intérieur deux blocs de code. Ces deux blocs seront définis à l'aide des mots réservés *get* et *set* :

- Le bloc de code *get* est exécuté automatiquement lors de la lecture de la donnée membre.
- Le bloc de code *set* est exécuté automatiquement lors de l'affectation d'une valeur à la donnée membre.

La classe *Personne* peut être améliorée de la manière suivante :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Personne
{
    class Program
    {
      static void Main(string[] args)
      {
      }
      }
      public partial class Personne
      {
            private string m_strNom;
```

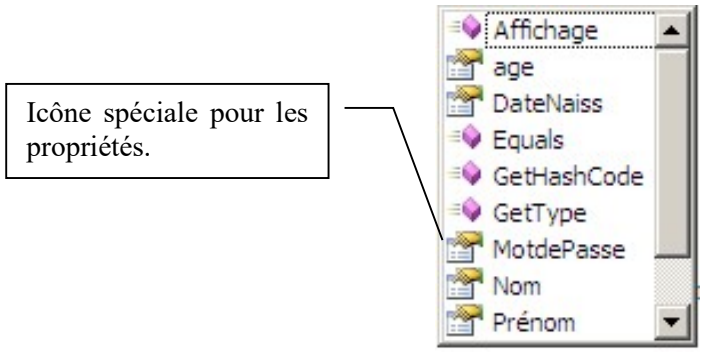
```
        private string m_strPrenom;
        private DateTime m_oDateNaiss;
        public int CalculAge()
        {
            return DateTime.Now.Year - m_oDateNaiss.Year;
        }
        public string prNom
        {
            get
            {
                return m_strNom;
            }
            set
            {
                m_strNom = value;
            }
        }
        public string prPrénom
        {
            get
            {
                return m_strPrenom;
            }
            set
            {
                m_strPrenom = value;
            }
        }
        public DateTime prDateNaiss
        {
            get
            {
                return m_oDateNaiss;
            }
            set
            {
                m_oDateNaiss = value;
            }
        }
    }
}
```

Propriété pour gérer la donnée membre *m_strNom*.

Type retourné par le *get*.

Value permet d'accéder à la valeur à assigner à la donnée membre.

IntelliSense va présenter la liste des propriétés avec une icône spécifique :



La création des propriétés permet maintenant d'accéder de manière indirecte aux données membres de la classe.

Extension des propriétés de la classe *Personne*. Imaginons les nouvelles règles suivantes :

- Le nom doit toujours être stocké en majuscules,
- Le prénom doit toujours être stocké en minuscules,
- La date de naissance doit toujours être supérieure à 1900.

```
public string Nom
        {
            get
            {
                return m_strNom;
            }
            set
            {
                m_strNom = value.ToUpper();
            }
        }
        public string Prénom
        {
            get
            {
                return m_strPrenom;
            }
            set
            {
                m_strPrenom = value.ToLower();
            }
        }
        public DateTime DateNaiss
        {
            get
            {
                return m_oDateNaiss;
            }
            set
            {
                if (value.Year >= 1900)
                    m_oDateNaiss = value;
            }
        }
```

Comme toute autre méthode de la classe, les propriétés ont accès aux données privées de la classe.

Lecture seule et écriture seule

Il peu parfois être intéressant de restreindre les accès possibles à une propriété. Elles peuvent donc être définies en lecture seule ou en écriture seule.

Le bloc de code *get* doit être omis pour une propriété en écriture seule. Pour une propriété en lecture seule, c'est le bloc *set* qui doit être omis. Pour mettre cela en application, imaginons d'ajouter la donnée membre *MotdePasse* dans la classe *Personne* qui sera en écriture seule. On peut également ajouter la propriété *âge* de la personne qui pourra se déduire au départ de la date de naissance et qui sera accessible en lecture seule.

```
static void Main(string[] args)
{
    Personne oPers = new Personne ();

    oPers.prPrenom = "Marc";
    Console.WriteLine("Le prénom est {0}", oPers.prPrenom);
}
public partial class Personne
{
```

L'usage de la propriété *Prenom* dans sa fonction de mise à jour. Il y a appel implicite de la fonction *Set*.

```
        private string m_strNom;
        private string m_strPrenom;
        private DateTime m_oDateNaiss;
        private string m_strMotdePasse;

        public int prAge
        {
            get
            {
                return DateTime.Now.Year - m_oDateNaiss.Year;
            }
        }

        public string prMotdePasse
        {
            set
            {
                m_strMotdePasse = value;
            }
        }
```

Seule la fonction *get* est définie dans la propriété *age*. Seule la lecture sera autorisée.

Propriété indexée

Les propriétés indexées permettent un accès de type tableau à des groupes d'éléments. Les propriétés indexées diffèrent légèrement des propriétés normales car elles attendent un paramètre indiquant l'élément du groupe auquel il faut accéder.

Ajoutons à la classe *Personne*, la liste des enfants de cette personne et en définissant cette propriété comme propriété indexée.

Le code la classe *Personne* devient :

```
 public partial class Personne
{
        private string m_strNom;
        private string m_strPrenom;
        private DateTime m_oDateNaiss;
        private string m_strMotdePasse;
        private Personne[] m_oEnfants = new Personne[10];

        public Personne this[int dIndex]
        {
            get
            {
                return m_oEnfants[dIndex];
            }
            set
            {
                m_oEnfants[dIndex] = value;
            }
        }
 }
```

Voici le programme principal permettant de tester le code ci-dessus:

```
static void Main(string[] args)
{
        Personne oPers = new Personne ();
        Personne oEnf1 = new Personne();
```

```
        Personne oEnf2 = new Personne();

        oPers.prNom = "Dupond";
        oPers.prPrenom = "Paul";
        oPers.prDateNaiss = new DateTime(1954, 12, 23);

        oEnf1.prNom = "Dupond";
        oEnf1.prPrenom = "Pascal";
        oEnf1.prDateNaiss = new DateTime(1979, 10, 5);

        oEnf2.prNom = "Dupond";
        oEnf2.prPrenom = "André";
        oEnf2.prDateNaiss = new DateTime(1984, 1, 15);

        oPers[0] = oEnf1;
        oPers[1] = oEnf2;

        Console.WriteLine("Mr {0} {1} né le {2} a 2 enfants", oPers.prPrenom,
        oPers.prNom, oPers.prDateNaiss);

        Console.WriteLine("{0} {1} qui a {2} ans", oPers[0].prNom,
        oPers[0].prPrenom,
        oPers[0].prAge);

        Console.WriteLine("{0} {1} qui a {2} ans", oPers[1].prNom,
        oPers[1].prPrenom,
        oPers[1].prAge);
    }
```

7.1.4. Création de Méthodes

7.1.4.A. Définition

Les méthodes sont des procédures ou des fonctions définies à l'intérieur d'une classe. Elles sont en général utilisées pour manipuler les données membres de la classe. Pour pouvoir utiliser une méthode, il suffit de préfixer le nom de la méthode par le nom de l'objet sur lequel la méthode va s'exécuter.

Ajoutons par exemple à la classe *Personne* la méthode *Affichage*:

```
    static void Main(string[] args)
{
        Personne oPers = new Personne ();
        Personne oEnf1 = new Personne();
        Personne oEnf2 = new Personne();

        oPers.prNom = "Dupond";
        oPers.prPrenom = "Paul";
        oPers.prDateNaiss = new DateTime(1954, 12, 23);

        oEnf1.prNom = "Dupond";
        oEnf1.prPrenom = "Pascal";
        oEnf1.prDateNaiss = new DateTime(1979, 10, 5);

        oEnf2.prNom = "Dupond";
        oEnf2.prPrenom = "André";
        oEnf2.prDateNaiss = new DateTime(1984, 1, 15);

        oPers[0] = oEnf1;
```

```
            oPers[1] = oEnf2;

            oPers.Affichage();

            Console.WriteLine("{0} {1} qui a {2} ans", oPers[0].prNom,
            oPers[0].prPrenom,
            oPers[0].prAge);

            Console.WriteLine("{0} {1} qui a {2} ans", oPers[1].prNom,
            oPers[1].prPrenom,
            oPers[1].prAge);

        }
        public partial class Personne
        {
            private string m_strNom;
            private string m_strPrenom;
            private DateTime m_oDateNaiss;
            private string m_strMotdePasse;
            private Personne[] m_oEnfants = new Personne[10];
            public void Affichage()
            {
                Console.WriteLine("Mr {0} {1} né le {2} a 2 enfants",
                prPrenom,
                prNom, prDateNaiss);
            }
        }
```

En POO, on essaie de réutiliser au maximum les fonctions déjà définies.

7.1.4.B. Surcharge de Méthodes

La surcharge de méthode est la création, au sein d'une classe, de méthodes ayant un nom identique mais un nombre de paramètres différent ou des types de paramètres différents. Cela permet de conserver un nom cohérent pour plusieurs signatures de méthodes ayant un but similaire mais pour lesquelles, juste l'un ou l'autre détail va changer.

Les informations suivantes n'interviennent pas dans la signature de la fonction :

- Le type et le nombre des paramètres,
- Le type de retour de la fonction,
- Les modificateurs *out* ou *ref* appliqués aux paramètres de la méthode.

Dans la classe *Personne*, on peut par exemple surcharger la fonction *Affichage* en vue de prendre en compte la langue d'affichage:

```
    public partial class Personne
    {
        private string m_strNom;
        private string m_strPrenom;
        private DateTime m_oDateNaiss;
        private string m_strMotdePasse;
        private Personne[] m_oEnfants = new Personne[10];
        public void Affichage()
        {
            Console.WriteLine("Mr {0} {1} né le {2} a 2 enfants",
            prPrenom,
            prNom, prDateNaiss);
        }

        public void Affichage(string strLangue)
        {
```

```
            switch (strLangue)
            {
                case "fr":
                    Console.WriteLine("Mr {0} {1} né le {2} a 2
                    enfants", prPrenom,
                    prNom, prDateNaiss);
                    break;
                default:
                    Console.WriteLine("Mr {0} {1} was born {2} has
                    two children", prPrenom,
                    prNom, prDateNaiss);
                    break;
            }
        }
```

7.1.4.C. Substitution de Méthodes

Les classes dérivées héritent des propriétés et des méthodes de leur classe de base. On peut donc les réutiliser, à partir d'une classe dérivée, sans aucune modification.

Cependant, si le fonctionnement d'une propriété ou d'une méthode héritée n'est pas adapté à la nouvelle classe, on a la possibilité de la substituer par une nouvelle implémentation dans la classe dérivée.

Dans ce cas, dans la classe dérivée, il faudra utiliser le mot réservé *override* lors de la substitution. Il est également impératif que la classe de base ait autorisé cette substitution par l'utilisation du mot réservé *virtual*.

Sans indication particulière, la méthode ou la propriété ne peut pas être substituée.

Les méthodes substituées doivent avoir la même signature que la méthode de la classe de base.

Pour tester la substitution, imaginons une classe *Salarie* qui est héritée de la classe *Personne*. Dans cette classe *Salarie*, on peut substituer la méthode *Affichage* comme suit :

```
    public class Salarie : Personne
     {
        private float m_fSalaire;

        public override void Affichage()
        {
            base.Affichage();
            Console.WriteLine("Le salaire de cette personne est
            {0}", m_fSalaire);
        }
        public float prSalaire
        {
            set
            {
                m_fSalaire = value;
            }
            get
            {
                return m_fSalaire;
            }
        }
     }
```

Avec cette déclaration, la méthode *Affichage* de la classe *Personne* n'est plus visible aux utilisateurs de la classe *Salarie*. Seule la méthode *Affichage* de la classe *Salarie* est accessible.

Cependant, le code de la méthode *Affichage* de la classe dérivée peut quand même avoir accès à la fonction *Affichage* de la classe de base en utilisant le mot réservé *base*. Dans une hiérarchie de classes, le mot réservé *base* fera toujours référence à la classe parente et non pas à la classe où le mot réservé *virtual* a été indiqué.

Dés qu'une méthode est indiquée comme substituable dans une classe, elle le sera pour toutes les classes dérivées quel que soit le degré de parenté. Le mot réservé *sealed* peut être utilisé pour bloquer à partir d'un niveau donné cette fonctionnalité.

Si dans la classe *Salarie* nous avions défini la fonction *Affichage* comme suit :

```
public class Salarie : Personne
 {
     private float m_fSalaire;

     public sealed override void Affichage()
     {
         base.Affichage();
         Console.WriteLine("Le salaire de cette personne est
           {0}", m_fSalaire);
     }
     public float Salaire
     {
         set
         {
             m_fSalaire = value;
         }
         get
         {
             return m_fSalaire;
         }
     }
 }
```

Alors, la fonction *Affichage* n'aurait plus pu être substituée dans les classes dérivées de la classe *Salarie*.

À l'inverse, on peut exiger qu'une classe héritée substitue une méthode définie dans une classe de base. Cette méthode doit être marquée avec le mot réservé *abstract*. Pour une telle méthode, il ne doit pas y avoir d'implémentation mais juste sa définition.

```
public abstract string EtatCivil ();
```



Une telle méthode est appelée **méthode abstraite**. Attention, elle exige que la classe dans laquelle elle est définie soit également marquée comme abstraite avec le mot réservé *abstract*.

7.1.4.D. Masquage de Méthode

Lorsque des éléments d'un programme partagent le même nom, l'un d'eux peut masquer l'autre. Dans un tel cas, l'élément masqué n'est plus accessible et le compilateur utilise à la place l'élément masquant. Le masquage peut se faire entre des éléments de type différent. Seul le nom de l'élément est utilisé pour assurer le masquage.

Lors du masquage, il convient d'utiliser le mot réservé *new* devant le nom du membre assurant le masquage.

On peut par exemple, masquer la propriété *age* dans une classe dérivée de la classe *Personne* :

```
public class Salarie : Personne
{
```

```
        private float m_fSalaire;

        public sealed override void Affichage()
        {
            base.Affichage();
            Console.WriteLine("Le salaire de cette personne est
            {0}", m_fSalaire);
        }
        public new int age()
        {
            return DateTime.Now.Year - DateNaiss.Year;
        }
        public float Salaire
        {
            set
            {
                m_fSalaire = value;
            }
            get
            {
                return m_fSalaire;
            }
        }
```

Pour la classe *Salarie*, il n'y a dorénavant plus qu'un seul élément appelé *age*. Tout ce qui exister comme élément, dans la ou les classes de base de cette classe, et qui est nommé *age* est masqué et inaccessible. Le seul élément visible est la fonction *age* déclarée dans la classe ; la propriété *age* héritée de la classe *Personne* est masquée.

Cette technique est à utiliser avec la plus grande prudence car en fonction de l'emplacement où se trouve une instruction, le même nom peut faire référence à des éléments de nature différente. L'usage des préfixes permettra d'éviter cette pratique plus que douteuse…

7.1.4.E. Méthode partielle

7.1.4.F. Méthode d'extension

Les méthodes d'extension permettent l'ajoute de fonctionnalités à une classe déjà définie sans bien sûr avoir à modifier le code de cette classe. Elles sont simplement écrites à l'extérieur de la classe et sont ensuite appelées exactement de la même façon que les méthodes disponibles directement dans la classe. Il y a cependant quelques règles à respecter :

- Elles peuvent être de type procédure ou fonction mais pas de type propriété.
- Le premier paramètre doit être précédé du mot réservé *this*.
- Le type du premier paramètre de la méthode détermine le type étendu par cette méthode.
- Au moment de l'exécution, ce premier paramètre représente l'objet de la classe sur lequel la méthode est appelée.
- Elles doivent être définies dans une classe *static*.
- Elles doivent elles-mêmes être *static*.

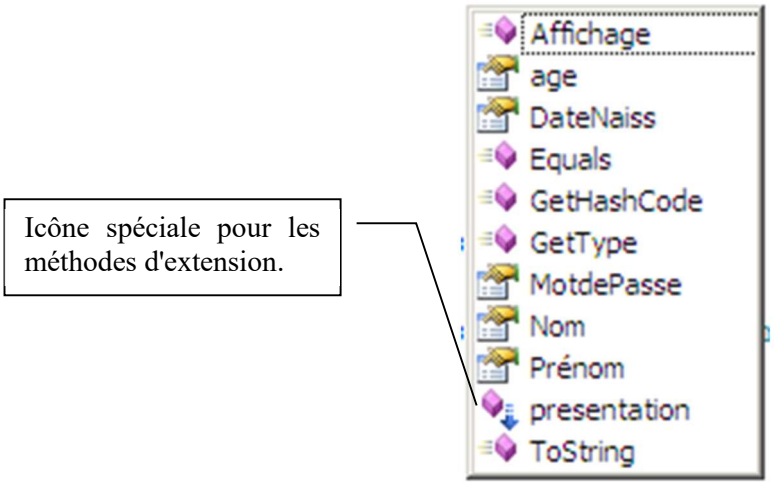
Dans l'exemple ci-dessous, on ajoute une méthode à la classe *Personne* :

```
static class MyExtensions
 {
  public static void presentation(this Personne oP)
  {
      Console.WriteLine("Méthode d'extension");
      Console.WriteLine("Nom : {0}", oP.Nom);
      Console.WriteLine("Prénom : {0}", oP.Prénom);
```

```
            Console.WriteLine("Date de naissance : {0}",
            oP.DateNaiss);
        }
    }
```

Cette classe d'extension peut être ajoutée dans le même Namespace que la classe étendue ou dans un autre Namespace à la seule condition d'utiliser la clause *using*.

Lorsqu'IntelliSense présente la liste des méthodes ou propriétés accessibles au départ d'un objet, les méthodes d'extension y sont référencées de manière spécifique :



Remarque relative à l'utilisation d'une méthode d'extension :

Les méthodes d'extension ont été introduites dans C# 3.0 comme une solution pour permettre au mapping syntaxique (plus d'info sur ça ici) de LINQ de fonctionner correctement avec IEnumerable, sans avoir à modifier IEnumerable (ce qui poserait d'énormes problèmes au code existant). Malheureusement si elles sont pratiques pour ce cas précis (et sont effectivement la meilleure solution), dans 95% des cas, elles posent de graves problèmes de versionning, et doivent donc être utilisées avec une extrême prudence. Voici un exemple du problème.

Supposons qu'un éditeur tierce (ou même un autre développeur de votre équipe) développe un assembly x.dll. Dans cet assembly nous avons une classe MyClass :

```
public class MyClass
{
    // ...
}
```

On décide d'écrire une méthode d'extension pour MyClass qui s'appelle par exemple Add :

```
public static void Add( this MyClass target, object item)
{
    // ...
}
```

On utilise cette méthode d'extension un peu partout dans votre code.

L'assembly x.dll passe alors en version 2.0, et il se trouve que l'éditeur ajoute une méthode Add à MyClass, qui a un comportement différent de votre méthode d'extension (ce qui sera toujours le cas, à moins d'une grosse coïncidence) :

```
public class MyClass
{
    // ...

    public void Add( object item)
    {
        // ...
    }
}
```

Le code va compiler sans problème (on n'aura même pas de Warning), mais le comportement de votre code sera changé. Si on utilise la méthode Add pour un objet de type MyClass, c'est maintenant la méthode d'instance qui sera appelée au lieu de la méthode d'extension.

Ce problème très grave n'a pas vraiment de solution. Il y a quelques usages à respecter pour réduire le risque, mais il existera toujours :

- N'utilisez les méthodes d'extensions qu'en dernier recours
- Ne définissez des méthodes d'extension que pour des types qui ont relativement peu de chances de changer, comme des types basiques du Framework .NET (comme

7.1.4.G. Les constructeurs et destructeurs

Les constructeurs sont des méthodes particulières d'une classe par différents aspects. Le constructeur est une méthode portant toujours le même nom que la classe elle-même. Il ne retourne aucun type, même pas *void*. Il n'est jamais appelé explicitement dans le code mais de manière implicite à la création d'une instance de classe.

Comme pour une méthode classique, un constructeur peut demander des paramètres. Un constructeur d'une classe qui n'attend pas de paramètre sera appelé le constructeur par défaut de la classe. Le rôle principal du constructeur est principalement l'initialisation des données membres d'un objet. Les constructeurs peuvent également être surchargés.

Voici la classe *Personne* munie de constructeurs :



```
public Personne(string strNom, string strPrenom, string
                strPswd)
  {
      prNom = strNom;
      prPrenom = strPrenom;
      prMotdePasse = strPswd;
  }

public Personne()
{
    prNom = "";
    prPrenom = "";
    prMotdePasse = "";
}
```

Pour intialiser les données membres, on a préférré utiliser les propriétés que d'accéder directement aux données membres. Pourquoi ? Parce qu'en Orienté Objet, on essaie d'utiliser au maximum les méthodes disponibles.

Dans une classe dérivée, on peut aussi trouver des constructeurs. Si dans la classe dérivée, on ajoute un constructeur par défaut, les règles suivantes doivent être respectées :

- Dans une classe dérivée, si on ne précise pas quel constructeur de la classe de base il faut utiliser, alors, le constructeur par défaut sera automatiquement sélectionné.
- Si une classe de base n'offre pas de constructeur par défaut, la classe de base est obligée de référencer un constructeur existant.
- Un constructeur de la classe dérivée peut invoquer un constructeur de la classe de base en utilisant le mot réservé *base*. À sa suite, on trouvera la liste des paramètres de construction. Le compilateur cherchera dans la classe de base, le constructeur offrant une signature correspondant à la liste des paramètres utilisés.

Voici l'exemple d'un constructeur de la classe *Salarie* :

```
public Salarie()
    : base()
{
    prSalaire = 0;
}
```

Le comportement aura été le même si nous avions écrit le code suivant :

```
public Salarie()
{
    prSalaire = 0;
}
```

Dans la classe *Salaire*, on peut également surcharger notre constructeur et invoquer un constructeur plus sophistiqué de la classe de base :

```
public Salarie(string strNom, string strPrenom, string
    strPswd, float fSalaire)
    : base(strNom, strPrenom, strPswd)
{
    prSalaire = fSalaire;
}
```

Les destructeurs sont d'autres méthodes particulières d'une classe. Comme les constructeurs, ils sont appelés implicitement mais uniquement lors de la destruction d'une instance de classe. La signature d'un destructeur est imposée. Le destructeur porte toujours le même nom que la classe en le faisant précéder du signe ~ et ne peut prendre aucun paramètre.

Du fait de cette signature imposée, il ne peut y avoir qu'un seul destructeur pour une classe, donc pas de surcharge possible pour les destructeurs.

La déclaration du destructeur est la suivante :

```
~Salarie()
{
}
```

Le destructeur doit permettre la libération des ressources utilisées par la classe. On peut, par exemple, y trouver du code permettant la fermeture d'un fichier ouvert par l'objet ou la fermeture d'une connexion ouverte avec une base de données.



Attention : pas de mot réservé *public* devant le destructeur.

7.1.4.H. Les membres statiques

Les membres statiques sont des données membres, des méthodes ou des propriétés qui sont accessibles par tous les objets d'une classe. On les appelle également des membres partagés.

Ils sont très utiles lorsqu'on a à gérer, dans une classe, des informations qui ne sont pas spécifiques à un objet mais à la classe elle-même. Par opposition aux données membres associées aux objets et qui existent pour chaque objet, les données membres statiques n'existent qu'en un et un seul exemplaire. Indépendamment du nombre d'objets qui existent.

Les membres statiques sont utilisables dans le code en y faisant référence par le nom de la classe.

Les méthodes statiques suivent les mêmes règles et peuvent servir à la création de bibliothèques de fonctions. L'exemple classique est la classe *Math* dans laquelle de nombreuses fonctions statiques sont définies. Les méthodes statiques possèdent cependant une sévère limitation car elles ne peuvent utiliser que des variables locales ou des données membres statiques de la classe. Donc, les méthodes statiques ne peuvent pas utiliser les données membres de la classe.



Les membres statiques doivent être déclarés avec le mot *static*. Comme pour tous les autres membres de la classe, on peut spécifier une visibilité.

Attention : une variable locale à une fonction ne peut pas être statique.

7.2. Utilisation d'une classe

L'utilisation d'une classe, dans une application, passe par deux étapes :

- La déclaration de l'objet,
- La création de l'objet.

7.2.1. Création d'un objet (instance de classe)

Les objets (appelés aussi instances de classe) sont des variables de type référence. Elles diffèrent des variables classiques par le fait que la variable ne contient pas directement les données mais contient une référence sur l'emplacement mémoire qui contient les données. Comme pour toutes les variables, elles doivent être déclarées avant leur utilisation.

La déclaration s'effectue de manière identique à celle d'une variable classique.

```
Personne oEnf1;
```

Après cette étape, l'objet existe mais ne référence aucun espace mémoire valide.



La deuxième étape consiste réellement à créer l'instance de la classe. Le mot réservé *new* est utilisé à cet effet. Il attend comme paramètre le nom de la classe dont il est chargé de créer une instance. L'opérateur *new* fait une demande au système pour obtenir la mémoire nécessaire au stockage de l'objet. Le constructeur de l'objet sera ensuite appelé afin d'initialiser la nouvelle instance créée.

```
Personne oEnf1 = new Personne();
```

Ou :

```
Personne oEnf1;
oEnf1 = new Personne();
```

Dans l'exemple ci-dessous, le constructeur par défaut sera invoqué. Si on souhaite utiliser un autre constructeur, on doit simplement placer une liste de paramètres entre les parenthèses suivant le nom de la classe :

```
oSalarie.prDateNaiss = new DateTime(1954, 12, 23);
```

7.2.2. Destruction d'un objet

La destruction d'un objet (instance de classe) est automatique dans une application. Le Common Language Runtime surveille à intervalles réguliers que toutes les instances de classes, créées dans l'application, sont encore accessibles : y a-t-il encore dans l'application une variable ou une propriété permettant l'accès à cette instance.

Si aucun moyen d'accéder à cet objet n'est trouvé, l'objet est alors marqué comme étant orphelin. Lorsque les réserves mémoire de l'application deviennent trop faibles, le Garbage Collector intervient et élimine les objets orphelins. C'est lors de cette élimination que les destructeurs de chacun des objets sont appelés.

Il n'y a aucuns moyens de précipiter les choses en demandant de manière explicite la libération de l'espace mémoire utilisé. Il est cependant possible de forcer le Garbage Collector à intervenir avec la ligne de code suivante :

```
GC.Collect();
```

Dans ce cas, le Garbage Collector (GC) intervient pour tous les objets orphelins. L'inconvénient de cette solution est qu'elle est relativement gourmande en ressources et cela pour ne récupérer parfois que quelques dizaines d'octets en mémoire, voir aucun s'il n'y a pas d'objets créés à supprimer.

Cette situation est parfois problématique lorsque l'objet utilise une ressource externe comme, par exemple, une connexion vers un serveur de base de données. Si la fermeture de la connexion est prévue dans le destructeur de la classe, il peut se passer assez longtemps entre le moment où l'objet devient inaccessible et celui où le destructeur est appelé.

Pour pallier ce problème, il est possible de mettre en œuvre une autre solution. Le code chargé de la libération des ressources est placé dans une autre méthode et cette méthode est appelée explicitement dans le code. En général, cette méthode est nommée *Dispose*. Pour être certain que les ressources sont bien libérées, on peut également prévoir un appel à cette méthode dans le destructeur de la classe.

Un autre problème peut alors survenir : si la méthode a été appelée explicitement dans le code de l'application, elle le sera à nouveau, de manière implicite, lorsque le GC entrera en action. On doit donc faire en sorte que le code de cette méthode *Dispose* puisse être exécuté deux fois sans poser de problèmes.

On peut également indiquer au GC, qu'il ne doit pas exécuter le destructeur de cet objet. Pour cela, dans la méthode *Dispose*, on doit le prévenir que le travail de nettoyage est déjà réalisé, en appelant la méthode *SuppressFinalize*.

Le code de la méthode *Dispose* et du destructeur doit alors avoir la forme suivante :

```
 public void Dispose()
{
    GC.SuppressFinalize(this);
}
~Personne()
{
    Dispose();
}
```

7.2.3. Liaison tardive et liaison précoce

Le compilateur C# réalise une opération appelée liaison lorsqu'un objet est affecté à une variable. Cette liaison est dite précoce lorsque la variable est créée à partir d'une classe spécifique. Cette fonctionnalité permet au compilateur d'effectuer des optimisations sur le code généré.

L'affectation d'un objet peut aussi se réaliser à une variable de type *Object*. Ce type de variable est capable de référencer n'importe quel autre type de classe. Dans ce cas, la liaison est dite tardive car le type réel de l'objet ne sera découvert qu'à l'exécution de l'application. Cette technique est à éviter car elle génère un code moins efficace et surtout elle ne permet pas de bénéficier de la complétion automatique du code dans l'éditeur ni de l'aide dynamique. En effet, dans ce cas Visual C# ne peut pas déterminer le type réel de l'objet manipulé.

Cependant, certaines fonctions retournent un type Object, mais pour pouvoir le manipuler, il convient de prendre quelques précautions. La première solution consiste à n'utiliser, avec la valeur renvoyée par la fonction, que des membres de la classe Object. Cette solution est relativement limitative quant aux fonctionnalités disponibles.

La deuxième solution consiste à affecter, à une variable d'un type particulier, la valeur renvoyée par la fonction. Cette solution permet d'utiliser toutes les fonctionnalités de l'objet retourné par la fonction. Cependant, il faut être certain que l'objet retourné est bien un objet de la classe que l'on souhaite manipuler.

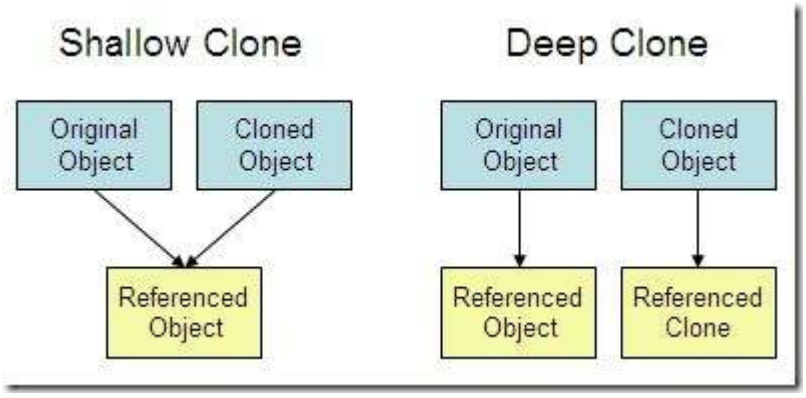
7.2.4. Clonage d'objets

En C#, il faut faire attention à la copie/clonage d'objets car au contraire du C, C++, cette opération ne donne pas toujours les résultats attendus quand on y prend pas garde. En C#, on parle généralement de deux types de copie :

- Shallow,

- Deep.

Une copie de type *shalow* va juste copier les références et pas les objets eux-mêmes. Au contraire, une copie de type *Deep* va réaliser une copie totale de l'objet. On peut schématiser cette situation de la manière suivante :



Pour réaliser le clonage d'un objet, il existe plusieurs méthodes que nous détaillerons ci-dessous.

7.2.4.A. Interface ICloneable

L'interface *ICloneable* contient une méthode *Clone* qui est utilisée pour réaliser une copie de l'objet :

```
public interface ICloneable
{
    object Clone();
}
```

Le problème de cette interface est que la méthode Clone ne spécifie pas de manière explicite si elle réalise un clonage de type shallow ou deep. La documentation MSDN semble dire que la méthode Clone réalise une copie de type deep mais cela n'est pas dit explicitement : *The ICloneable interface contains one member, Clone, which is intended to support cloning beyond that supplied by MemberWiseClone… The MemberwiseClone method creates a shallow copy…*

7.2.4.B. Type-Safe Clone

Un autre inconvénient de l'interface *Icloneable* est que la méthode *Clone* retourne un objet et donc, il nécessite un cast comme montré ci-dessous :

```
Person oJoe = new Person();
oJoe.Name = "Joe Smith";
Person oJoeClone = (Person)oJoe.Clone();
```

Une possibilité d'éviter le cast est de créer sa propre méthode : on parlera de *type-safe Clone* méthode. Attention que la méthode *ICloneable.Clone* doit aussi être fournie afin de satisfaire l'interface *Icloneable*.

```
public class Person : ICloneable
{
    public string Name;
    object ICloneable.Clone()
    {
        return this.Clone();
    }

    public Person Clone()
    {
```

```
                return (Person)this.Clone();
            }
        }
```

7.3. L'héritage

7.3.1. Introduction

L'héritage est une puissante fonctionnalité d'un langage orienté objet mais peut parfois être utilisée mal à propos. Deux types de relations peuvent être utilisés entre deux classes. On peut avoir la relation *IS-A* et la relation *HAS-A*.

La relation d'héritage doit être envisagée lorsque la relation *IS-A* peut être appliquée entre deux classes. Par exemple, on peut dire : *Un Client est une Personne*. On peut donc envisager une relation d'héritage entre ces deux classes.



La mise en œuvre est très simple au niveau du code puisque, dans la déclaration de la classe, il suffit juste de spécifier le caractère ":" suivi par le nom de la classe parente. Visual C# n'acceptant pas l'héritage multiple, on ne doit spécifier qu'un seul nom de classe de base.

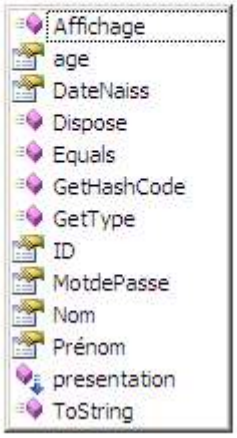
```
class Client : Personne
{
    protected int m_dID;
  public int ID
  {
      get
      {
          return m_dID;
      }
      set
      {
          m_dID = value;
      }
  }
}
```

La classe peut ensuite être utilisée et propose toutes les fonctionnalités définies dans la classe *Client* plus celles héritées de la classe *Personnes* :

```
Client oCli;

oCli = new Client();

oCli.
```



7.3.2. L'emploi du mot réservé Base

Il est légitime de vouloir ensuite modifier le fonctionnement de certaines méthodes héritées pour les adapter à la classe *Client*. Par exemple, la méthode *Affichage* peut être substituée pour tenir compte des nouvelles données membres disponibles dans la classe.

```
public void Affichage(string strLangue)
 {
     switch (strLangue)
     {
         case "fr":
             Console.WriteLine("Mr {0} {1} né le {2} a 2
                 enfants", prPrenom,
             prNom, prDateNaiss);
             Console.WriteLine("Le code client est : {0}",
                 m_dID);
             break;
         default:
             Console.WriteLine("Mr {0} {1} was born {2} has two
                 children", prPrenom,
             prNom, prDateNaiss);
             Console.WriteLine("The client code is : {0}",
                 m_dID);
             break;
     }
 }
```

La signature de la fonction doit être la même.

Ce code fonctionne très bien mais ne respecte pas l'un des principes de base de la programmation orientée objet qui veut que l'on réutilise au maximum ce qui existe déjà.

Dans l'exemple ci-dessus, on disposait déjà d'une fonction d'affichage dans la classe de base *Personne* qui permettait d'afficher les données membres de cette classe. Il aurait donc été très judicieux de réutiliser cette fonction et obtenir ce qui suit :

```
public void Affichage(string strLangue)
 {
```

```
            base.Affichage();

            switch (strLangue)
            {
                case "fr":
                    Console.WriteLine("Le code client est : {0}",
                        m_dID);
                    break;
                default:
                    Console.WriteLine("The client code is : {0}",
                        m_dID);
                    break;
            }
        }
```

Il ne faut surtout pas ouvlier le mot réservé *base*. Sinon, on appelle la fonction de la classe *Cient* et on obtient un appel récursif.

Pour ce qui concerne la construction d'un objet client, on peut utiliser la même logique et faire appel au constructeur de la classe de base :

```
        public Client(string strNom, string strPrenom, string strPswd,
            int dID) : base(strNom, strPrenom, strPswd)
        {
            m_dID = dID;
        }
```

7.3.3. Les classes abstraites

Les classes abstraites sont des classes qui peuvent uniquement être utilisée comme classe de base dans une relation d'héritage. Il est impossible de créer une instance d'une classe abstraite.

Elles servent essentiellement de modèle pour la création de classes devant avoir une base commune. Les classes abstraites peuvent contenir des données membres, des propriétés et des méthodes comme une classe ordinaire.

Cette technique facilite l'évolution de l'application, car si une nouvelle fonctionnalité doit être disponible dans les classes dérivées, il suffit de l'ajouter dans la classe de base.

Il est aussi possible de ne pas fournir d'implémentation pour une classe abstraite et ainsi laisser à l'utilisateur de la classe le soin de créer l'implémentation dans la classe dérivée. Dans ce cas, il sera obligatoire pour la classe dérivée de fournir une implémentation de la fonction membre non implémentée.

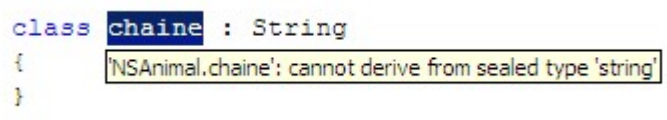
Pour qu'une classe devienne une classe abstraite, on doit utiliser le mot réservé *abstract* dans la déclaration de la classe.

```
    public abstract class Animal
      {

      }
```

7.3.4. Classes finales

Les classes finales sont des classes ordinaires qui peuvent être instanciées mais ne sont pas utilisables comme classe de base dans une relation d'héritage il s'agit de classe feuille dans l'arborescence des classes. C'est le cas de plusieurs classes du Framework.NET, comme par exemple la classe *string*.



Une classe finale doit être définie avec le mot réservé *sealed*:

```
public sealed class Tortue : Animal
{
}
```

7.3.5. Les classes anonymes

Visual C# permet la possibilité de créer des objets sans écrire de code pour la définition de la classe correspondante. C'est le compilateur qui va prendre en charge la génération de la classe. Évidemment, il va falloir l'aider un petit peu et lui fournir quelques informations pour qu'il puisse générer la classe et en créer une instance.

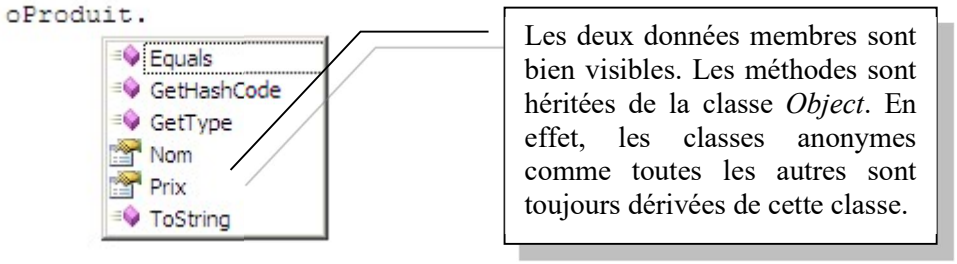
Lors de la déclaration d'une instance de classe anonyme, on va devoir fournir la liste des propriétés souhaitées ainsi que leurs valeurs.

Exemple:

```
var oProduit = new { Nom = "biscuit", Prix = 1.56 };
```

L'objet *oProduit* fait référence à une classe contenant deux propriétés : *Nom* et *Prix*. Ces deux deux données membres sont initialisées respectivement avec les valeurs *biscuit* et *1.56*.

Pour le vérifier, on peut utiliser *Intellisense*:



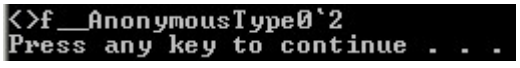
Dans une classe anonyme, les seules méthodes que l'on pourra y trouver seront celles héritées de la classe *Object*. Uniquement des données membres et rien d'autre.

Comment le compilateur fait-il pour s'y retrouver s'il existe plusieurs classes anonymes ? Pendant la compilation, il va générer automatiquement un nom pour chacune de ces classes.

Pour le vérifier, il suffit d'afficher le type de l'objet *oProduit* :

```
Console.WriteLine(oProduit.GetType().Name);
```

On obtient le résultat suivant :



Il existe donc bien un nom pour ces classes. Par contre, ce nom n'est connu qu'après la compilation. Il est donc hors de question de l'utiliser dans le code. Cela serait d'ailleurs une très mauvaise idée car il est susceptible de change à chaque compilation.

Avant de créer une nouvelle classe, le compilateur vérifie s'il n'a pas déjà traité une définition identique. Pour cela, il se base sur les éléments suivants :

- Le nombre des données membres,
- Le nom des données membres,
- Le type des données membres,
- L'ordre des données membres dans la définition.

Si tous ces évènements sont identiques, alors il réutilisera la classe qu'il a déjà générée pour créer un nouvel objet :

```
var oProduit = new { Nom = "biscuit", Prix = 1.56 };
var oProduit2 = new { Nom = "confiture", Prix = 2.24 };

Console.WriteLine(oProduit.GetType().Name);
Console.WriteLine(oProduit2.GetType().Name);
```

Le résultat est :

```
<>f__AnonymousType0`2
<>f__AnonymousType0`2
Press any key to continue . . .
```

Tout est normal car les deux objets ont exactement la même définition. Avec une toute petite modification, le résultat sera différent :

```
var oProduit = new { Nom = "biscuit", Prix = 1.56 };

var oProduit2 = new { Prix = 2.24, Nom = "confiture" };

Console.WriteLine(oProduit.GetType().Name);
Console.WriteLine(oProduit2.GetType().Name);
```

```
<>f__AnonymousType0`2
<>f__AnonymousType1`2
Press any key to continue . . .
```

Cette fois, le compilateur a généré deux classes différentes car les définitions sont différentes.

Pour comparer deux objets de classe anonyme, on doit utiliser la méthode *Equals*. Cette méthode est héritée de la classe *Object* mais substituée par le compilateur au moment de la génération de la classe anonyme. Cette méthode indiquera que deux instances sont identiques si toutes les conditions suivantes sont respectées :

- Les deux objets sont déclarés dans le même assembly.
- Les données membres doivent avoir le même nom, le même type et être déclarés dans le même ordre.
- Les valeurs des données membres doivent être identiques dans les deux objets.

Vérification :

```
var oProduit = new { Nom = "biscuit", Prix = 1.56 };

var oProduit2 = new { Nom = "biscuit", Prix = 1.56 };

Console.WriteLine(oProduit.Equals(oProduit2));
```

Ce code va afficher *true* car les deux objets repsectent toutes les conditions énumérées ci-dessus.

7.4. Les Interfaces

Avec les classes abstraites, on a vu qu'il était possible d'obliger une classe dérivée à fournir l'implémentation d'une fonction membre.

Si on a plusieurs classes qui doivent implémenter la même méthode, il est plus pratique d'utiliser les interfaces.

Comme les classes, les interfaces permettent de définir un ensemble de données membres, de méthodes et d'évènements. Cependant, ces interfaces, ne contiennent aucune implémentation. Les implémentations doivent se trouver au niveau des classes elles-mêmes.

L'interface peut être vue comme un contrat que l'on signe. En déclarant qu'une classe implémente une interface, on s'engage à fournir dans la classe tout ce qui a été défini dans l'interface.

Il faut être prudent dans l'usage des interfaces ; il ne faut jamais modifier une interface qui est déjà utilisée au risque de devoir réécrire le code de toutes les classes qui en dépendent.

Pour pouvoir utiliser une interface, il faut la définir au préalable. La déclaration est semblable à la déclaration d'une classe mais en utilisant le mot réservé *interface*.

Dans la définition d'une interface, on peut utiliser le caractère : pour définir une relation d'héritage. Les seules indications que l'on peut trouver dans une interface sont des définitions de méthodes et des évènements. Dans les méthodes, il ne doit y avoir aucun code.

```
interface Comparable
{
 int Compare (Object o1);
}
```

Cette interface va obliger à créer dans les classes qui l'implémenteront une fonction permettant de comparer l'instance courante d'un objet avec l'objet qui sera passé en paramètre. La fonction retournera :

- une valeur égale à 1 si l'objet passé comme paramètre est supérieur à l'instance courante,
- une valueur égale à zéro si les deux objets sont égaux,
- une valeur égale à -1 si l'instance courante est supérieure à l'objet passé en paramètre.

Quels critères vont être utilisés pour dire qu'un objet est supérieur ou non à un autre ?

Dans la description de l'interface, on ne doit pas y penser puisqu'il n'y a aucune implémentation dans l'interface. Cela sera de la responsabilité de chaque classe rattachée à cette interface qui devra implémenter la méthode *Compare* et donc de décider comment comparer deux de ses objets.

Par exemple, on pourrait rattacher la classe *Client* à l'interface *Comparable*. Dans ce cas, la méthode *Compare* pourrait comparer deux clients sur base de leur nom :

```
interface Comparable
{
 int Compare(Object o);
}

class Client : Personne, Comparable
{
 protected int m_dID;

 public int Compare(Object o)
 {
     Client oCli;

     if (o is Client)
     {
         oCli = (Client)o;
     }
     else
     {
         throw new InvalidCastException();
     }

     return Nom.CompareTo(oCli.Nom);
 }
}
```

La classe *Client* est rattachée à l'interface *Comparable*.

La classe *Client* doit implémenter toutes les fonctions membres définies dans l'interface.

L'objet universel *o* doit être référencé au compilateur comme étant un objet de la classe *Client*.

Si le paramètre reçu n'est pas un *Client*, alors on provoque une erreur en générant une exception.

On retourne le résultat de la comparaison entre les deux noms.

Quelle va être l'utilité principale d'une interface ?

Il arrive fréquemment que l'on ait besoin de trier des éléments dans une application. Pour y arriver, on a deux solutions :

- créer une fonction de tri spécifique pour chaque type d'élément que l'on veut trier,
- créer une fonction de tri universelle et faire en sorte que les éléments que l'on utilise soient triables par cette routine.

Les interfaces vont aider à mettre en œuvre la deuxième solution. Pour pouvoir trier des éléments, et quelle que soit la méthode utilisée pour le tri, on aura besoin de comparer deux éléments.

Pour être certain que la routine de tri fonctionnera sans problème, il faut s'assurer que les éléments qu'elle devra trier auront la possibilité d'être comparés les uns aux autres. On ne peut garantir cela que si tous les éléments implémentent l'interface *Comparable*. On va l'exiger dans la déclaration de la routine de tri:

```
public static void Tri (Comparable[] iTab)
{
    Comparable iTemp;

    for (int dI = 0; dI < iTab.Length; dI++)
    {
        for (int dJ = dI + 1; dJ < iTab.Length; dJ++)
        {
            if (iTab[dJ].Compare(iTab[dI]) < 0)
            {
```

```
                    iTemp = iTab[dJ];
                    iTab[dJ] = iTab[dI];
                    iTab[dI] = iTemp;
                }
            }
        }
    }
```

Cette fonction de tri est définie dans la classe *Program*. Cette fonction va être capable de trier n'importe quel tableau d'objets à condition qu'ils soient définis dans une classe implémentant l'interface *Comparable*.

On obtient donc notre classe de tri universelle mais la condition est qu'elle ne fonctionne que pour les classes qui implémentent l'interface *Comparable*.

L'usage de la fonction de tri va se faire comme suit :

```
Client[] oTab = new Client [5];

oTab[0] = new Client("Dupont", "Jean", "pswd1", 123);
oTab[1] = new Client("Michel", "André", "pswd2", 124);
oTab[2] = new Client("Léonard", "Robert", "pswd3", 125);
oTab[3] = new Client("Muller", "Luc", "pswd4", 126);
oTab[4] = new Client("Durand", "Pascal", "pswd5", 127);

Tri (oTab);

for (int dI = 0; dI < oTab.Length; dI++)
{
    Console.WriteLine(oTab[dI].Nom);
}
```

Le résultat affiché est :

```
DUPONT
DURAND
LÉONARD
MICHEL
MULLER
Press any key to continue . . .
```

Cette fonction de tri universelle ne fonctionne que pour les classes qui implémentent l'interface *Comparable*.

7.5. Les événements

Les méthodes permettent de communiquer avec les objets qui composent une application mais les objets ont également la possibilité de faire part de leur réaction face en générant des événements. Ces événements doivent ensuite être pris en compte pour réagir à ce qui vient de se passer dans l'application.

Les événements sont très largement utilisés dans la conception d'une interface graphique d'une application car ils permettent d'avoir des informations sur les actions effectuées par l'utilisateur de l'application.

7.5.1. Déclaration et déclenchement d'événements

Voyons tout d'abord, comment répondre à un événement à l'intérieur d'une classe. La première chose à faire est de déclarer l'événement dans la classe. Cette déclaration s'effectue de la même manière que celle des variables internes de la classe.

On utilise pour cela le mot réservé *event* suivi du type de délégué chargé de gérer l'événement et enfin du nom de l'événement. Il est recommandé de toujours utiliser le type de délégué *EventHandler* pour rester cohérent avec le Framework.NET. Ce type de délégué à la signature suivante :

```
public delegate void EventHandler (Object sender, EventArgs e)
```

La méthode qui sera par la suite chargée de gérer l'événement devra bien sûr respecter cette signature. Le premier paramètre permet d'identifier l'objet qui est à l'origine de l'événement. Le deuxième paramètre est utilisé pour fournir à la procédure chargée du traitement des informations complémentaires sur l'origine de l'événement.

S'il n'y a pas d'informations spécifiques à fournir, on pourra utiliser directement une instance de cette classe. Si des informations supplémentaires sont à fournir, il faut créer une classe héritant de la classe *EventArgs*. Cette classe comportera les données membres permettant de représenter les informations de l'événement. Il faut dans ce cas utiliser la version générique du délégué *EventHandler*.

Pour illustrer cela, on va ajouter à la classe *Client* deux événements. Le premier se déclenchera lors de l'ajoute d'une nouvelle commande à un client et fournira comme informations le nombre de commandes du client. Le deuxième événement se déclenchera lorsque le nombre de commandes d'un client sera égal à cinq ; ce dernier ne fournira pas d'informations supplémentaires.

Commençons par créer la classe représentant les informations du premier événement.

```
public class AjouteCmdeEventArgs : EventArgs
{
    private int m_dNbreCmde;
    public AjouteCmdeEventArgs(int dNbreCmde)
    {
        m_dNbreCmde = dNbreCmde;
    }
    public int NbreCmde
    {
        get
        {
            return m_dNbreCmde;
        }
    }
}
```

On peut maintenant déclarer les deux événements dans la classe *Client*.

```
public event EventHandler<AjouteCmdeEventArgs> NewCmde;
public event EventHandler MaxCmde;
```

Il faut maintenant écrire une méthode permettant le déclenchement de l'événement uniquement s'il y a un gestionnaire pour le traiter.

```
protected virtual void DeclencheNewCmde(AjouteCmdeEventArgs e)
 {
     EventHandler<AjouteCmdeEventArgs> handler = NewCmde;
     if (handler != null)
     {
         handler(this, e);
     }
 }

 protected virtual void DeclencheMaxCmde(EventArgs e)
 {
     EventHandler handler = MaxCmde;
```

```
        if (handler != null)
        {
            handler(this, e);
        }
    }
```

Il ne reste plus maintenant qu'à déclencher les événements au bon moment. Dans l'exemple de la classe *Client*, cela se fera dans la méthode *AjouteCmde*.

```
public void AjouteCmde()
 {
     m_dNbreCmde++;

     AjouteCmdeEventArgs e = new
                             AjouteCmdeEventArgs(m_dNbreCmde);
     DeclencheNewCmde(e);
     if (m_dNbreCmde == 5)
     {
         DeclencheMaxCmde(new EventArgs());
     }
 }
```

Un événement a été déclenché au niveau de l'objet. Mais faut-il encore y réagir. On va voir dans la section suivante que l'utilisateur de l'objet peut décider comment il va gérer cet événement.

7.5.2. Gérer les événements

Avant de voir comment gérer les événements, on va vérifier qu'ils ne perturbent pas le bon fonctionnement du code. On va essayer le code suivant qui utilise la classe *Client* sans se préoccuper des événements :

```
Client oCli2;

oCli2 = new Client("Dupont", "jean", "debu", 1452);

oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
```

Le code ci-dessus fonctionne parfaitement. Ceci permet de bien voir qu'il n'est absolument pas obligatoire de gérer tous les événements déclenchés dans une application. C'est une information rassurante quand on sait le nombre impressionnant d'événements déclenchés dans une application Windows.

Commet générer un événement ?

7.5.2.A. Ajoute et suppression de gestionnaire d'événement

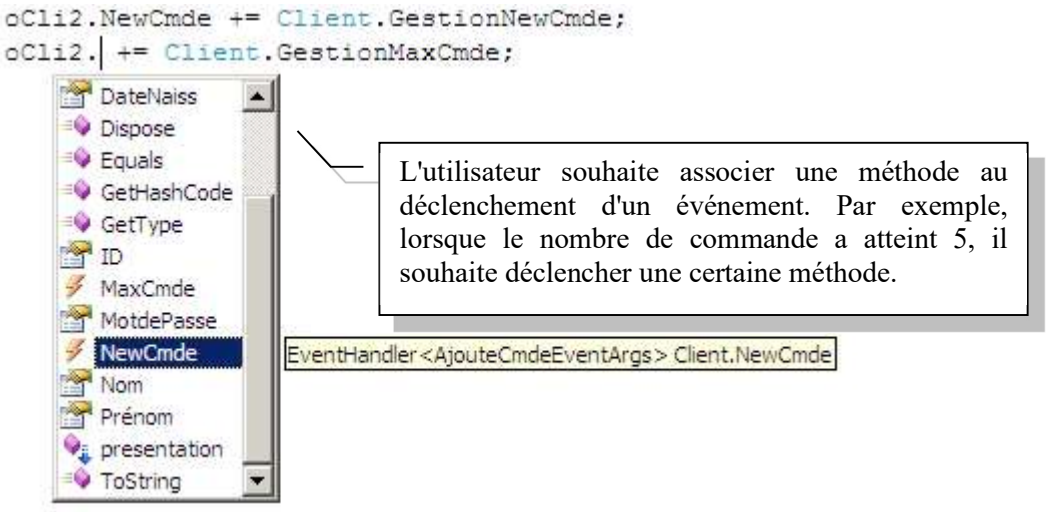
Si on est intéressé par les événements que les objets de l'application vont déclencher, on doit au préalable écrire les procédures chargées de traiter ces événements. Ces procédures doivent correspondre à la signature des événements. Dans l'exemple utilisé, il y a deux procédures à écrire :

```
 public static void GestionNewCmde(object oSender,
                                   AjouteCmdeEventArgs e)
{
      Console.WriteLine("une nouvelle commande");
}
```

```
public static void GestionMaxCmde(object oSender, EventArgs e)
{
      Console.WriteLine("le client a cinq commandes");
}
```

Il faut ensuite connecter ces procédures aux événements qu'elles vont gérer.

Pour cela, on doit utiliser l'opérateur += pour associer la procédure à l'événement déclenché par une instance de classe.



Voici donc les deux expressions permettant d'associer une méthode à un événement :

```
oCli2.NewCmde += Client.GestionNewCmde;
oCli2.MaxCmde += Client.GestionMaxCmde;
```

Avec l'exécution des expressions suivantes :

On créé une association entre un événement et une méthode au niveau de l'objet !

```
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
oCli2.AjouteCmde();
```

Les messages suivants vont être affichés :

```
une nouvelle commande
une nouvelle commande
une nouvelle commande
une nouvelle commande
une nouvelle commande
le client a cinq commandes
une nouvelle commande
une nouvelle commande
une nouvelle commande
Press any key to continue . . .
```

Les deux méthodes gérant les deux événements ont bien été déclenchées. Le lien réalisé entre les événements et les méthodes chargées de les gérer va rester actif pendant toute la durée de vie des objets.

Si on souhaite interrompre le lien, on doit utiliser l'opérateur -= :

```
oCli2.NewCmde -= Client.GestionNewCmde;
oCli2. -= Client.GestionMaxCmde;
```

7.5.2.B. Événement et héritage

Les événements suivent les mêmes règles d'héritage que les données membres et les méthodes. Un événement déclaré dans une classe sera accessible dans toutes les classes dérivées.

7.6. Les délégués

Il peut être parfois utile de pouvoir passer comme paramètre à une fonction ou à une procédure non pas un type simple ou un objet mais directement une procédure ou une fonction.

Par exemple, si on veut choisir le critère suivant lequel on va trier les clients, on doit écrire une fonction de comparaison pour chacun des critères possibles. Lors de l'appel à la procédure de tri, on va spécifier quelle est la méthode qui devra être utilisée pour effectuer la comparaison.

7.6.1. Déclaration et création d'un délégué

La première étape consiste à déclarer le délégué. Cette déclaration va utiliser le mot réservé *Delegate* suivi par la signature de la fonction ou de la procédure.

```
public delegate int Comparaison (Client oCli1, Client oCli2);
```

Il faut ensuite créer une procédure ou une fonction qui respecte la signature du délégué. Dans l'exemple utilisé, on va créer deux fonctions capables de comparer deux clients.

Nous en créerons une pour effectuer la comparaison sur l'ID et une deuxième pour la comparaison sur le nombre de commandes.

```
public class Client : Personne, Comparable
 {
  protected int m_dID;
  private int m_dNbreCmde;

  public static int CompareID(Client oCli1, Client oCli2)
  {
      if (oCli1.m_dID < oCli2.m_dID)
      {
          return -1;
      }
      if (oCli1.m_dID > oCli2.m_dID)
      {
          return 1;
      }
      else
      {
          return 0;
      }
  }

  public static int CompareCmd(Client oCli1, Client oCli2)
  {
```

```
        if (oCli1.m_dNbreCmde < oCli2.m_dNbreCmde)
        {
            return -1;
        }
        if (oCli1.m_dNbreCmde > oCli2.m_dNbreCmde)
        {
            return 1;
        }
        else
        {
            return 0;
        }
    }
```

Il faut maintenant modifier la fonction de tri pour qu'en plus du tableau à trier, elle prenne, comme paramètre, la fonction utilisée pour la comparaison de deux éléments du tableau. La fonction de tri devient donc :

```
public static void Tri(Client[] tCli, Comparaison fctCompare)
{
    Client oCli;

    for (int dI = 0; dI < tCli.Length - 1; dI++)
    {
        for (int dJ = 0; dJ < dI; dJ++)
        {
            if (fctCompare.Invoke(tCli[dI], tCli[dJ]) < 0)
            {
                oCli = tCli[dI];
                tCli[dI] = tCli[dJ];
                tCli[dI] = oCli;
            }
        }
    }
}
```

Deux modifications dans le code :

- L'ajoute de *fctCompare* comme délégué dans la fonction de tri,
- L'utilisation de ce délégué pour comparer deux éléments du tableau en utilisant la méthode *invoke*.

Les éléments du tableau ne sont plus obligés d'implémenter l'interface *Comparable* car un moyen de les comparer deux à deux est passé à la fonction.

7.6.2. Utilisation des délégués

Pour utiliser la fonction de tri, on doit maintenant lui fournir deux paramètres : le tableau à trier et une instance d'un délégué pour trier le tableau. On doit donc créer une instance de délégué.

```
comparaison cmp = compareCode;
```

On doit ensuite appeler la fonction de tri en lui passant comme deuxième paramètre l'instance du délégué.

```
Tri(tab, cmp);
```

Si on souhaite utiliser un autre critère de tri, on doit simplement créer une autre fonction respectant la signature du délégué et construire un délégué à partir de cette fonction.

7.7. Les types génériques

7.7.1. Introduction

Les types génériques sont des éléments d'un programme qui s'adaptent, automatiquement, pour réaliser la même fonctionnalité sur différents types de données. Lorsque l'on créé un élément générique, on n'a pas besoin de concevoir une version différente pour chaque type de donnée avec lequel on souhaite réaliser une fonctionnalité.

Pour faire une analogie avec un objet courant, on va prendre l'exemple d'un tournevis. En fonction du type de vis à utiliser, on peut prendre un tournevis spécifique pour le type de vis. Une technique fréquemment utilisée par les bricoleurs consiste à acquérir un tournevis universel avec de multiples embouts. En fonction du type de vis, il choisit l'embout adapté. Le résultat est le même que s'il dispose d'une multitude de tournevis différents : il peut visser et dévisser.

Lorsque l'on utilise un type générique, on va le paramétriser avec un type de données. Ceci permet au code de s'adapter automatiquement et de réaliser la même action indépendamment du type de données. Une alternative pourrait être l'utilisation du type universel *Object*. L'utilisation des types génériques présente plusieurs avantages par rapport à cette solution :

- Elle impose la vérification des types de données au moment de la compilation et évite les vérifications qui doivent être effectuées manuellement avec l'utilisation du type *Object*.
- Elle évite les opérations de conversion, du type *Object* vers un type plus spécifique et inversement, consommatrices de ressources.
- Elle évite l'utilisation de la liaison *tardive*, incontournable avec le type *Object*.
- L'écriture du code est facilitée par l'environnement de développement grâce à *IntelliSense*.
- Elle favorise l'écriture d'algorithmes indépendants des types de données.

Les types génériques peuvent cependant imposer certaines restrictions concernant le type de donnée utilisé. Ils peuvent par exemple, imposer que le type utilisé implémente une ou plusieurs interfaces, qu'il soit un type référence ou possède un constructeur par défaut.

Il est important de bien comprendre quelques termes utilisés avec les génériques :

Le type générique. C'est la définition d'une classe, structure interface ou procédure pour laquelle on spécifie au moins un type de données, au moment de sa déclaration.

Le type paramètre. C'est l'emplacement réservé pour le type de données dans la déclaration du type générique.

Le type argument. C'est le type de donnée qui remplace le type de paramètre, lors de la construction d'un type à partir d'un type générique.

Les contraintes. Ce sont les conditions que l'on impose qui limitent le type argument que l'on peut fournir.

Le type construit. C'est la classe, interface, structure ou procédure déclarée à partir d'un type générique pour lequel on a spécifié des types arguments.

7.7.2. Les classes génériques

Une classe qui attend un type de paramètre est appelé classe générique. On peut générer une classe construite en fournissant à la classe générique un type argument pour chacun de ces types paramètre.

7.7.2.A. Définition d'une classe générique

On peut définir une classe générique qui fournit les mêmes fonctionnalités sur différents types de données. Pour cela, on doit fournir un ou plusieurs types de paramètre dans la définition de la classe.

Prenons l'exemple d'une classe, capable de gérer une liste d'éléments avec les fonctionnalités suivantes :

- Ajouter un élément,
- Supprimer un élément,
- Se déplacer sur le premier élément,
- Se déplacer sur le dernier élément,
- Se déplacer sur l'élément suivant,
- Se déplacer sur l'élément précédent,
- Obtenir le nombre d'éléments.

On doit d'abord définir la classe comme une classe ordinaire :

```
class ListeGenerique
{
}
```

La transformation de cette classe en classe générique s'effectue en ajoutant un type de paramètre immédiatement après le nom de la classe :

```
class ListeGenerique<TypeDeDonnee>
{
}
```

Si plusieurs types de paramètres sont nécessaires, ils doivent être séparés par des virgules.

Si le code doit réaliser d'autres opérations que des assignations, on doit ajouter des contraintes sur le type de paramètre. Par exemple, si le paramètre doit implémenter une interface particulière, on va utiliser la syntaxe suivante :

```
class ListeGenerique<TypeDeDonnee> where TypeDeDonnee : IComparable
{
}
```

S'il n'y a pas de contrainte spécifiée, les seules opérations autorisées seront celles supportées par le type *Object*.

Voyons le code complet de la classe :

```
class ListeGenerique<TypeDeDonnee>
{
 private TypeDeDonnee[] m_tListe;
 private int m_dCurPos;
 private int m_dPosition;
 private int m_dTaille;
 private bool m_bFull = false;

 public ListeGenerique(int dTaille)
 {
     m_tListe = new TypeDeDonnee[dTaille];
     m_dTaille = dTaille;
     m_dPosition = 0;
     m_dCurPos = 0;
 }
```

```
public void Add(TypeDeDonnee oElem)
{
    if (m_bFull)
        return;

    m_tListe[m_dCurPos] = oElem;
    m_dCurPos++;
    m_bFull = (m_dCurPos == m_dTaille);
}

public void Delete(int dPos)
{
    if (dPos >= m_dCurPos || dPos < 0)
        return;

    // Décalage des éléments d'une position vers le haut
    for (int dI = dPos; dI < m_dCurPos - 1; dI++)
    {
        m_tListe[dI] = m_tListe[dI + 1];
    }

    m_dCurPos--;
}

public int Taille
{
    get
    {
        return m_dTaille;
    }
}

public TypeDeDonnee Premier()
{
    if (m_dCurPos == 0)
    {
        throw new Exception("Liste vide");
    }
    m_dPosition = 0;
    return m_tListe[0];
}

public TypeDeDonnee Dernier()
{
    if (m_dCurPos == 0)
    {
        throw new Exception("Liste vide");
    }
    m_dPosition = m_dCurPos - 1;
    return m_tListe[m_dPosition];
}

public TypeDeDonnee Suivant()
{
    if (m_dCurPos == 0)
    {
        throw new Exception("Liste vide");
    }
    if (m_dPosition == m_dCurPos)
    {
```

```
            throw new Exception("Pas d'élément suivant");
        }
        m_dPosition++;
        return m_tListe[m_dPosition];
    }

    public TypeDeDonnee Precedent()
    {
        if (m_dCurPos == 0)
        {
            throw new Exception("Liste vide");
        }
        if (m_dPosition == 0)
        {
            throw new Exception("Pas d'élément précédent");
        }
        m_dPosition--;
        return m_tListe[m_dPosition];
    }
}
```

7.7.2.B. Utilisation d'une classe générique

Pour pouvoir utiliser une classe générique, on doit d'abord générer une classe construite en fournissant un type d'argument pour chacun de ces types paramètres. Par exemple : *ListeGenerique<int>*.

On peut alors créer un objet dans la classe construite par un des constructeurs disponibles. Voici un exemple en réutilisant la classe *ListeGenerique* définie précédemment :

```
ListeGenerique<int> oListe = new ListeGenerique<int>(5);
```

Cette déclaration a créé un objet *oListe* pouvant contenir un maximum de 5 entiers. Les méthodes de la classe sont alors disponibles :

```
oListe.Add(10);
oListe.Add(11);
oListe.Add(12);
oListe.Add(13);
oListe.Add(14);
```

Le compilateur vérifie bien sûr que nous utilisons la classe correctement, notamment en vérifiant les types de données qu'on lui confie :

Voici la fonction *Menu()* permettant de tester le bon fonctionnement de l'objet *oListe* :

```
static public void Menu(ListeGenerique<int> oListe)
{
    char cChoix;

    Console.SetCursorPosition(1, 24);
    Console.WriteLine("p = Premier, < = Précédent, > = Suivant, d =
    Dernier, f = Fin");
    cChoix = Console.ReadKey().KeyChar;

    while (cChoix != 'f')
    {
        Console.Clear();
        Console.SetCursorPosition(1, 1);
```

```
            try
            {
                switch (cChoix)
                {
                    case 'p':
                        Console.WriteLine("Le premier est {0}",
                                                    oListe.Premier());
                        break;
                    case '<':
                        Console.WriteLine("Le premier est {0}",
                                                    oListe.Precedent());
                        break;
                    case '>':
                        Console.WriteLine("Le premier est {0}",
                                                    oListe.Suivant());
                        break;
                    case 'd':
                        Console.WriteLine("Le premier est {0}",
                                                    oListe.Dernier());
                        break;
                }
            }
            catch (Exception e)
            {
                Console.ForegroundColor = ConsoleColor.Red;
                Console.WriteLine(e.Message);
                Console.ForegroundColor = ConsoleColor.White;
            }
            Console.SetCursorPosition(1, 24);
            Console.WriteLine("p = Premier, < = Précédent, > = Suivant, d
                                = Dernier, f = Fin");
            cChoix = Console.ReadKey().KeyChar;
        }
    }
```

Permet de récupérer l'exception générée dans les méthodes de la classe *ListeGenerique*.

On pourrait aussi vérifier que la classe fonctionne bien si on lui demande de travailler avec des chaînes de caractères :

```
ListeGenerique<string> oListe = new ListeGenerique<string>(5);

oListe.Add("Premier");
oListe.Add("Deuxième");
oListe.Add("Troisième");
oListe.Add("Quatrième");
oListe.Add("Cinquième");
```

7.7.3. Les Procédures et les Fonctions génériques

Une procédure ou une fonction générique sont des méthodes définies avec au moins un type paramètre. Ceci permet au code appelant de spécifier le type de données dont il a besoin à chaque appel de la procédure ou de la fonction.

Une telle méthode peut cependant être utilisée sans indiquer d'information pour le type argument. Dans ce cas, le compilateur essaie de déterminer le type, en fonction des arguments passés à la méthode. Cette solution doit toutefois être utilisée avec précaution, car si le compilateur ne peut pas déterminer le type des arguments, il provoquera une erreur de compilation

7.7.3.A. Création d'une procédure ou d'une fonction générique

La déclaration d'une procédure ou d'une fonction générique doit contenir au moins un type paramètre. Ce type paramètre est défini par un identifiant. Dans l'exemple précédent, *TypeDeDonnee*. Cet identifiant est ensuite utilisé dans le reste du code, à chaque fois que l'on a besoin d'utiliser le type paramètre.

On va maintenant créer une fonction générique capable de rechercher un élément particulier dans un tableau de n'importe quel type. Cette fonction va utiliser un type paramètre indiquant la nature des éléments présents dans le tableau.

Pour pouvoir rechercher un élément dans le tableau, on va devoir le comparer avec ceux présents dans toutes les cases du tableau. Pour s'assurer que cette comparaison sera possible dans tous les cas de figure, on va ajouter une contrainte sur le type paramètre. Il doit implémenter l'interface *IComparable* afin de garantir que la méthode *CompareTo* utilisée dans la fonction soit disponible pour chaque élément du tableau.

La déclaration de la fonction va prendre la forme suivante :

```
public static int RechercheGenerique<TypeDeDonnee>(TypeDeDonnee[] tTab,
     TypeDeDonnee oElemRecherche) where TypeDeDonnee : IComparable
```

Après avoir vérifié que le tableau contient au moins un élément, on va comparer l'élément recherché avec celui présent dans chaque case du tableau. S'il y a égalité, la fonction retourne l'index où l'élément a été trouvé, sinon la fonction retourne -1.

Pour effectuer la comparaison, on va utiliser la fonction *CompareTo* définie dans le *TypeDeDonnee*. ON est certain que l'implémentation de cette fonction s'y trouve puisque ce type implémente l'interface *IComparable*.

Voici le code de la fonction définie comme fonction statique définie dans la classe *Program*:

```
public static int RechercheGenerique<TypeDeDonnee>(TypeDeDonnee[] tTab,
     TypeDeDonnee oElemRecherche) where TypeDeDonnee : IComparable
 {
     // Test si le tableau a plusieurs dimensions
     if (tTab.Rank > 1)
         return -1;

     // Test si le tableau est vide
     if (tTab.Length == 0)
         return -1;

     for (int dI = 0; dI <= tTab.GetUpperBound(0); dI++)
     {
         if (tTab[dI].CompareTo(oElemRecherche) == 0)
             return dI;
     }

     return -1;
 }
```

Donne la valeur d'index maximale.

7.7.3.B. Utilisation d'une procédure ou d'une fonction générique

L'utilisation d'une procédure ou d'une fonction générique est identique à celle d'une procédure ou d'une fonction classique, hormis la nécessité de spécifier un type argument pour le ou les types paramètre.

Le code suivant permet de tester le bon fonctionnement de la fonction :

```
int[] tTab = { 12, 45, 85, 47, 62, 95, 81 };
int dResult;
```

```
dResult = RechercheGenerique<int>(tTab, 81);

if (dResult == -1)
   Console.WriteLine("Valeur non trouvée");
else Console.WriteLine("Valeur trouvée en position {0}", dResult);
```

7.8. Les Collections

Les applications ont très fréquemment besoin de manipuler de grandes quantités d'information. De nombreuses structures sont disponibles en Visual C# pour faciliter la gestion de ces informations. Elles sont regroupées sous le terme *Collection*.

7.8.1. Les Collections prédéfinies

Les différentes classes permettant la gestion de collections sont réparties dans deux espaces de nom :

- System.Collections
- System.Collections.Generic

Le premier contient les classes normales alors que le deuxième contient les classes génériques équivalentes permettant la création de collections fortement typées.

Ces collections fortement typées sont spécialisées dans la gestion d'un type précis de données. Bien que ces nombreuses classes fournissent des fonctionnalités différentes, elles ont quand même des points communs, dus au fait qu'elles implémentent les mêmes interfaces.

Par exemple, toutes ces classes sont capables de fournir une méthode *enumerator* permettant de parcourir l'ensemble de la collection. C'est cet objet qui est d'ailleurs utilisé par l'instruction *for each* de Visual C#.

7.8.1.A. La classe Array

La classe *Array* ne fait pas partie de l'espace de nom *System.Collection* mais elle peut quand même être considérée comme une collection car elle implémente l'interface *IList*.

Les tableaux créés à partir de la classe *Array* sont de taille fixe. Cette classe contient une multitude de méthodes partagées, permettant l'exécution de nombreuses fonctionnalités sur les tableaux. Deux propriétés sont très utiles pour l'utilisation de la classe *Array* :

- *Length* qui retourne le nombre total d'éléments dans le tableau.
- *Rank* qui contient le nombre de dimensions du tableau.

7.8.1.B. Les classes ArrayList et List

La classe *ArrayList* ou sa version générique *List* sont des évolutions de la classe *Array*. Elles apportent de nombreuses améliorations par rapport à cette dernière :

- La taille d'un *ArrayList* est dynamique et est automatiquement ajustée en fonction des besoins.
- Elle propose des méthodes permettant l'ajoute, l'insertion et la suppression de plusieurs éléments simultanément en une seule opération.

Par contre, sur certains points, la classe *ArrayList* est moins efficace qu'un simple tableau :

- Les *ArrayList* n'ont qu'une seule dimension.

- Un tableau de données d'un type spécifique est plus efficace qu'un *ArrayList* dont les éléments sont gérés en tant qu'*Object*. L'utilisation de la version générique *List* permet d'obtenir des performances équivalentes.

Comme toute classe, un *ArrayList* doit être instancié avant de pouvoir être utilisé. Deux constructeurs sont disponibles :

- Le premier est le constructeur par défaut et créé un objet *ArrayList* de taille zéro. Il sera ensuite dimensionné automatiquement, lors de l'ajoute d'éléments. Cette solution n'est pas conseillée car l'agrandissement de l'objet *ArrayList* consomme beaucoup de ressources.
- Si on a une estimation du nombre d'éléments à stocker, il est préférable d'utiliser le deuxième constructeur qui attend comme paramètre la capacité initiale de l'*ArrayList*. Ceci évite de redimensionner l'objet à chaque ajoute d'un élément.

La taille indiquée initialement n'est pas définitive et l'*ArrayList* pourra contenir plus d'éléments qu'initialement prévu.

Autres propriétés et méthodes intéressantes :

- La propriété *Capacity* permet de connaître le nombre d'éléments que l'*ArrayList* peut contenir.
- La propriété *Count* indique le nombre actuel d'éléments dans l'*ArrayList*.
- Les méthodes *Add* et *AddRange* ajoutent des éléments à la fin de la liste.
- Les méthodes *Insert* et *InsertRange* permettent de choisir l'emplacement où va s'effectuer l'ajoute.
- La propriété *Item* s'utilise pour atteindre un élément à une position donnée.
- La suppression d'éléments se fait par la méthode *RemoveAt* ou *RemoveRange* : la première attend comme paramètre l'index de l'élément à supprimer, la deuxième exige en plus le nombre d'éléments à supprimer.
- La méthode *Clear* est plus radicale et supprime tous les éléments.

Voici un exemple de fonctionnement de cette classe :

```
ArrayList oList;

oList = new ArrayList();

Console.WriteLine("Capacité initiale de la liste {0}", oList.Capacity);
Console.WriteLine("Nombre d'éléments de la liste {0}", oList.Count);
Console.WriteLine("Ajoute d'un client");

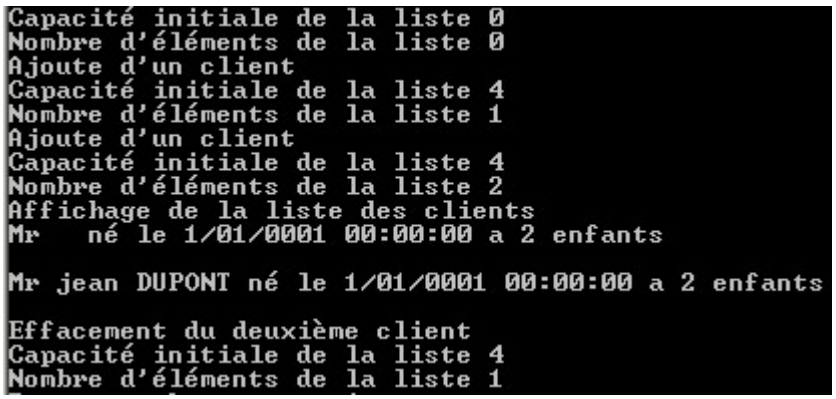
oList.Add(oCli);
Console.WriteLine("Capacité initiale de la liste {0}", oList.Capacity);
Console.WriteLine("Nombre d'éléments de la liste {0}", oList.Count);
Console.WriteLine("Ajoute d'un client");

oList.Add(oCli2);
Console.WriteLine("Capacité initiale de la liste {0}", oList.Capacity);
Console.WriteLine("Nombre d'éléments de la liste {0}", oList.Count);

Console.WriteLine("Affichage de la liste des clients");
foreach (Client oC in oList)
{
    oC.Affichage();
    Console.WriteLine();
}

Console.WriteLine("Effacement du deuxième client");
oList.RemoveAt(1);
Console.WriteLine("Capacité initiale de la liste {0}", oList.Capacity);
Console.WriteLine("Nombre d'éléments de la liste {0}", oList.Count);
```

Voici le résultat affiché par l'exécution de ce code :



La capacité de la liste ne diminue pas lors de la suppression d'un élément, même lorsque la liste est vide.

Tri d'un objet de la classe ArrayList

Le tri d'un objet se réalise en quatre étapes :

1) La déclaration d'un objet *ArrayList* :

```
ArrayList tLigFact = new ArrayList(100);
```

2) Choisir le type des données à mémoriser dans le tableau :

```
etLigFact stLigFact = new etLigFact();
tLigFact.Add(stLigFact);
```

3) Le type de donnée inséré doit être hérité de l'interface *IComparable* et doit disposer d'une méthode *CompareTo* :

```
public struct etLigFact : IComparable
{
  public long ldFactID;
  public float fMontPaye;
  public float fMontTVAC;
  public DateTime dteDate;

  public int CompareTo(object stLigFact)
  {
      return dteDate.CompareTo(((etLigFact)stLigFact).dteDate);
  }
}
```

4) Invoquer la méthode de tri sur le tableau :

```
tLigFact.Sort();
```

7.8.1.C. Hashtable et Dictionary

Une *Hashtable* ou sa version générique *Dictionary* enregistre les informations sous forme de couple (clés, valeur). La *Hashtable* est constituée en interne de compartiments contenant les éléments de la collection

Pour chaque élément de la collection, un code est généré par une fonction de hachage, basée sur la clé de chaque élément. Le code est ensuite utilisé pour identifier le compartiment dans lequel est stocké l'élément. Lors de la

recherche d'un élément dans la collection, l'opération inverse est effectuée. Le code de hachage est généré à partir de la clé de l'élément recherché. Cette clé est ensuite utilisée à identifier le compartiment dans lequel se trouve l'élément recherché.

Pour qu'une *Hashtable* puisse stocker un objet, celui-ci doit être capable de fournir son propre code de hachage.

7.8.1.D. Queue (File)

Ce type de collection est utilisé lorsqu'on a besoin d'un espace de stockage temporaire. Lorsqu'un élément est récupéré à partir de la collection, il est en même temps supprimé de la collection.

Les collections de type *Queue* sont organisées dans l'ordre FIFO (First In First Out). Les trois principales opérations disponibles sont :

- *Enqueue* pour ajouter un élément à la fin de la queue.
- *Dequeue* pour obtenir l'élément le plus ancien de la queue et le supprimer.
- *Peek* pour obtenir l'élément le plus ancien de la queue mais sans le supprimer.

7.8.1.E. Stack (Pile)

Les collections de type *Stack* utilisent le même principe que les *queues* : lorsqu'un élément est récupéré de la collection, il est supprimé. La seule distinction par rapport à la classe *Queue* est l'ordre dans lequel les éléments sont récupérés. Au lieu d'utiliser l'ordre FIFO, les *Stacks* implémentent l'ordre LIFO (Last In First Out).

L'exemple classique de ce type de gestion est la pile d'assiettes de la cuisine. On y empile les assiettes par-dessus et quand on en a besoin, on prend celle qui se trouve au sommet de cette pile.

Les trois principales opérations sont :

- *Push* pour ajouter un élément au sommet de la pile,
- *Pop* pour obtenir l'élément au sommet de la pile et le supprimer,
- *Peek* pour obtenir l'élément au sommet de la pile sans le supprimer.

8. Gestion des erreurs et débogage du code

8.1. Traitement des exceptions

8.1.1. Récupération d'exceptions

La gestion des exceptions donne la possibilité de protéger un bloc de code contre les erreurs d'exécution qui pourrait s'y produire. Le code dangereux doit être placé dans un bloc *try*.

Si une exception est déclenchée dans ce bloc de code, Visual C# regarde les instructions *catch* qui suivent. S'il en existe une capable de traiter l'exception, le code correspondant est exécuté, sinon la même exception peut être déclenchée pour éventuellement être récupérée par un bloc *try*, de plus haut niveau.

Une instruction *finally* permet de marquer un groupe d'instructions, exécutées avant la sortie bu bloc *try*, qu'une erreur se soit produite ou non.

La syntaxe générale est la suivante :

```
try
{
      …
      Instructions dangereuses
      …
}
catch (Exception1 e1)
{
 …
 Code exécuté si une exception de type Exception1 se produit
 …
}
catch (Exception1 e2)
{
 …
 Code exécuté si une exception de type Exception2 se produit
 …
}
 finally
{
 …
 Code exécuté dans tous les cas avant la sortie du bloc try
 …
}
```

Cette structure a un fonctionnement très similaire au *switch* déjà étudié.

Chaque type d'erreur est associé à une classe d'exception. Lorsqu'une erreur de ce type se produit, une instance de la classe *Exception* correspondante est créée. On peut donc déterminer, pour chaque instruction *catch*, quel type d'exception elle doit traiter.

Voici un exemple de code susceptible de générer une erreur d'entrée/sortie :

```
public static void OpenFichier()
{
      FileStream oFichier;

      try
      {
```

```
            oFichier = new FileStream("c:\\data.txt", FileMode.Open);
        }
        catch (IOException e)
        {
            Console.WriteLine("Erreur d'ouverture de fichier");
        }
        finally
        {
            Console.WriteLine("Fin de la procdure d'ouverture de fichier");
        }
    }
```

Étant donné que le fichier référencé, n'existe pas, les messages suivants vont être générés :

```
Erreur d'ouverture de fichier
Fin de la procdure d'ouverture de fichier
```

L'erreur dans l'ouverture du fichier a généré une exception qui a été rattrapée par la fonction. Il faut remarquer que le code de l'option *Finally* a été exécuté à la fin. Sans l'usage du *try-catch*, voici le message d'erreur que le programme aurait généré :

```
Unhandled Exception: System.IO.FileNotFoundException: Could not find file 'c:\data.txt'.
File name: 'c:\data.txt'
   at System.IO.__Error.WinIOError(Int32 errorCode, String maybeFullPath)
   at System.IO.FileStream.Init(String path, FileMode mode, FileAccess access, Int32 rights, Boolean useRights, FileSh
rSize, FileOptions options, SECURITY_ATTRIBUTES secAttrs, String msgPath, Boolean bFromProxy)
   at System.IO.FileStream..ctor(String path, FileMode mode)
   at NSClient.Program.OpenFichier() in H:\User\Marc\Cours C#\Exemples et Exercices\Personne\Personne\Program.cs:line
   at NSClient.Program.Main(String[] args) in H:\User\Marc\Cours C#\Exemples et Exercices\Personne\Personne\Program.cs
Press any key to continue . . .
```

Et le programme se serait arrêté !

Si, parmi tous les *catch*, aucun ne correspond à l'exception générée, l'exception est propagée dans le code des fonctions (ou procédures) appelantes à la recherche d'une instruction *catch* capable de prendre en compte l'exception générée. Si aucun bloc *catch* n'est trouvé alors une erreur d'exécution est déclenchée.

Si le paramètre indiqué à l'instruction *catch* est la classe de base *exception* alors elle sera capable de capturer toutes les exceptions créées à partir de cette classe ou de cette sous-classe. Évidemment, l'inconvénient sera que l'on obtiendra des informations très générales sur l'exception générée. En utilisant une classe plus spécifique, il sera possible de récupérer des données plus précises sur la nature de l'erreur.

```
    public static void OpenFichier()
    {
        FileStream oFichier;

        try
        {
            oFichier = new FileStream("c:\\data.txt", FileMode.Open);
        }
        catch (Exception e)
        {
            Console.WriteLine("Erreur d'ouverture de fichier");
        }
        finally
        {
            Console.WriteLine("Fin de la procdure d'ouverture de fichier");
        }
    }
```

Les différentes classes disposent des propriétés suivantes permettant d'avoir plus d'informations sur l'origine de l'exception :

- *Message* : Chaîne de caractères associée à l'exception,
- *Source* : Nom de l'application qui a déclenché l'exception,
- *StackTrace* : Liste de toutes les méthodes par lesquelles l'application est passée avant le déclenchement de l'erreur.
- *TargetSite* : Nom de la méthode ayant déclenché l'exception,
- *InnerException* : obtient l'exception originale, si deux exceptions sont déclenchées en cascade.

8.1.1.A. Création et déclenchement d'exceptions

Les exceptions sont avant tout des classes, il est donc possible de créer de nouvelles classes d'exceptions en les rattachant à une classe d'exception existante.

Pour respecter les conventions du Framework .Net, il est conseillé de garder le préfixe *Exception* dans le nom de la nouvelle classe. Ainsi, par exemple, on pourrait créer la nouvelle classe suivante :

```
class NewException: Exception
{
      public NewException()
            : base()
      {
      }
      public NewException(String strMsg)
            : base(strMsg)
      {
      }
      public NewException(string strMsg, Exception e, int dCode)
            : base(strMsg, e)
      {
            m_dCode = dCode;
      }
      private int m_dCode;
}
```

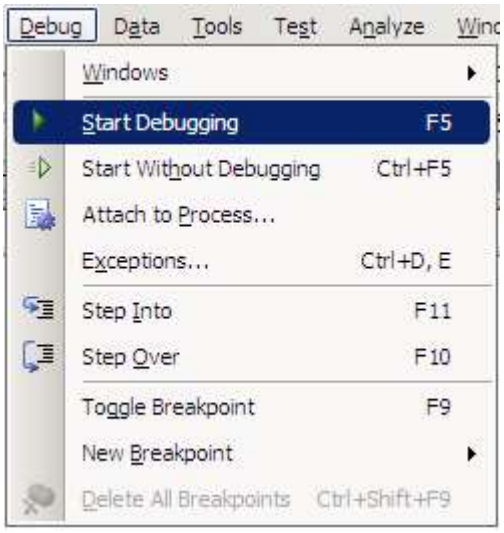
On peut ensuite utiliser cette nouvelle classe pour générer une exception personnalisée :

```
try
{
      oFichier = new FileStream("c:\\data.txt", FileMode.Open);
}
catch (IOException eIO)
{
      throw new NewException("erreur ouverture", eIO, 1024);
}
catch (NewException eNew)
{
      Console.WriteLine("{0} {1}", eNew.Message, eNew.dCode);
}
finally
{
      Console.WriteLine("Fin de la procdure d'ouverture de fichier");
}
```

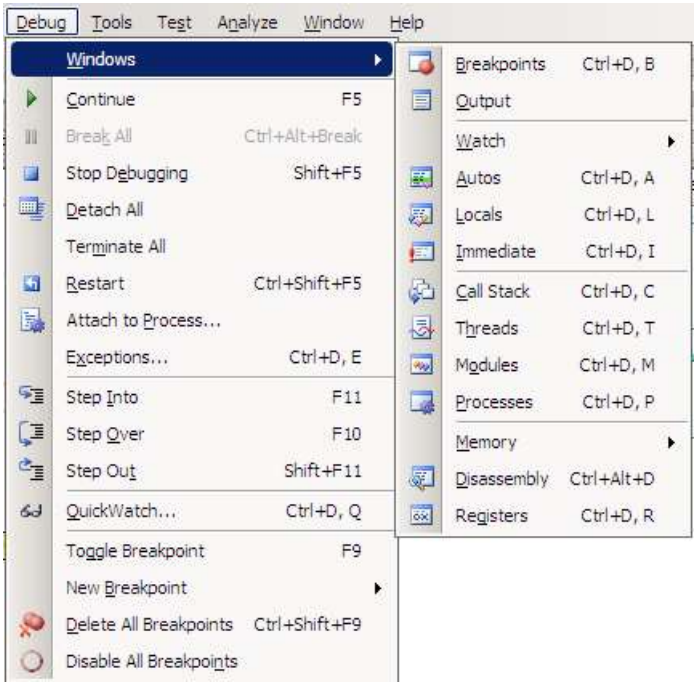
!!! provoque une erreur

8.2. Les outils de débogage

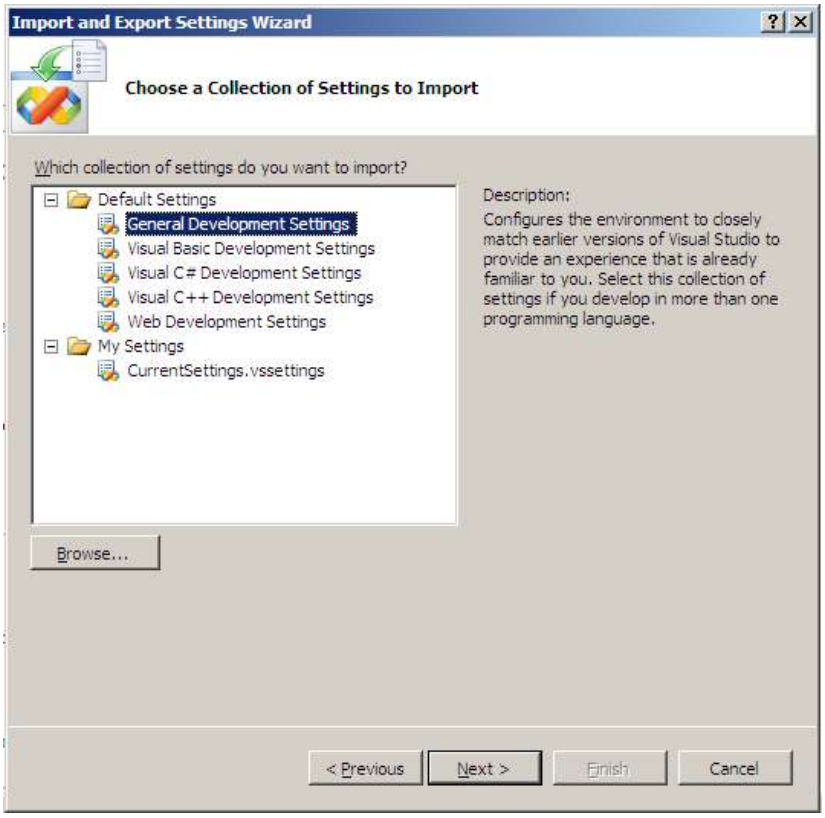
Les erreurs de logique étant les plus difficiles à trouver, Visual Studio .Net propose toute une série d'outils de débogage à la fois performants et faciles d'utilisation. Ils permettent de contrôler le déroulement de l'exécution de l'application, de visualiser et de modifier le contenu des variables, de visualiser le contenu de la mémoire à un emplacement particulier, de vérifier la liste de toutes les fonctions utilisées,… Ces différents outils sont accessibles par la barre d'outils *Déboguer* :



En cours d'exécution, le programme peut être débogué grâce à un menu plus complet :



En fonction de la configuration de l'environnement de Visual Studio, certains outils ne seront peut-être pas disponibles. On peut reconfigurer Visual Studio pour intégrer ces outils supplémentaires par le menu *Outils/Importation et exportation de paramètres.* Les différentes boîtes de dialogue proposent de sauvegarder l'environnement actuel avant de le modifier, puis de choisir un environnement type à importer.



Parmi les configurations disponibles, c'est la configuration *Paramètres de développement généraux* qui propose le plus de fonctionnalités. On va considérer que c'est cette configuration qui est installée.

8.2.1. Contrôle de l'exécution

8.2.1.A. Démarrage de la solution

Un projet dans Visual Studio peut être dans trois états distincts :

- En conception,
- En exécution,
- En mode arrêt (l'exécution a été interrompue).

Le lancement de l'exécution peut s'effectuer par la barre d'outils ou par le raccourci-clavier [F5] ou [ctrl][F5]. Avec cette dernière option, l'application est exécutée en mode normal et aucun outil de débogage n'est disponible.

Si la solution contient plusieurs projets, l'un d'entre eux doit être configuré comme le projet de démarrage de la solution. Ce projet doit également avoir un objet de démarrage configuré, c'est par son exécution que va débuter l'application.

8.2.1.B. Arrêter la solution

L'arrêt de l'application peut s'effectuer en fermant toutes les fenêtres ; pour une application Windows, dés que la dernière fenêtre est fermée, l'application s'arrête ou par la combinaison des touches [ctrl][C] pour une application console. La barre d'outils ou le raccourci-clavier [ctrl][alt][Pause] permettent aussi d'arrêter l'application.

8.2.1.C. Interrompre la solution

L'interruption de l'exécution s'effectue avec la combinaison de touches [ctrl][alt][Pause] ou par la barre d'outils.

L'interruption se produit sur l'instruction suivant celle en cours d'exécution au moment de la demande d'arrêt. La fenêtre de code devient à nouveau visible avec un repère en face de la ligne où l'exécution s'est interrompue.

Cette méthode n'est pas très pratique car il faut avoir beaucoup de chance pour interrompre l'exécution à un endroit précis. Les points d'arrêt abordés plus tard dans ce chapitre offrent une bien meilleure solution pour interrompre l'exécution du code.

8.2.1.D. Poursuivre l'exécution

Une fois en mode arrêt, on a de nombreuses possibilités pour continuer l'exécution de l'application.

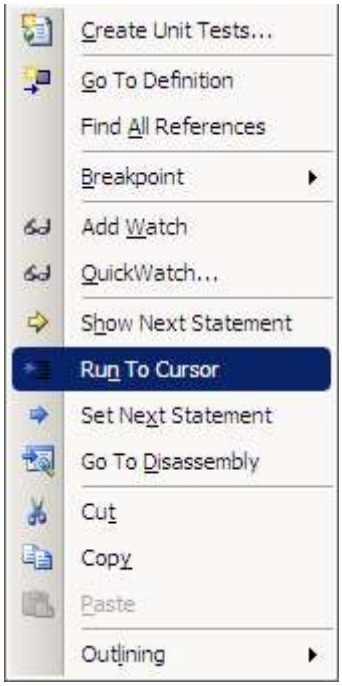
La première possibilité permet de reprendre l'exécution normale de l'application en utilisant la même technique que pour le démarrage du programme (F5). Cependant, une technique plus courante lors du débogage est l'exécution pas à pas.

Trois solutions sont disponibles :

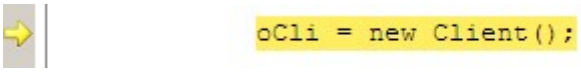
- **Pas à pas détaillé [F11]**. Lorsque l'exécution est arrêtée sur un appel de fonction ou de procédure, le Pas à pas détaillé va permettre d'entrer dans le code de la fonction ou de la procédure et de l'exécuter pas à pas.
- **Pas à pas principal [F10]**. Lorsque l'exécution est arrêtée sur un appel de fonction ou de procédure, le Pas à pas principal va permettre de ne pas entrer dans le code de la fonction ou de la procédure. La fonction ou la procédure est exécutée comme une instruction simple.
- **Pas à pas sortant [shift][F11]**. Quand on se trouve dans une fonction ou une procédure, ce mode permet l'exécution du code jusqu'à la fin d'une procédure ou d'une fonction, sans décomposer ligne par ligne, puis repasse en mode arrêt sur la ligne suivant l'appel de la fonction ou de la procédure.



Une dernière solution permet facilement d'exécuter un bloc de code puis de s'arrêter sur une ligne spécifique. Pour cela un menu contextuel sur la fenêtre de code offre la possibilité de relancer l'exécution jusqu'à l'emplacement du curseur, sans s'arrêter sur toutes les lignes se trouvant entre la ligne actuelle et la position du curseur.



Inversément, si on souhaite ignorer l'exécution d'un bloc de code ou au contraire à nouveau exécuter un bloc de code, il est possible de déplacer le point d'exécution pour désigner la prochaine instruction exécutée. Il suffit de déplacer la flèche jaune affichée dans la marge, en face de la prochaine instruction à exécuter :



Attention, cette commande doit être utilisée avec précaution car il faut se souvenir des points suivants :

- Les instructions situées entre l'ancien et le nouveau point d'exécution ne seront pas exécutées.
- Déplacer le point d'exécution en arrière n'annule pas les instructions déjà exécutées.
- Le nouveau point d'exécution ne peut être déplacé qu'à l'intérieur d'une fonction ou d'une procédure.

8.2.2. Les points d'arrêt et les TracePoint

Arrêter l'exécution du code se réalise de manière bien plus précise avec l'usage des points d'arrêt. De plus, ils peuvent aussi être conditionnels. Différents types de condition sont pris en charge pour leur activation (condition, nombre de passage, …).

Les TracePoints sont pratiquement identiques aux points d'arrêt mis à part que pour un TracePoint, on peut spécifier l'action exécutée lorsque le point d'arrêt est atteint. Ce peut être le passage en mode arrêt de l'application et/ou l'affichage d'un message. Les messages sont consultables dans la fenêtre *Output*.

!!!Les points d'arrêt ou les TracePoints sont affichés dans l'environnement Visual Studio, par une série d'icônes.

8.2.2.A. Placer un point d'arrêt

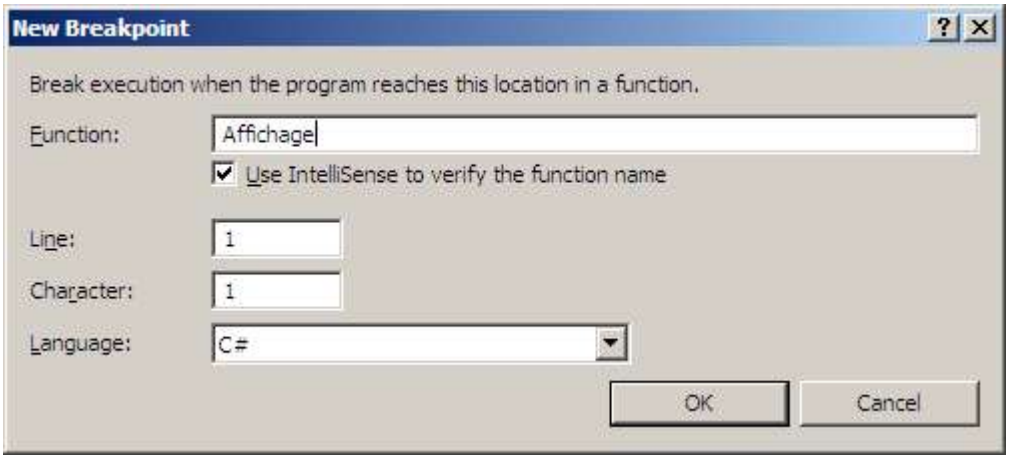
Pour placer un point d'arrêt, plusieurs possibilités existent :

- Effectuer un clic sur la marge de gauche de la fenêtre de code,
- Positionner le curseur sur la ligne correspondante et utiliser le raccourci clavier : [ctrl][B],
- Utiliser l'option *Insérer un point d'arrêt* dans le menu contextuel de la fenêtre de code.

Toutes ces techniques insèrent un point d'arrêt et matérialisent son emplacement par un point rouge dans la marge de gauche et il y a un surlignement en rouge de la ligne de code référencée :

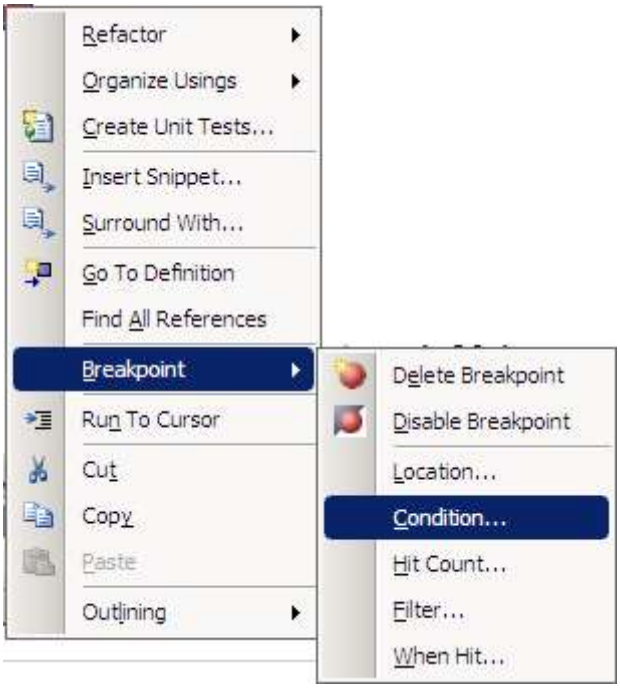


L'option *Interrompre à la fonction* du menu *Déboguer – Nouveau point d'arrêt* permet de placer un point d'arrêt sur une procédure ou sur une fonction en saisissant simplement son nom :

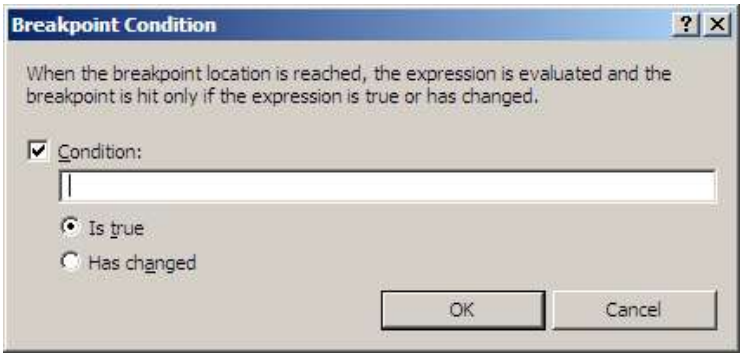


Attention: la boîte de dialogue propose de préciser sur quelle ligne de la fonction on souhaite placer un point d'arrêt mais cette fonctionnalité n'est pas disponible pour les points d'arrêt sur des fonctions;

Les points d'arrêt ainsi placés sont inconditionnels. Dés que l'exécution arrive sur cette ligne, l'application passe en mode arrêt. On peut perfectionner le fonctionnement des points d'arrêt en y ajoutant des conditions, un nombre de passage ou en le transformant en *TracePoint*. Pour ajouter une condition, il suffit d'utiliser le menu contextuel au départ du point d'arrêt qui a été défini.



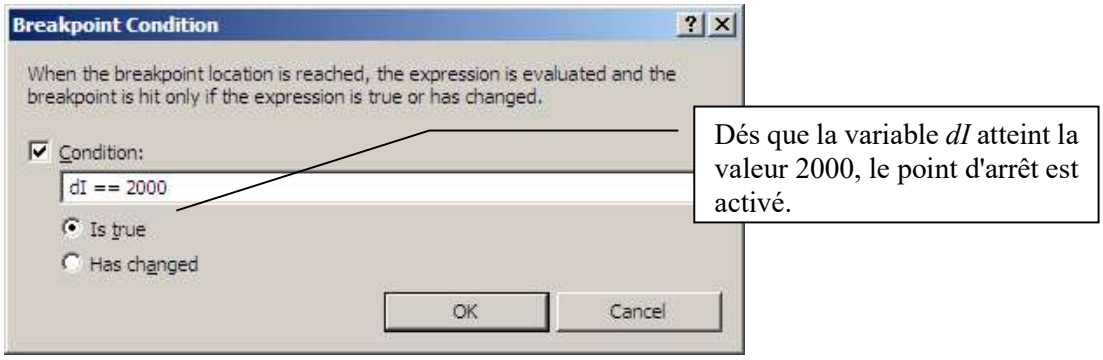
Le passage en mode arrêt peut être soumis à condition. La boîte de dialogue suivante permet de préciser les conditions d'exécution du point d'arrêt :



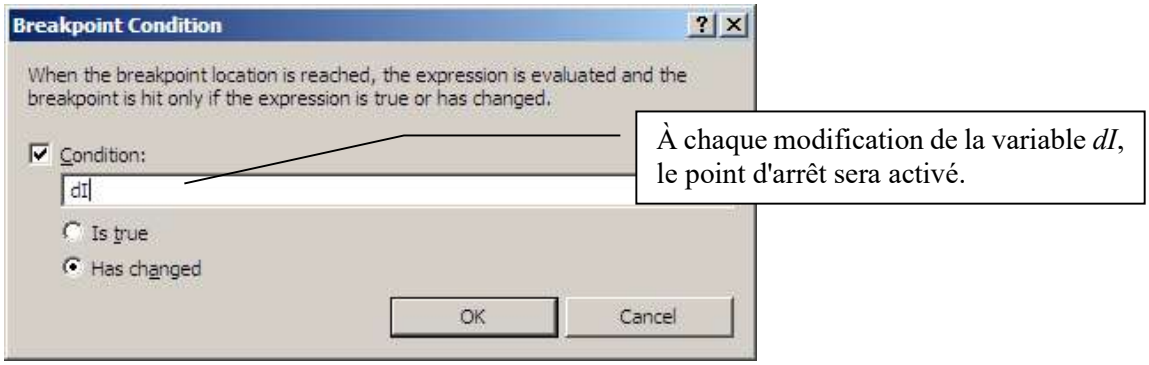
On doit saisir dans cette boîte de dialogue, une expression qui sera évaluée à chaque passage sur le point d'arrêt. L'exécution s'arrêtera alors :

- Si le résultat de l'évaluation de la condition est vrai. Par exemple, dans une boucle incrémentant la variable *dI* de 0 à 10000, on pourrait imaginer la condition suivante :

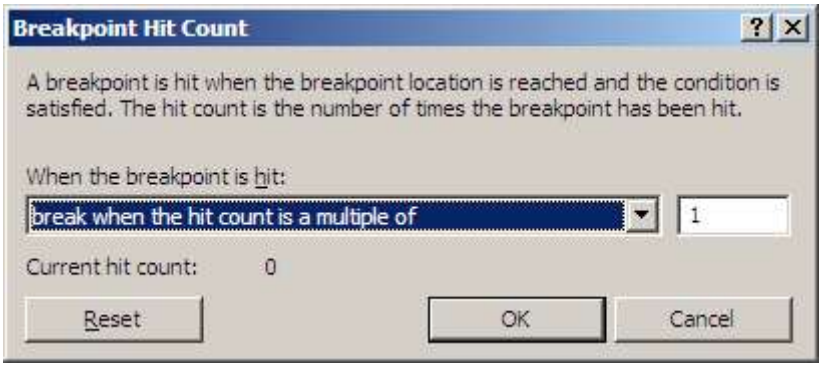
```
for (int dI = 0; dI < 10000; )
    dI++;
```



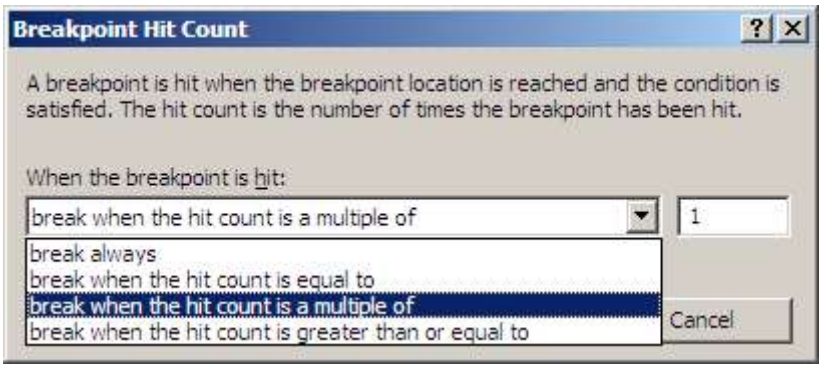
- Si le résultat de l'évaluation de la condition a été modifié depuis le dernier passage sur ce point d'arrêt. À noter que dans ce cas, deux passages sont au moins nécessaire pour provoquer l'arrêt de l'application (le premier servant simplement à mémoriser le résultat de l'expression). En réutilisant l'exemple de la boucle précédent, on pourrait imaginer la condition suivante :



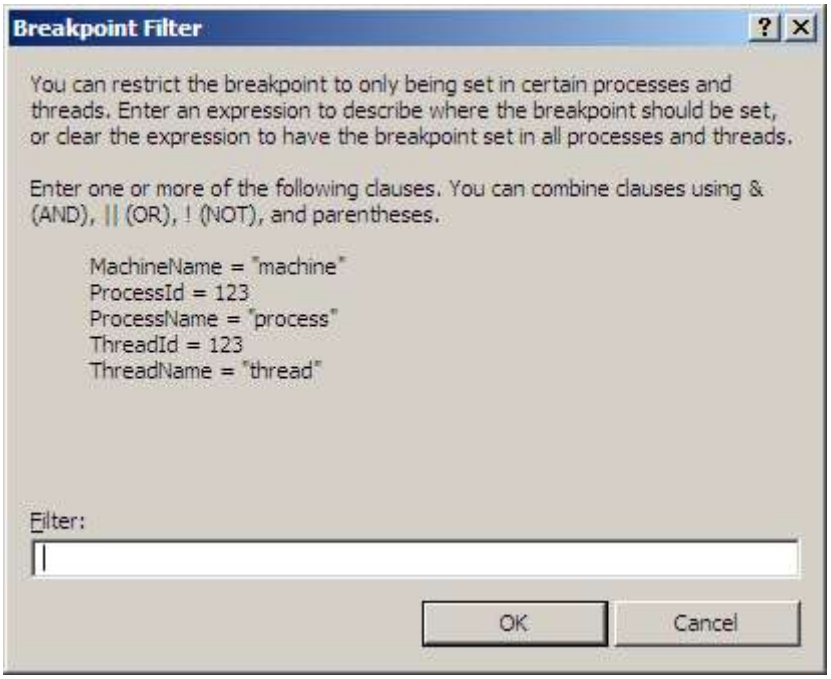
Les points d'arrêt sont également capables de compter le nombre de fois où ils sont atteints et de s'exécuter pour un nombre particulier de passages.



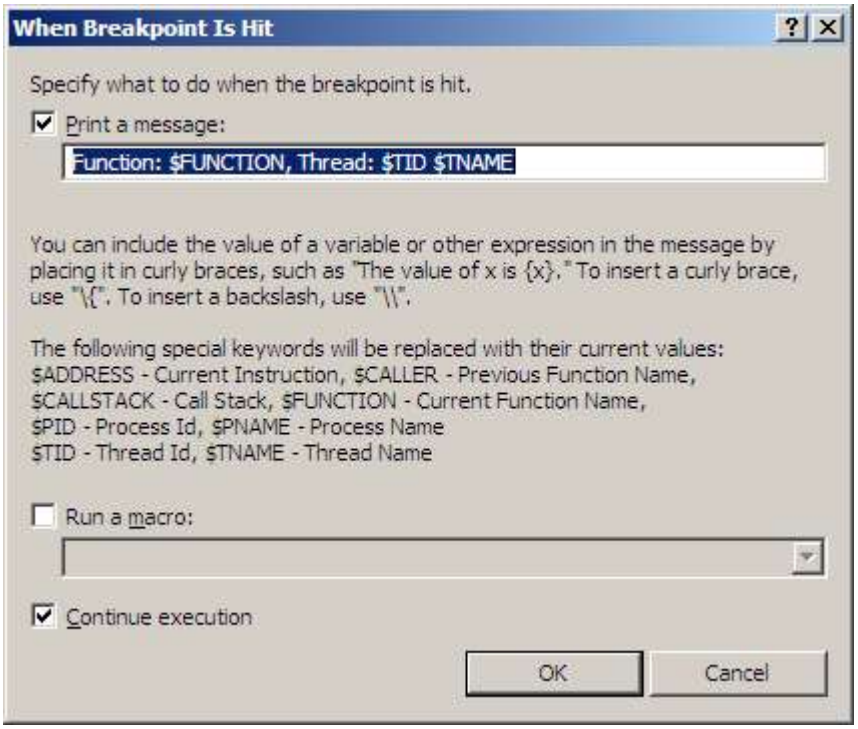
Cette boîte de dialogue permet de définir le nombre de passages sur le point d'arrêt pour que celui-ci arrête effectivement l'application. Quatre options sont disponibles pour la condition d'arrêt sur le nombre de passages:



Les filtres permettent d'ajouter des critères supplémentaires pour l'exécution d'un point d'arrêt. Ces critères portent sur le nom de la machine où s'exécute l'application, ainsi que le process ou le thread.



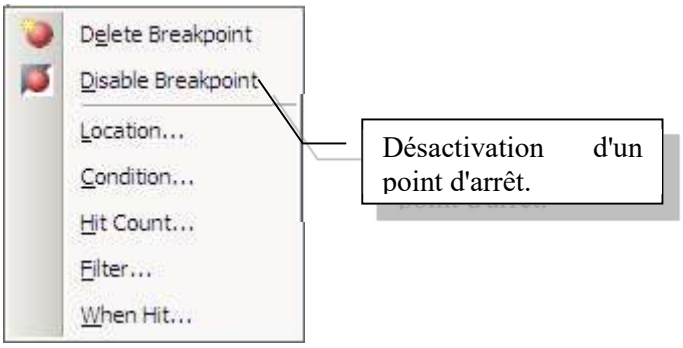
Un point d'arrêt peut être transformé en *TracePoint* en précisant une action particulière à exécuter lorsqu'il sera atteint.



Cette boîte de dialogue attend le libellé du message à afficher lorsque le point d'arrêt est atteint. Le *TracePoint* peut aussi accepter l'exécution d'une macro. Pour que le point d'arrêt soit transformé en *TracePoint*, l'option *Continuer l'exécution* doit être activée.

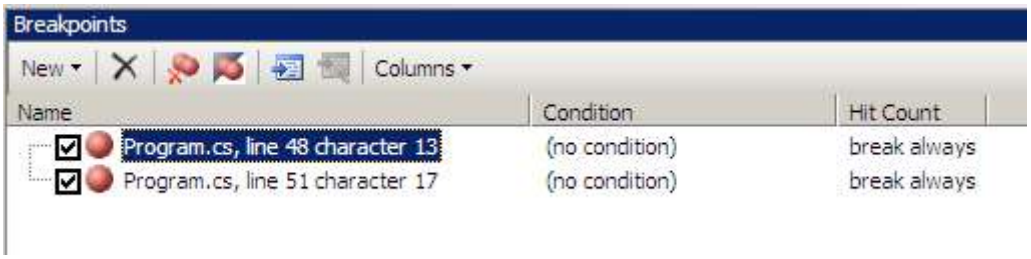
8.2.2.B. Activer, désactiver, supprimer un point d'arrêt

Les points d'arrêt peuvent également être momentanément désactivés en utilisant le menu contextuel :



Le point d'arrêt peut ensuite de nouveau être réactivé en utilisant le même menu contextuel. Ce même menu contextuel permet aussi la suppression d'un point d'arrêt mais il est plus rapide d'effectuer un double clic sur le point d'arrêt lui-même. Le menu *deboguer* permet aussi de supprimer tous les points d'arrêt.

Pour faciliter le travail lors du deboguage d'une application, une fenêtre propose une liste de tous les points d'arrêt définis dans le projet. Cette fenêtre est accessible via le menu *Deboguer/Fenêtres/Point d'arrêt*. Cette fenêtre propose un menu contextuel permettant de faire les actions principales sur un point d'arrêt.



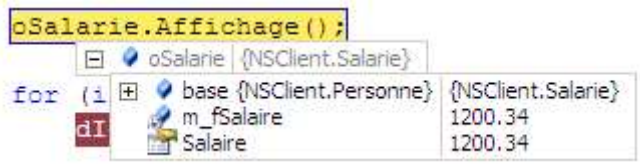
Les points d'arrêt sont conservés lorsque l'on ferme le projet.

8.2.3. Examen du contenu des variables

L'intérêt du débogueur est de pouvoir suivre le fonctionnement de l'application au cours de son exécution. Il est également primordial de pouvoir visualiser, lorsque l'application est en mode arrêt, les valeurs contenues dans les différentes variables de l'application.

8.2.3.A. Data Tips

Les *DataTips* fournissent un moyen rapide pour visualiser le contenu d'une variable. Il suffit de déplacer le curseur de la souris au dessus du nom d'une variable et après, un court instant, une fenêtre présentant le contenu de la variable s'affiche. Si la variable est un type complexe, un objet par exemple, le *DataTips* propose un petit signe + permettant d'ouvrir et ainsi d'accéder à la structure de la variable. Les informations visualisées sont également modifiables directement dans le *DataTips*. Le *DataTips* disparaît automatiquement dés que le curseur de la souris quitte le nom de la variable.



Pour visualiser le code masqué par le *DataTips* sans le faire disparaître, on peut utiliser la touche [ctrl] qui va le rendre transparent.

8.2.3.B. Fenêtre automatique

La fenêtre automatique affiche les variables utilisées dans l'instruction courante et dans l'instruction précédente. Cette fenêtre est accessible par le menu *Deboguer/Fenêtres/Automatique*.

Cette fenêtre permet également la modification du contenu d'une variable en double cliquant sur la valeur, dans la fenêtre, et en validant la modification, après saisie par la touche [enter]. L'application continuera son exécution avec cette nouvelle valeur pour la variable.

8.2.3.C. Fenêtre variables locales

La fenêtre *Variables locales* est accessible par le même *Deboguer/Fenêtres/Variables locales* et possède un fonctionnement identique à la fenêtre *Automatique* mis à part qu'elle affiche toutes les variables dans la portée courante.

8.2.3.D. Les fenêtres Espion

La fenêtre *Espion* permet l'affichage des variables semblant intéressantes pour le debogage de l'application. Cette fenêtre ou plutôt ces fenêtres, puisque quatre fenêtres *Espion* sont disponibles, sont affichées par le menu *Deboguer/Fenêtres/Espion* puis *Espion 1* à *4*.

On doit ensuite configurer la fenêtre en ajoutant les variables et expressions que l'on souhaite visualiser. En effectuant un double clic dans la colonne *Nom*, on peut saisir ce que l'on souhaite afficher dans la fenêtre. On peut aussi effectuer un glisser-déplacer depuis la fenêtre de code. Si on saisit un nom de variable complexe, l'ensemble des données membres est affiché dans la fenêtre sous forme d'arborescence.

Le contenu des variables ne sera affiché que si l'application est en mode arrêt sur une ligne de code où cette variable peut être utilisée. Par exemple, le contenu des variables locales à une procédure ou fonction n'est affiché que si le code est arrêté dans cette procédure ou fonction.

Dans le cas contraire, la fenêtre *Espion* indique simplement que cette variable n'est pas déclarée dans la portion de code où l'on se trouve, en l'affichant en caractères grisés.

8.2.3.E. La fenêtre Espion Express

La fenêtre *Espion express* propose le même principe de fonctionnement et est accessible par le menu *Deboguer/Espion Express*. Dans ce cas, la variable ou l'expression sur laquelle se trouve le curseur est affiché dans la fenêtre *EspionExpress*. Cette fenêtre étant modale, on devra obligatoirement la fermer avant de poursuivre le debogage de l'application.

Le bouton *Ajouter un espion* permet d'ajouter rapidement l'expression dans la fenêtre *Espion* pour pouvoir l'étudier dans la suite du débogage.

8.2.4. Les autres fenêtres de débogage

De nombreuses autres fenêtres sont disponibles pour le débogage mais certaines ne sont pas vraiment utiles pour le développement d'applications avec Visual C#. Elles sont plutôt réservées pour le test d'applications développées avec d'autres langages comme C++ par exemple.

8.2.5. Autres techniques de débogage : la compilation conditionnelle

On peut utiliser la compilation conditionnelle pour spécifier des portions de code qui seront ou non compilées, en fonction de la valeur d'une constante que l'on aura au préalable définie. Par exemple, on peut tester plusieurs solutions pour résoudre un problème et on cherche la plus efficace.

Le bloc de code dont la compilation est soumise à condition doit être encadré par les instructions *#if condition* et *#endif*. En fonction de la valeur de la condition, le bloc de code sera ou non compilé. IL faut bien sûr que la ou les macros utilisées dans la condition soient initialisées avant son apparition dans une commande *if*.

```
#define VersionTest

#if VersionTest
    …
#endif
```

Les macros peuvent être déclarées avec l'instruction *#define* ou encore dans les propriétés du projet. Si elles sont déclarées dans le code, elles doivent obligatoirement l'être sur les premières lignes du fichier.

Attention : ces macros ne peuvent être utilisées que pour la compilation conditionnelle et ne sont pas accessibles dans le code.

9. Les Applications Windows

9.1. Les différents types d'application

Les applications Windows sont basées sur une ou plusieurs fenêtres constituant l'interface entre l'utilisateur et l'application. Pour développer ce type d'application, on a à sa disposition dans le framework .NET un ensemble de classes permettant la conception de l'interface de l'application. Ces éléments sont fréquemment regroupés sous le terme Technologie Windows Forms. Une application basée sur les Windows Forms utilisera un ou plusieurs formulaires pour construire l'interface utilisateur de l'application.

Sur ces formulaires (ou feuilles), on placera des contrôles afin de définir exactement l'aspect de l'interface de l'application. Les formulaires sont créés à partir de classes du framework .NET que l'on spécialisera par l'ajout de fonctionnalités. Le formulaire ainsi créé étant lui-même une classe, il sera possible de le réutiliser, dans une autre application, en lui ajoutant des fonctionnalités supplémentaires par une relation d'héritage.

Les Windows Forms peuvent être créés directement par le code mais l'environnement de développement Visual Studio propose toute une panoplie d'outils graphiques pour faciliter la tâche du programmeur. On va utiliser cette dernière technique.

9.1.1. Modes de présentation des fenêtres

Dans une application Windows, trois styles de présentation sont disponibles pour les fenêtres de l'application.

9.1.1.A. Interface mono document (SDI)

Une seule fenêtre est disponible dans l'application. Pour pouvoir ouvrir un nouveau document, le document actif de l'application doit obligatoirement être fermé. Le bloc-notes de Windows est une application SDI.

9.1.1.B. Interface multi documents (MDI)

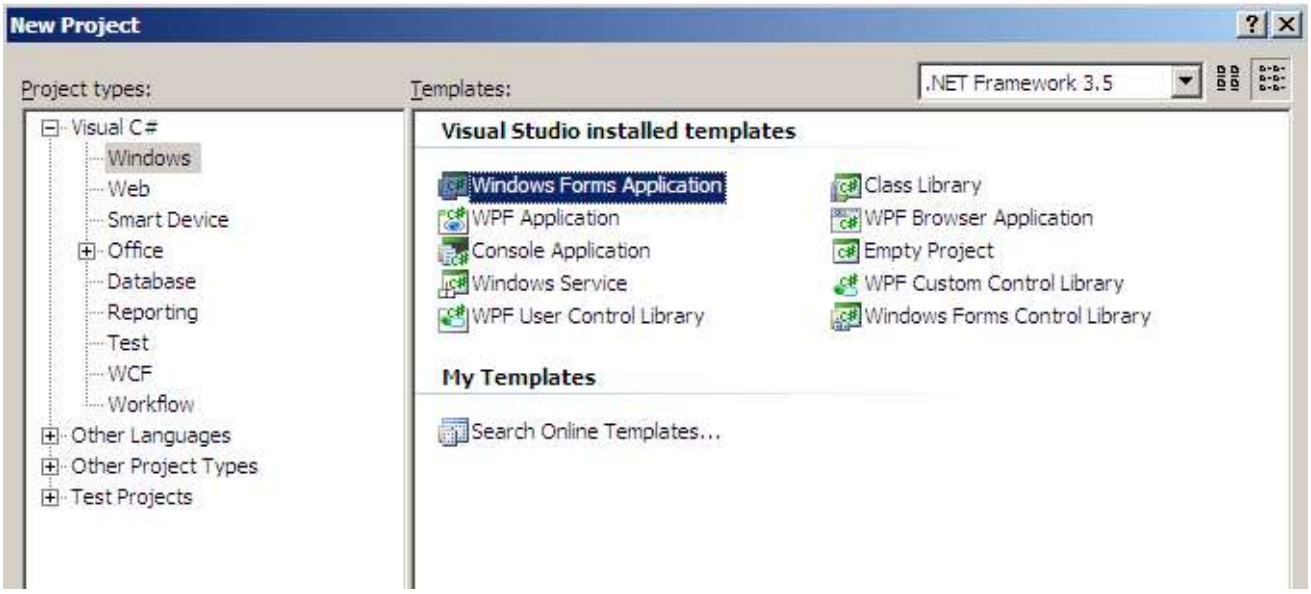
L'application est formée d'une fenêtre principale (la fenêtre mère) dans laquelle vont apparaître plusieurs fenêtres (fenêtres filles) contenant les documents sur lesquels on souhaite travailler. En règle générale, ce type d'application dispose d'un menu permettant la réorganisation des différentes fenêtres filles. Ce type de présentation est utilisé par la plupart des applications bureautiques.

9.1.1.C. Interface de type explorateur

Ce style d'interface se développe de plus en plus au détriment des deux autres. Dans ce cas, la fenêtre est divisée en deux zones. La zone de gauche présente une vue, sous forme d'arbre, des éléments pouvant être manipulés par l'application. La zone de droite présente l'élément sélectionné dans l'arbre et permet sa modification. De nombreux outils d'administration utilisent cette présentation.

9.2. Les fenêtres

Quand on commence une nouvelle application Windows Forms, l'environnement de développement ajoute automatiquement au projet un formulaire. Ce formulaire sert de point de départ pour l'application. On peut immédiatement lancer l'application et tout fonctionne. Évidemment, l'application ne permet pas de réaliser grand-chose mais elle a toutes les fonctionnalités d'une application Windows et cela sans écrire la moindre ligne de code. Le code existe mais il a été généré automatiquement par Visual Studio. Comme ce code ne doit jamais être modifié manuellement, les fichiers le contenant sont masqués dans l'explorateur de solutions.



Pour les afficher, on peut utiliser le bouton :



On va pouvoir constater que de nombreux fichiers existent déjà dans le projet. Les fichiers réservés de Visual Studio ont tous l'extension *.designer.cs*. On peut bien entendu les visualiser.

Le fichier associé à la première fenêtre de l'application est nommé *form1.designer.cs*. Il va contenir la description de toutes les actions, traduites en code Visual C#, que l'on va réaliser pour personnaliser les caractéristiques de la fenêtre.

Voici le contenu du fichier :

```
namespace WindowsFormsApplication1
{
    partial class Form1
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

         /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be
        disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                  components.Dispose();
            }
            base.Dispose(disposing);
```

```
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.components = new System.ComponentModel.Container();
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.Text = "Form1";
        }

        #endregion
    }
}
```

Il contient la définition de la classe correspondent à la fenêtre héritant de la classe *System.Windows.Forms.Form*. Cette définition comporte une petite particularité par rapport à la définition d'une classe tel que vu jusqu'à présent.

Le mot réservé *partial* est spécifié devant le nom de la classe. Il signifie au compilateur que le fichier ne contient qu'une partie de la définition de la classe, l'autre partie étant disponible dans le fichier *form1.cs*. Cette technique permet de répartir les rôles :

- Visual Studio se charge de générer, dans le fichier *form1.designer.cs*, le code correspondant à la personnalisation de la fenêtre.
- On est responsable du code, contenu dans le fichier *form1.cs*, chargé de la personnalisation du fonctionnement de la fenêtre.

Dans le fichier *form1.designer.cs*, on trouve la définition partielle de la classe :

```
partial class Form1
```

Dans le fichier *form1.cs*, on trouve l'autre partie :

```
public partial class Form1 : Form
{
 public Form1()
 {
     InitializeComponent();
 }
}
```

Cette solution limite les risques de modification involontaire de la partie de code réservée à Visual Studio. L'élément le plus important est formé par la méthode *InitializeComponent*. Cette méthode est appelée automatiquement à la création d'une instance de la fenêtre, lors de l'appel au constructeur.

Si on ajoute un constructeur surchargé dans la classe *Form1*, on est responsable de cet appel. Le plus simple dans ce cas est de faire appel au constructeur par défaut.

```
Public Form1 (int dI):this()
{
}
```

On doit également bien respecter, dans le constructeur par défaut, l'emplacement des initialisations particulières. Si elles sont placées avant l'appel à la méthode *InitializeComponent*, celle-ci risque de les modifier. Avant l'appel de la méthode *InitializeComponent*, les éléments graphiques de la fenêtre ne sont pas disponibles car c'est le rôle principal de cette méthode de les créer et d'initialiser certaines de leurs propriétés.

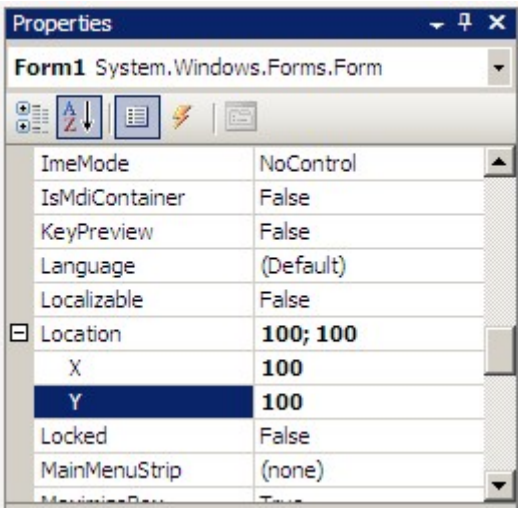
Une méthode *dispose* est également créée pour pouvoir supprimer tous les objets instanciés par la classe. Cette méthode commence par supprimer les objets créés puis elle appelle la méthode *dispose* de la classe de base.

Maintenant que l'on a fait le tour du code généré automatiquement, on va maintenant regarder comment modifier l'apparence et le comportement de la fenêtre par l'intermédiaire de ses propriétés.

9.2.1. Dimension et position des fenêtres

La position de la feuille sur l'écran (ou sur son conteneur) est modifié par la propriété *Location*. Cette propriété est une structure composée de deux membres, indiquant les coordonnées du coin supérieur gauche de la feuille par rapport au coin supérieur gauche de l'écran. Les membres de cette structure peuvent être modifiés dans la fenêtre de propriété de Visual C#.

Quand on modifie des propriétés, du code est ajouté pour prendre en compte ces modifications. Par exemple, si on veut que la fenêtre apparaisse aux coordonnées 100,100, on modifie la propriété *Location* :



Si on regarde dans le code, on retrouve l'implémentation de la modification introduite :

```
private void InitializeComponent()
 {
     this.SuspendLayout();
     //
     // Form1
     //
     this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
     this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
     this.ClientSize = new System.Drawing.Size(292, 273);
     this.Location = new System.Drawing.Point(100, 100);
     this.Name = "Form1";
     this.Text = "Form1";
     this.ResumeLayout(false);

 }
```

Les dimensions de la fenêtre sont modifiables par la propriété *size* qui contient deux membres *Width* et *Height* indiquant la largeur et la hauteur de la fenêtre.

!!!(p237)Résumons cela par un petit schéma :

Les unités sont des pixels, pour toutes les propriétés, concernant les dimensions et positions d'objet. Les propriétés *Left, Top, Height* et *Width* sont disponibles dans le code mais pas dans la fenêtre des propriétés.

Ces propriétés sont mises à jour pendant l'exécution de l'application, si la fenêtre est déplacée ou redimensionnée. Elles sont accessibles par le code de l'application.
La largeur et la hauteur de la fenêtre peuvent évoluer entre les limites fixées par les propriétés *MinimumSize* et *MaximumSize.* Par défaut ces deux propriétés sont initialisées à 0 indiquant qu'il n'y a pas de limite fixée pour la taille de la fenêtre.

Deux autres propriétés indiquent le comportement de la fenêtre au démarrage de l'application. La propriété *StartPosition* permet d'imposer une position à la fenêtre, au démarrage de l'application. Les valeurs possibles sont résumées dans le tableau suivant :

Valeur de la propriété	Effet sur la fenêtre
Manual	Les propriétés *Location* et *Size* sont utilisées pour l'affichage de la fenêtre.
CenterParent	La fenêtre est centrée dans la fenêtre mère.
CenterScreen	La fenêtre est centrée sur l'écran.
WindowsDefaultLocation	Le système positionne automatiquement les fenêtres en partant du coin supérieur gauche de l'écran. Les fenêtres sont décalées vers le bas de l'écran à chaque affichage d'une nouvelle fenêtre. La taille de chaque fenêtre est spécifiée par la propriété *Size*.
WindowsDefaultBounds	Même principe que ci-dessus mais la taille est déterminée par le système d'affichage de la fenêtre.

Table 17: Valeurs possibles de la propriété StartPosition

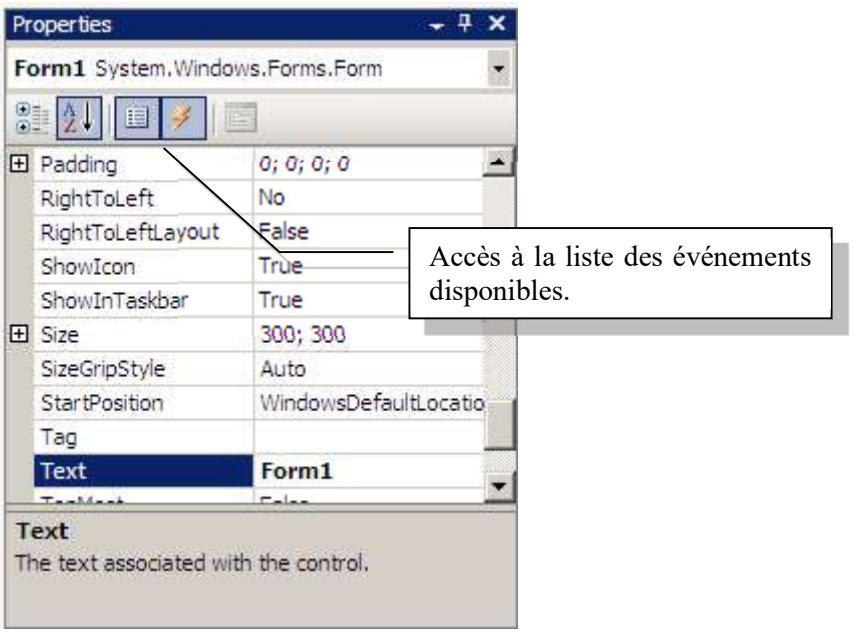
La propriété WindowsState indique l'état de la feuille. Les trois valeurs possibles sont :

Valeur de la propriété	Effet sur la fenêtre
Normal	Taille définie par la propriété *Size*.
Minimised	Fenêtre en icône sur la barre des tâches.
Maximised	Fenêtre en plein écran.

Table 18: Valeurs possibles de la propriété WindowsState

Ces propriétés peuvent être modifiées par le code de l'application. Par contre, il est plus efficace d'utiliser les méthodes *SetLocation* et *SetSize* qui permettent le dimensionnement et le positionnement de la feuille. L'utilisation de ces méthodes ou la manipulation directe des propriétés déclenche les évènements *Resize* et *Move* sur la feuille correspondante.

Pour ajouter le code de gestion de ces événements, on doit créer une méthode respectant la signature des délégués et associer chaque méthode à l'événement. Ce travail est grandement facilité par Visual Studio par l'intermédiaire de la fenêtre des propriétés.



Il suffit juste d'effectuer un double-clic sur le nom de l'événement qui est intéressant pour que Visual Studio génère un squelette de méthode et associe automatiquement cette méthode à l'événement. Cette association est réalisée dans la méthode *InitializeComponenet* :

```
private void InitializeComponent()
 {
     this.SuspendLayout();
     //
     // Form1
     //
     this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
     this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
     this.ClientSize = new System.Drawing.Size(292, 273);
     this.Location = new System.Drawing.Point(100, 100);
     this.Name = "Form1";
     this.Text = "Form1";
     this.MinimumSizeChanged += new
                 System.EventHandler(this.Form1_MinimumSizeChanged);
     this.ResumeLayout(false);

 }
```

La méthode *Form1_MinimumSizeChanged* est décrite dans la classe *Form1* :

```
public partial class Form1 : Form
{
 public Form1()
 {
     InitializeComponent();
 }

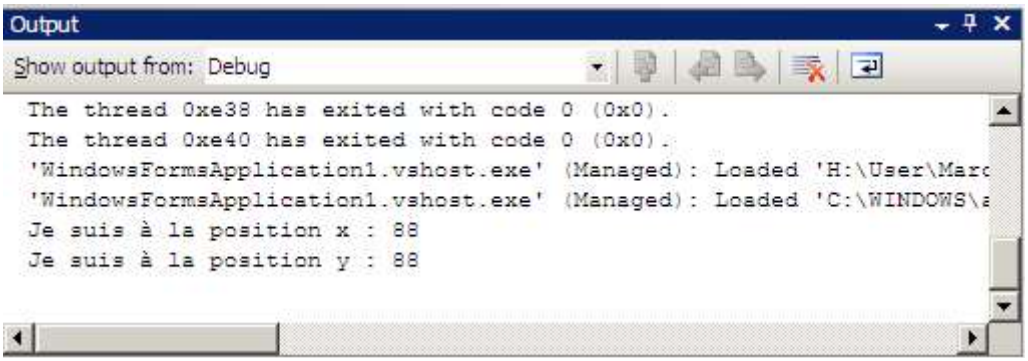
 private void Form1_MinimumSizeChanged(object sender, EventArgs e)
 {

 }
}
```

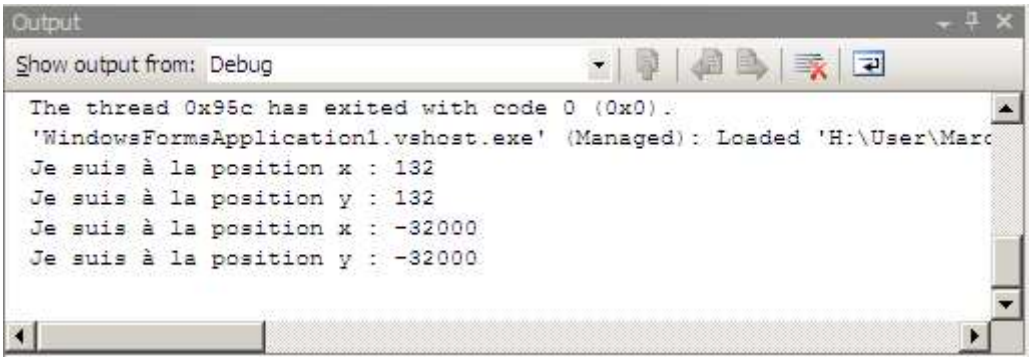
Il reste maintenant à indiquer les actions à réaliser à chaque fois que l'événement sera généré.

```
private void Form1_Move(object sender, EventArgs e)
 {
     Console.WriteLine("Je suis à la position x : {0}", this.Location.X);
     Console.WriteLine("Je suis à la position y : {0}", this.Location.Y);
 }
```

L'affichage des passages va se retrouver dans la fenêtre *Output* :



Si on réduit la fenêtre en icône au moyen du bouton de minimalisation, on obtiendra les valeurs suivantes : -32000 et -32000 :



Les positions de X et Y sont négatives ! On peut en fait utiliser les valeurs comprises dans la limite de celles acceptables pour un entier. Seule la partie de la fenêtre comprise entre zéro et la largeur et hauteur de l'écran sera visible. On peut utiliser la méthode *GetBounds* de l'objet *Screen* pour obtenir la taille de l'écran.

```
private void Form1_Move(object sender, EventArgs e)
 {
     Console.WriteLine("Je suis à la position x : {0}", this.Location.X);
     Console.WriteLine("Je suis à la position y : {0}", this.Location.Y);
     Console.WriteLine(" sur un écran de {0} par {1}",
           Screen.GetBounds(this).Width,
           Screen.GetBounds(this).Height);
 }
```

Pour que l'utilisateur puisse déplacer ou modifier la taille de la fenêtre, il doit disposer des outils nécessaires :

- Une barre de titre pour pouvoir agripper la fenêtre et la déplacer,
- Une bordure pour pouvoir la dimensionner,
- Des boutons pour pouvoir l'agrandir, la réduire et l'afficher avec sa taille normale.

Pour pouvoir redimensionner la fenêtre, celle-ci doit disposer d'une bordure de type *sizeable* affectée à sa propriété *FormBorderStyle*.

Pour être déplacée, une fenêtre doit avoir une barre de titre. Cette barre de titre peut être masquée par la propriété *ControlBox* positionnée sur *false*. Dans ce cas, même le titre de la fenêtre n'est plus affiché. Dans le cas où la barre de titre est affichée, l'état des différents boutons apparaissant dessus peut être contrôlé par les propriétés suivantes :

- *MinimizeBox*: Affichage ou non de mise en icône de la fenêtre,
- *MaximizeBox*: Affichage ou non du bouton d'agrandissement de la fenêtre,
- *HelpButton*: Affichage ou non du bouton d'aide. Visible seulement si les deux boutons précédents ne sont pas visibles.

9.2.2. Couleurs et Polices utilisées sur les fenêtres

La propriété *BackColor* indique la couleur de fond utilisée sur la fenêtre. Cette couleur sera également utilisée pour tous les contrôles qui seront, par la suite, placés sur la feuille. La propriété *ForColor* indique la couleur des éléments qui seront tracés directement sur la feuille ou la couleur de la légende des contrôles placés sur cette feuille. Il existe quatre possibilités pour affecter une valeur à ces propriétés de couleur :

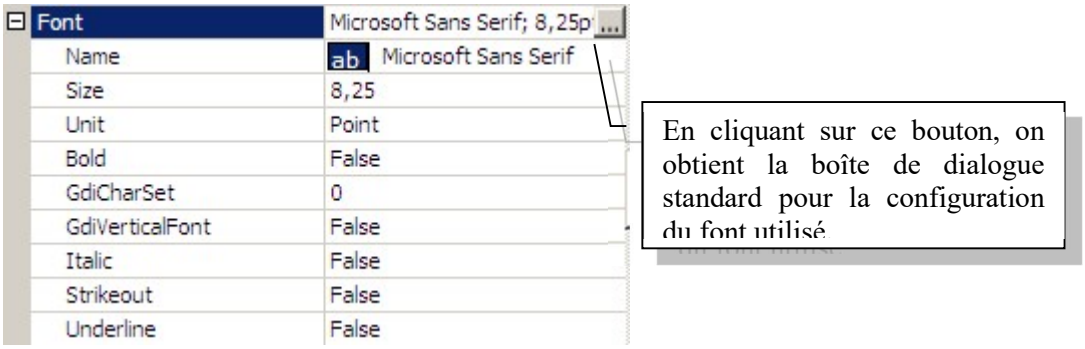
- Par la fenêtre de propriétés, en choisissant une couleur dans l'onglet *Personnaliser*.
- Par la fenêtre de propriétés, en choisissant une couleur web. Ces couleurs correspondent aux couleurs disponibles en langage HTML.
- Par la fenêtre de propriétés, en choisissant une couleur système. Dans ce cas, l'application s'adaptera automatiquement à l'environnement du poste de travail sur lequel elle est installée. Si l'utilisateur a configuré son poste pour avoir des boutons de couleur rose fluo, il retrouvera la même apparence dans l'application.
- En fabriquant une couleur avec un peu de rouge, un peu de vert, un peu de bleu. Pour mélanger tout cela et obtenir la couleur finale, il faut utiliser la méthode *FromARGB* qui prend comme paramètre la quantité de rouge, la quantité de vert, la quantité de bleu et fournit la couleur résultante. Les quantités de chaque couleur sont des valeurs comprises entre 0 et 255. La valeur 255 correspondant à la couleur pure.

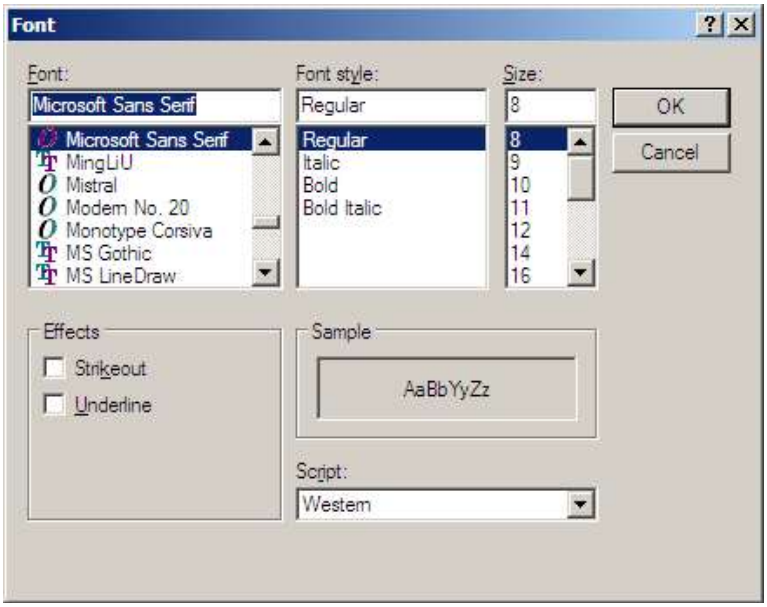
La propriété *Opacity* permet de régler la transparence de la feuille. La valeur doit être comprise entre zéro (fenêtre transparente) et un (fenêtre opaque). Attention, cette propriété ne sera prise en compte par certains Windows.

On peut également faire de la tapisserie en indiquant une image de fond pour la fenêtre en utilisant la propriété *BackgroundImage*. Si l'image n'est pas assez grande pour couvrir la fenêtre, elle sera reproduite en mosaïque.

De la même manière, on peut spécifier qu'une couleur sera considérée transparente sur la fenêtre. Pour cela, on doit affecter à la propriété *TransparencyKey* la valeur de cette couleur. Par exemple, on pourrait indiquer que la couleur blanche soit transparente.

La propriété *Font* permet de spécifier les caractéristiques de la police de caractères, utilisée pour l'affichage du texte, directement sur la fenêtre. Cette police sera également utilisée pour tous les contrôles que l'on placera sur la fenêtre.





9.2.3. Les fenêtres MDI

Les applications MDI sont constituées de deux types de feuilles :

- Les feuilles mères,
- Les feuilles filles.

Dans Visual C#, la même classe est utilisée pour les deux types de fenêtre. Dans le premier cas, on indiquera simplement le fait que la fenêtre est une fenêtre MDI en positionnant sur *true* sa propriété *IsMidiContainer*. Pour ajouter ensuite une fenêtre fille, il convient bien sûr d'abord de créer la fenêtre puis de l'associer à une fenêtre mère par sa propriété *MdiParent*.

Le code suivant créé trois fenêtres et les transforme en fenêtres filles MDI:

```
public FenetresMDI()
{
     InitializeComponent();
     Form oForm1, oForm2, oForm3;

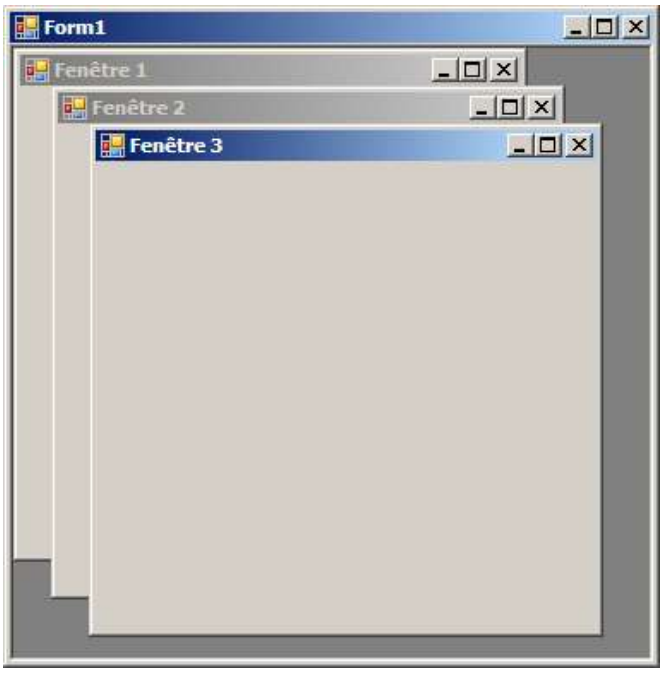
     IsMdiContainer = true;

     oForm1 = new Form();
     oForm1.Text = "Fenêtre 1";
     oForm1.MdiParent = this;
```

```
        oForm1.Show();

        oForm2 = new Form();
        oForm2.Text = "Fenêtre 2";
        oForm2.MdiParent = this;
        oForm2.Show();

        oForm3 = new Form();
        oForm3.Text = "Fenêtre 3";
        oForm3.MdiParent = this;
        oForm3.Show();
    }
```

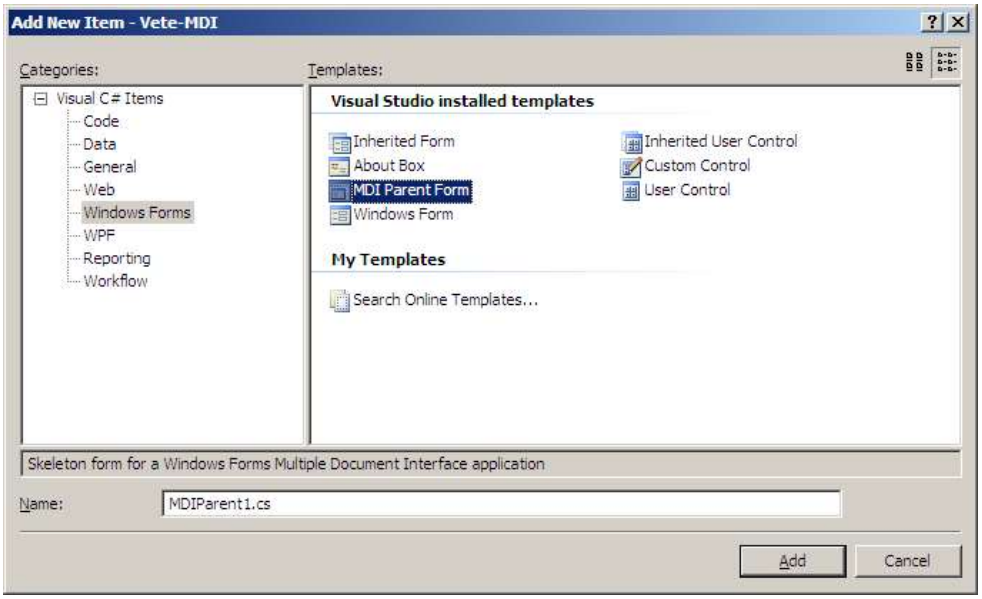


Pour obtenir des fenêtres filles bien rangées dans leur fenêtre mère, on peut faire appel à la méthode *LayoutMdi* en lui passant comme paramètre l'une des constantes prédéfinies de l'énumération *MdiLayout* :

```
this.LayoutMdi(MdiLayout.TileVertical);
```

Pour créer une application MDI, il faut :

- Créer un nouveau projet de type "Windows Form".
- Ensuite, dans le menu *project*, il faut ajouter un nouveau formulaire Windows. Dans cette option, il y aura la possibilité de sélectionner un formulaire de type MDI :



- Définir ce formulaire comme étant le parent. *IsMdiContainer = yes*.
- Modifier le formulaire de démarrage dans la fonction *main* se trouvant dans le fichier *Program.cs* :

```
static void Main()
{
     Application.EnableVisualStyles();
     Application.SetCompatibleTextRenderingDefault(false);
     Application.Run(new FormPrinc());
}
```

Pour permettre de visualiser les formulaires qui sont ouverts à un instant donné, il faut créer un menu avec le nom *Windows*. Dans le menu, il y a la propriété : *MdiWindowsListItem*.



Il est possible d'ajouter des options d'organisation des fenêtres ouvertes. Chaque option doit déclencher une fonction qui va changer la propriété : *LayoutMdi* :

```
private void CascadeToolStripMenuItem_Click(object sender, EventArgs e)
  {
     LayoutMdi(MdiLayout.Cascade);
  }

  private void TileVerticalToolStripMenuItem_Click(object sender, EventArgs
e)
  {
     LayoutMdi(MdiLayout.TileVertical);
  }

  private void TileHorizontalToolStripMenuItem_Click(object sender, EventArgs
e)
  {
     LayoutMdi(MdiLayout.TileHorizontal);
  }

  private void ArrangeIconsToolStripMenuItem_Click(object sender, EventArgs
e)
  {
```

```
        LayoutMdi(MdiLayout.ArrangeIcons);
    }

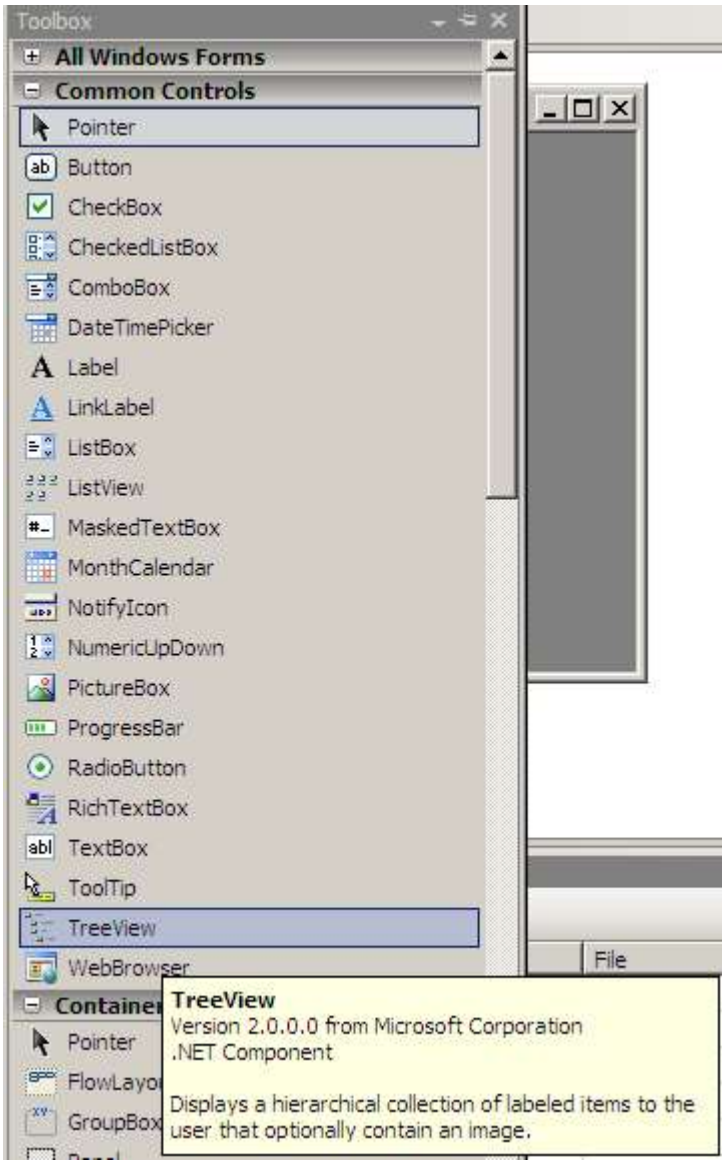
    private void CloseAllToolStripMenuItem_Click(object sender, EventArgs e)
    {
        foreach (Form childForm in MdiChildren)
        {
            childForm.Close();
        }
    }
```

9.2.4. Les fenêtres de type explorateur

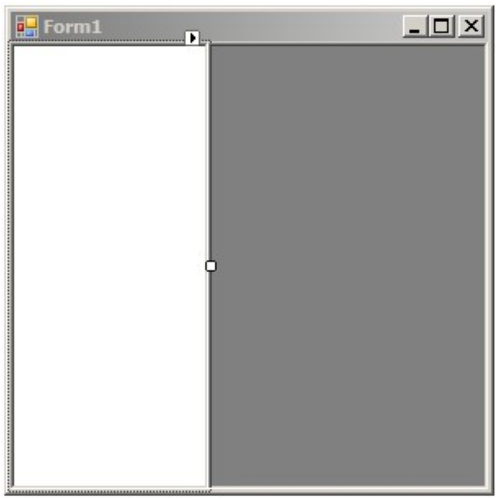
Pour illustrer une autre manipulation possible des fenêtres MDI, on va les utiliser pour réaliser une application de style Explorateur.

Pour la fenêtre principale :

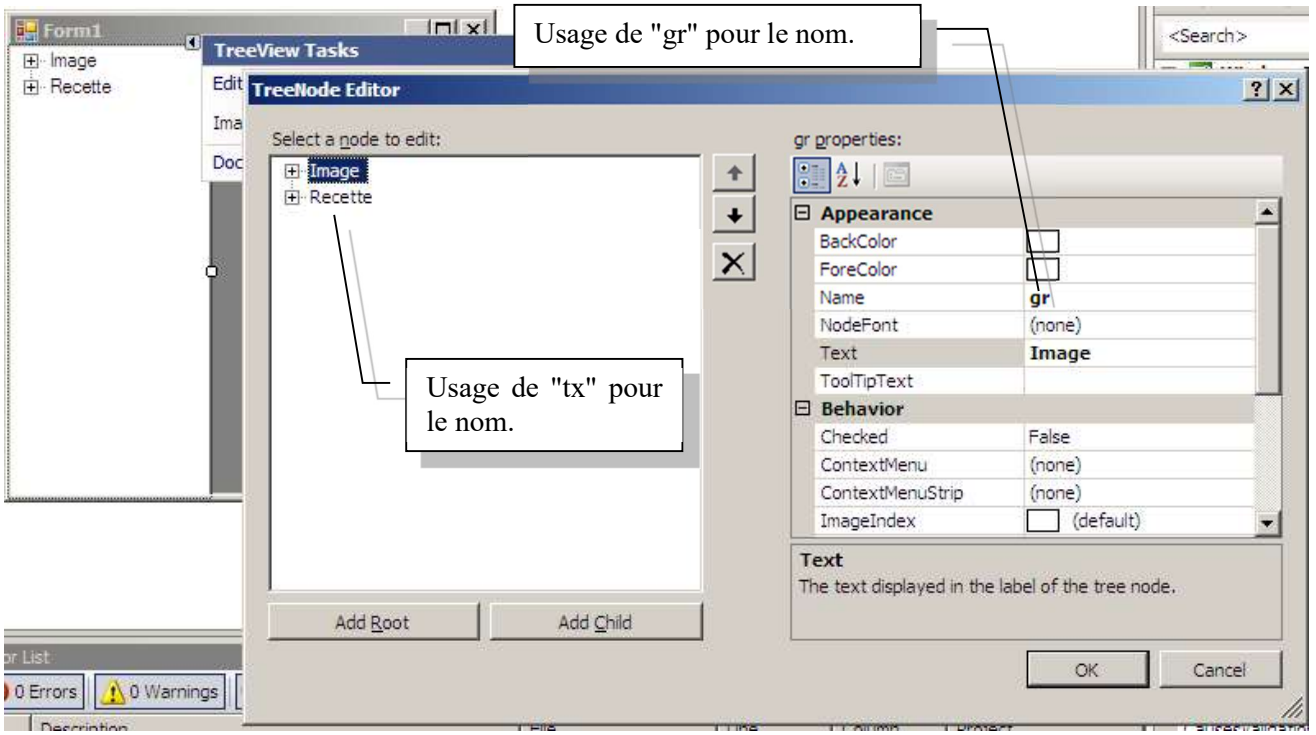
- Modifier la propriété *IsMdiContainer* sur *true* pour activer la fonctionnalité de fenêtre MDI.
- Ajouter un contrôle *TreeView*.



- Modifier la propriété *Dock* du contrôle *TreeView* sur *Left* pour qu'il soit associé à la bordure gauche de la fenêtre.

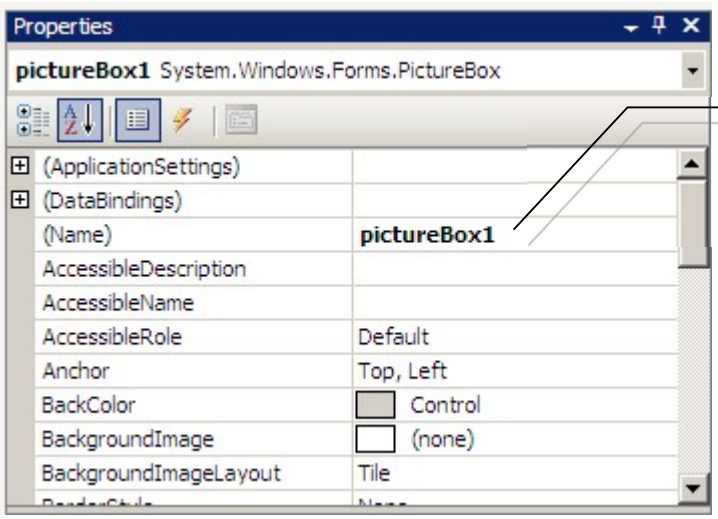


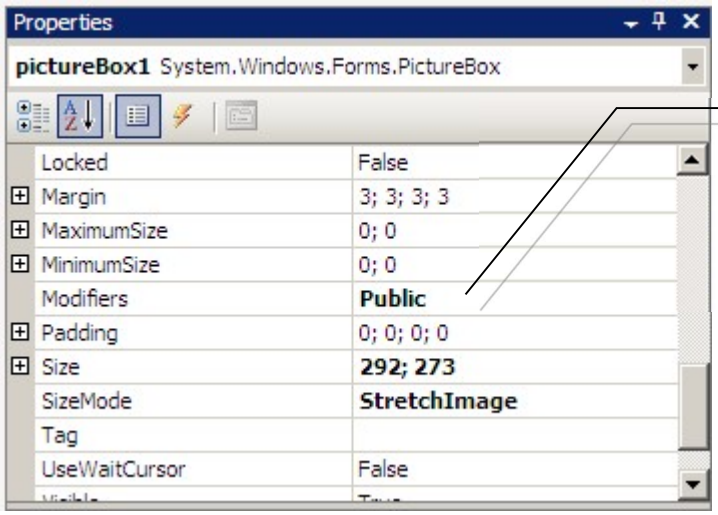
- Ajouter les éléments dans le contrôle *TreeView* en utilisant l'éditeur de nœuds:



Pour la fenêtre graphique associée à l'entrée principale "image" :

- Ajouter une nouvelle feuille à l'application,
- Modifier la propriété *BorderStyle* du contrôle *TreeView* sur *none*,
- Ajouter un contrôle *PictureBox*,

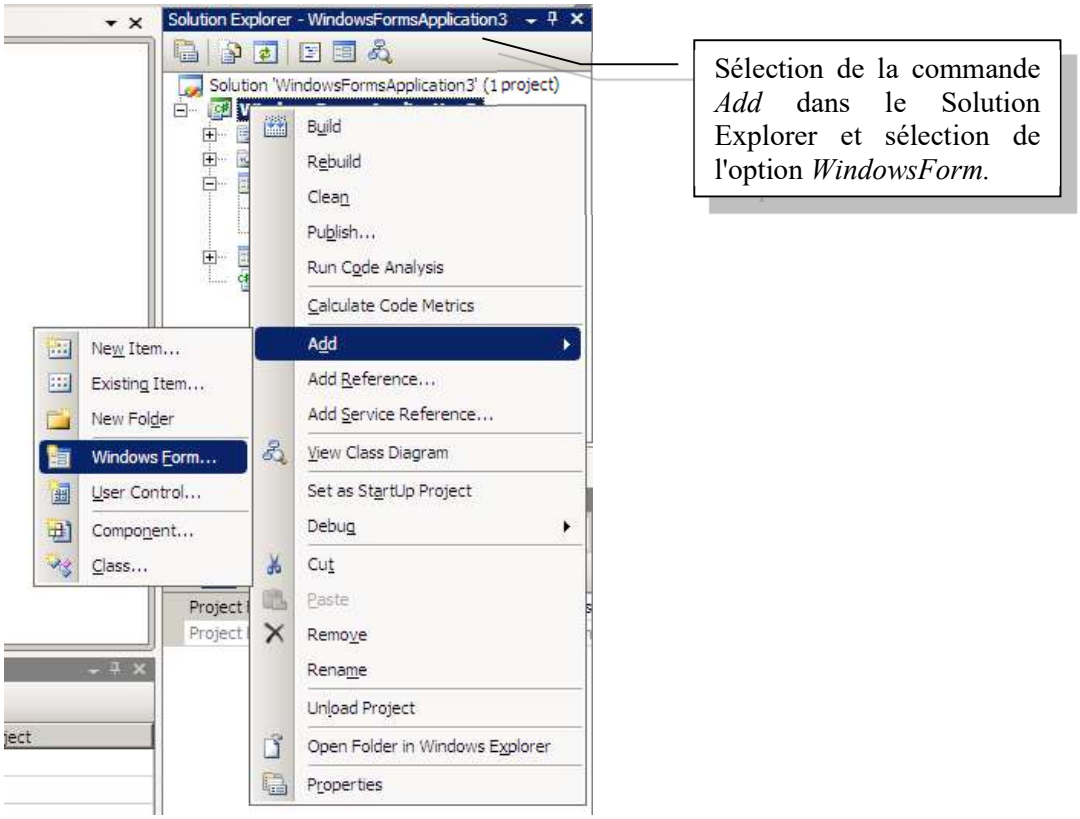




- Modifier la propriété *dock* de ce contrôle sur *Right* pour qu'il occupe toute la surface disponible de droite de la fenêtre,
- Modifier la propriété *SizeMode* de ce contrôle sur *StretchImage* pour que l'image s'adapte à la taille du contrôle (donc de la fenêtre).

Pour la fenêtre texte associée à l'entrée principale "recette":

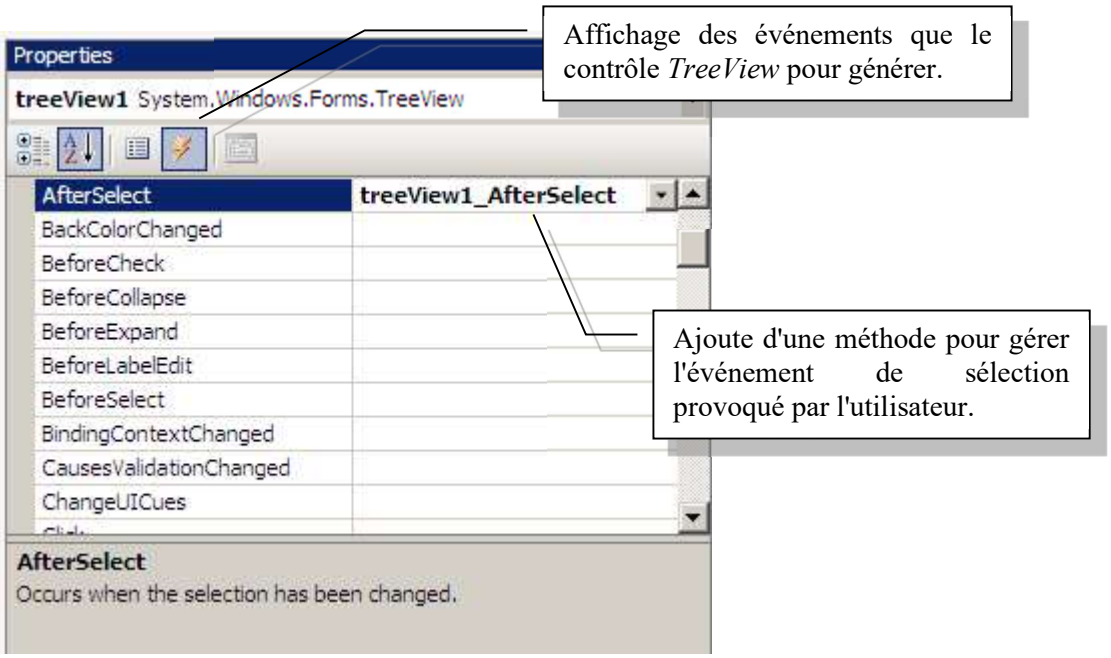
- Ajouter une nouvelle feuille à l'application :



- Ajouter un contrôle *RichTextBox*.
- Modifier la propriété *dock* de ce contrôle sur *Fill* pour qu'il occupe toute la surface disponible de la fenêtre.

Voici maintenant quelques lignes de code pour afficher la bonne fenêtre lors d'une sélection dans le contrôle *TreeView.* Voici ci-dessous ces quelques lignes de code :

1) Il faut ajouter une méthode qui va gérer l'événement provoqué par l'utilisateur lorsque celui-ci va sélectionner une feuille de l'arbre :



2) Ajoute du code qui va gérer cet événement :

```
private void treeView1_AfterSelect(object sender, TreeViewEventArgs e)
{
    if (e.Node.Parent != null)
    {
        foreach (Form oF in this.MdiChildren)
            oF.Close();

        switch (e.Node.Parent.Name)
        {
            case "gr":
                Form2 fGR;

                fGR = new Form2();
                fGR.MdiParent = this;
                fGR.Show();
                fGR.Dock = DockStyle.Fill;
                fGR.pictureBox1.Image = Image.FromFile("..." +
                                        e.Node.Name);;
                break;
            case "tx":
                Form3 fTX;

                fTX = new Form3();
                fTX.MdiParent = this;
                fTX.Show();
                fTX.Dock = DockStyle.Fill;
                fTX.richTextBox1.LoadFile("" + e.Node.Name);
                break;
        }
    }
}
```

Test pour savoir si c'est un nœud enfant qui a été sélectionné. Si oui, il faut fermer les fenêtres enfants. Il ne pourrait en fait ici n'y en avoir qu'une seule.

pictureBox1 doit être accessible.

Test de la racine qui a été sélectionnée.

Indique le nom de l'image a afficher dans le contrôle.

9.2.5. Quelques trucs

9.2.5.A. Comment empêcher la fermeture d'un formulaire?

Pour empêcher la fermeture d'un formulaire lorsque l'utilisateur en a fait la demande, il faut procéder comme suit :

1. Dans le fichier *Designer_* du formulaire, il faut ajouter les lignes suivantes :

```
using System.ComponentModel;

this.Closing += new CancelEventHandler(this.FormConsult_Close);
```

2. Dans le fichier *.cs* du formulaire, il faut ajouter la fonction *FormConsult_Close*:

```
private void FormConsult_Close(object sender, CancelEventArgs e)
{
    e.Cancel = true;
}
```

9.2.5.B. Ne pas afficher l'icône de fermeture d'un formulaire?

Lorsqu'un formulaire est affiché, comment peut-on éviter d'afficher l'icône 'X' relative à la fermeture du formulaire ? On peut le faire en ajoutant le code suivant à la classe qui gère le formulaire :

```
protected override CreateParams CreateParams
```

```
{
    get
    {
        CreateParams param = base.CreateParams;
        param.ClassStyle = param.ClassStyle | 0x200;
        return param;
    }
}
```

9.3. Les événements clavier et souris

La gestion du clavier et de la souris s'effectue exclusivement en utilisant les différents événements que ces deux périphériques sont capables de déclencher. La plupart des contrôles sont capables de gérer ces événements, il faut simplement connaître à quel moment ils sont déclenchés et quelles informations ils fournissent.

9.3.1. Les événements clavier

Les possibilités d'action de l'utilisateur sur son clavier sont limitées : il peut simplement frapper une touche du clavier.

Dans Visual C#, cette action est décomposée en trois événements distincts :

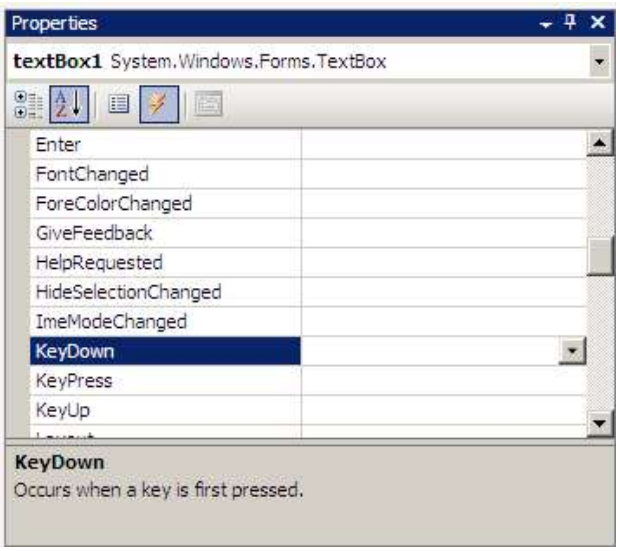
- *KeyDown*: Cet événement se produit lors de l'enfoncement de la touche.
- *KeyUp*: Cet événement se produit lors du relâchement de la touche.
- *KeyPress*: Cet événement se produit lors de l'enfoncement, uniquement si la touche correspond à un caractère ASCII.

Dans les événements *KeyDown* et *KeyUp*, un argument de type *KeyEventArgs* donne des informations complémentaires sur l'événement. Les propriétés suivantes sont notamment disponibles :

- [alt] indique l'état de la touche [alt] du clavier au moment où l'événement se produit.
- [ctrl] et [shift] fournissent des informations identiques pour les touches [ctrl] et [shift].
- *KeyCode* indique le numéro de la touche sur le clavier.

L'événement *KeyDown* est principalement utilisé pour travailler avec les touches de fonction. On peut par exemple convertir en majuscules le texte d'un contrôle *TextBox* si l'utilisateur frappe la combinaison des touches [shift][ctrl][alt][F8]. Il faut pour cela :

- Ajouter un contrôle de type *TextBox* et une méthode pour traiter l'événement *KeyDown*:



- Créer le contenu de la méthode :

```
private void textBox1_KeyDown(object sender, KeyEventArgs e)
{
     if (e.Alt && e.Control && e.Shift && (e.KeyCode == Keys.F8))
     {
          textBox1.Text = textBox1.Text.ToUpper();
     }
}
```

Une autre solution est possible en utilisant la propriété *Modifiers* qui regroupe l'état des touches [shift],[alt] et [ctrl] :

```
private void textBox1_KeyDown(object sender, KeyEventArgs e)
{
     if (e.Modifiers == (Keys.Shift|Keys.Control|Keys.Alt) &&
        (e.KeyCode == Keys.F8))
     {
          textBox1.Text = textBox1.Text.ToUpper();
     }
}
```

Enfin, la dernière possibilité consiste à utiliser la propriété *KeyData* qui regroupe l'état des touches [shift],[alt], [ctrl] et une touche de fonction :

```
private void textBox1_KeyDown(object sender, KeyEventArgs e)
{
     if (e.KeyData == (Keys.Shift|Keys.Control|Keys.Alt|Keys.F8)
     {
          textBox1.Text = textBox1.Text.ToUpper();
     }
}
```

L'événement *KeyPress* informe simplement, par l'intermédiaire du paramètre *e* de type *KeyPressEventArgs*, sur le caractère qui vient d'être saisi. On peut par exemple utiliser cet événement pour vérifier que tous les caractères saisis sont numériques et si ce n'est pas le cas, d'émettre un bip :

```
using System.Media;

private void textBox1_KeyPress(object sender, KeyPressEventArgs e)
```

```
{
        bool bResult;
        int dI;

        bResult = int.TryParse((e.KeyChar.ToString()), out dI);!!!pourquoi int?

        if (!bResult)
        {
            SystemSounds.Beep.Play();
        }
}
```

Voir en annexe.

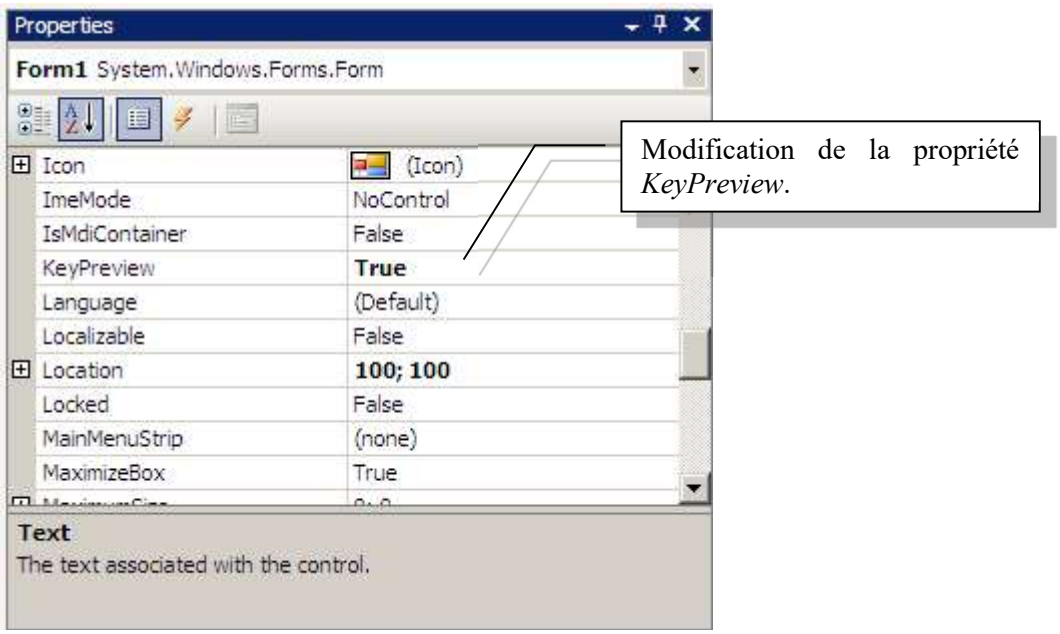
Cette procédure informe simplement l'utilisateur qu'il a saisi un caractère invalide, mais le caractère apparaît tout de même dans la zone de texte. Il serait plus judicieux d'éviter que le carcatère atteigne la zone de texte. La propriété *Handled* du paramètre permet, en lui affectant la valeur *true*, d'indiquer que l'événement clavier vient d'être géré et qu'il ne doit pas être pris en compte par le contrôle :

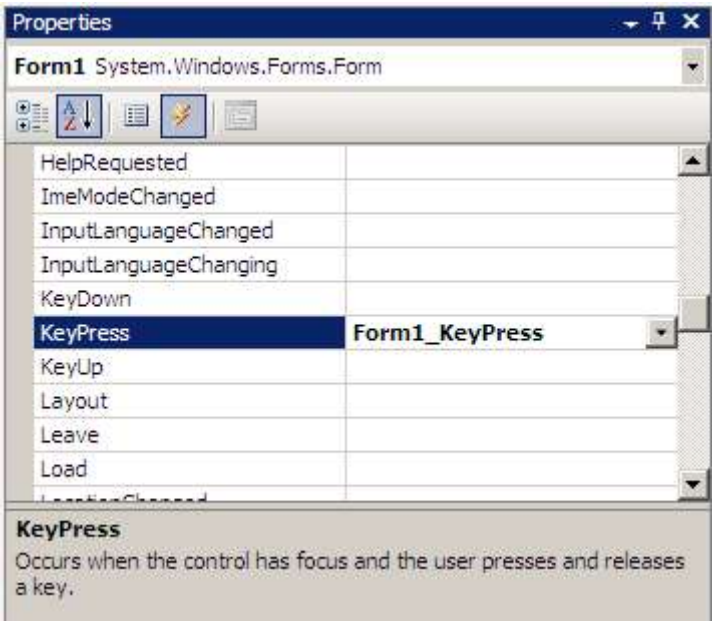
```
private void textBox1_KeyPress(object sender, KeyPressEventArgs e)
  {
        bool bResult;
        int dI;

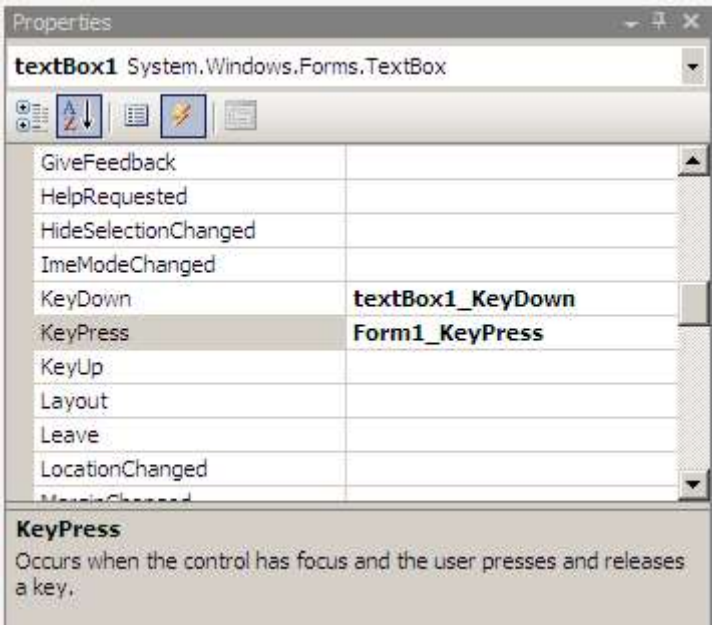
        bResult = int.TryParse((e.KeyChar.ToString()), out dI);

        if (!bResult)
        {
            SystemSounds.Beep.Play();
            e.Handled = true;                !!! ne fonctionne pas
        }
  }
```

Si plusieurs contrôles *TypeBox* doivent avoir le même fonctionnement, on peut simplement recopier ce code dans les événements *KeyPress* de chacun de ces contrôles. Une solution plus élégante permet de centraliser le traitement dans l'événement *KeyPress* de la fenêtre. Pour cela, il convient d'indiquer à la propriété *KeyPreview* de la fenêtre qu'elle recevra les événements clavier avant le contrôle de la fenêtre qui a le focus. Il faut simplement tester le contrôle actif de la feuille et véifier s'il doiot subir le traitement.








```
private void Form1_KeyPress(object sender, KeyPressEventArgs e)
{
     if (this.ActiveControl == textBox1)
     {
          bool bResult;
          int dI;

          bResult = int.TryParse((e.KeyChar.ToString()), out dI);

          if (!bResult)
          {
               SystemSounds.Beep.Play();
               e.Handled = true;
          }
     }
}
```

9.3.2. Les événements souris

Ces événements sont principalement liés à l'utilisation des boutons ou de la molette de la souris. L'événement *click* est le plus utilisé des événements souris. Il correspond à un appui suivi d'un relâchement du bouton principal de la souris. Il faut bien parler de bouton principal car en fonction de la configuration du poste de travail, ce bouton peut aussi bien être le bouton droit ou le bouton gauche suivant que l'utilisateur soit gaucher ou droitier.

L'événement *DoubleClick* n'est pas pris en charge par tous les contrôles, ainsi, par exemple, les contrôles *Button* ne sont pas capables de gérer le double clic, ils génèrent en fait deux événements *click* à la suite.

Des événements souris plus élémentaires sont aussi disponibles :

- *MouseDown* : lorsqu'un bouton de la souris est enfoncé,
- *MouseUp* : lorsqu'un bouton de la souris est relâché,
- *MouseWheel* lorsque la molette est actionnée.

Pour ces trois événements, un paramètre de type *MouseEventArgs* est fourni. Par l'intermédiaire des propriétés disponibles dans cette classe, on obtient les informations suivantes :

- Le bouton à l'origine de l'événement avec la propriété *Button*,
- Le nombre de fois où le bouton est enfoncé ou relâché avec la propriété *Clicks*,
- Le nombre de crans de déplacement de la roulette avec la propriété *Delta*. Cette propriété est un entier positif ou négatif, suivant le sens de rotation de la molette. Chaque déplacement de la molette d'un cran incrémente ou décrémente cette propriété d'une valeur de 120,
- Les propriétés *x* et *y* indiquent l'emplacement sur le contrôle où l'événement souris vient de se produire.

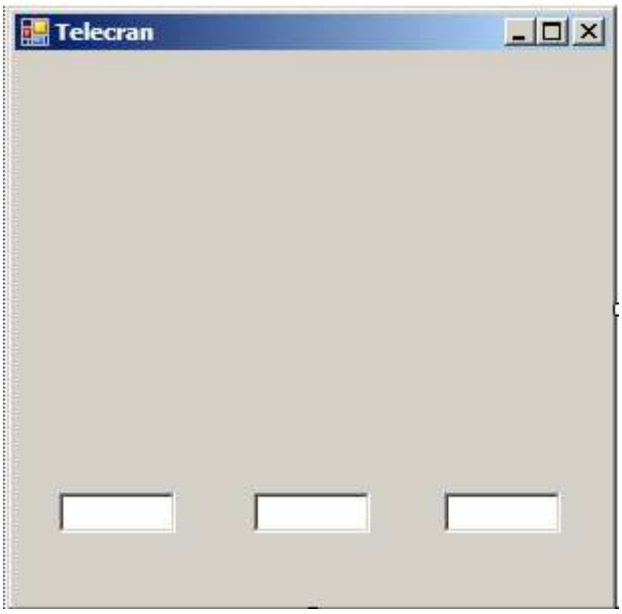
Les déplacements de la souris génèrent quatre événements :

- *MouseEnter* : lorsque la souris entre au dessus d'un contrôle,
- *MouseMove* : lorsque la souris se déplace sur le contrôle,
- *MouseLeave* : lorsque la souris quitte la surface du contrôle,
- *MouseHover* : lorsque la souris reste au-dessus d'un contrôle pendant une seconde.

Comme pour les événements liés aux boutons de la souris, un paramètre de type *MouseEventArgs* est fourni.

Dans l'exemple suivant, à chaque déplacement de la souris, un point est tracé sur la feuille avec la couleur indiquée par les trois zones de texte (rouge, vert, bleu). Pour modifier la couleur, il suffit de déplacer la souris au-dessus de la zone de texte correspondante (le focus se place automatiquement grâce à l'événement *MouseHover*) et d'actionner la molette de la souris pour incrémenter ou décrémenter le pourcentage.

Création d'un nouveau projet de type WindowsForm avec le nom *Telecran*. Renommer le nom *Form1* généré par défaut en *Telecran*. Ensuite, ajouter trois contrôles de type *TextBox* sur la feuille :



Dans chacun des trois *TextBox*, se trouvera le pourcentage de la couleur à utiliser. Lorsque la molette de la souris sera utilisée, alors ce pourcentage sera augmenté ou diminué selon le sens utilisé avec la molette. Il va falloir ajouter une méthode qui va gérer cet événement. Cependant, cet événement sera lié à chacun des trois *textBox* car ce n'est que lorsque le contrôle à le focus que le mouvement de la molette lui sera associé. Pour ajouter, cette méthode, cela n'est pas possible d'utiliser les *properties* du contrôle car l'événement *MouseWheel* n'y est pas visible. Il faut ajouter cette méthode dans le fichier *telecran.designer.cs* comme suit :

```
            //
            // txtRouge
            //
            this.txtRouge.Location = new System.Drawing.Point(22, 220);
            this.txtRouge.Name = "txtRouge";
            this.txtRouge.Size = new System.Drawing.Size(58, 20);
            this.txtRouge.TabIndex = 2;
            this.txtRouge.MouseWheel += new
                System.Windows.Forms.MouseEventHandler(this.m_txtRouge_MouseWheel);
```

Et voici les fonctions à ajouter dans le fichier *Telecran.cs* :

```
    public partial class Telecran : Form
{
    private int m_dRouge;
    private int m_dVert;
    private int m_dBleu;

    public Telecran()
    {
        InitializeComponent();
    }

    private void Telecran_MouseMove(object sender, MouseEventArgs e)
    {
        Graphics oGr;
        Pen oCrayon;

        if (e.Button == MouseButtons.Left)
        {
```

```
            oGr = Graphics.FromHwnd(this.Handle);

            oCrayon = new Pen(Color.FromArgb((int)(m_dRouge * 2.55),
                                             (int)(m_dBleu * 2.55),
                                             (int)(m_dVert * 2.55)));
            oGr.DrawEllipse(oCrayon, e.X, e.Y, 1, 1);
        }
    }

    private void txtRouge_MouseWheel(object sender, MouseEventArgs e)
    {
        m_dRouge += e.Delta / 120;

        if (m_dRouge > 100)
        {
            m_dRouge = 100;
            txtRouge.Text = "100%";
        }
        else if (m_dRouge < 0)
        {
            m_dRouge = 0;
            txtRouge.Text = "0%";
        }
        else txtRouge.Text = m_dRouge + "%";
    }

    private void txtVert_MouseWheel(object sender, MouseEventArgs e)
    {
        m_dVert += e.Delta / 120;

        if (m_dVert > 100)
        {
            m_dVert = 100;
            txtVert.Text = "100%";
        }
        else if (m_dVert < 0)
        {
            m_dVert = 0;
            txtVert.Text = "0%";
        }
        else txtVert.Text = m_dVert + "%";
    }

    private void txtBleu_MouseWheel(object sender, MouseEventArgs e)
    {
        m_dBleu += e.Delta / 120;

        if (m_dBleu > 100)
        {
            m_dBleu = 100;
            txtBleu.Text = "100%";
        }
        else if (m_dBleu < 0)
        {
            m_dBleu = 0;
            txtBleu.Text = "0%";
        }
        else txtBleu.Text = m_dBleu + "%";
    }

    private void txtRouge_MouseHover(object sender, EventArgs e)
```

```
        {
            txtRouge.Focus();
        }

        private void txtBleu_MouseHover(object sender, EventArgs e)
        {
            txtBleu.Focus();
        }

        private void txtVert_MouseHover(object sender, EventArgs e)
        {
            txtVert.Focus();
        }

    }
```

9.4. Le Drag and Drop

Le *Drag and Drop* est une fonctionnalité largement utilisée dans les applications Windows. Elle permet le déplacement d'informations dans une application ou entre applications, en *accrochant* un élément au curseur de la souris et en le lâchant ensuite au dessus de sa destination.

On peut pratiquement accrocher n'importe quel élément au curseur de la souris pour ensuite le déplacer ou le copier. Pour guider l'utilisateur sur ces possibilités, le curseur de la souris est modifié en fonction des contrôles qui sont survolés par la souris.

Une opération de *Drag and Drop* se déroule en trois étapes :

- Accrochage d'un élément à la souris,
- Déplacement de la souris vers la destination,
- Largage au dessus de la destination.

Lors de l'opération de largage, on a deux possibilités :

- L'élément accroché à la souris est déplacé. Dans ce cas, il disparaît du contrôle de départ au moment du largage au-dessus du contrôle de destination.
- L'élément accroché est copié. Dans ce cas, c'est une copie qui est larguée au-dessus du contrôle de destination.

Pour étudier en détail l'opération de *Drag and Drop*, on va utiliser un exemple visant à réaliser la copie d'un texte entre deux contrôles de type texte.

9.4.1. Démarrage du Drag and Drop

En général, une opération de *Drag and Drop* est démarrée lorsque la souris se déplace au-dessus d'un contrôle et le sélectionne sans relâchement du bouton de la souris. On considère, dans ce cas, que l'utilisateur vient d'accrocher le contrôle à la souris.

Dans le code, cela se traduit simplement par l'appel de la méthode *DoDragDrop* du contrôle de départ. L'appel de cette méthode nécessite deux paramètres :

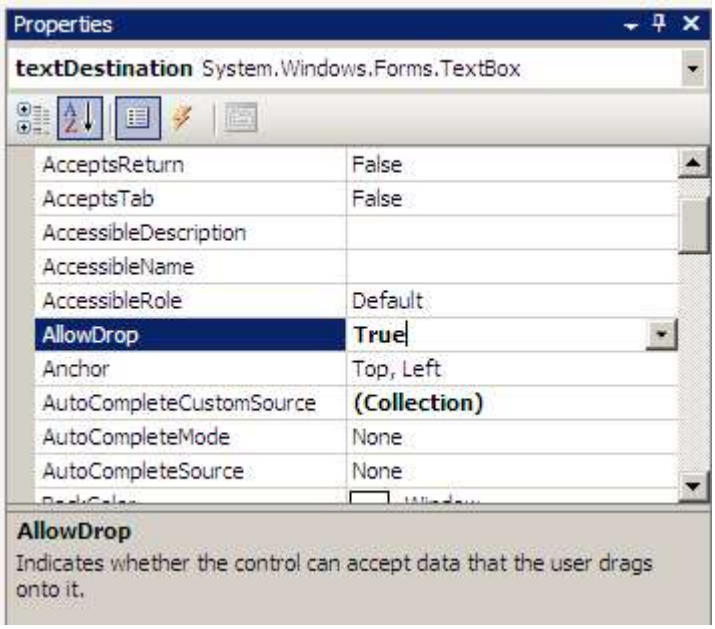
- L'élément que l'on accroche au curseur de la souris (une chaîne de caractères, une image bitmap ou une image métafile),
- Les opérations autorisées pour l'objet accroché (copie, déplacement,…).

```
    private void textSource_MouseUp(object sender, MouseEventArgs e)
```

```
{
    if (e.Button == MouseButtons.Left)
    {
        textSource.DoDragDrop(textSource.Text,
                              DragDropEffects.Move |
                              DragDropEffects.Copy);
    }
}
```

9.4.2. Configuration des contrôles pour la réception

Il convient avant tout de configurer les contrôles de destination pour qu'ils acceptent la réception d'un élément en modifiant la propriété *AllowDrop* sur *true* :



Ensuite, on doit gérer l'événement *DragEnter* qui se produit lorsque la souris entre sur la surface du contrôle avec un élément *accroché*. Dans la gestion de cet événement, il faut déterminer ce qui est *accroché*. Dans la gestion de cet événement, il faut déterminer ce qui est accroché au curseur de la souris pour savoir si le contrôle peut l'accepter. Enfin, il faut déterminer si l'utilisateur souhaite faire un déplacement ou une copie de l'élément accroché au curseur de la souris.

Pour effectuer cela dans le code, dans l'événement *DragEnter*, on a à disposition le paramètre *e* qui est une instance de la classe *DragEventArg*. Cette classe fournit de nombreuses propriétés pour aider à prendre les bonnes décisions :

- ***AllowEffect***: Permet de savoir quelles sont les opérations autorisées par le contrôle à l'origine du *Drag and Drop*.
- ***Data***: Contient les informations accrochées au curseur de la souris. Par l'intermédiaire de cette propriété, on peut obtenir le type d'information en appelant la méthode *GetDataPresent* ou obtenir les données en appelant la méthode *GetData*.
- ***Effect*** : indique l'action autorisée par le contrôle de destination. Cette propriété est utilisée pour contrôler l'apparence du curseur de la souris.
- ***KeyState*** : indique l'état des touches [shift],[ctrl],[alt] permettant de savoir si l'utilisateur souhaite effectuer un déplacement ou une copie.

- ***X, Y*** : indique les coordonnées de la souris sur le contrôle.

```
private void textDestination_DragEnter(object sender, DragEventArgs e)
  {
      if (e.Data.GetDataPresent(DataFormats.Text))
      {
          if ((e.KeyState & 8) == 8)
          {
              e.Effect = DragDropEffects.Copy;
          }
          else
          {
              e.Effect = DragDropEffects.Move;
          }
      }
  }
```

9.4.3. Récupération de l'élément accroché

Lorsque l'utilisateur relâche le bouton de la souris, l'événement *DragDrop* se déclenche sur le contrôle de destination. Dans cet événement, il faut récupérer l'élément accroché et le placer dans le contrôle de destination. S'il s'agit d'un déplacement, il faut aussi supprimer l'information du contrôle source. Problème ! Qui est le contrôle source ? On n'a aucune information sur son identité.

La solution consiste à stocker, dans une variable une référence vers le contrôle à l'origine de l'opération de *Drag and Drop* au début de l'opération.

Pour tester quelles sont les touches qui sont utilisées au moment du *Drag and Drop,* on peut utiliser la propriété *KeyState* qui retournera les valeurs suivantes :

Bit	Key
1	The left mouse button.
2	The right mouse button.
4	The SHIFT key.
8	The CTRL key.
16	The middle mouse button.
32	The ALT key.

Pour tester les bits qui ont été positionnés, il suffit de faire une opération de ET binaire entre la valeur retournée par le *KeyState* et la valeur à tester :

- `((e.KeyState & 8) == 8)` : Permet de tester si la touche <ctrl> a été utilisée.
- `((e.KeyState & (8+32)) == (8+32)` : Permet de tester si les touches <ctrl> et <alt> ont été utilisées.

Voici le contenu complet de la classe :

```
public partial class DragDrop : Form
{
```

```
        private TextBox ctrlSource;

        public DragDrop()
        {
            InitializeComponent();
        }

       private void textDestination_DragEnter(object sender, DragEventArgs e)
        {
            if (e.Data.GetDataPresent(DataFormats.Text))
            {
                if ((e.KeyState & 8) == 8)
                {
                    e.Effect = DragDropEffects.Copy;
                }
                else
                {
                    e.Effect = DragDropEffects.Move;
                }
            }
        }

        private void textSource_MouseUp(object sender, MouseEventArgs e)
        {
            if (e.Button == MouseButtons.Left)
            {
                textSource.DoDragDrop(textSource.Text,
                                      DragDropEffects.Move |
                                      DragDropEffects.Copy);
            }

            ctrlSource = textSource;
        }

        private void textDestination_DragDrop(object sender, DragEventArgs e)
        {
            textDestination.Text = (String)e.Data.GetData(DataFormats.Text);

            if ((e.KeyState & 8) != 8)
            {
                ctrlSource.Clear();
            }
        }
    }
```

9.5. Les boîtes de dialogue

Les boîtes de dialogue sont des fenêtres qui ont une fonction spéciale dans une application. Elles sont, en général, utilisées pour demander la saisie d'informations à l'utilisateur. Pour s'assurer que ces informations sont bien saisies avant de continuer l'exécution de l'application, les boîtes de dialogues sont souvent affichées en mode modal, c'est-à-dire que le reste de l'application est bloqué tant que la boîte de dialogue est affichée.

Il arrive fréquemment que dans une application, on ait besoin des mêmes informations : un nom de fichier à ouvrir, une police de caractère à choisir,… Pour éviter de devoir à recréer, à chaque fois, une nouvelle boîte de dialogue, on a à notre disposition une série de boîtes de dialogue prédéfinies.

9.5.1. La boîte de message

Les boîtes de message permettent de passer une information à l'utilisateur et lui donnent la possibilité de répondre par l'intermédiaire des boutons de commande de la boîte de message.

La boîte de message est disponible par l'intermédiaire de la méthode *show* disponible dans la classe *MessageBox*. Cette méthode prend de nombreux paramètres pour configurer la boîte de dialogue :

- Le premier paramètre correspond au message affiché.
- Le deuxième paramètre spécifie le titre de la boîte de message,
- Les paramètres suivants doivent être choisis parmi des constantes prédéfinies pour indiquer respectivement :
    - Les boutons disponibles sur la boîte de message,
    - L'icône affichée sur la boîte de message,
    - Le bouton sélectionné par défaut à l'affichage de la boîte de message.

Les constantes disponibles sont :

- Pour les boutons :

CONSTANTE	SIGNIFICATION
OK	Bouton OK seul
OKCancel	Boutons OK et Annuler
AbortRetryIgnore	Boutons Abandonner, réessayer et Ignorer
YesNoCancel	Boutons Oui, Non et Annuler
YesNo	Boutons Oui et Non
RetryCancel	Boutons réessayer et Annuler

- Pour les icônes :

CONSTANTE	SIGNIFICATION
IconInformation	Icône d'information
IconExclamation	Icône d'attention
IconError	Icône d'erreur
IconQuestion	Icône interrogative

- Pour le bouton par défaut :

CONSTANTE	SIGNIFICATION
DefaultButton1	Premier bouton
DefaultButton2	Deuxième bouton
DefaultButton3	Troisième bouton

Le code suivant :

```
static void Main()
  {
      Application.EnableVisualStyles();
      Application.SetCompatibleTextRenderingDefault(false);
      Application.Run(new Form1());

      DialogResult oReponse;

      oReponse = MessageBox.Show("Voulez-vous enregistrer en quittant
                                  l'application",
                                  "Fin du Programme",
                                  MessageBoxButtons.YesNoCancel,
                                  MessageBoxIcon.Question,
                                  MessageBoxDefaultButton.Button1);

  }
```

Va afficher la boîte de message suivante :



Comme on pose une question à l'utilisateur, on doit récupérer sa réponse pour décider de la conduite à tenir dans l'application. Pour cela, la méthode *Show* renvoie une valeur indiquant le bouton utilisé pour fermer la boîte de message. Ici encore, une série de constantes est définie pour identifier chaque cas possible :

VALEUR RENVOYEE	BOUTON UTILISE
Ok	Bouton OK
Cancel	Bouton Cancel
Abort	Bouton Abandonner
Retry	Bouton réessayer
Ignore	Bouton Ignorer
Yes	Bouton Oui
No	Bouton Non

On peut tester la réponse comme suit :

```
switch (oReponse)
{
        case DialogResult.Yes:
            break;
        case DialogResult.No:
```

```
                break;
            case DialogResult.Cancel:
                break;
    }
```

9.5.2. Les boîtes de dialogue de Windows

De nombreuses boîtes de dialogue sont déjà définies au niveau du système Windows lui-même. Pour pouvoir les utiliser dans les applications, on a à sa disposition une série de classes.

Regardons comment les configurer et les utiliser dans une application.

9.5.2.A. Dialogue pour l'ouverture d'un fichier

Cette boîte de dialogue permet la sélection d'un ou de plusieurs noms de fichier avec la possibilité de se déplacer dans l'arborescence de la machine. La classe utilisée est la classe *OpenFileDialog*. On peut donc créer une instance dans l'application :

```
OpenFileDialog dlgOpen;

dlgOpen = new OpenFileDialog();
```

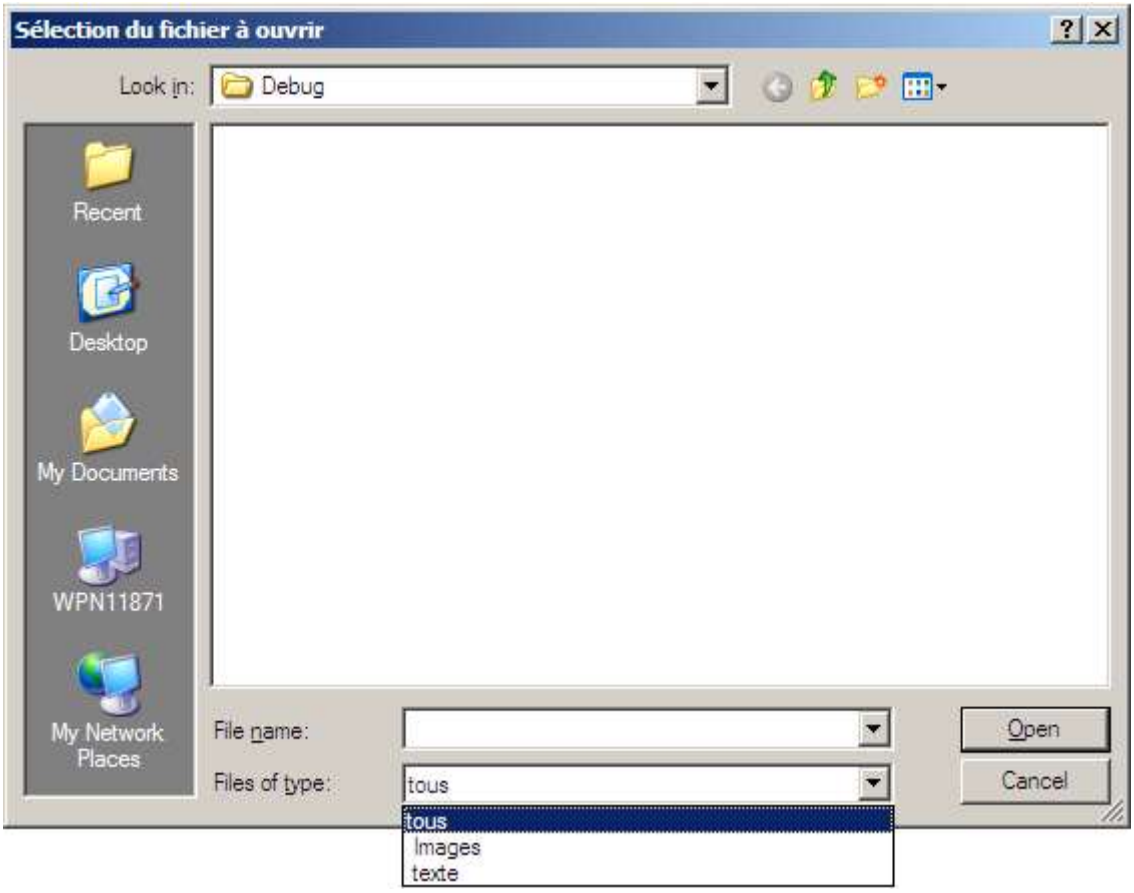
Il faut également configurer la boîte de dialogue. La propriété *InitialDirectory* indique le répertoire sur lequel se trouve la boîte de dialogue à son ouverture. Il est possible de n'afficher que certains fichiers dans les répertoires qui seront parcourus, il faut pour cela configurer la propriété *Filter*. Cette propriété stocke l'information sous forme d'une chaîne de caractères. La description et l'extension sont séparées dans la chaîne par le caractère "|".

Si plusieurs extensions sont disponibles pour une même description, elles doivent être séparées par un point-virgule dans la chaîne. On peut également indiquer si une extension doit être ajoutée aux noms de fichiers saisis manuellement, si ceux-ci n'en comportent pas.

La propriété *DefaultExt* contient l'extension à ajouter et *AddExtension* indique si cette extension est ajoutée automatiquement. Dans la mesure où l'on permet à l'utilisateur de saisir manuellement le chemin et le nom du fichier à ouvrir, on peut confier à la boîte de dialogue le soin de vérifier que ces informations sont correctes. Les propriétés *CheckFileExist* et *CheckPathExist* gèrent ces vérifications. On peut également autoriser les sélections multiples par l'intermédiaire de la propriété *Multiselect*.

Enfin, pour afficher la boîte de dialogue, on utilise la méthode *ShowDialog*:

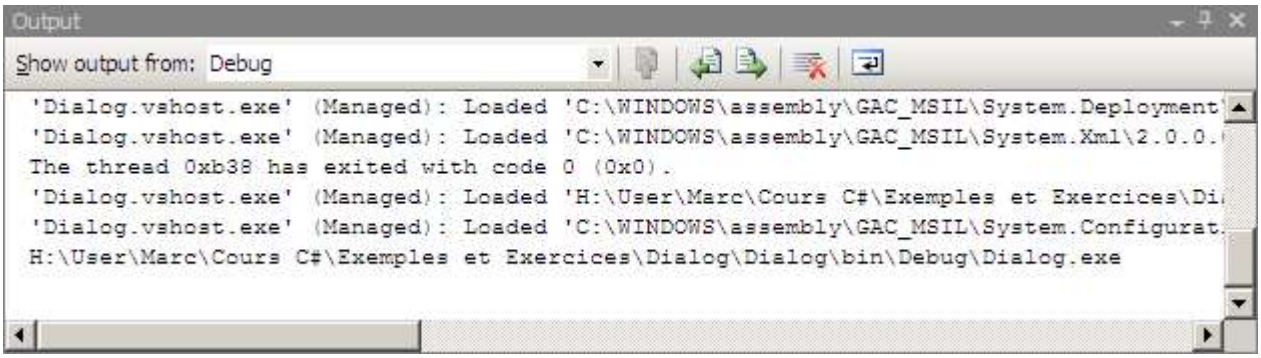
```
dlgOpen.Title = "Sélection du fichier à ouvrir";
dlgOpen.Filter = "tous|*.*|Images|*.bmp;*.gif;*.jpg|texte|*.txt";
dlgOpen.DefaultExt = "toto";
dlgOpen.AddExtension = true;
dlgOpen.CheckFileExists = false;
dlgOpen.Multiselect = true;
dlgOpen.ShowDialog();
```



Les noms du ou des fichiers sélectionnés sont disponibles dans la propriété *FileNames*. Cette propriété est un tableau de chaînes de caractères avec, dans chaque élément du tableau, le nom complet d'un des fichiers sélectionnés.

```
foreach (string strNomFichier in dlgOpen.FileNames)
{
     Console.WriteLine(strNomFichier);
}
```

Les messages écrits sur la console s'affichent dans la fenêtre *Output* :



Le test sur le bouton de sortie utilisé est fait par :

```
if (dlgOpen.ShowDialog() == DialogResult.OK)
```

9.5.2.B. Dialogue d'enregistrement de fichier

La boîte de dialogue d'enregistrement de fichier est similaire à la précédente, mis à part la propriété *Multiselect* qui disparaît et les propriétés *CreatePrompt* et *OverwritePrompt* permettant d'afficher un message d'avertissement, si le nom du fichier saisi n'existe pas ou au contraire s'il existe déjà.

9.5.2.C. Dialogue de choix de répertoire

Cette boîte de dialogue est utilisée pour la sélection ou la création d'un répertoire. Elle est créée à partir de la classe *FolderBrowserDialog*. Cette dernière comporte très peu de propriétés. La plus utilisée est certainement la propriété *SelectedPath* permettant la récupération du chemin d'accès au répertoire sélectionné.

Le répertoire racine de la boîte de dialogue est indiqué par la propriété *RootFolder*. Cette propriété reçoit une des valeurs de l'énumération *Environment.SpecialFolder* représentant les principaux répertoires caractéristiques du système comme, par exemple, le répertoire *Mes Documents*. Si cette propriété est utilisée la sélection ne pourra se faire que dans un sous répertoire du répertoire racine. L'ajout d'un bouton permettant la création d'un nouveau répertoire peut être autorisé en modifiant la propriété *ShowNewFolderButton*. L'affichage de la boîte de dialogue se fait de manière classique par la méthode *ShowDialog* :

```
FolderBrowserDialog dlgChoixRep;

dlgChoixRep = new FolderBrowserDialog();
dlgChoixRep.RootFolder = Environment.SpecialFolder.MyDocuments;
dlgChoixRep.ShowDialog();

MessageBox.Show(dlgChoixRep.SelectedPath, "Répertoire sélectionné");
```

9.5.2.D. Dialogue de choix d'une couleur

La boîte de dialogue de choix de couleur créée à partir de la classe *ColorDialog* peut avoir deux configurations différentes.

Une version simple où seules les couleurs de base sont disponibles.

Une version complète dans laquelle l'utilisateur pourra créer des couleurs personnalisées.

La propriété *Color* permet d'initialiser la boîte de dialogue avant son affichage et ensuite de récupérer la couleur choisie par l'utilisateur. On peut interdire l'utilisation des couleurs personnalisées ou au contraire afficher la boîte de dialogue complète dés son ouverture. Pour interdire l'affichage des couleurs personnalisées, on modifie la propriété *AllowFullOpen*. Pour forcer l'affichage complet, on utilise la propriété *FullOpen*.

L'affichage de la boîte de dialogue s'effectue toujours par la méthode *ShowDialog*. Pour conserver une qualité d'affichage correcte, on peut également n'autoriser que l'utilisation de couleurs pures (les couleurs obtenues par juxtaposition de différents pixels seront éliminées des choix possibles). Cette option est à utiliser si on a une carte graphique limitée à 256 couleurs.

L'exemple suivant modifie la couleur de fond la feuille :

```
private void Form1_MouseClick(object sender, MouseEventArgs e)
{
     ColorDialog dlgColor;

     dlgColor = new ColorDialog();
     dlgColor.FullOpen = true;
     dlgColor.SolidColorOnly = true;
     dlgColor.Color = this.BackColor;
```

```
            dlgColor.ShowDialog();
            this.BackColor = dlgColor.Color;
        }
```

9.5.2.E. Dialogue de choix d'une police

La classe de base utilisée pour la sélection d'une police est la classe *FontDialog*. La propriété *Font* permet de définir la police de caractères utilisées pour initialiser la boîte de dialogue ou, après fermeture, de récupérer la police sélectionnée. On peut également afficher une boîte de dialogue simplifiée sans le choix de couleur ou des effets.

Pour cela, les propriétés *ShowColor* et *ShowEffects* contrôlent l'affichage de ces paramètres dans la boîte de dialogue. Afin de garantir que les paramètres sélectionnés correspondent bien à une police existant sur la machine, on peut utiliser la propriété *FontMustExist*. Cette propriété obligera la boîte de dialogue à vérifier l'existence d'une police correspondante sur le système avant de se fermer. Certaines polices proposent plusieurs jeux de caractères. On peut autoriser les utilisateurs à choisir l'un de ces jeux de caractères en modifiant la propriété *AllowScriptChange*. La taille de la police sélectionnée peut également être limitée par les propriétés *MaxSize* et *MinSize*.

Pour se rendre compte de l'effet de la police sélectionnée, un aperçu sur quelques caractères est disponible. Si cet aperçu n'est pas suffisant, on a la possibilité d'afficher un bouton *Appliquer* sur la boîte de dialogue par l'intermédiaire de la propriété *ShowApply*. Ce bouton déclenche un événement *Apply* sur la boîte de dialogue. Dans la gestion de cet événement, on peut utiliser la propriété *Font* de la boîte de dialogue pour visualiser l'effet de la police actuellement sélectionnée sur le texte.

```
FontDialog dlgFont;

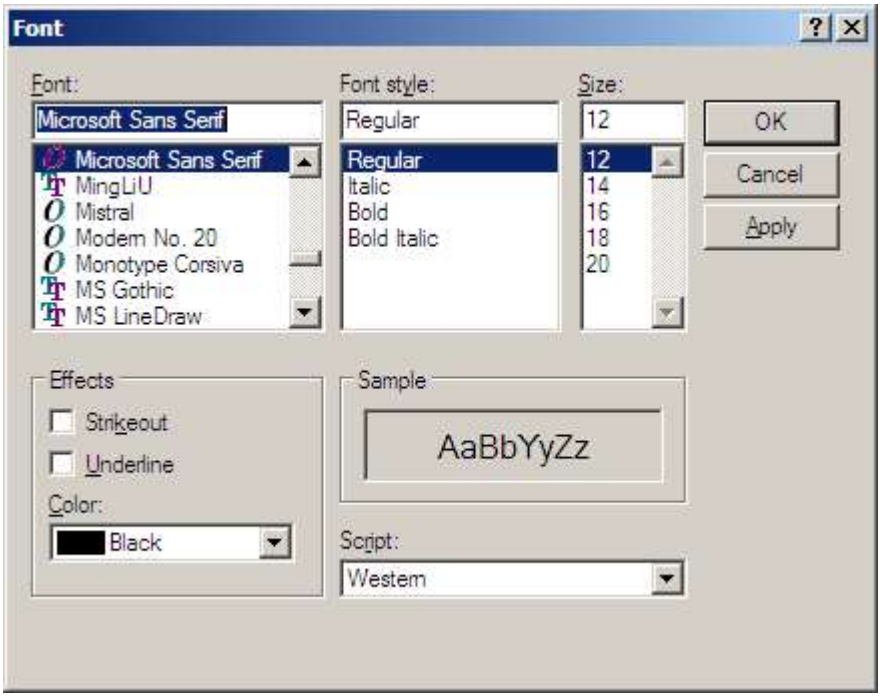
dlgFont = new FontDialog();
dlgFont.ShowApply = true;
dlgFont.ShowColor = true;
dlgFont.ShowEffects = true;
dlgFont.MaxSize = 20;
dlgFont.MinSize = 12;
dlgFont.FontMustExist = true;
dlgFont.AllowScriptChange = true;
dlgFont.Apply += new EventHandler(dlgFont_Apply);
dlgFont.ShowDialog();
```

L'application récupère les choix de l'utilisateur dés que celui-ci a utilisé un des boutons présent sur la boîte de dialogue :

```
private void cmdChangeFont_Click(object sender, EventArgs e)
  {
      DialogResult oResult = dlgFont.ShowDialog();

      if (oResult == DialogResult.OK)
      {
          m_fntStd = new Font(dlgFont.Font.Name, dlgFont.Font.Size,
                     dlgFont.Font.Style);
      }
  }
```

La fonction ci-dessus est déclenchée lorsque l'utilisateur déclenche un évènement indiquant qu'il souhaite modifier de police de caractère. Dans ce cas, on lui affiche une boîte de dialogue de type *choix de caractères*.



9.5.2.F. Dialogue de mise en page

Par l'intermédiaire de cette boîte de dialogue, on va pouvoir configurer les paramètres de mise en page du document (marges, orientation,…).

Cette boîte de dialogue est créée à partir de la classe *PageSetupDialog*. Pour travailler, cette classe a besoin de deux classes auxiliaires : la classe *PageSettings* sert à stocker la configuration de mise en page, la classe *PrinterSettings* sert à stocker la configuration de l'imprimante sélectionnée. Il faut créer une instance de ces deux classes et les associer aux propriétés *PageSettings* et *PrinterSettings* de la boîte de dialogue. On devra importer l'espace de noms *System.Drawing.Printing* pour pouvoir utiliser ces deux classes.

L'utilisation des différentes rubriques de la boîte de dialogue peut être interdite par la modification des propriétés suivantes :

- *AllowMargins* pour la modification des marges,
- *AllowOrientation* pour la modification de l'orientation,
- *AllowPaper* pour le choix du papier,
- *AllowPrinter* pour le choix de l'imprimante.

Les choix de l'utilisateur seront ensuite récupérés à l'aide des propriétés *PageSettings* et *PrinterSettings* de la boîte de dialogue.

```
using System.Drawing.Printing;

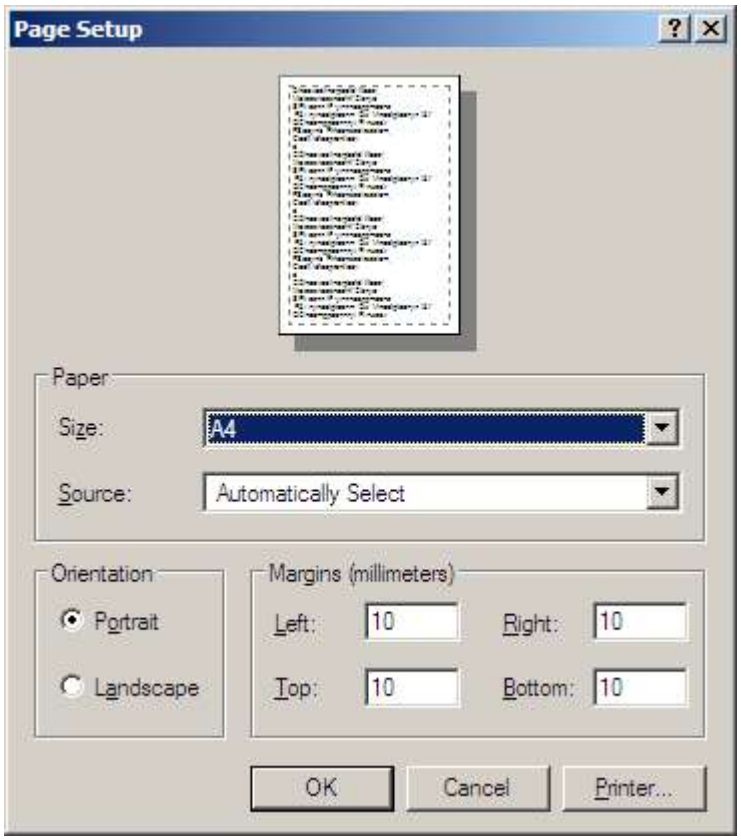
PageSetupDialog dlgPageSetUp;
PageSettings oConfigPage;
PrinterSettings oConfigPrt;

dlgPageSetUp = new PageSetupDialog();
oConfigPage = new PageSettings();
oConfigPrt = new PrinterSettings();

dlgPageSetUp.PageSettings = oConfigPage;
dlgPageSetUp.PrinterSettings = oConfigPrt;
dlgPageSetUp.AllowPrinter = true;
```

```
dlgPageSetUp.ShowDialog();

MessageBox.Show("Vous avez choisi d'imprimer avec l'imprimante " +
         dlgPageSetUp.PrinterSettings.PrinterName +
         " sur du papier " +
         dlgPageSetUp.PageSettings.PaperSize.PaperName +
         " en format " +
         (dlgPageSetUp.PageSettings.Landscape ? "paysage" : "portrait"));
```





9.5.2.G. Dialogue de configuration d'impression

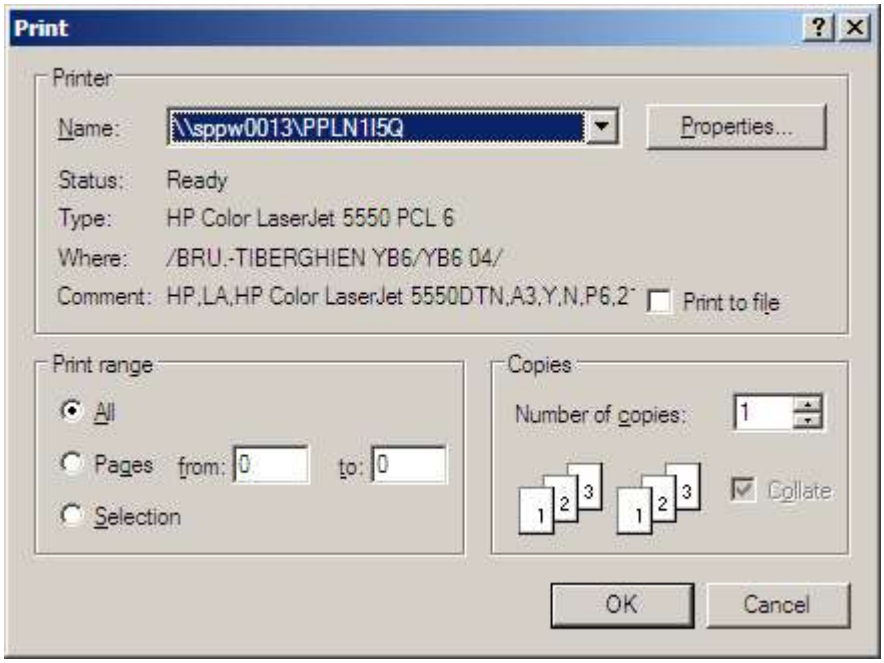
Avec cette boîte de dialogue, on peut configurer les paramètres d'impression du document. On va utiliser une instance de la classe *PrintDialog*.

Comme pour la boîte de dialogue de mise en page, la boîte de dialogue de configuration d'impression a besoin d'une instance de la classe *PrinterSettings* pour stocker les informations de configuration de l'imprimante.

Les différentes rubriques peuvent être interdites d'utilisation, par la modification des propriétés suivantes :

- *AllowSelection* : autorise l'utilisation du bouton *Sélection*. En général, ce bouton est accessible uniquement s'il y a quelque chose de sélectionné dans le document que l'on veut imprimer.
- *AllowSomePages* : autorise la sélection d'une page de début et d'une page de fin pour l'impression du document. Ce bouton doit être disponible si le document contient plusieurs pages.
- *AllowPrintToFile* : indique si la case à cocher *Impression dans un fichier* est disponible. Cette fonctionnalité permet par exemple la récupération d'un fichier au format *PostScript* pour l'importer dans une autre application.

Le résultat des différentes options sélectionnées est disponible après fermeture de la boîte de dialogue, par l'intermédiaire de la propriété *PrinterSettings*.



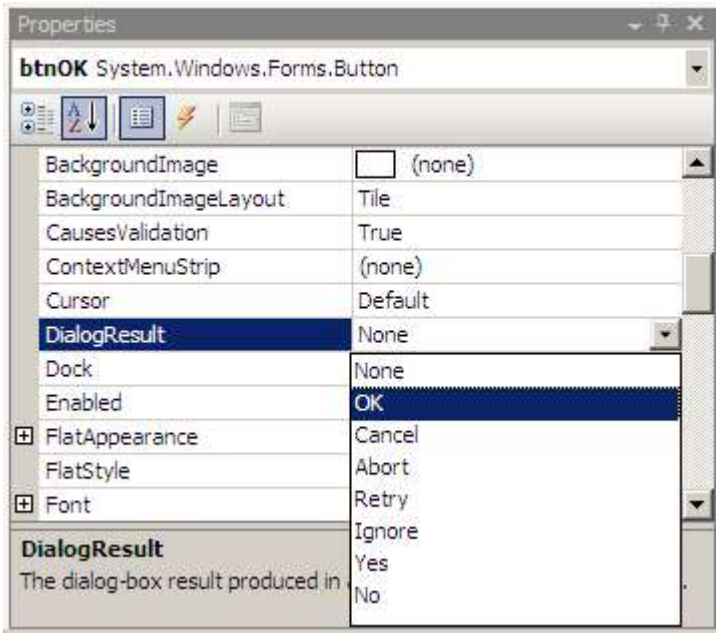
9.5.3. Les boîtes de dialogue personnalisées

Après ce bref aperçu des boîtes de dialogues prédéfinies, on va voir maintenant que l'on peut créer des boîtes de dialogue.

La base de création d'une boîte de dialogue est une fenêtre classique sr laquelle on va pouvoir ajouter des contrôles (voir sections suivantes). L'affichage de la boîte de dialogue se fera par l'appel de la méthode *ShowDialog*.

À la fermeture de la boîte, il faut pouvoir déterminer quel bouton a provoqué la fermeture de la boîte de dialogue. C'est en fait la méthode *ShowDialog* qui va fournir la solution. Elle retourne en effet une des valeurs de l'énumération *System.Windows.Forms.DialogResult*.

La valeur retournée n'est bien sûr pas prise au hasard. Au moment de la conception de la boîte de dialogue, on doit fournir la valeur retournée pour chacun des boutons provoquant la fermeture de la boîte de dialogue. On peut le faire en modifiant la propriété *DialogResult* de la boîte de dialogue dans l'événement *Click* de chacun des boutons ou en modifiant la propriété *DialogResult* des boutons concernés par la fermeture de la boîte de dialogue.



Attention: lorsque le clic sur un bouton renvoi une valeur, alors, cela provoque automatiquement la fermeture de la dialog box sur laquelle il se trouve. Par contre, un bouton qui renvoi la valeur *none* ne provoque pas la sortie de la dialog.

À noter que dans ce cas, il n'y a pas besoin de gérer l'événement *Click* sur le bouton pour provoquer la fermeture de la boîte de dialogue. Si les deux solutions sont utilisées simultanément, la propriété *DialogResult* de la boîte de dialogue sera prioritaire pour déterminer la valeur renvoyée par la méthode *ShowDialog*.

```
Form2 dlgTexte;
DialogResult oDialogResult;

dlgTexte = new Form2();
oDialogResult = dlgTexte.ShowDialog();

if (oDialogResult == DialogResult.OK)
   MessageBox.Show("Sortie de la dialog avec le OK");
```

Au-delà de recevoir l'identification du bouton utilisé, on souhaite peut-être retourner d'autres valeurs en résultat de la Dialog Box. Cela est possible en ajoutant des fonctions *public* qui permettront de retourner la valeur de certains contrôles situés sur la *DialogBox*.

9.6. Utilisation des contrôles

Les contrôles vont permettre de créer l'interface entre l'application et son utilisateur. C'est par leur intermédiaire que l'utilisateur pourra agir sur le fonctionnement de l'application en saisissant du texte, en choisissant des options, en lançant l'exécution d'une partie spécifique de l'application.

Les contrôles seront disponibles, en Visual C#, par l'intermédiaire d'une série de classes qui devront être instanciées au cours de l'exécution de l'application.

Ces différentes classes sont issues d'une hiérarchie qui commence par la classe de base *Control*. Cette classe assure les fonctions élémentaires des contrôles (positions, dimensions,…). Chaque classe dérivée ajoute des fonctionnalités supplémentaires et ainsi de suite jusqu'à la classe finale de la hiérarchie.

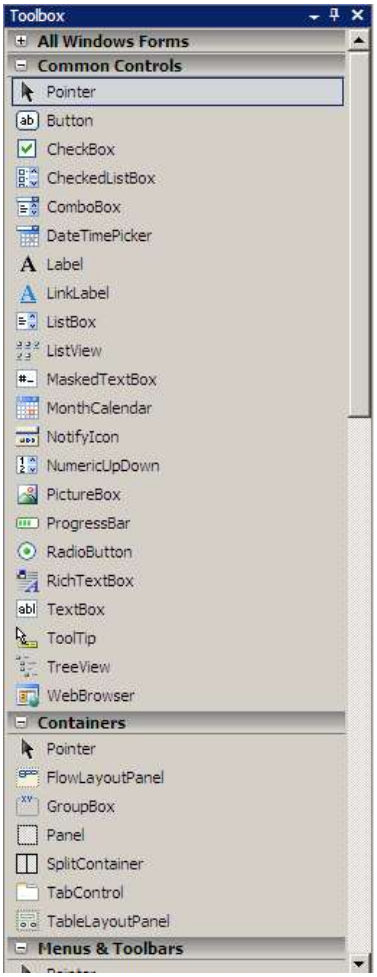
9.6.1. Ajoute des contrôles

Les contrôles peuvent être ajoutés sur une fenêtre de deux manières différentes. La plus simple, et la plus rapide également, passe par l'utilisation de la boîte à outils. Ici encore, il existe trois possibilités pour l'ajoute des contrôles.

- Effectuer un double clic sur le contrôle dans la boîte à outils. Cette méthode permet d'en placer un exemplaire, avec une taille par défaut au centre de la fenêtre.

- Effectuer un glisser-déplacer entre la boîte à outils et la fenêtre. Dés que l'on survole la feuille, le curseur de la souris indique par la présence d'un petit signe plus (+) que l'on va ajouter quelque chose sur la feuille. La position à laquelle on va lâcher le bouton de la souris correspondra à la position du coin supérieur gauche du contrôle. Il sera dimensionné avec les valeurs par défaut.

- Sélectionner le contrôle dans la boîte à outils, puis cliquer sur la fenêtre à l'endroit où on veut placer le coin supérieur gauche du contrôle. Puis sans relâcher le bouton de la souris, il faut agrandir le rectangle jusqu'à la taille désirée du contrôle.



Si on souhaite placer plusieurs exemplaires du même contrôle sur la fenêtre, il est possible de bloquer la sélection dans la boîte à outils en utilisant la touche [ctrl] lorsque l'on sélectionne le contrôle dans la boîte à outils. On pourra alors placer plusieurs exemplaires du même contrôle sans avoir à le resélectionner dans la boîte à outils en conservant la touche [ctrl] enfoncée.

Certains contrôles n'ont pas d'interface visible au moment de la conception de la fenêtre. Pour éviter d'encombrer la surface de la fenêtre, ils sont placés dans une zone située en bas de la fenêtre de conception graphique. C'est le cas par exemple des contrôles *ImageList* et *Timer* que l'on verra plus loin dans ce chapitre.

Il est possible d'ajouter des contrôles à la boîte à outils. Ces contrôles peuvent être des contrôles .net ou des contrôles ActiveX. L'utilisation de contrôles ActiveX va entraîner quelques inconvénients pour l'application. Le code sera moins efficace : des opérations supplémentaires seront nécessaires pour accéder au contrôle ActiveX. Le déploiement de l'application nécessitera aussi des modifications dans la base de registre des machines afin d'enregistrer les contrôles ActiveX.

Les contrôles ajoutés sur la feuille sont nommés automatiquement par Visual Studio au fur et à mesure de l'ajout. Par contre, les noms utilisés par défaut ne sont pas très explicites.

Pour la lisibilité du code, il va être primordial de renommer les contrôles de préférence dès la création ou, au plus tard, avant de les utiliser dans le code. Il suffit simplement de changer la propriété *name* de chacun d'entre eux à l'aide de la fenêtre de propriétés.

Il n'y a pas de règle absolue à respecter pour les noms des contrôles. Une solution fréquemment utilisée consiste à associer un préfixe représentatif du type du contrôle à un nom explicite pour l'application. Voici un tableau avec quelques préfixes :

Préfixe	Contrôle
cbo	ComboBox

lst	ListBox
chk	CheckBox
opt	RadioButton
btn	Button
txt	TextBox
lbl	Label
dte	DateTime
mtxt	Masked Text Box
pct	PictureBox
cmd	Commande dans les menus
menu	Menus
dlg	Dialog Box
form	Formulaire
dtep	DateTime Picker
prb	Progress Bar
tms	TimeSpan
col	Collection
o	Classes construites
st	Structures
pr	Propriétés
sr	StreamReader
fnt	Font
pd	PrintDocument
prv	Print Preview
tree	TreeView
dir	Directory ou DirectoryInfo
dtg	DataGridView

Table 19: Liste des préfixes pour les Contrôles

9.6.2. Position et dimension des contrôles

Après avoir placé les contrôles sur la fenêtre, il est possible de les repositionner ou de les redimensionner. Lorsque l'on déplace la souris au-dessus d'un contrôle, le curseur change d'apparence pour indiquer la possibilité de déplacer le contrôle.

Il suffit de cliquer sur le contrôle puis de déplacer la souris. Un rectangle noir suit alors le curseur de la souris, pour représenter la future position du contrôle. Des lignes de guidage sont affichées pendant le déplacement du contrôle pour faciliter son alignement avec les autres contrôles déjà placés sur la fenêtre. Les lignes bleues représentent les alignements possibles sur les bordures des autres contrôles. Les lignes roses représentent les alignements possibles sur les libellés des contrôles. Le contrôle sera effectivement déplacé au moment où on relâchera le bouton de la souris.

L'utilisation des flèches du clavier est également possible et apporte plus de précision dans les déplacements. On peut également modifier la position d'un contrôle par sa propriété *Location* dans la fenêtre des propriétés. Cette propriété est d'ailleurs modifiée lorsqu'on déplace un contrôle avec la souris ou le clavier.

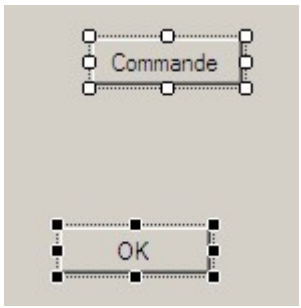
Enfin, la dernière possibilité est de modifier les propriétés *Left* et *Top* du contrôle par le code. Le code suivant permet de déplacer le bouton à une position aléatoire à chaque fois que l'on clique dessus :

```
private void btnOK_Click(object sender, EventArgs e)
 {
     btnOK.Left = new Random().Next(0, (this.ClientSize.Width -
                                        btnOK.Size.Width));
```

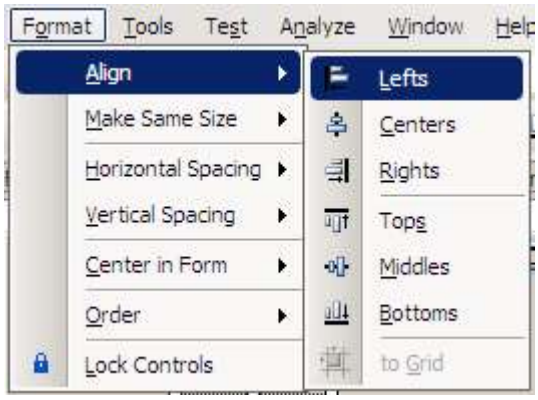
```
            btnOK.Top = new Random().Next(0, (this.ClientSize.Height -
                                                btnOK.Size.Height));
        }
```

Des fonctionnalités plus évoluées permettent le positionnement des contrôles les uns par rapport aux autres. Pour pouvoir les utiliser, il faut au préalable sélectionner plusieurs contrôles sur la feuille. Pour cela, deux solutions sont possibles :

- Dessiner un rectangle de sélection, avec la souris autour des contrôles.
- Cliquer sur les contrôles les uns après les autres en maintenant la touche [ctrl] enfoncée. Le premier contrôle sélectionné apparaît avec des poignées de sélection blanches.



Les options du menu *Format* sont maintenant disponibles et fournissent de nombreuses options pour positionner les contrôles. Le contrôle qui apparaît dans la sélection avec des poignées de sélection blanches est considéré comme référence pour l'alignement.





De nombreuses options sont disponibles pour organiser le placement des contrôles sur une feuille.

Le redimensionnement des contrôles est également très simple à mettre en œuvre puisqu'il suffit de sélectionner le ou les contrôles à redimensionner et de placer le curseur de la souris sur l'une des poignées de sélection, pour faire apparaître une flèche indiquant dans quelle direction on peut redimensionner le contrôle.

Il faut alors cliquer sur le carré correspondant et déplacer la souris jusqu'à ce que le contrôle ait atteint la taille désirée. On peut également utiliser les flèches du clavier en association avec la touche [shift] pour le dimensionnement des contrôles.

Le redimensionnement par le code utilise la méthode *SetBounds* qui permet à la fois de fixer la position et la taille du contrôle. Le code suivant diminue la taille du bouton à chaque fois que l'on clique dessus :

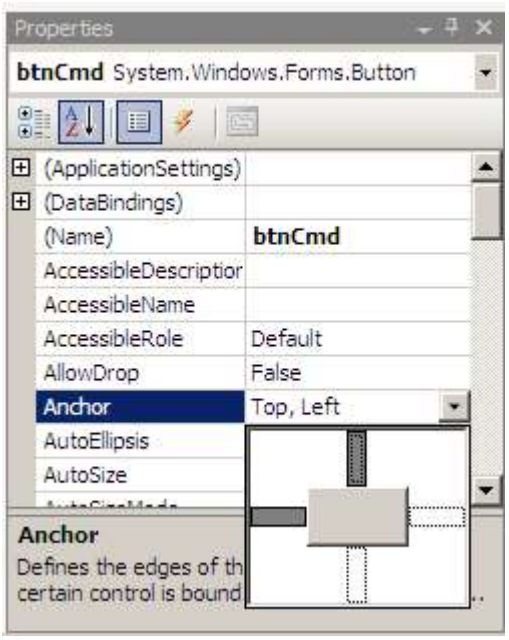
```
        private void btnOK_Click(object sender, EventArgs e)
```

```
{
     btnOK.SetBounds(btnOK.Left, btnOK.Top, btnOK.Width - 5,
                     btnOK.Height - 5);
}
```

Après de nombreux efforts pour positionner et dimensionner les contrôles, il serait dommage qu'une erreur de manipulation vienne remettre tout en cause. Pour éviter ces soucis, il est possible de verrouiller les contrôles sur la feuille, par le menu *Format/Verrouille les contrôles*. Cette commande bloque le déplacement et le redimensionnement de tous les contrôles présents sur la feuille ainsi que le redimensionnement de la feuille elle-même. Les contrôles peuvent ensuite être déverrouillés par la même option de menu. On peut également déverrouiller les contrôles individuellement par la propriété *Locked*.

Si on conçoit une application dans laquelle l'utilisateur peut redimensionner la fenêtre au moment de l'exécution, les contrôles doivent suivre les modifications de taille de la fenêtre. Pour autoriser le redimensionnement automatique d'un contrôle, on peut utiliser la propriété *Anchor* du contrôle. Par l'intermédiaire de cette propriété, on indique que la distance entre les bords du contrôle et les positions d'ancrage sera conservée lors du redimensionnement de la fenêtre. À la création, les contrôles sont ancrés aux bords haut et gauche de la feuille.

La modification de cette propriété s'effectue par un petit assistant, disponible dans la fenêtre de propriétés.

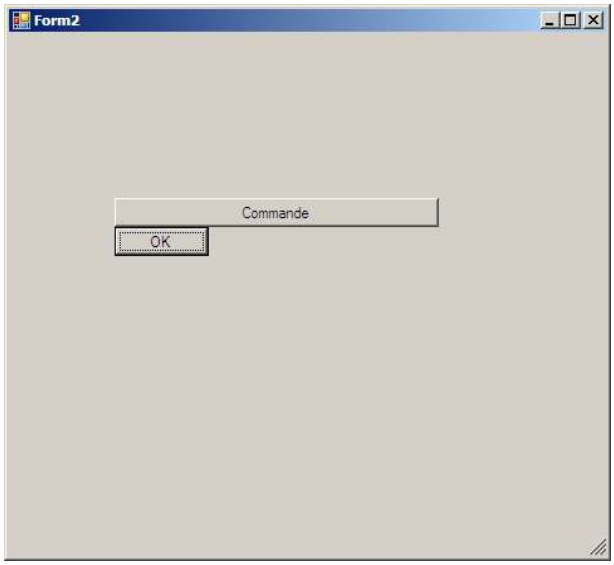


Pour modifier la propriété *Anchor*, il faut sélectionner la branche de l'étoile correspondant au côté avec lequel on veut réaliser un ancrage, ou supprimer un ancrage existant.

Par exemple, pour la fenêtre suivante, les contrôles sont ancrés à gauche et à droite :

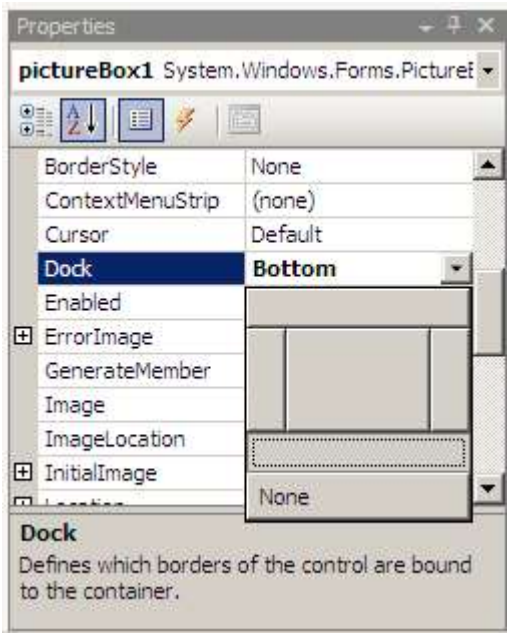


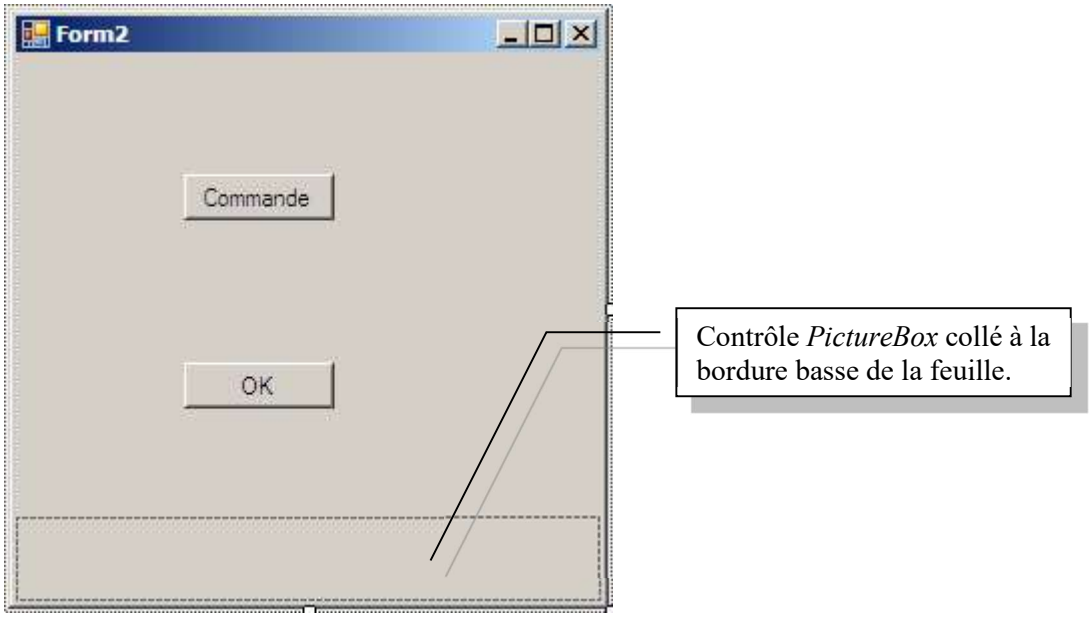
Si on redimensionne la fenêtre, les contrôles suivent l'agrandissement horizontal de la feuille.



On peut également indiquer qu'un contrôle doit adapter une ou plusieurs de ces dimensions à celle de son conteneur. Pour cela, on utilise la propriété *Dock* du contrôle en indiquant sur quelle bordure de son conteneur le contrôle va adapter une de ses dimensions.

Par exemple, on peut placer un contrôle *PictureBox* en demandant à l'amarrer à la bordure basse de la fenêtre.





9.6.3. Passage du focus entre contrôles

Lorsque l'on conçoit une application, on doit penser aux personnes réfractaires à l'utilisation de la souris en leur permettant quand même d'utiliser l'application. Il convient donc de concevoir l'application pour qu'elle puisse fonctionner uniquement en utilisant le clavier.

Dans une application Windows, on dit d'un contrôle qu'il détient le *focus* lorsqu'il est prêt à recevoir la saisie de l'utilisateur. Le focus peut se déplacer de contrôle en contrôle en utilisant la touche [tab]. Deux propriétés des contrôles règlent le passage du focus par la touche [tab] :

- La propriété *TabStop* indique si un contrôle pourra recevoir le focus par l'utilisation de la touche [tab].

- La propriété *TabIndex* indique l'ordre dans lequel le focus sera passé entre les contrôles.

Par défaut, les propriétés *TabIndex* sont numérotées dans l'ordre dans lequel les contrôles sont créés. Pour modifier cet ordre, on peut modifier directement la propriété *TabIndex* de chaque contrôle ou utiliser le menu *Affichage(View)/Ordre de Tabulation(Tab Order)*. Les contrôles sont alors affichés avec, dans leur coin supérieur gauche, la valeur de leur propriété *TabIndex* :



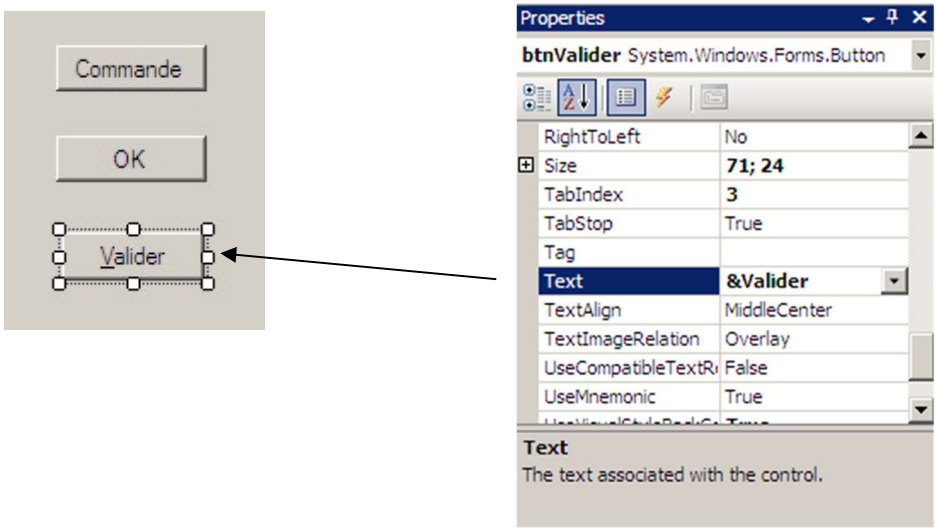
On doit ensuite cliquer sur les contrôles, dans l'ordre dans lequel on veut que le focus soit transféré.

On peut revenir en mode normal en utilisant à nouveau le menu *Affichage/Ordre de Tabulation* ou en utilisant simplement la touche [esc].

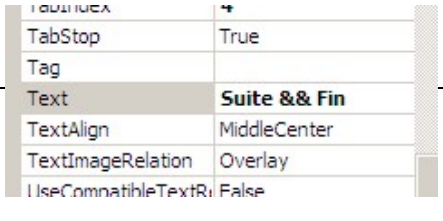
9.6.4. Les raccourcis clavier

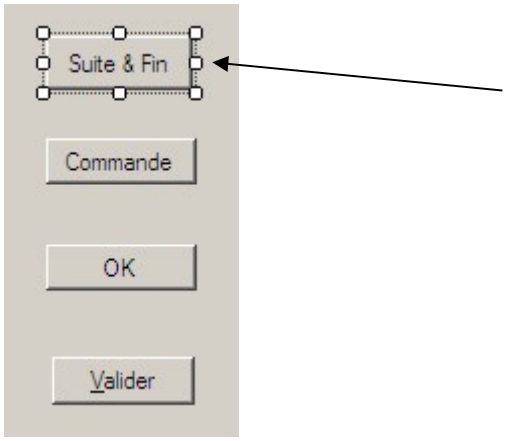
Certains utilisateurs pressés souhaitent pouvoir se déplacer directement sur un contrôle particulier sans avoir à passer le focus sur tous ceux qui le précèdent dans l'ordre des tabulations. On peut, pour cela, ajouter un raccourci-clavier qui sera activé par l'intermédiaire de la touche [alt] et un caractère.

Pour spécifier le caractère à utiliser pour activer le contrôle, il faut l'ajouter dans la propriété *Text* du contrôle avec le caractère '&' placé devant le caractère utilisé pour le raccourci-clavier associé au contrôle. Cela provoque l'activation du raccourci et le soulignement du caractère dans le texte apparaissant sur le contrôle.



Si dans le texte associé au contrôle, on souhaite utiliser le caractère '&' lui-même, alors il faut le doubler à l'intérieur de la zone de texte.





Pour certains contrôles (boutons, case à cocher, …), l'utilisation du raccourci-clavier est équivalent à un clic de la souris et lance l'action correspondante. Pour les autres contrôles, le raccourci-clavier place simplement le focus sur le contrôle correspondant.

Pour les contrôles n'ayant pas de légende, il faudra passer par l'intermédiaire d'un contrôle *label* qui lui servira de légende et activera également le raccourci-clavier. À voir plus tard dans ce chapitre.

9.7. Les Contrôles

Chaque contrôle utilisable dans Visual C# est représenté au moyen d'une classe dans laquelle on va pouvoir créer des instances et cela pour concevoir l'interface de l'application. La majorité des contrôles dérivent de la classe *Control* et de ce fait, héritent d'un bon nombre de propriétés, de méthodes et d'évènements.

Voici la liste des contrôles les plus utilisés.

9.7.1. La classe Control

9.7.1.A. Dimensions et position

Les propriétés *Left, Top, Width* et *Height* permettent le positionnement des contrôles. Ces propriétés peuvent être modifiées individuellement et acceptent des valeurs de type *integer*.

Il est donc possible dans le code d'utiliser la syntaxe suivante :

```
txtBox.Left = 100;
txtBox.Top = 50;
txtBox.Width = 150;
txtBox.Height = 50;
```

Deux autres propriétés permettent de travailler avec la position et la taille d'un contrôle : la propriété *Location* accepte un objet de type *point* grâce auquel on peut spécifier la position du contrôle. De la même manière, la propriété *Size* qui accepte un objet de type *Size*, gère les dimensions du contrôle. Les lignes précédentes peuvent être remplacées par :

```
txtBox.Location = new Point(100, 50);
txtBox.Size = new Size(150, 50);
```

Une troisième possibilité permet de manipuler, à la fois, la position et la taille des contrôles : la propriété *Bounds* attend une instance de la classe *Rectangle* pour définir les caractéristiques du contrôle. Le code peut donc se résumer en une seule ligne :

```
txtBox.Bounds = new Rectangle(100, 50, 150, 50);
```

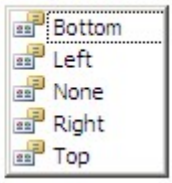
La méthode *SetBounds* permet également de modifier les positions et dimensions des contrôles sans avoir à créer de nouvelle instance de la classe *Rectangle* mais en modifiant celle déjà associée au contrôle.

```
txtBox.SetBounds(100, 50, 150, 50);
```

La modification de ces propriétés entraîne le déclenchement des événements *Resize* et *Move* sur le contrôle. Ces événements sont déclenchés lorsque la valeur des propriétés est modifiée dans le code, mais aussi lorsque, par exemple, la modification de la fenêtre entraîne un repositionnement ou un redimensionnement du contrôle.

Le comportement des contrôles lorsque la fenêtre est redimensionnée est spécifié par les propriétés *Anchor* et *Dock*. Pour les modifier dans par le code, il suffit d'y affecter une des valeurs définies dans les énumérations *AnchorStyle* et *DockStyle*.

```
txtBox.Anchor = AnchorStyles.
```



Jusqu'à présent, les positions avec lesquelles on a travaillé étaient des positions exprimées par rapport au coin supérieur gauche du conteneur du contrôle. Dans certains cas, il peut être utile d'obtenir les coordonnées d'un point du contrôle non pas par rapport au coin supérieur gauche du contrôle mais par rapport au coin supérieur gauche de l'écran. La méthode *PoinToScreen* permet cette conversion. Elle attend comme paramètre un objet de la classe *Point* avec les coordonnées exprimées par rapport au contrôle et renvoie une nouvelle instance de la classe *Point* avec les coordonnées exprimées par rapport à l'écran.

Le code suivant convertit, en coordonnées écran, la position supérieure gauche d'un contrôle de type *TextBox* :

```
private void txtBox_Click(object sender, EventArgs e)
  {
      Console.WriteLine("Contrôle/Fenêtre");
      Console.WriteLine(txtBox.Location);

      Point p = new Point(0, 0);

      p = txtBox.PointToScreen(txtBox.Location);
      Console.WriteLine("Contrôle/Ecran");
      Console.WriteLine(p);
  }
```

```
Contrôle/Fenêtre
{X=180,Y=71}
Contrôle/Ecran
{X=644,Y=351}
```

L'opération inverse peut être réalisée par la méthode *PointToClient* qui elle, prend comme paramètre un point en coordonnées écran et renvoie un point exprimé en coordonnées liées au contrôle. SI l'on effectue l'opération inverse, c'est-à-dire en partant des coordonnées écran, on obtient bien la même valeur :

```
Console.WriteLine("Controle/Fenêtre à partir de l'écran" +
                  txtBox.PointToClient(p).ToString());
```

```
Contrôle/Fenêtre
{X=180,Y=71}
```

```
Contrôle/Ecran
{X=476,Y=277}
Controle/Fenêtre à partir de l'écran{X=180,Y=71}
```

9.7.1.B. Apparence des contrôles

La couleur de fond du contrôle peut être modifiée par la propriété *BackColor* tandis que la couleur du texte du contrôle est modifiée par la propriété *ForeColor*.

On peut affecter à ces propriétés des valeurs définies dans l'espace de nom *System.Drawing.Color* pour obtenir des couleurs prédéfinies :

```
txtBox.BackColor = System.Drawing.Color.Yellow;
```

Il est aussi possible de créer une nouvelle couleur en utilisant la fonction *FromArgb* et en spécifiant comme paramètre la quantité de chacune des couleurs de base (Rouge, Vert, Bleu).

```
txtBox.BackColor = System.Drawing.Color.FromArgb(127, 0, 127);
```

La police est modifiable par la propriété *Font* du contrôle. On peut, pour l'occasion, créer une nouvelle instance de la classe *Font* et l'affecter au contrôle. Il y a treize constructeurs différents pour la classe *Font* donc treize façon différentes de créer une police de caractère. On utilisera la plus simple en indiquant, simplement, le type de police et la taille.

```
txtBox.Font = new Font(System.Drawing.FontFamily.GenericMonospace, 16);
```

Après avoir effectué des modifications sur ces propriétés, il est possible de revenir à une configuration normale en appelant les méthodes *ResetBackColor*, *ResetForeColor, ResetFont*. Les propriétés correspondantes sont réinitialisées avec les valeurs définies pour le conteneur du contrôle.

La propriété *BackgroundImage* permet de spécifier une image qui sera utilisée comme fond pour le contrôle. Si l'image n'est pas assez grande pour recouvrir le contrôle, elle est représentée en mosaïque :

```
btnCmd.BackgroundImage = new Bitmap("C:\\Image.jpg");
```

Pour annuler l'affichage de l'image, il suffit d'assigner *null* à la propriété :

```
btnCmd.BackgroundImage = null;
```

La propriété *Cursor* permet de choisir l'apparence du curseur lorsque la souris se trouve sur la surface du contrôle. Plusieurs curseurs sont prédéfinis dans Windows. !!! page 285

Ces curseurs sont rangés dans une collection *Cursors* et peuvent être utilisés directement en les affectant à la propriété *Cursor* du contrôle :

```
txtBox.Cursor = Cursors.WaitCursor;
```

Si parmi les curseurs, aucun ne convient, on peut utiliser un curseur personnalisé en créant une instance de la classe *Cursor* et en l'affectant à la propriété *Cursor* du contrôle.

```
txtBox.Cursor = new Cursor("xxx.cur");
```

!!! chercher plus d'infos la dessus

La détection de l'entrée et de la sortie de la souris sur le contrôle et la modification du curseur en conséquence est gérée automatiquement par le contrôle lui-même.

Comme pour la police de caractère, il est possible de restaurer le curseur par défaut en appelant la méthode *ResetCursor*.

La modification de la plupart des propriétés des contrôles déclenche un événement. Ces événements sont identifiés par le nom de la propriété suivi du suffixe *Changed*. Ils peuvent être utilisés pour sauvegarder les préférences de l'utilisateur lorsqu'il personnalise l'application.

9.7.1.C. Comportement des contrôles

Les contrôles placés sur une feuille peuvent être masqués en modifiant la propriété *Visible* ou ils peuvent être désactivés en modifiant la propriété *Enabled*. Dans ce cas, le contrôle est toujours visible mais apparaît avec un aspect grisé pour indiquer à l'utilisateur que ce contrôle est inactif pour le moment :



Les contrôles dans cet état ne peuvent bien sûr pas recevoir le focus dans l'application. On peut vérifier cela en examinant la propriété *CanFocus* qui renvoie un booléen. On peut également vérifier si un contrôle détient actuellement le focus, en vérifiant la propriété *Focused* ou la propriété *ContainsFocus*. Cette dernière est à utiliser avec les contrôles conteneurs (c'est-à-dire les contrôles qui peuvent contenir d'autres contrôles), dans ce cas cette propriété est positionnée sur *true* si l'un des contrôles placés à l'intérieur du conteneur a la focus.

Le focus peut être placé sur un contrôle sans l'intervention de l'utilisateur, en appelant la méthode focus du contrôle :

```
btnCmd.Focus();
```

Pour surveiller le passage du focus d'un contrôle à l'autre, quatre événements sont à disposition :

- *Enter* indique que le focus arrive sur un des contrôles du conteneur,
- *GotFocus* indique qu'un contrôle particulier à reçu le focus,
- *LostFocus* indique qu'un contrôle a perdu le focus,
- *Leave* indique que le focus n'est plus sur un des contrôles du conteneur.

Par exemple, pour bien visualiser qu'un contrôle a le focus, on peut utiliser le code suivant qui modifie la couleur du texte lorsque le contrôle reçoit ou perd le focus :

```
this.txtBox.LostFocus += new System.EventHandler(this.txtBox_LostFocus);
this.txtBox.GotFocus += new System.EventHandler(this.txtBox_GotFocus);

private void txtBox_LostFocus(object sender, EventArgs e)
{
     txtBox.ResetForeColor();
}

private void txtBox_GotFocus(object sender, EventArgs e)
{
     txtBox.ForeColor = Color.Green;
}
```

!!! pourquoi les fonctions LostFocus et GotFocus ne sont pas ajoutables via les properties du contrôle ?

Dans certains cas, il est souhaitable de vérifier la saisie de l'utilisateur dans un formulaire avant de continuer l'application. Cette vérification peut être effectuée à la fermeture du formulaire ou au fur et à mesure de la saisie, dans les différents contrôles du formulaire.

Chaque contrôle peut être configuré pour permettre la vérification de la saisie en modifiant la propriété *CausesValidation* sur *True*. Juste avant que le contrôle ne perde le focus, l'événement *Validating* est déclenché pour permettre la vérification de la saisie de l'utilisateur. Si la saisie n'est pas correcte, on peut bloquer le passage du focus vers un autre contrôle en modifiant la propriété *Cancel* de l'objet *CancelEventArg* qui est passé comme paramètre.

Dans ce cas, le focus reste sur le contrôle pour lequel la saisie n'est pas correcte. Par contre, si la saisie est correcte, l'événement *Validated* est déclenché sur le contrôle et le focus se déplace sur le contrôle suivant.

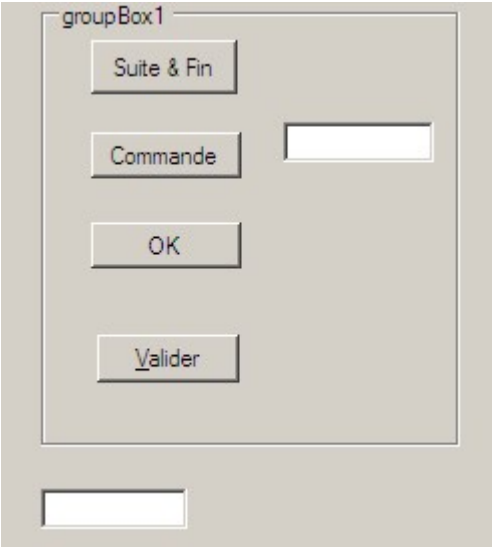
Par exemple, pour saisir un numéro de téléphone, on peut vérifier que seules des valeurs numériques ont été saisies. En cas d'erreur, on génère un beep, on modifie la couleur du texte et on bloque le passage du focus sur un autre contrôle.

```
private void txtNumTel_Validating(object sender, CancelEventArgs e)
{
     bool bResult;
     long ldI;

     bResult = long.TryParse(txtNumTel.Text, out ldI);
     if (!bResult)
     {
         //SystemSounds.Beep.Play();
         txtNumTel.ForeColor = Color.Red;
         e.Cancel = true;
     }
}

private void txtNumTel_Validated(object sender, EventArgs e)
{
     txtNumTel.ResetForeColor();
}
```

Deux propriétés sont parfois utiles lorsque l'on travaille avec des contrôles conteneur. La propriété *HasChildren* permet de savoir si des contrôles sont placés dans le conteneur. Si c'est le cas, la collection *Controls* contient la liste des tous ces contrôles. On peut par exemple modifier la couleur de texte de tous les contrôles d'un conteneur lorsque le focus est placé sur l'un d'entre eux.




```
private void grBox1_Enter(object sender, EventArgs e)
{
     if (grBox1.HasChildren)
     {
         foreach (Control c in grBox1.Controls)
         {
             c.ForeColor = Color.YellowGreen;
         }
     }
}

private void grBox1_Leave(object sender, EventArgs e)
{
     if (grBox1.HasChildren)
     {
         foreach (Control c in grBox1.Controls)
         {
             c.ResetForeColor();
         }
     }
}
```

L'opération inverse est également possible c'est-à-dire qu'à partir d'un contrôle, on peut récupérer les propriétés de son conteneur. La propriété *Parent* fournit une référence vers le conteneur du contrôle. On peut par exemple faire en sorte que la couleur de fond de chaque contrôle change en même temps que celle de son conteneur.

```
private void grBox1_BackColorChanged(object sender, EventArgs e)
{
     foreach (Control c in grBox1.Controls)
     {
         c.BackColor = c.Parent.BackColor;
     }
}
```

Maintenant que l'on a exploré les propriétés communes aux différents contrôles disponibles, on va les étudier un par un afin d'étudier leurs propriétés spécifiques.

9.7.2. Les contrôles d'affichage d'informations

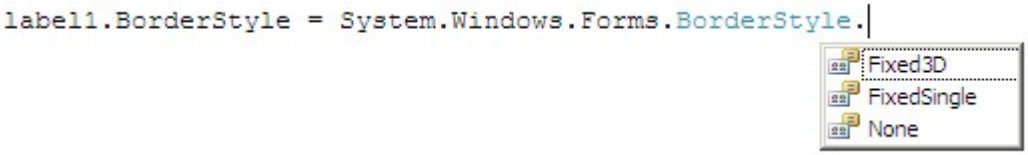
9.7.2.A. Le contrôle Label

Le contrôle *Label* est utilisé pour afficher, sur un formulaire, un texte qui ne sera pas modifiable par l'utilisateur. Il sert essentiellement à fournir une légende à des contrôles qui n'en possèdent pas. Dans ce cas, il permettra également de fournir un raccourci-clavier pour atteindre le contrôle.

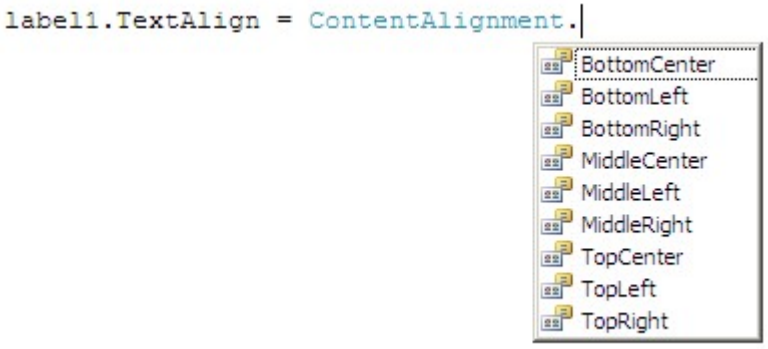
Le texte affiché par le contrôle est indiqué par la propriété *Text*. Cette propriété pourra bien sûr être modifiée par le code de l'application. Il faut cependant être prudent car, par défaut, le contrôle conservera la taille qu'on lui a donnée à la conception. Si la nouvelle chaîne de caractères affectée à la propriété *Text* est plus longue que celle spécifiée au moment de la conception, seul le début du texte sera visible.

Pour éviter ce problème, il faut demander au contrôle *Label* d'adapter sa largeur en fonction du texte à afficher, en modifiant la propriété *AutoSize* sur *True*.

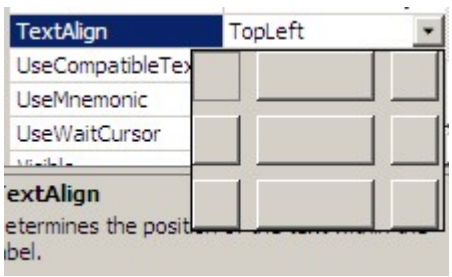
Par défaut, le contrôle *Label* n'a pas de bordure. On peut en ajouter une en modifiant la propriété *BorderStyle*, en utilisant une des trois valeurs disponibles :



On a aussi la possibilité d'indiquer la position du texte dans le contrôle par l'intermédiaire de la propriété *TextAlign*. On utilisera une des constantes prédéfinies :



Par la fenêtre de propriété, il suffit de cliquer sur la position désirée pour le texte à l'intérieur du contrôle :



À noter cependant que la propriété *textAlign* modifiera la position du texte uniquement si la propriété *AutoSize* est positionnée sur *False*.

Les contrôles *Label* peuvent également afficher des images. On peut indiquer l'image à afficher à l'aide de la propriété *Image*. Une autre solution consiste à utiliser un contrôle *ImageList* qui servira, en quelque sorte, de stockage pour les images de l'application. Dans ce cas, on indique par l'intermédiaire de la propriété *ImageList*, dans quel contrôle on va chercher l'image, et par la propriété *ImageIndex* à quel endroit elle se trouve dans le contrôle *ImageList*.

Si on utilise un contrôle *ImageList*, la propriété *Image* du contrôle sera ignorée. Comme pour le texte, on peut modifier la position de l'image dans le contrôle par la propriété *ImageAlign* avec les mêmes constantes que pour la propriété *TextAlign*.

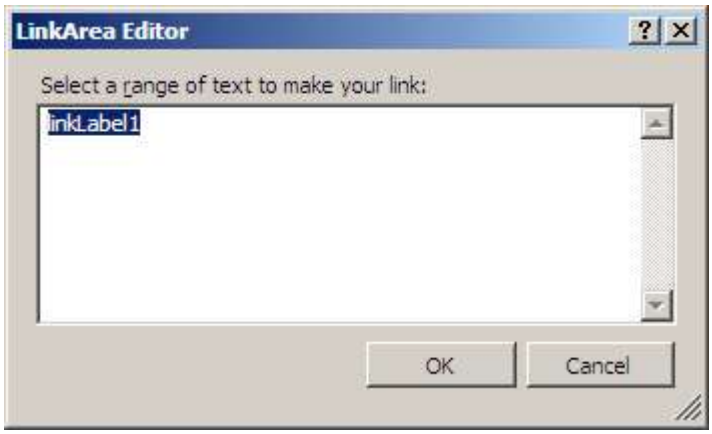
On a indiqué que le contrôle *Label* pouvait être utilisé comme raccourci-clavier pour un autre contrôle. Pour cela, trois précautions sont à prendre :

- Comme pour les autres contrôles, il faut ajouter un '&' dans la propriété *Text* pour le caractère utilisé comme raccourci.
- Indiquer au contrôle *Label* son rôle de gestionnaire de raccourci-clavier en modifiant la propriété *UseMnemonic* sur *True*.
- Vérifier que le contrôle, qui doit recevoir le focus, est immédiatement après le contrôle *Label* dans l'ordre des tabulations (propriété *TabIndex*).

9.7.2.B. Le contrôle LinkLabel

Le contrôle *LinkLabel* hérite de toutes les caractéristiques du contrôle *Label* et ajoute simplement des fonctionnalités de lien style *web*. Les propriétés supplémentaires par rapport au contrôle *Label* gèrent les différents paramètres du lien.

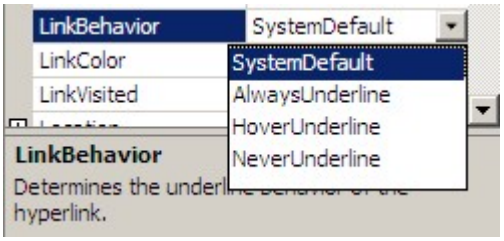
La propriété *LinkArea* indique quelle portion du texte activera le lien. Cette propriété peut être modifiée, par l'intermédiaire de la fenêtre de propriétés, avec un petit utilitaire dans lequel on doit spécifier la portion de texte constituant le lien.



Les couleurs utilisées pour le lien sont modifiables par trois propriétés :

- *LinkColor* : couleur du lien à l'état normal.
- *VisitedLinkColor* : couleur du lien après une première utilisation.
- *ActiveLinkColor* : couleur du lien au moment où l'on clique dessus.

L'apparence du lien est modifiable par la propriété *LinkBehavior*.



Les quatre valeurs possibles permettent respectivement :

- D'utiliser la même configuration, pour les liens, que le navigateur,
- D'avoir les liens toujours soulignés,
- D'avoir les liens soulignés lorsque la souris les survole,
- De ne jamais avoir les liens soulignés.

Lorsque l'utilisateur clique sur le lien, l'événement *LinkClicked* est déclenché dans l'application. On doit modifier la propriété *LinkVisited* en la positionnant sur *True* pour indiquer que ce lien a déjà été utilisé dans l'application. L'action exécutée peut être l'ouverture d'un site Web dans le navigateur par défaut, comme dans l'exemple suivant :

```
private void lnkLabel1_LinkClicked(object sender,
                                   LinkLabelLinkClickedEventArgs e)
{
     System.Diagnostics.Process.Start("http://microsoft.com");
     lnkLabel1.LinkVisited = true;
}
```

9.7.2.C. Le contrôle StatusStrip

Le contrôle *StatusStrip* est généralement utilisé pour présenter des informations à l'utilisateur, concernant le fonctionnement de l'application. Les informations peuvent être affichées sous forme de texte, de barre de progression, de menu ou de bouton de commande associé à un menu. Un éditeur spécifique accessible par la propriété *Items* du contrôle permet sa configuration.

Chaque élément ajouté sur le contrôle *StatusStrip* doit ensuite être configuré individuellement. Les propriétés des éléments pouvant être utilisés pour la construction d'un *StatusStrip* sont très semblables à celles des contrôles normaux. Par exemple, l'élément *ToolStripStatusLabel* est pratiquement identique au contrôle *LinkLabel*.

!!! ajouter plus d'infos

9.7.2.D. Le contrôle ToolTip

Ce contrôle permet l'affichage d'une bulle d'aide associée à un contrôle. Ce contrôle n'a pas d'interface visible. Il sera donc placé dans la zone située en dessous de la fenêtre de conception. Il effectue beaucoup de travail sans aucun effort de programmation. Il surveille, par exemple, en permanence où se trouve la souris. Si celle-ci est sur un contrôle, il vérifie s'il y a une info-bulle associée au contrôle et si c'est le cas, il affiche l'info-bulle pendant la durée spécifiée par la propriété *AutoPopDelay*.

Pour pouvoir fonctionner, le contrôle *ToolTip* doit associer une chaîne de caractères à chacun des contrôles de l'interface. Pour cela, dés qu'un contrôle *ToolTip* est disponible sur une feuille, une propriété *ToolTip* est ajoutée à chacun des contrôles, permettant ainsi de spécifier le texte de l'info-bulle associée au contrôle.

Les chaînes de caractères associées à chaque contrôle peuvent également être indiquées par l'intermédiaire du code, en appelant la méthode *SetToolTip* et en y indiquant comme paramètre, le nom du contrôle et la chaîne de caractères, qui lui est associée.

```
toolTip1.SetToolTip(txtBox, "Nouveau help pour ce contrôle");
```

Cette technique permet de conserver des légendes relativement courtes pour les contrôles tout en fournissant suffisamment d'informations sur l'utilisation de l'application.

9.7.2.E. Le contrôle ErrorProvider

Ce contrôle permet d'indiquer facilement à l'utilisateur des problèmes sur les données qu'il a saisies, sur un formulaire. En général, il intervient lors de la phase de validation des données du formulaire, en affichant en face de chaque contrôle une petite icône afin d'attirer l'attention de l'utilisateur. 

Des informations supplémentaires peuvent être fournies par une info-bulle associée au contrôle *ErrorProvider*. Un même contrôle *ErrorProvider* peut être utilisé pour tous les contrôles d'un formulaire.

L'activation du contrôle *ErrorProvider* peut s'effectuer à la fermeture du formulaire lorsque l'utilisateur clique sur le bouton *OK*, mais il est également possible de surveiller la saisie au fur et à mesure où elle est effectuée en gérant par exemple les événements *Validating*. Cet événement est déclenché par un contrôle au moment où celui-ci perd le focus. On peut, dans ce cas, vérifier immédiatement la valeur saisie dans le contrôle et réagir en conséquence en affichant le contrôle *ErrorProvider*. Pour cela, on appelle la méthode *SetError* en spécifiant le nom du contrôle qui pose problème et la chaîne de caractères affichée dans l'info-bulle associée au contrôle.

S'il n'y a pas d'erreur, il faut réinitialiser la chaîne pour faire disparaître l'icône du contrôle *ErrorProvider*.

```
private void txtNumTel_Validating(object sender, CancelEventArgs e)
{
     bool bResult;
     long ldI;

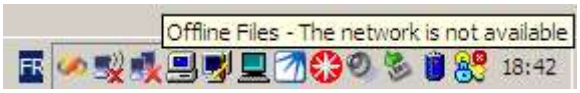
     bResult = long.TryParse(txtNumTel.Text, out ldI);
     if (!bResult)
     {
          //SystemSounds.Beep.Play();
          txtNumTel.ForeColor = Color.Red;
          e.Cancel = true;
          errorProvider1.SetError(txtNumTel, "valeur numérique obligatoire");
     }
}

private void txtNumTel_Validated(object sender, EventArgs e)
{
     txtNumTel.ResetForeColor();
     errorProvider1.SetError(txtNumTel, "");
}
```

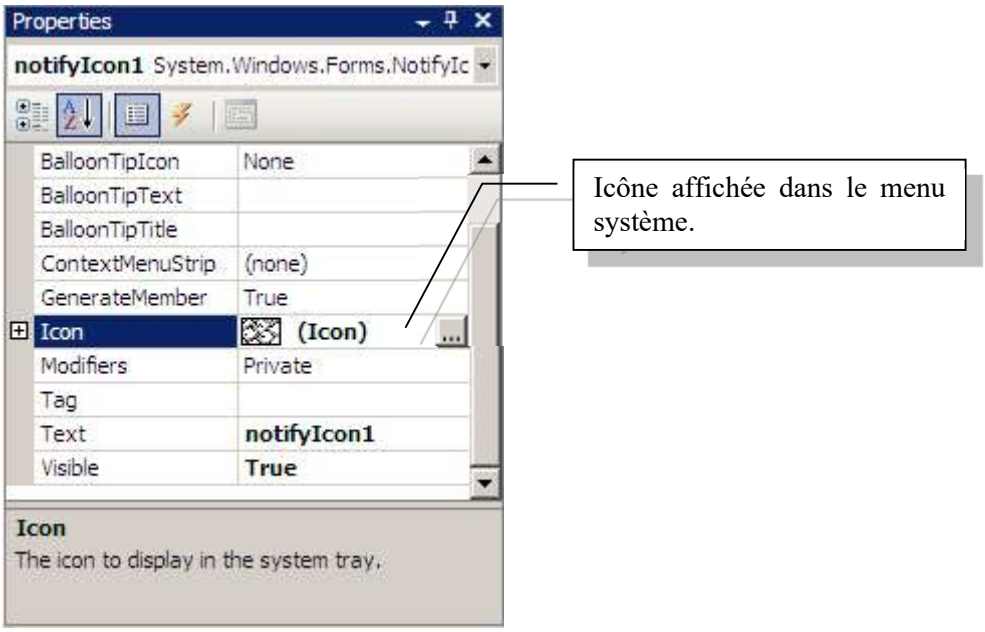


9.7.2.F. Le contrôle NotifyIcon

Ce contrôle est principalement utilisé pour afficher des informations sur le fonctionnement d'un processus s'exécutant en tâche de fond dans l'application. Il est affiché dans la zone de statut du système. La propriété *Icon* du contrôle détermine l'icône affichée. La propriété *Text* représente la légende affichée lorsque la souris survole le contrôle.



En gérant l'événement *DoubleClick* du contrôle, on peut afficher une boîte de dialogue permettant la configuration du processus associé au contrôle.



Fonction exécutée quand on double-clique sur l'icône :

```
private void notifyIcon1_DoubleClick(object sender, EventArgs e)
{

}
```

Il est également possible d'associer un menu contextuel, en renseignant la propriété *ContextMenuStrip*. Ce menu peut, par exemple, contrôler le fonctionnement du processus auquel est associé le contrôle.

9.7.2.G. Le contrôle HelpProvider

Le contrôle *HelpProvider* assure la liaison entre un fichier d'aide et l'application. Le fichier d'aide doit être généré par l'outil *Html Help Workstation*, disponible en téléchargement sur le site de Microsoft. On utilisera ici un fichier existant sur le système : le fichier C:\Windows\Help\Charmap.chm correspondant à l'utilitaire Table des Caractères.

Ce fichier doit être associé au contrôle par la propriété *HelpNamespace*. La présence d'un contrôle *HelpProvider* sur une fenêtre, ajoute automatiquement trois propriétés à chaque contrôle présent sur la fenêtre :

- *HelpKeyword* : indique le mot clé associé au contrôle dans le fichier d'aide.
- *HelpNavigator* : indique l'action exécutée lors de l'affichage de l'aide.
- *HelpString* : contient la chaîne de caractère affichée lors de l'utilisation du bouton  d'une boîte de dialogue. Pour que ce bouton soit disponible sur la boîte de dialogue, il faut modifier la propriété *HelpButton* de la fenêtre sur *True* et masquer les boutons d'agrandissement et de réduction de la fenêtre en modifiant les propriétés *MaximizeBox* et *MinimizeBox* sur *False*.

9.7.2.H. Le contrôle ProgressBar

Ce contrôle est utilisé pour informer l'utilisateur sur la progression d'une action lancée dans l'application. Il affiche cette information sous la forme d'une zone rectangulaire qui sera plus ou moins remplie selon l'état d'avancement de l'action exécutée. L'aspect de la *ProgressBar* est contrôlé par sa propriété *Style*. Trois valeurs sont possibles :

- *Continuous* : la progression est affichée par une barre bleue pleine.
- *Blocks* : la progression est affichée par une série de petits rectangles.
- *Marquee* : cette présentation est identique à la précédente avec, en plus, un défilement à l'intérieur de la ProgressBar.

La position de la barre de progression est contrôlée par la propriété *Value*. Cette propriété peut évoluer entre les deux extrêmes indiqués par les propriétés *Minimum* et *Maximum*.

Trois techniques sont disponibles pour faire évoluer la barre de progression :

- Modifier directement la propriété *Value* du contrôle. À noter que dans ce cas, si la valeur de cette propriété dépasse les extrêmes, une exception est déclenchée.
- Utiliser la méthode *PerformStep* qui incrémente à chaque appel la propriété *Value* de la valeur contenue dans la propriété *Step*. Le contrôle vérifie, dans ce cas, la valeur contenue dans la propriété *Value* et s'assure qu'elle ne dépasse pas les extrêmes.
- Utiliser la méthode *Increment* en indiquant comme paramètre, la valeur utilisée comme incrément pour la propriété *Value*. La valeur de la propriété *Value* est également vérifiée lors de l'exécution de cette méthode.

Si la *ProgressBar* a le style *Marquee*, la propriété *Value* n'a aucun effet sur la taille de la barre de progression et les méthodes *PerformStep* et *Increment* ne doivent pas être utilisées, sinon une exception est déclenchée. La réinitialisation du contenu de la *ProgressBar* se fait au moyen de l'expression suivante :

```
prbDelProp.Value = 0;
```

L'exemple suivant présente une horloge originale où l'heure est affichée par trois *ProgressBar* :

Création d'un contrôle *Timer*:




```
public partial class Horloge : Form
{
     public Horloge()
     {
           InitializeComponent();
           pgbHeure.Minimum = 0;
           pgbHeure.Maximum = 23;
           pgbHeure.Style = ProgressBarStyle.Continuous;

           pgbMinute.Minimum = 0;
           pgbMinute.Maximum = 59;
           pgbMinute.Style = ProgressBarStyle.Continuous;

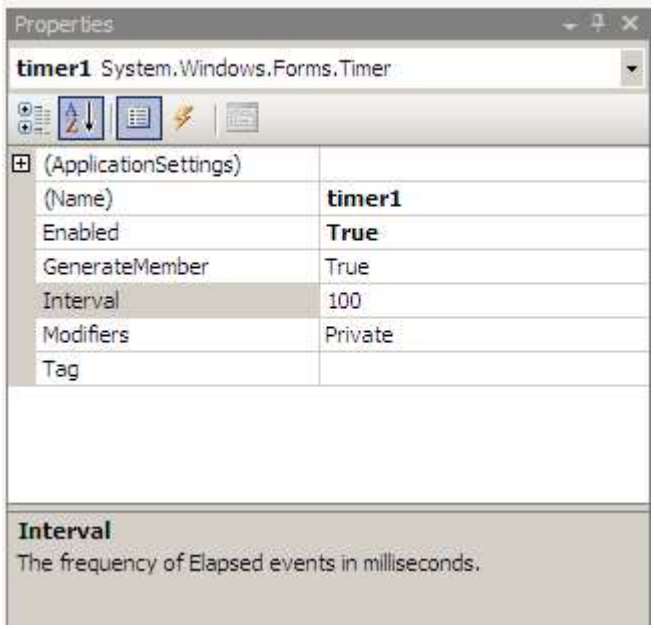
           pgbSeconde.Minimum = 0;
           pgbSeconde.Maximum = 59;
           pgbSeconde.Style = ProgressBarStyle.Continuous;

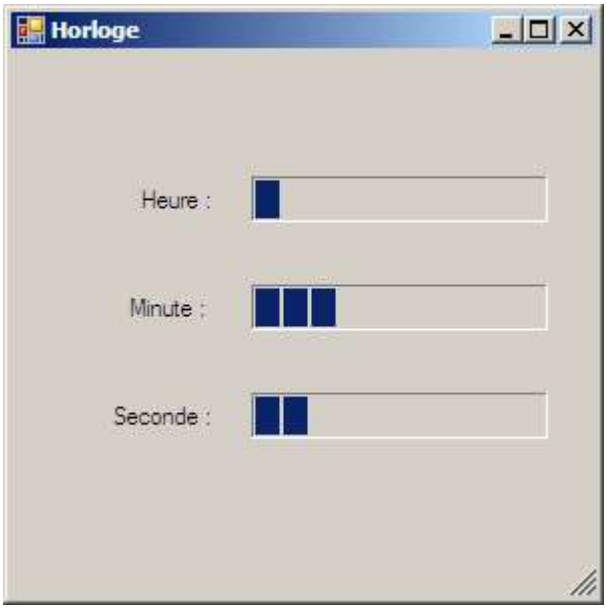
           timer1.Interval = 10;
           timer1.Enabled = true;
```

```
        }

        private void timer1_Tick(object sender, EventArgs e)
        {
                pgbHeure.Value = DateTime.Now.Hour;
                pgbMinute.Value = DateTime.Now.Minute;
                pgbSeconde.Value = DateTime.Now.Second;
        }
}
```

Activation du contrôle *Timer* :





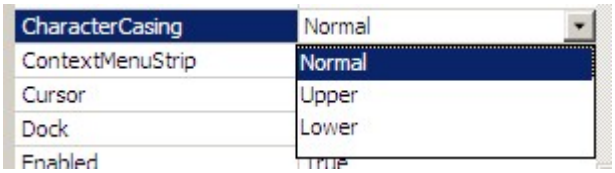
9.7.3. Les contrôles d'édition de texte

9.7.3.A. Le contrôle TextBox

Le contrôle *TextBox* est utilisé pour permettre à l'utilisateur de saisir des informations. Le contrôle peut être configuré pour saisir du texte sur une ou plusieurs lignes. La taille maximale du texte varie de 2000 à 32 caractères, suivant la configuration du contrôle (simple ligne ou multi-lignes). Le contrôle est également capable de gérer la sélection de texte et les opérations avec le presse papier.

De nombreuses propriétés et méthodes sont disponibles pour travailler avec ce contrôle. Le texte affiché dans le contrôle peut être modifié ou récupéré par la propriété *Text*. Le format d'affichage du texte est modifiable par différentes propriétés.

La propriété *AutoSize* permet de demander au contrôle *TextBox* de se redimensionner, en fonction de la taille de la police de caractères. Cette propriété est presque toujours positionnée sur *True*. La propriété *CharacterCasing* autorise le contrôle à modifier tous les caractères saisis soit en minuscules, soit en majuscules.



La propriété *Lines* permet de récupérer le texte saisi, ligne par ligne. Cette propriété est un tableau de chaînes de caractères qui contient autant de cases qu'il y a de lignes. Elle n'a d'intérêt que si le contrôle est configuré pour accepter la saisie sur plusieurs lignes avec la propriété *Multiline* positionnée sur *True*.

Dans ce cas, il faut également prévoir la possibilité de faire défiler le texte, en ajoutant des barres de défilement avec la propriété *ScrollBars*. Les différentes possibilités permettront d'avoir une barre de défilement horizontale, verticale ou les deux. Attention cependant car la barre de défilement verticale ne sera visible que si la propriété *WordWrap* est positionnée sur *False* sinon le contrôle gère lui-même le retour à la ligne lorsque la longueur de la ligne dépasse la longueur du contrôle. Par contre, dans ce cas, les retours chariot ajoutés automatiquement ne sont pas insérés dans le texte.

La longueur maximale du texte du contrôle est fixée par la propriété *MaxLength*. À noter que dans le cas d'un contrôle multi-ligne, les caractères retour chariot et saut de ligne sont également comptés. Cette propriété est fréquemment utilisée lorsque l'on utilise *TextBox* pour la saisie d'un mot de passe. Dans ce cas, la propriété *PasswordChar* indique le caractère utilisé à l'affichage, pour masquer la saisie de l'utilisateur. En général, on utilise le caractère '*' ou '#'. Cette propriété n'influence bien sûr que l'affichage, et les caractères saisis par l'utilisateur sont toujours récupérés dans la propriété *Text*.

La gestion de la sélection du texte se fait automatiquement par le contrôle. La propriété *SelectText* permet de récupérer la chaîne de caractères actuellement sélectionnée dans le contrôle. Les propriétés *SelectionStart* et *SelectionLength* indiquent respectivement le caractère de début de la sélection (le premier caractère à l'indice 0) et le nombre de caractères de la sélection. Ces propriétés sont également utilisées pour insérer du texte dans le contrôle : la propriété *SelectionStart* indique, dans ce cas, le point d'insertion et la propriété *SelectedText*, le texte à insérer. Pour l'ajout de texte à la suite de celui déjà présent dans le contrôle, il est plus pratique d'utiliser la méthode *AppendText* en passant comme paramètre la chaîne de caractères à ajouter.

Le remplacement d'une portion de texte dans le contrôle *TextBox* se fait en deux étapes. Il faut d'abord sélectionner le texte que l'on veut remplacer à l'aide des propriétés *SelectionStart* et *SelectionLength*, pour indiquer le texte de remplacement avec la propriété *SelectedText*. Le texte remplacé et le texte de remplacement ne doivent pas forcément avoir la même taille. La sélection de texte peut également s'effectuer avec la méthode *Select*, en indiquant le caractère de début de la sélection et le nombre de caractères de la sélection.

La sélection de la totalité du texte peut être effectuée avec la méthode *SelectAll*. Par exemple, on peut forcer la sélection de tout le texte lorsque le contrôle reçoit le focus.

De manière classique, dés qu'un contrôle perd le focus, la sélection du texte qu'il y avait à l'intérieur du contrôle n'est plus visible. La propriété *HideSelection* positionnée sur *False* permet de conserver la sélection visible, même si le contrôle n'a plus le focus.

Pour la gestion du presse-papiers, le contrôle *TextBox* dispose d'un menu contextuel permettant d'effectuer les opérations courantes. On a cependant la possibilité d'appeler les méthodes *copy, cut* and *paste* pour gérer les opérations de copier-coller d'une autre manière, par exemple un menu de l'application.

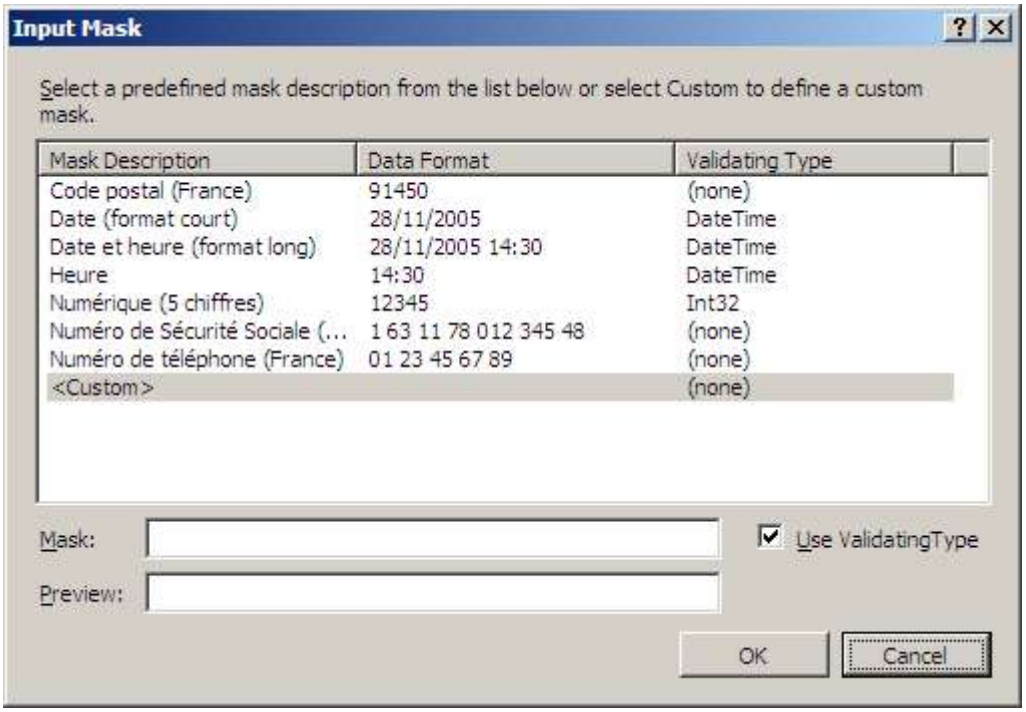
Les opérations couper et coller ne seront cependant pas possibles si le contrôle *TextBox* est configuré en lecture seule avec la propriété *ReadOnly* positionnée sur *True*, la modification du texte par l'utilisateur est bien dans ce cas impossible.

Tout le monde ayant droit à l'erreur, le contrôle *TextBox* propose la méthode *Undo* permettant d'annuler la dernière modification de texte effectuée sur le contrôle. Cette méthode est déjà utilisable par l'option *Annuler* du menu contextuel du contrôle *TextBox* ou par le raccourci clavier [ctrl][z]. elle peut bien sûr être appelée par un autre menu de l'application. Il n'y a qu'un seul niveau d'*Undo*.

Est également disponible sur ce contrôle, l'événement *TextChanged* qui se produit lorsque la propriété *Text* du contrôle est modifiée par le code de l'application ou par l'utilisateur;

9.7.3.B. Le contrôle MaskedTextBox

Ce contrôle est une amélioration du contrôle *TextBox* car il permet de vérifier automatiquement que les informations saisies correspondent à ce qui est attendu par l'application. La propriété *Mask* détermine le format des informations pouvant être saisies dans le contrôle. L'éditeur accessible par la fenêtre de propriétés, permet de choisir un masque existant ou de configurer un masque spécifique.



Pour la propriété *Mask*, certains caractères ont une signification particulière :

Caractère	Signification
0	Représente un chiffre obligatoire (0 à 9)
9	Représente un chiffre ou un espace optionnel
L	Représente une lettre obligatoire (de a à z ou A à Z)
?	Représente une lettre optionnelle
C	Représente un caractère quelconque
.	Représente le séparateur décimal
,	Représente le séparateur des milliers
:	Représente le séparateur horaire
/	Représente le séparateur de dates
$	Représente le symbole monétaire
<	Les caractères suivants seront transformés en minuscules
>	Les caractères suivants seront transformés en majuscules
|	Annule l'effet des deux caractères > et <
\	Caractère d'échappement faisant perdre sa signification spéciale au caractère suivant
-	Tous les autres caractères sont affichés tels quels dans le contrôle

Table 20: caractères de formatage utilisables dans le contrôle de type MaskedTextBox

Le masque suivant peut par exemple être utilisé pour introduire une adresse IP :

```
000\.000\.000\.000
```

9.7.3.C. Le contrôle RichTextBox

Le contrôle *RichTextBox* permet l'affichage, la saisie et la manipulation de texte avec mise en forme. Il possède les mêmes fonctionnalités que le contrôle *TextBox* mais il est capable de gérer des polices de caractères différentes, des images,… Il propose en fait toutes les fonctions de base d'une application de traitement de texte.

Chargement et enregistrement de fichier

Les méthodes *LoadFile* et *SaveFile* permettent le chargement et l'enregistrement depuis ou vers un fichier. Le seul paramètre obligatoire pour ces deux fonctions représente le chemin complet vers le fichier à charger ou à sauvegarder. Le format de fichier utilisé par défaut par ces deux fonctions est le format *rtf* (Rich Text Format).

Si d'autres formats de fichier doivent être utilisés, on doit alors le spécifier par un deuxième paramètre qui est une constante de l'énumération *RichTextBoxStreamType*. Dans le cas d'une lecture de fichier, il est important que les informations contenues dans le fichier soient en accord avec la constante utilisée.

Par exemple, la lecture d'un fichier texte normal avec la ligne de code suivante va provoquer une exception:

```
rtbTextBox1.LoadFile("string", RichTextBoxStreamType.RichText);
```

Par contre, il n'y a pas de problème pour l'enregistrement car c'est le contrôle *RichTextBox* qui gère lui-même le format des informations inscrites dans le fichier. Le seul risque est de perdre les informations inscrites dans le fichier. Le seul risque est de perdre les informations de mise en page contenues dans le contrôle si un format d'enregistrement, inadapté est utilisé.

Par exemple,, la ligne suivante fera perdre toutes les informations de formatage lors de l'enregistrement du fichier.

```
rtbTextBox1.LoadFile("string", RichTextBoxStreamType.PlainText);
```

Ajout de texte

Le contrôle *RichTextBox* gère son contenu grâce à deux propriétés :

- La propriété *Text* qui, comme pour les autres contrôles, définit les informations affichées dans le contrôle. C'est le texte brut qui est stocké dans cette propriété.
- La propriété *Rtf* contient elle aussi le texte affiché par le contrôle mais elle contient en plus les informations concernant la présentation du texte.

Pour ajouter du texte à un contrôle, on peut définir la propriété *Text* ou la propriété *Rtf*. Dans ce second cas, il faut également inclure les caractères de formatage utilisés par le contrôle pour la mise en forme du texte. Les codes *Rtf* étant loin d'être simples, cette solution est rarement utilisée. Une solution plus souple existe.

La méthode *AppendText* permet d'ajouter une chaîne de caractères au contenu du contrôle *RichTextBox*. Le formatage de la chaîne à ajouter doit être défini au préalable par l'intermédiaire des propriétés de la sélection.

La propriété *SelectFont* détermine la police utilisée pour l'affichage du texte ajouté. Pour ajouter du texte en gras et souligné, il faut donc modifier les caractéristiques de la police. Plus exactement, il faut créer la police désirée car il n'est pas possible de modifier les caractéristiques d'une police existante puisque ses propriétés sont en lecture seule.

```
rtbTextBox1.SelectionFont.Bold = true;
```
Property or indexer 'System.Drawing.Font.Bold' cannot be assigned to -- it is read only

Par contre la syntaxe suivante est correcte pour modifier les caractéristiques de la police existante :

```
rtbTextBox1.SelectionFont = new Font(rtbTextBox1.SelectionFont,
                                     FontStyle.Bold |
                                     FontStyle.Underline);
```

Les couleurs du texte et du fond sont déterminées par les propriétés *SelectionColor* et *SelectionBackColor*.

```
rtbTextBox1.SelectionBackColor = Color.Beige;
rtbTextBox1.SelectionBackColor = Color.Red;
```

L'alignement du texte est représenté par la propriété *SelectionAlignment* à laquelle on doit affecter une des valeurs de l'énumération *HorizontalAlignment* :

```
rtbTextBox1.SelectionAlignment = HorizontalAlignment.Center;
```

9.7.4. Les contrôles de déclenchement d'actions

9.7.4.A. Le contrôle Button

Le contrôle *Button* est principalement utilisé, dans une application pour lancer l'exécution d'une action. Cette action peut être l'exécution d'une portion de code ou la fermeture d'une boîte de dialogue. Comme pour les contrôles vus jusqu'à présent, le libellé du bouton est modifiable par la propriété *Text* du contrôle. Cette propriété peut contenir un '&' pour créer un raccourci-clavier.

Lorsque le bouton est utilisé pour la fermeture d'une boîte de dialogue, on doit indiquer par l'intermédiaire de la propriété *DialogResult*, la valeur qui sera renvoyée lorsque la boîte de dialogue sera fermée par ce bouton. Les valeurs utilisables pour cette propriété, correspondent aux boutons standards rencontrés sous Windows.

Lorsque l'on crée une boîte de dialogue, il y a généralement au moins un bouton de validation et un bouton d'annulation. On peut indiquer que, pour cette boîte de dialogue, l'utilisation de la touche [enter] sera équivalente à un clic sur le bouton de validation. L'utilisation de la touche [esc] sera équivalente à l'utilisation du bouton d'annulation. Il convient pour cela, d'indiquer dans la propriété *AcceptButton* de la fenêtre, le nom du contrôle correspondant au bouton de validation et dans la propriété *CancelButton*, le nom du contrôle utilisé pour l'annulation.

9.7.4.B. Le contrôle MenuStrip

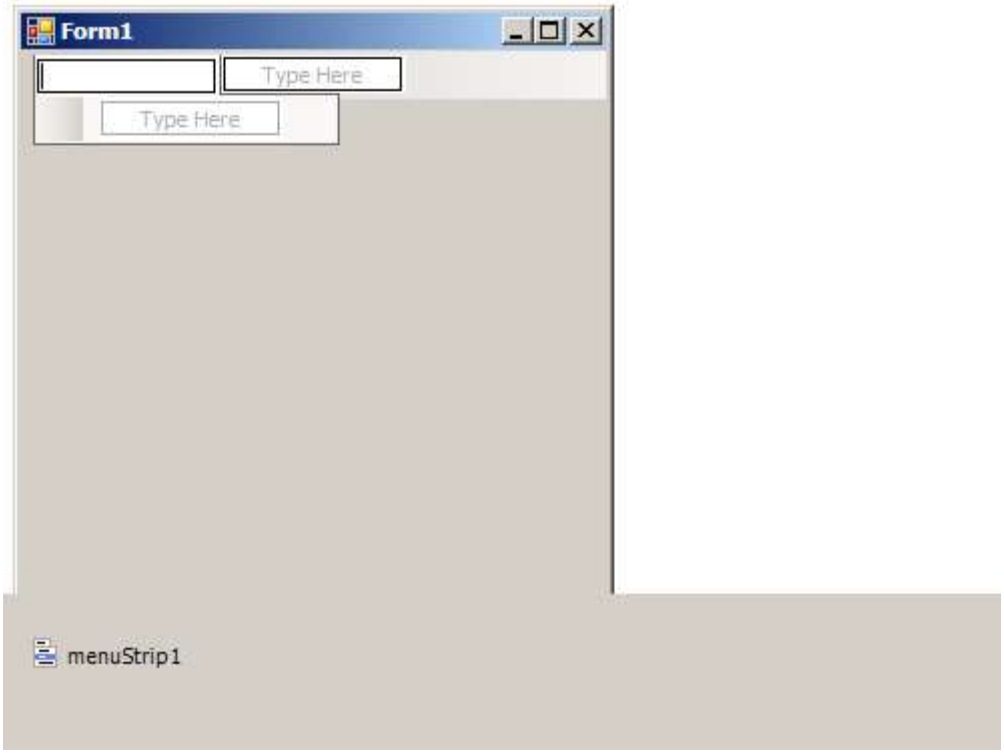
La majorité des applicatons disposent d'une barre de menu. Par l'intermédiaire de ces menus, l'utilisateur pourra activer les différentes fonctionnalités de l'application. Les menus sont constitués d'éléments de menus permettant le lancement d'actions dans l'application ou l'apparition d'un sous-menu. Dans ce cas, un petit triangle à côté du libellé de menu prévient l'utilisateur de la présence d'un sous-menu.

Les menus peuvent également servir à activer ou non une option, dans ce cas, un petit repère est affiché devant le libellé du menu pour marquer l'activation de l'option. Pour les inconditionnels du clavier, il est également possible d'associer à un menu une combinaison de touches équivalentes à un clic sur le menu.

La première chose à faire est d'insérer un contrôle *MenuStrip* sur la feuille. Ce contrôle se place à deux endroits :

- Dans la zone réservée aux contrôles invisibles,
- Directement sur la fenêtre, à la place d'une barre de menu standard.

C'est à partir de cet endroit que l'on va pouvoir composer le menu. Le concepteur de menu propose les différentes zones dans lesquelles on peut saisir les informations constituant le menu.



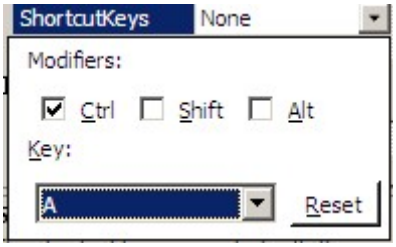
On peut ainsi composer complètement la barre de menu et visualiser son apparence au fur et à mesure de sa conception.

À chaque fois que l'on insère un élément, le concepteur crée une nouvelle instance de la classe *ToolStripMenuItem* et l'insère dans la collection *Items* pour les titres de menus et dans la collection *DropDownItems* pour les éléments de menu et de sous-menu.

Configuration des propriétés de chacun des éléments de manu :

- *Text* : contient le libellé du menu.
- *Visible* : permet de masquer un élément de menu.
- *Enabled* : permet d'interdire l'utilisation de l'élément. Il apparaît alors en grisé dans le menu.
- *Checked* : indique si l'élément de menu est coché ou non.
- *CheckOnClick* : permet de changer automatiquement l'état du menu à chaque clic; la valeur de la propriété *Checked* est, dans ce cas, inversée à chaque clic sur le menu.
- *ShortcutKeys* : définit la combinaison de touches équivalentes à un clic sur le menu.

Cette propriété est facilement modifiable par la fenêtre de propriétés :



La propriété *ShowShortcutKeys* autorise l'affichage du raccourci à côté du libellé du menu.

Ces propriétés sont aussi modifiables par le code :

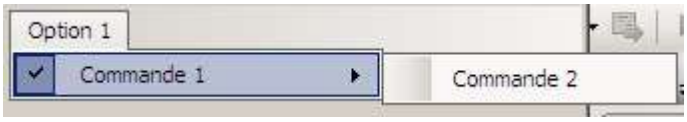
```
private void cmdToolStripMenuItem_Click(object sender, EventArgs e)
{
     cmdToolStripMenuItem.Checked = true;
}
```



Dans certains cas, il est nécessaire d'ajouter dynamiquement des éléments pendant le fonctionnement de l'application pour ajouter par exemple la liste des derniers fichiers utilisés dans l'application.

La solution consiste à ajouter, à la collection *DropDownItems*, un nouvel élément en utilisant la méthode *Add*. La commande ajoutée le sera comme sous-menu de l'option de menu sur laquelle la commande s'exécute :

```
cmdToolStripMenuItem.DropDownItems.Add("Commande 2");
```



Cette opération ajoute bien un élément au menu, mais il ne sera pas d'une grande utilité. Il est en effet impossible de récupérer les événements *Click* sur ce menu.

Pour résoudre ce problème, on doit utiliser une syntaxe différente de la méthode *Add* permettant d'associer un gestionnaire d'événement au nouvel élément ajouté. Il faut d'abord créer la procédure qui sera utilisée pour la gestion de cet événement :

```
cmdToolStripMenuItem.DropDownItems.Add("Commande 2", null,
                                       cmdCommande2_Click);

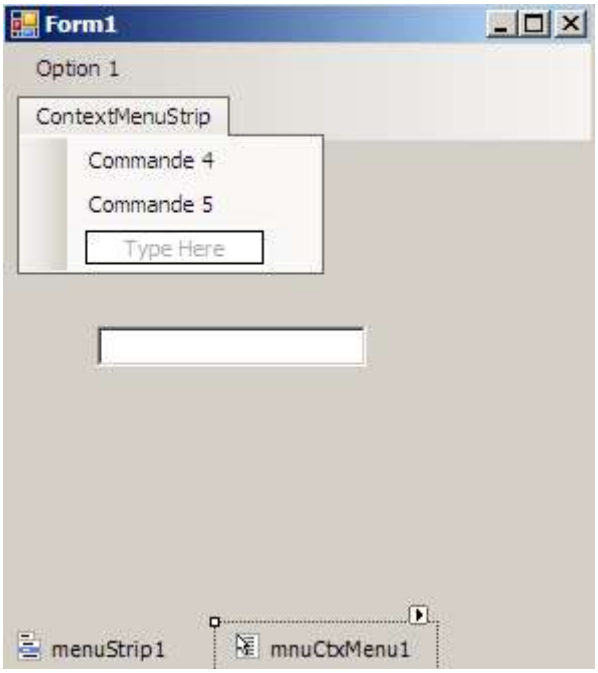

 private void cmdCommande2_Click(object sender, EventArgs e)
 {

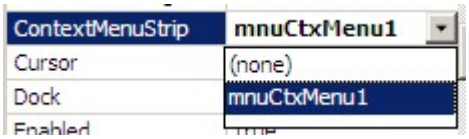
 }
```

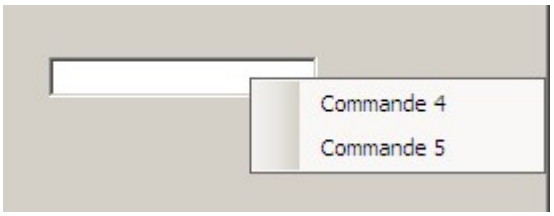
9.7.4.C. Le menu ContextMenuStrip

Le menu contextuel permet d'associer aux différents contrôles de l'interface de l'application, un menu qui sera affiché lors d'un clic avec le bouton droit de la souris sur le contrôle. Les menus doivent d'abord être conçus en ajoutant sur la feuille des contrôles *ContextMenuStrip*.

Ce contrôle apparaît, sur la barre de menu de l'application, juste pour permettre de saisir les différents éléments du menu. À noter qu'un menu contextuel ne comporte pas de titre mais permet simplement la saisie d'éléments de menu et de sous-menus. Il faut ensuite associer, à chaque contrôle, le menu contextuel correspondant par l'intermédiaire de la propriété *ContextMenuStrip*.



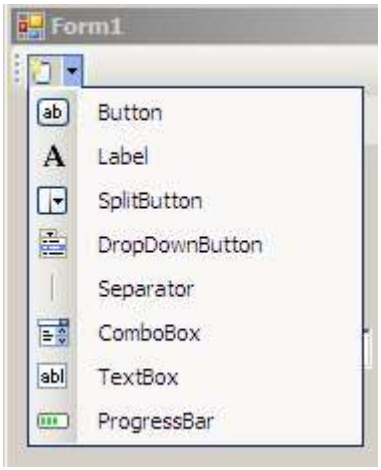




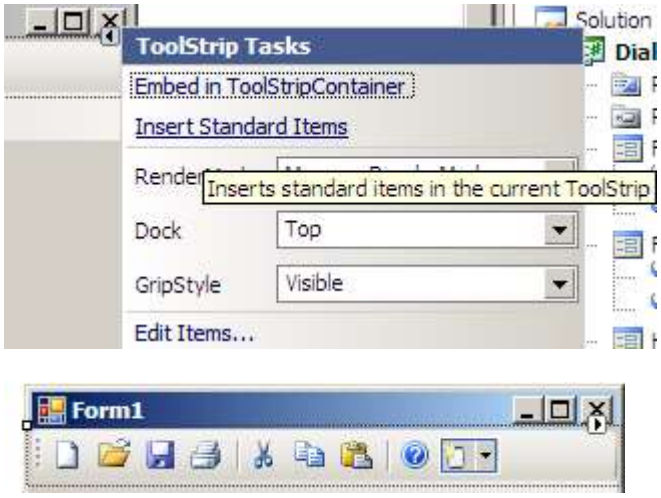
Il est aussi possible d'associer un menu contextuel à la feuille.

9.7.4.D. Le contrôle ToolStrip

Le contrôle *ToolStrip* et toutes les classes associées sont utilisés pour la création de barre d'outils. Ce contrôle sert en fait de conteneur pour les éléments constituant une barre d'outils. Ces éléments sont ajoutés au contrôle *ToolStrip* par l'intermédiaire du menu qui est affiché en cliquant sur la petite flèche présente à gauche sur le contrôle.



Cette solution permet de créer une barre d'outils de A à Z en y ajoutant tous les éléments un à un. Dans beaucoup d'applications, les barres d'outils comportent un ensemble de boutons standards. Pour faciliter la tâche, le contrôle *ToolTrip* contient un assistant pour générer automatiquement une barre d'outils avec toutes les fonctionnalités de base d'une application. On doit pour cela cliquer sur la petite flèche présente sur la droite du contrôle et choisir l'option *Ajouter des éléments standards*. La barre d'outils prend alors l'apparence suivante :

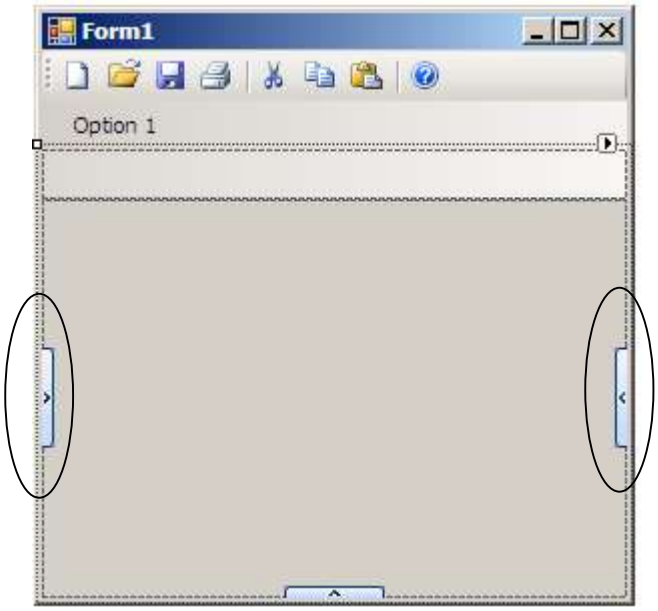


Les boutons classiques sont ajoutés automatiquement à la barre d'outils. La *magie* de l'assistant s'arrête ici car il n'y a, bien sûr, pas de code généré automatiquement pour réagir aux clics sur ces différents boutons. On doit écrire ce code comme pour tous les autres contrôles de l'interface de l'application.

9.7.4.E. Le contrôle ToolStripContainer

Le but de ce contrôle est de faciliter la conception d'une fenêtre lorsque celle-ci doit contenir plusieurs barres d'outils. Ce contrôle est généralement placé sur une fenêtre avec la propriété *Dock* positionnée sur *Fill* pour occuper toute la surface disponible de la fenêtre.

Il dispose sur chacune des bordures d'un conteneur pour accueillir les différentes barres d'outils. Par défaut ces conteneurs ne sont pas visibles et il faut donc les activer avant de pouvoir y ajouter des éléments. Pour cela, on doit cliquer sur les petites flèches présentes sur les onglets en périphérie du contrôle.



Une nouvelle zone devient active sur laquelle il est possible de placer un ou plusieurs *ToolStrip*. Attention, ces différentes zones ne peuvent contenir que des contrôles *ToolStrip*. La zone centrale peut par contre contenir n'importe quel contrôle.

L'avantage de l'utilisation du contrôle *ToolStripContainer* réside dans la possibilité de modifier la disposition des barres d'outils pendant l'exécution de l'application. Pour déplacer une barre d'outils, l'utilisateur doit la saisir en cliquant sur les trois points présents sur la barre d'outils et réaliser un glisser déplacer vers une bordure de la fenêtre. La barre d'outils vient alors s'ancrer sur la bordure correspondante.

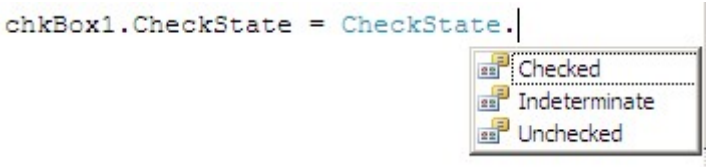
!!! page 311 : infos supplémentaires

9.7.5. Les contrôles de sélection

9.7.5.A. Le contrôle CheckBox

Le contrôle *CheckBox* est utilisé pour proposer à l'utilisateur plusieurs options parmi lesquelles il pourra en choisir une ou plusieurs. Le contrôle *CheckBox* peut en général prendre deux états : coché lorsque l'option est sélectionnée ou non coché lorsque l'option correspondante n'est pas sélectionnée.

Une troisième option correspond à un état indéterminé de la case à cocher ; dans ce cas, elle apparaît en grisé. L'état de la case à cocher peut être vérifié ou modifié par la propriété *Checked*. Cette propriété fournit un booléan qui reflète l'état de la case. Si la case à cocher est configurée pour fonctionner avec trois états possibles, en positionnant la propriété *ThreeState* sur *True*, la propriété *CheckState* indique l'état de la case à cocher par l'intermédiaire d'une des valeurs définies dans l'énumération.



Comme pour les autres contrôles, la propriété *TextAlign* permet de modifier la position du texte sur le contrôle. La propriété *CheckAlign*, suivant le même principe, permet de modifier la position de la case à cocher par rapport au texte.

Par défaut, le contrôle *CheckBox* gère lui-même l'affichage de la coche et la modification des propriétés *Checked* et *CheckState* en fonction des actions de l'utilisateur. On peut prendre la responsabilité de la gestion de l'état de la case à cocher, en modifiant la propriété *AutoCheck* sur *False*. Dans ce cas, il faut gérer l'événement *Click* et modifier par le code, dans la gestion de cet événement, la propriété *Checked* ou la propriété *CheckState*. L'exemple de code, ci-après, permet de modifier les caractéristiques de la police de caractère d'un contrôle *TextBox*.

Dans cet exemple, on travaille avec l'événement *CheckedChanged* du contrôle *CheckBox* qui indique chaque changement de l'état de la case à cocher.




```
private void chkGras_CheckedChanged(object sender, EventArgs e)
{
      FontStyle fntStyle;

      fntStyle = txtTexte.Font.Style;

      if (chkGras.Checked)
      {
          fntStyle = fntStyle | FontStyle.Bold;
      }
      else
      {
          fntStyle = fntStyle & ~FontStyle.Bold;
      }

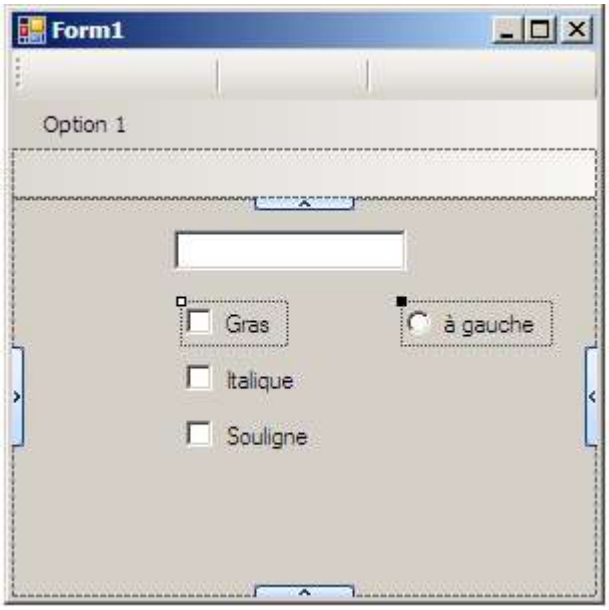
      txtTexte.Font = new Font(txtTexte.Font, fntStyle);
}
```

9.7.5.B. Le contrôle RadioButton

Le contrôle *RadioButton* permet également de proposer à l'utilisateur différentes options parmi lesquelles il ne pourra en sélectionner qu'une seule. Comme son nom l'indique, ce contrôle fonctionne comme les boutons permettant de sélectionner une station sur un poste de radio (on ne peut pas écouter trois stations de radio en même temps. Il faut évidemment qu'il s'agisse d'une vieille radio ayant des boutons !).

Les propriétés sont strictement identiques à celles disponibles dans le contrôle *CheckBox* hormis la propriété *Checkstate* car ce contrôle ne peut avoir que deux états et un booléan est suffisant pour représenter l'état du contrôle.

Complétons l'exemple précédent en ajoutant des options pour l'alignement du texte :




```
private void rdoButton1_CheckedChanged(object sender, EventArgs e)
{
     if (rdoButton1.Checked)
     {
         txtTexte.TextAlign = HorizontalAlignment.Left;
     }
}
```

Il est conseillé, lorsque l'interface de l'application contient des contrôles de type *RadioButton*, qu'il y en ait toujours un de sélectionné par défaut à l'affichage du formulaire.

Complétons l'application en y ajoutant le choix de la couleur du texte :




```
private void rdoButtonBleu_CheckedChanged(object sender, EventArgs e)
{
```

```
        if (rdoButtonBleu.Checked)
        {
            txtTexte.ForeColor = Color.Blue;
        }
    }
```

À ce stade de l'application, on va avoir quelques soucis avec l'application, car il ne sera pas possible de sélectionner une option d'alignement et une option de couleur en même temps. En effet, les contrôles de type *radioButton* sont conçus de telle sorte qu'une seule option peut être sélectionnée. La solution à ce problème sera l'utilisation d'un conteneur pour isoler les contrôles les uns des autres. C'est ce que va voir plus tard avec les contrôles de type *GroupBox* et *Panel*.

9.7.5.C. Le contrôle ListBox

Le contrôle *ListBox* propose à l'utilisateur une liste de choix dans laquelle il pourra en sélectionner un ou plusieurs. Le contrôle gère automatiquement l'affichage des éléments avec, au besoin, l'ajout d'une barre de défilement sur le contrôle;

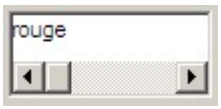
Les éléments affichés dans la liste peuvent être saisis au moment de la conception, dans la fenêtre des propriétés, ou modifiés par le code. En règle générale, les éléments de la liste sont des chaînes de caractères mais peuvent être de n'importe quel type.



Par défaut, le contrôle *ListBox* propose les éléments sous forme de liste avec un défilement vertical.



La propriété *MultiColumn* positionnée sur *true* permet d'avoir un défilement horizontal de la liste :



Dans ce cas, la propriété *ColumnWidth* indique la largeur de chacune des colonnes. La propriété *IntegraleHeight* évite d'avoir un élément de la liste partiellement visible. Si cette propriété est positionnée sur *true*, la liste se redimensionne pour n'afficher que des éléments complets.

On peut ajouter !!!

Les éléments de la liste sont gérés sous forme de collection, par la propriété *Items*. Des éléments sont ajoutés à la liste en utilisant la méthode *Add* de la collection et en passant, comme paramètre, l'objet à insérer dans la liste. Dans la majorité des cas, l'objet inséré est une chaîne de caractères mais n'importe quel autre type peut être utilisé. Il faut cependant que l'objet ajouté dispose d'une méthode *ToString* pour permettre l'affichage par le contrôle *ListBox*.

```
class Couleur
{
    private string strNom;
    private Color oValeur;

    public Couleur(string strN, Color oC)
    {
        strNom = strN;
        oValeur = oC;
```

```
        }

        public override string ToString()
        {
            return strNom;
        }
    }

    lstCouleur.Items.Add(new Couleur("Rouge", Color.Red));
```

Les éléments sont ajoutés à la fin de la liste sauf si la propriété *Sorted* est positionnée sur *True* car, dans ce cas, les éléments de la liste sont triés par ordre croissant. La méthode *Insert* ajoute un élément dans la liste, à un emplacement précis (le premier élément de la liste se trouve à l'emplacement zéro). Pour insérer un élément en deuxième position, on utilisera le code suivant :

```
    lstCouleur.Items.Insert(1, new Couleur("Jaune", Color.Yellow));
```

L'effacement d'un élément de la liste s'effectue avec la méthode *RemoveAt* en passant comme paramètre l'index de l'élément à supprimer.

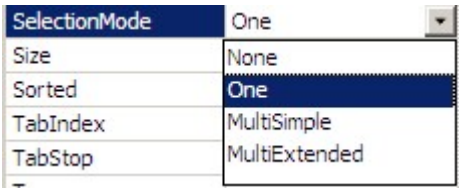
```
    lstCouleur.Items.RemoveAt(0);
```

On peut aussi effectuer un nettoyage complet de la liste avec la méthode *Clear* :

```
    lstCouleur.Items.Clear();
```

Deux méthodes sont utiles, lors de l'ajout d'éléments dans la liste, si l'on veut éviter les doublons. La méthode *FindString* recherche le premier élément de la liste qui commence par la chaîne de caractères spécifiée. Cette fonction renvoie l'index de l'élément correspondant au critère ou la valeur -1, si rien n'est trouvé. La même chose peut être effectuée avec la méthode *FondStrinExact* mais, dans ce cas, il faut une correspondance exacte des chaînes de caractères.

Par défaut, les *ListBox* n'autorisent que la sélection d'un élément. La propriété *SelectionMode* permet de choisir entre quatre modes possibles de sélection :



- *None* : risque d'être frustrant pour l'utilisateur car la *ListBox* sera pleinement fonctionnelle mais le clic de la souris sur un item de la liste ne provoquera aucun effet.
- *One* : est la valeur par défaut pour lequel chaque clic de souris sélectionne l'élément ou déselectionne l'élément précédemment sélectionné.
- *MultiSimple* : chaque clic sur un élément inverse son état. Il peut y avoir plusieurs éléments sélectionnés en même temps.
- *MultiExtended* : permet également la sélection de plusieurs éléments mais offre la possibilité de sélectionner plusieurs éléments successifs rapidement. En sélectionnant le premier, puis en utilisant la touche [shift] et en cliquant sur le dernier, tous les éléments présents entre les deux seront également sélectionnés.

Il faut bien sûr récupérer, dans le code, la ou les sélections de l'utilisateur. Dans le cas d'une sélection simple, la propriété *SelectedIndex* indique l'indice, dans la collection *Items*, de l'élément sélectionné. La propriété *SelectedItem* fournit une référence vers l'objet sélectionné dans la liste.

Pour les sélections multiples, le principe de fonctionnement est le même mais les informations sont renvoyées sous forme de collection. La collection *SelectedIndices* propose la liste des indices des éléments sélectionnés, alors que la collection *SelectedItems* donne la liste des objets sélectionnés. Dans les deux cas, il faut parcourir la liste pour extraire les éléments.

```
foreach (Couleur c in lstCouleur.SelectedItems)
{
          Console.WriteLine(c.ToString());
}
```

Le fait de stocker des objets dans la collection *Items* permet d'avoir accès, par l'intermédiaire de la propriété *SelectedItem*, à toutes les propriétés de l'objet. On peut ainsi par exemple, dans le cas des objets *Couleur*, d'utiliser la propriété *Valeur*.

Dans l'exemple suivant, on modifie la couleur de fond de la feuille en fonction de la sélection dans la *ListBox*. Pour cela, on va gérer l'événement *SelectedIndexChanged* qui se produit à chaque nouvelle sélection de l'utilisateur.

```
private void lstCouleur_SelectedIndexChanged(object sender, EventArgs e)
{
      this.BackColor = ((Couleur)lstCouleur.SelectedItem).Valeur;
}
```

Associé une clé primaire à chaque ligne

Quand on ajoute dans une *ListBox* des records provenant d'une table, la sélection d'une ligne de la table va peut-être nécessiter de pouvoir accéder au record correspondant dans la table. Dans ce cas, on ne peut pas se contenter de la chaîne de caractères qui se trouve dans la ligne de *ListBox*. Il faut la clé primaire. Mais où placer cette clé primaire sans qu'elle ne soit affichée ?

Voici la réponse :

```
using System.Web.UI.WebControls;

ListItem lstItem;

lstItem.Text = strNomAnim.ToUpper();
lstItem.Value = strAnimID;

cboAnim.Items.Add(lstItem);
```

Nécessaire d'ajouter la référence *System.Web*.

On ajoute le texte qui sera visible dans la liste et on place la clé primaire dans *value*.

Lorsqu'une ligne de la liste a été sélectionnée, voici comment retrouver l'information :

```
ListItem lstItem;
string strAnimID;

lstItem = (ListItem) cboAnim.SelectedItem;

strAnimID = lstItem.Value;
```

SelectedItem retourne une valeur de type *Object*.

9.7.5.D. Le contrôle NumericUpDown

Le contrôle *NumericUpDown* est l'association d'une zone de texte et de deux boutons. Ces deux boutons servent à incrémenter ou à décrémenter la valeur affichée dans la zone de texte. Ce contrôle est très simple à utiliser. La valeur qu'il est chargé de gérer est stockée dans la propriété *value*.

Elle peut évoluer entre deux extrêmes représentés par les propriétés *Minimum* et *Maximum*. Le pas d'incrémentation est défini par la propriété *Increment*. Toutes ces valeurs peuvent être des valeurs décimales. Le format d'affichage peut être modifié par la propriété *DecimalsPlaces* qui permet de spécifier le nombre de décimales affichées.

Cette propriété intervient uniquement au niveau de l'affichage et ne change pas la valeur de la propriété *Value*. Voici comme exemple un petit morceau de code qui simule le fonctionnement d'un thermostat de chauffage. Ce n'est évidemment pas la température du bureau qui sera modifiée par l'application mais la couleur de fond de la fenêtre : du bleu pour le plus froid et le rouge pour le plus chaud.
!!!

9.7.5.E. Le contrôle TrackBar

Ce contrôle propose un aspect plus visuel du contrôle *NumericDown* tout en fournissant les mêmes fonctionnalités. Il se présente sous la forme d'un potentiomètre sur lequel on peut déplacer un curseur pour ajuster une valeur numérique. Contrairement au contrôle *NumericDown*, il n'y a pas d'affichage de la valeur et celle-ci ne peut être qu'une valeur entière. Les deux valeurs extrêmes sont définies par les propriétés *Minimum* et *Maximum*.

Deux pas d'incrémentation sont également disponibles avec les propriétés *SmallChange* et *LargeChange*. La propriété *SmallChange* est utilisée pour incrémenter ou décrémenter la valeur lors de l'utilisation des flèches de déplacement su clavier. La propriété *LargeChange* est utilisée suite à la frappe des touches [page suivante] ou [page précédente]. Si le curseur du potentiomètre est déplacé à l'aide de la souris, la propriété *Value* correspond à l'emplacement du curseur. Aucun des deux pas d'incrémentation n'est utilisé dans ce cas. Pour faciliter l'ajustement d'une valeur, des repères sont placés sur le potentiomètre. Leur espacement est déterminé par la propriété *TickFrequency*.

9.7.5.F. Le contrôle DomainUpDown

Ce contrôle a pratiquement le même fonctionnement que le contrôle *Listbox*. Il se présente comme une zone de texte à laquelle sont associés deux boutons permettant le déplacement dans une liste d'éléments. La principale propriété de ce contrôle est la propriété *Items* qui contient les éléments proposés dans la liste. Les seules différences avec le contrôle *Listbox* sont liées aux sélections multiples qui sont impossibles pour ce contrôle et la propriété *Wrap* permettant le parcours de la liste de manière circulaire.

9.7.5.G. Le contrôle CheckedListBox

Ce contrôle est une amélioration du contrôle *ListBox* et de ce fait fonctionne pratiquement de la même manière. La seule limitation par rapport au contrôle *ListBox* est qu'il n'accepte pas les sélections multiples. L'évolution par rapport au contrôle *ListBox* réside dans les cases à cocher présentes devant les éléments de la liste.



Ces cases peuvent être cochées indépendamment de la sélection des éléments; la récupération des cases cochées est possible par l'intermédiaire de la propriété *CheckedItems*. Cette collection contient tous les éléments cochés de la liste.

```
private void clbBox_SelectedIndexChanged(object sender, EventArgs e)
{
      foreach (var elt in clbBox.CheckedItems)
      {
          Console.WriteLine(elt.ToString());
```

```
        }
    }
```

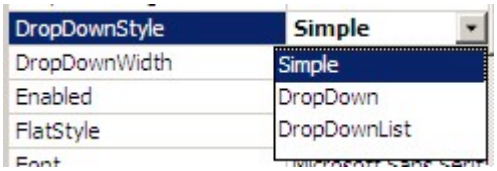
Une autre solution pour obtenir les éléments cochés de la liste consiste à parcourir la liste et à tester avec la méthode *GetItemChecked* si la case correspondante est cochée.

```
for (int dI = 0; dI < clbBox.Items.Count - 1; dI++)
{
        if (clbBox.GetItemChecked(dI))
        {
            Console.WriteLine(clbBox.Items[dI]);
        }
}
```

La propriété *CheckOnClick* détermine si l'activation ou la désactivation de la case à cocher se fait sur le premier clic d'un élément dans la liste (true) ou si un double clic est nécessaire pour changer l'état de la case à cocher (false).

9.7.5.H. Le contrôle ComboBox

Le contrôle *ComboBox* est l'association d'un contrôle *ListBox* et *TextBox*. L'utilisateur pourra, suivant la configuration du contrôle, choisir un élément dans la liste ou saisir du texte. La propriété *DropDownStyle* indique le mode de fonctionnement de la *ComboBox*.



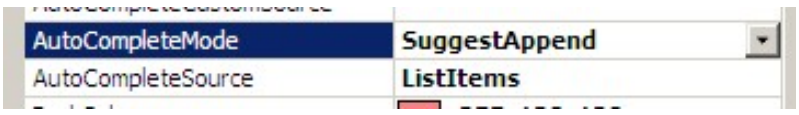
- *Simple* : l'utilisateur sélectionne un élément dans la liste ou saisit du texte dans la zone de texte. La liste est affichée en permanence.
- *DropDown* : même configuration mais la liste n'est affichée qu'à la demande de l'utilisateur.
- *DropDownList* : l'utilisateur ne peut plus saisir du texte dans la zone de texte mais simplement choisir un élément existant dans la liste, après avoir demandé son affichage.

L'état de l'affichage de la liste est contrôlé par la propriété *DroppedDown*. On peut, par exemple, écrire du code pour que la liste soit déroulée dés que le contrôle reçoit le focus.

```
void cmbComboBox.GotFocus(object sender, EventArgs e)
{
     cmbComboBox.DroppedDown = true;
}
```

Il n'est pas possible de réaliser des sélections multiples avec le contrôle *ComboBox*. Le texte peut être récupéré dans la propriété *Text*. Cette propriété est mise à jour même si l'utilisateur a sélectionné un élément dans la liste. Dans ce cas, la propriété *SelectedIndex* est égale à l'index de l'élément sélectionné. Si l'utilisateur a saisi du texte directement, cette propriété est égale à -1.

La manière de taper des caractères dans la zone d'édition pour se positionner sur une ligne de la ComboBox peut se faire en utilisant les propriétés :



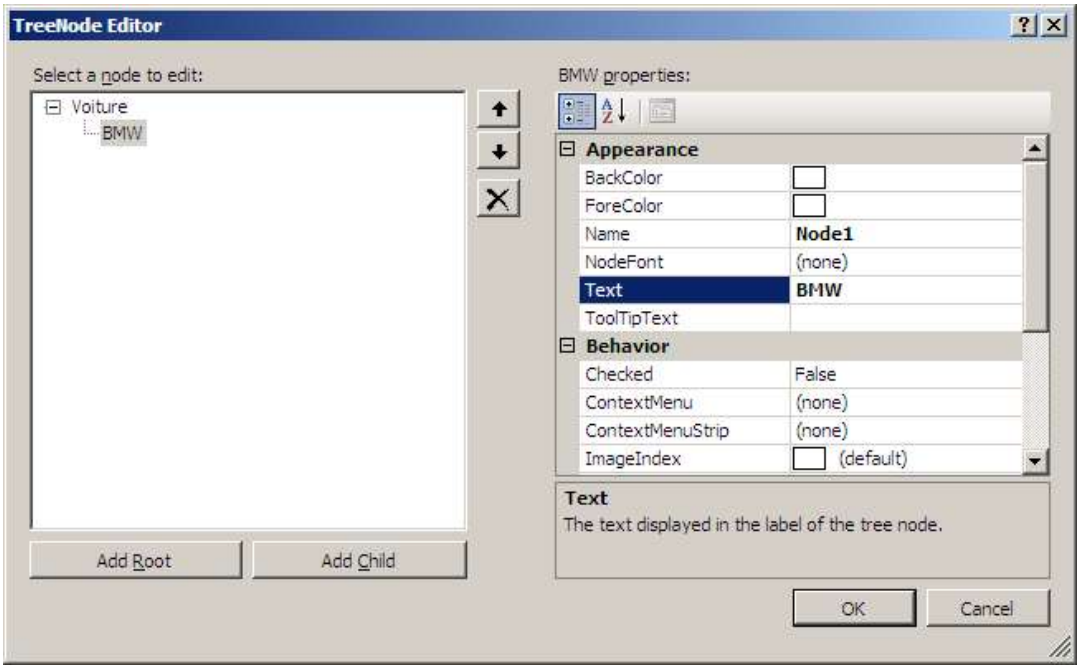
9.7.5.I. Le contrôle TreeView

Le contrôle *TreeView* permet la présentation d'informations, sous forme d'une arborescence identique à la présentation des fichiers et des dossiers dans l'explorateur de Windows. Chaque élément est représenté sous forme d'un nœud pouvant éventuellement contenir des nœud enfants.

Chaque élément peut être affiché sous forme développée ou réduite, par l'intermédiaire du signe (plus (+) ou moins (-)) affiché en face de chaque nœud.

La propriété *Nodes* contient la liste de tous les nœuds de premier niveau de l'arborescence. Chacun d'entre eux dispose également d'une propriété *Nodes*, qui stocke à son tour la liste de tous ses nœuds enfants et ainsi de suite jusqu'au dernier niveau de l'arbre.

Ces propriétés *Nodes* sont modifiables au moment de la conception, par un éditeur spécifique accessible par la fenêtre de propriétés.



Le bouton *Ajouter une racine* ajoute un élément racine à la propriété *Nodes* du contrôle *TreeView*. Pour chaque élément, il convient également d'indiquer :

- son libellé par la propriété *Text,*
- l'image affichée sur le nœud par la propriété *ImageIndex*,
- l'image affichée, lorsque le nœud est sélectionné dans l'arbre, par la propriété *SelectedImageIndex*.

Ces images sont extraites du contrôle *ImageList* que l'on aura, au préalable, associé à la propriété *ImageList* du contrôle. Ce contrôle *ImageList* doit bien sûr être créé et rempli avant la modification des propriétés du contrôle *TreeView* :

```
ImageList lstImage = new ImageList();
lstImage.Images.Clear();

foreach (FileInfo fileName in tFileName)
{
     lstImage.Images.Add(Image.FromFile(fileName.ToString()));
```

```
        }

        treePhotos.ImageList = lstImage;
```

Chaque nœud possède donc une image, pour son affichage normal, et une image, pour son affichage lorsqu'il est sélectionné. La valeur *Default* pour ces deux images indique que le nœud n'utilisera pas une image spécifique, mais les images par défaut qui sont indiquées dans les propriétés *ImageIndex* et *SelectedImageIndex* du contrôle *TreeView*. On peut de cette façon utiliser des images identiques pour tous les nœuds. Seuls ceux ayant un affichage particulier nécessiteront la modification des valeurs par défaut.

L'ajout de nœud dans l'arbre par le code est un petit peu plus complexe. Il faut créer un nouveau nœud en instanciant la classe *TreeNode* et en indiquant, dans l'appel du constructeur de la classe, la légende affichée sur le nœud. L'image et l'image de sélection sont également à préciser si on ne veut pas utiliser les valeurs par défaut. La création d'un nœud peut donc prendre une des deux formes suivantes :

```
TreeNode oNoeud;

oNoeud = new TreeNode("Audi");

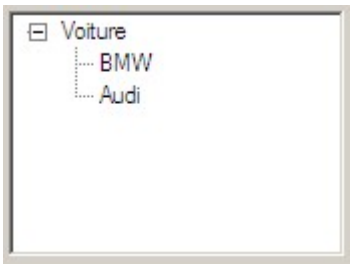
oNoeud = new TreeNode("Audi", 2, 3);
```

Le deuxième paramètre indique l'index de l'image normal, suivi de l'index pour l'affichage lorsque le nœud sera sélectionné.

Il faut ensuite raccrocher ce nouveau nœud à une branche de l'arbre en utilisant la méthode *Add* du nœud qui va devenir le parent du nouveau nœud.

Ainsi, pour ajouter un enfant à la racine de l'arbre, on peut utiliser le code suivant :

```
treeView.Nodes[0].Nodes.Add(oNoeud);
```

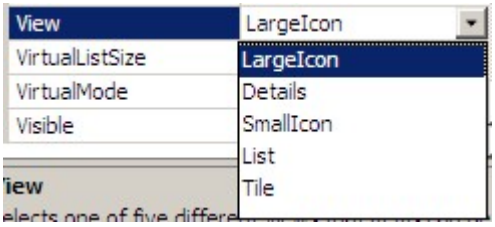


Lorsque l'on travaille avec le contrôle *TreeView*, on a fréquemment besoin de savoir sur quel élément du contrôle l'utilisateur a cliqué. Dans l'événement *AfterSelect* du contrôle *TreeView*, on dispose, par l'intermédiaire du paramètre *e*, une référence sur le nœud sur lequel l'utilisateur a cliqué. La propriété *Text* de cet élément permet d'identifier le nœud du contrôle et de réagir en conséquence :

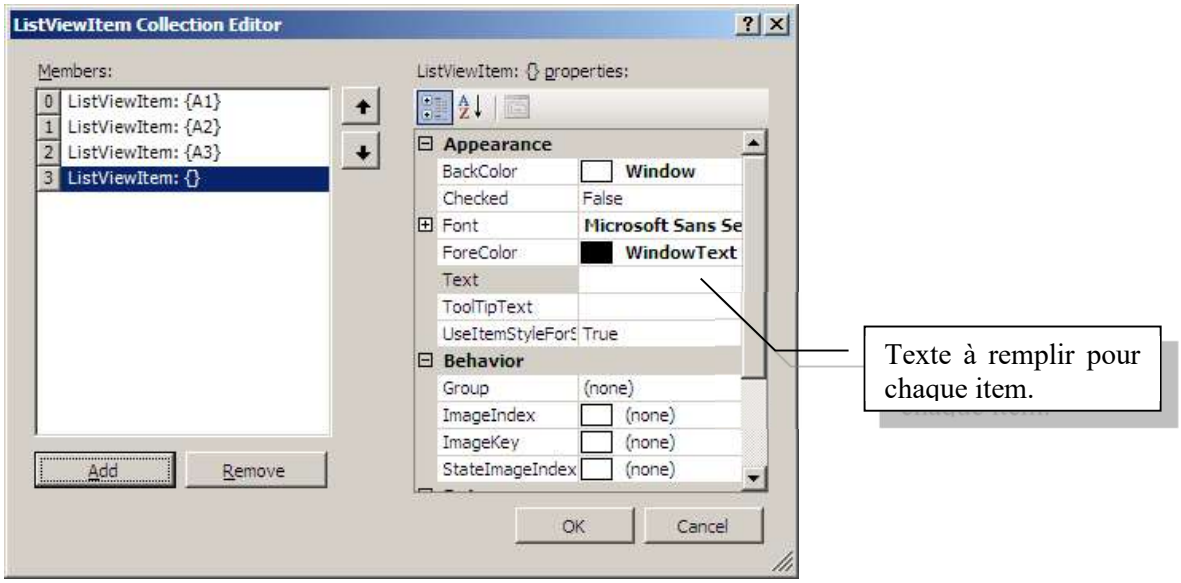
```
private void trvTreeView_AfterSelect(object sender, TreeViewEventArgs e)
{
      switch (e.Node.Text)
      {
          case "BMW":
              Console.WriteLine("BMW");
              break;
          case "Audi":
              Console.WriteLine("Audi");
              break;
      }
}
```

9.7.5.J. Le contrôle ListView

Ce contrôle fournit la possibilité de présenter des informations, à l'utilisateur, de quatre manières différentes. Comme le permet l'explorateur de Windows.

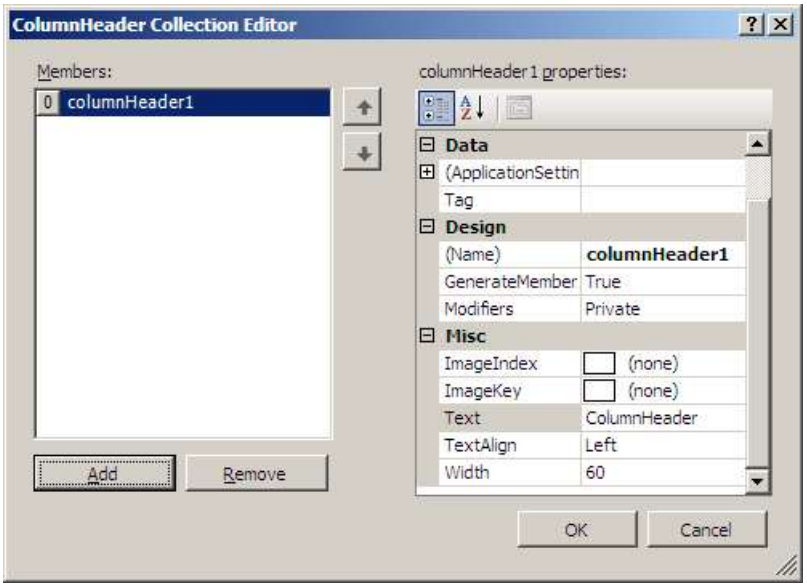


Comme pour le contrôle *TreeView*, ce contrôle dispose d'une collection *Items* stickant les informations à afficher. Cette collection peut être remplie par un éditeur spécifique accessible depuis la fenêtre de propriétés.



Cet éditeur permet d'ajouter ou de supprimer des éléments dans l'affichage du contrôle et pour chacun d'entre eux, la modification de la légende par la propriété *Text* et de l'image associée par la propriété *ImageIndex*. La propriété *SubItems* est également une collection utilisée lors de l'affichage détaillé. Cette collection contient les informations affichées dans chacune des colonnes du mode détaillé.

Les colonnes sont créées par la propriété *Columns* du contrôle. Un éditeur spécifique est également disponible pour la gestion des colonnes.



La propriété *Text* correspond au titre des colonnes. Chaque colonne est associée aux éléments qui auront été saisis dans la propriété *SubItems*.

Résumons tout cela avec l'exemple des véhicules avec leurs caractéristiques :

- Pour chaque véhicule à afficher, il faut insérer un élément dans la collection *Items* du contrôle *ListView*. Pour chacun de ces éléments, la propriété *SubItems* doit contenir les informations utilisées pour l'affichage en mode détail (le type du véhicule, son prix et sa puissance).

- Il faut ensuite ajouter trois colonnes à la propriété *Columns* du contrôle *ListView* (type, prix, puissance).

- Pour terminer la configuration, les propriétés *LargeImageList* et *SmallImageList* indiquent respectivement les contrôles *ImageList* où seront récupérées les images pour l'affichage en mode Grande Icônes ou Petites Icônes.

Le type de présentation de l'affichage est modifiable par la propriété *View*.

Remarques :

- Lors de l'insertion d'un item dans une *ListView*, celui-ci doit être recréé à chaque nouvelle insertion dans la liste.
- La propriété *SubItem* permet de préciser les valeurs à attribuer aux colonnes mais il n'y a pas de liaison explicite entre les colonnes et les *SubItems*. L'assignation se fait dans l'ordre.
- La propriété *Tag* peut être utilisée pour contenir une primary key.

La recherche des lignes sélectionnées dans la liste peut se faire au moyen d'une collection *SelectedListViewItemCollection* comme dans l'exemple suivant :

```
ListView.SelectedListViewItemCollection lstItem;

lstItem = lstvConsult.SelectedItems;

foreach (ListViewItem oItem in lstItem)
{
    ldConsultID = (long) oItem.Tag;
}
```

La sélection d'une ligne à l'intérieur de la liste par programme peut se faire par une des deux méthodes ci-dessous:

```
lstvConsult.Items[0].Selected = true;
lstvConsult.Selected();
```

L'accès aux valeurs des différentes colonnes se fait au moyen :

```
lstvPay.Items[dI].SubItems[index de la colonne].Text
```

Tri de la ListView avec un clic sur la colonne

Il est possible de trier automatiquement les items se trouvant dans une ListView en cliquant sur le nom de la colonne : comme cela se fait usuellement dans toutes les applications Windows.

Pour ce faire, il faut tout d'abord définir une nouvelle classe : ListViewColumnSorter.

```
using System.Collections;
using System.Windows.Forms;

public class ListViewColumnSorter : System.Collections.IComparer
    {
        private int m_dColumnToSort;
        private CaseInsensitiveComparer m_oObjectCompare;
        private SortOrder m_oSortOrder;

        public ListViewColumnSorter()
        {
            m_dColumnToSort = 0;
            m_oSortOrder = SortOrder.None;
            m_oObjectCompare = new CaseInsensitiveComparer();
        }

        // ************************************************************
        // Avec l'interface IComparer, il faut implémenter la méthode Compare
        // ************************************************************

        public int Compare(object oX, object oY)
        {
            int dCompareResult;
            ListViewItem oListViewItemX, oListViewItemY;

            oListViewItemX = (ListViewItem)oX;
            oListViewItemY = (ListViewItem)oY;

            // -------------------------
            // Compare les deux éléments
            // -------------------------

            dCompareResult =
m_oObjectCompare.Compare(oListViewItemX.SubItems[m_dColumnToSort].Text,
oListViewItemY.SubItems[m_dColumnToSort].Text);

            // ------------------------------------------------------------------
-------------------------------------------
            // Calcule la valeur correcte d'après la comparaison d'objets
            // Si tri croissant est sélectionné, renvoie des résultats normaux de
comparaison, sinon renvoie le résultats négatif
            // En cas d'égalité, on retourne zéro.
            // ------------------------------------------------------------------
-------------------------------------------
```

```
            if (m_oSortOrder == SortOrder.Ascending)
            {
                return dCompareResult;
            }
            else if (m_oSortOrder == SortOrder.Descending)
            {
                return (-dCompareResult);
            }
            else
            {
                return 0;
            }
        }

        public int prSortColumn
        {
            set
            {
                m_dColumnToSort = value;
            }
            get
            {
               return m_dColumnToSort;
            }
        }

        public SortOrder prOrder
        {
            set
            {
                m_oSortOrder = value;
            }
            get
            {
                return m_oSortOrder;
            }
        }
    }
```

Ensuite il faut ajouter dans la classe qui gère le formulaire sur lequel se trouve la ListView, les élements suivants :

1) La définition d'une donnée membre supplémentaire :

```
private ListViewColumnSorter m_lvwColumnSorter;
```

2) Dans le constructeur du formulaire, il faut ajouter la définition des colonnes de la ListView :

```
m_lvwColumnSorter = new ListViewColumnSorter();

this.lstvPay.ListViewItemSorter = m_lvwColumnSorter;

ColumnHeader oColumnHeader;

oColumnHeader = new ColumnHeader();
oColumnHeader.Text = "Titre de la colonne";
this.lstvPay.Columns.Add(oColumnHeader);
```

3) Ajouter une function qui sera invoquée suite au clic sur le nom de la colonne :

```
private void lstvPay_ColumnClick(object sender, ColumnClickEventArgs e)
  {
      int dSelectedColumn = e.Column;

      dSelectedColumn = 0;

      if (dSelectedColumn == m_lvwColumnSorter.prSortColumn)
      {
          // Inverser le sens de tri en cours pour cette colonne.
          if ( m_lvwColumnSorter.prOrder == SortOrder.Ascending)
          {
              m_lvwColumnSorter.prOrder = SortOrder.Descending;
          }
          else
          {
              m_lvwColumnSorter.prOrder = SortOrder.Ascending;
          }
      }
      else
      {
          // Définir le numéro de colonne à trier ; par défaut sur croissant.
          m_lvwColumnSorter.prSortColumn = dSelectedColumn;
          m_lvwColumnSorter.prOrder = SortOrder.Ascending;
      }

      // Procéder au tri avec les nouvelles options.
      this.lstvPay.Sort();
  }
```

Gestion d'un grand nombre d'items dans une ListView

Tout comme beaucoup d'autres Listes, le temps de chargement d'un grand nombre d'items dans une ListView est excessivement élevé. Une première option pour réduire le temps de chargement est d'utiliser les méthodes :

```
lstvMvt.BeginUpdate(); → à placer avant le début du chargement.

lstvMvt.EndUpdate();   → à placer après le chargement.
```

Mais, ces deux méthodes ne vont pas être suffisantes si plusieurs miliers d'items doivent être chargés. Pour réduire, le temps de chargement à presque rien, on va utiliser le mode virtuel. Avec ce mode, les items ne vont pas être réellement chargés dans la liste. Les items sont chargés dans une collection (peu importe laquelle), et la ListView va être une espèce de fenêtre qui va se déplacer sur cette collection. Seuls les items visibles dans la ListView sont réellement chargés.

Pour activer le mode virtuel au niveau de la ListView, il faut d'abord activer le VirtualMode :



Ensuite, il faut ajouter une méthode qui va réagir à l'évènement *RetrieveVirtualItems* :

```
private void lstvPay_RetrieveVirtualItem(object sender,
                  RetrieveVirtualItemEventArgs e)
  {
      ListViewItem oItem = new ListViewItem();
      etLigPaiement stLigPaiement = m_tListPay[e.ItemIndex];

      oItem.Text = stLigPaiement.strID;
```

```
            oItem.Tag = stLigPaiement.strFactID;

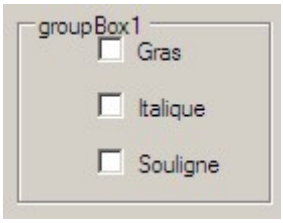
            oItem.SubItems.Add(stLigPaiement.strDate);
            oItem.SubItems.Add(stLigPaiement.strNom.ToUpper());
            oItem.SubItems.Add(stLigPaiement.strPrenom.ToUpper());
            oItem.SubItems.Add(stLigPaiement.strType.ToUpper());
            oItem.SubItems.Add(stLigPaiement.strMontTVAC + " €");
            oItem.SubItems.Add(stLigPaiement.strMontPaye + " €");
            oItem.SubItems.Add(stLigPaiement.strAutreTit.ToUpper());

            e.Item = oItem;
        }
```

9.7.6. Les contrôles de regroupement

9.7.6.A. Le contrôle GroupBox

Le contrôle *GroupBox* permet le regroupement logique et visuel de contrôles sur un formulaire. Le contrôle possède une bordure et une légende apparaissant sur la bordure haute. Ce contrôle facilite également le placement des contrôles sur la fenêtre, car après avoir placé les contrôles sur le *GroupBox*, ceux-ci se déplacent en bloc avec le contrôle *GroupBox*. Il permet également d'isoler les *RadioButtons*, les uns des autres, lorsqu'ils doivent fonctionner de manière indépendante.



9.7.6.B. Le contrôle Panel

Le contrôle *Panel* dispose des mêmes fonctionnalités que le contrôle *GroupBox*. Il ne dispose cependant pas de propriété *Text* et ne permet donc pas l'affichage d'une légende. Par contre, il possède une propriété *BorderStyle* permettant de supprimer l'affichage de la bordure. On a, dans ce cas, des contrôles qui sont regroupés logiquement sans indication visible de ce regroupement.

La propriété *AutoScroll* du contrôle permettra d'afficher automatiquement des barres de défilement si sa surface n'est pas suffisante pour afficher l'ensemble des contrôles qui lui sont confiés.

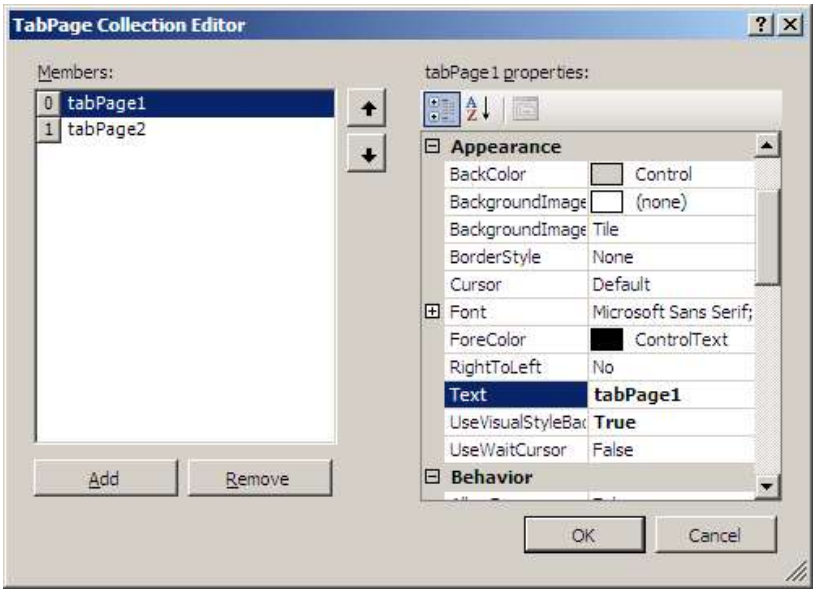
Pour ces deux contrôles, la modification de la propriété *Enabled* désactivera également l'ensemble des contrôles qui sont placés à l'intérieur, sans avoir à modifier cette propriété sur chacun des contrôles.



9.7.6.C. Le contrôle TabControl

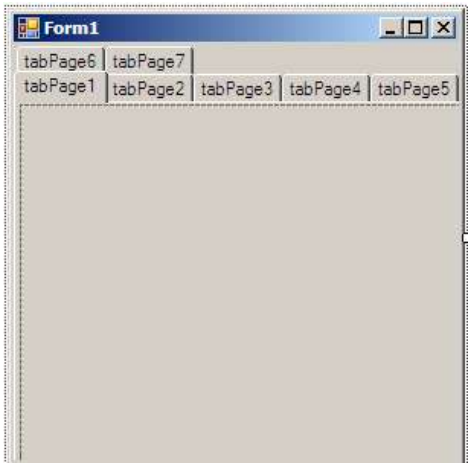
Le contrôle *TabControl* affiche plusieurs onglets similaires aux intercalaires d'un classeur. L'utilisation de ce contrôle est la création de boîte de dialogue à plusieurs onglets.

La propriété principale *TabPages* contient la liste de toutes les pages associées au contrôle. Un éditeur spécifique permet la manipulation de ces pages.



Chaque onglet est considéré comme une page, sur laquelle on pourra placer des contrôles. Pour chacun des onglets, on retrouve d'ailleurs beaucoup de propriétés déjà disponibles sur les fenêtres.

L'insertion de contrôles s'effectue en sélectionnant au préalable l'onglet de destination, puis en dessinant les contrôles dessus, de la même manière que sur une fenêtre classique. Si la boîte de dialogue contient de nombreux onglets, on peut choisir de les afficher sur plusieurs lignes en modifiant la propriété *Multiline* du contrôle.



Il est également possible d'interdire l'utilisation de tous les contrôles présents sur un onglet, en modifiant la propriété *Enabled* de l'onglet. Par exemple, pour interdire l'utilisation de l'onglet 3:

```
tabControl.TabPages[2].Enabled = false;
```

Comme pour beaucoup d'autres contrôles, une propriété *ImageList* permet d'associer des images bitmap à chacun des onglets, par l'intermédiaire de la propriété *ImageIndex* de chaque onglet.

9.7.6.D. Le contrôle SplitContainer

Le contrôle *SplitContainer* permet la création d'interface utilisateur comparable à l'explorateur Windows. Il sépare la fenêtre en deux parties par une barre horizontale ou verticale. Cette barre peut être déplacée par l'utilisateur pour redimensionner chacune des zones délimitées. Il est fréquemment associé avec un contrôle *TreeView* dans sa partie gauche et un ensemble de contrôles dans la partie droite permettant la modification de l'élément sélectionné dans le *TreeView*. Cette présentation est par exemple utilisée dans l'outil *MMC (Microsoft Management Console)*, par

lequel de nombreux paramètres de fonctionnement du système sont configurables. Chaque partie du contrôle est utilisable comme un containeur pour d'autres contrôles.

La propriété *Orientation* indique le sens dans lequel est découpée la surface du contrôle. Le dimensionnement de chacune des deux zones peut être interdit avec la propriété *IsSplitterFixed* positionnée sur *True*. Il est également possible d'interdire le redimensionnement d'une seule zone avec la propriété *FixedPanel*. Dans ce cas, si les dimensions du conteneur de contrôle (la fenêtre) sont modifiées, une seule zone est redimensionnée. La propriété *Orientation* détermine bien sûr le sens de découpage du contrôle.

!!!plus d'explication nécessaire

9.7.6.E. Le contrôle FlowLayoutPanel

Le contrôle *FlowLayoutPanel* organise automatiquement le placement des contrôles que l'on lui confie. En fonction de la valeur de la propriété *FlowDirection*, les contrôles seront disposés :

- De la gauche vers la droite,
- De la droite vers la gauche,
- Du haut vers le bas,
- Du bas vers le haut.

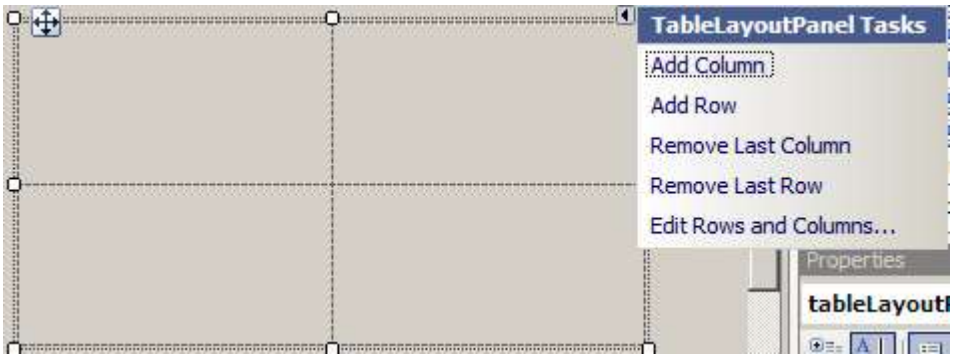
Si l'espace disponible n'est pas suffisant pour l'ajout d'un contrôle supplémentaire, la propriété *WrapContents* indique si une nouvelle colonne ou ligne est ajoutée, en fonction de l'orientation dans laquelle le contrôle *FlowLayoutPanel* travaille ou si les contrôles supplémentaires sont tronqués.

9.7.6.F. Le contrôle TableLayoutPanel

Il est parfois important qu'un formulaire conserve un aspect correct lorsqu'il est redimensionné. La première solution consiste à gérer un des événements survenant pendant le dimensionnement de la fenêtre, *Resize* ou *Layout* par exemple et de modifier les positions et dimensions des contrôles en conséquence. Cette solution est très lourde à mettre en œuvre et doit être répétée sur chaque formulaire.

L'autre solution, plus efficace est de demander à un gestionnaire de positionnement de gérer la taille et la position des contrôles qu'on lui confie. Le contrôle *TableLayoutPanel* est spécialisé pour ce travail. Ce conteneur organise son contenu sous forme d'une grille.

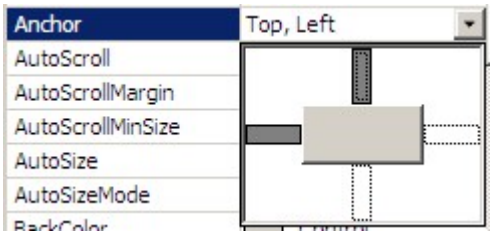
Il faut d'abord placer sur le formulaire un contrôle *TableLayoutPanel*. Par défaut, le contrôle est généré avec deux lignes et deux colonnes. Pour modifier cette disposition, il suffit simplement d'activer les options du contrôle en cliquant sur la petite flèche située sur le coin supérieur droit puis en utilisant l'option correspondante.



Chaque ligne et colonne peut également être modifiée après sa création en choisissant l'option *Modifier les lignes et les colonnes*.

Les contrôles peuvent ensuite être placés dans chacune des cases. Il ne peut y avoir qu'un seul contrôle par case.

Le principal intérêt de ce contrôle se situe pendant le redimensionnement du formulaire puisque les contrôles vont suivre automatiquement les modifications de taille du formulaire. Pour cela, le contrôle *TableLayoutPanel* doit être ancré sur les bordures gauche et droite du formulaire.



Il en va de même pour tous les contrôles placés dans le *TableLayoutPanel*. Après une modification de la taille de la fenêtre, on conservera toujours un aspect correct du formulaire.

Il est possible de faire chevaucher un contrôle sur plusieurs colonnes. Pour ce faire, il faut sélectionner le contrôle et dans ce celui-ci, choisir l'attribut *ColumnSpan*. Il suffit d'y spécifier le nombre de colonnes utiles pour le contrôle :



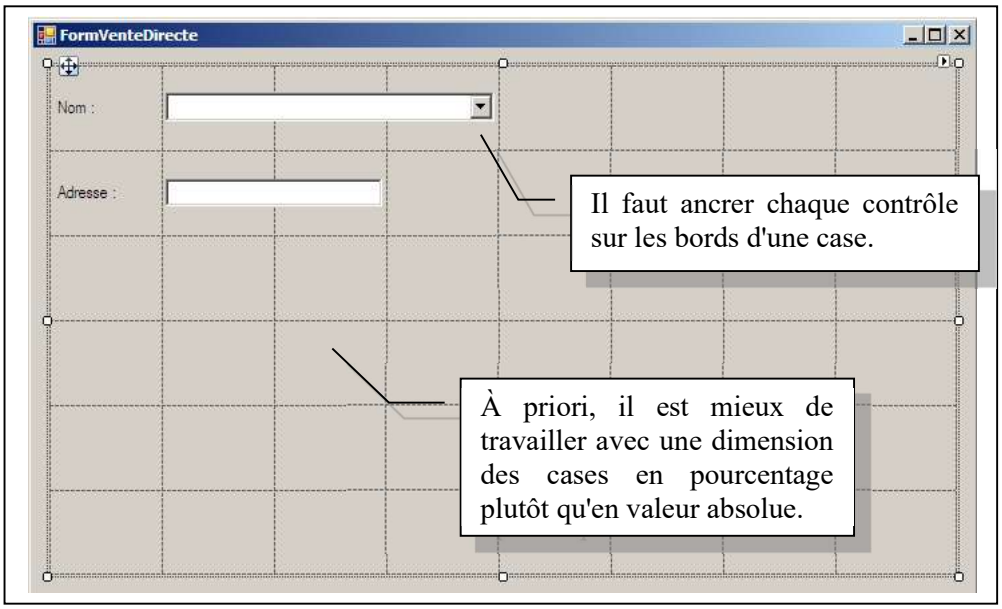
Pour l'alignement du contrôle à l'intérieur d'une case du tableau, il faut utiliser la propriété *Anchor* du contrôle.

Trucs et astuces pour manipuler ce type de contrôle

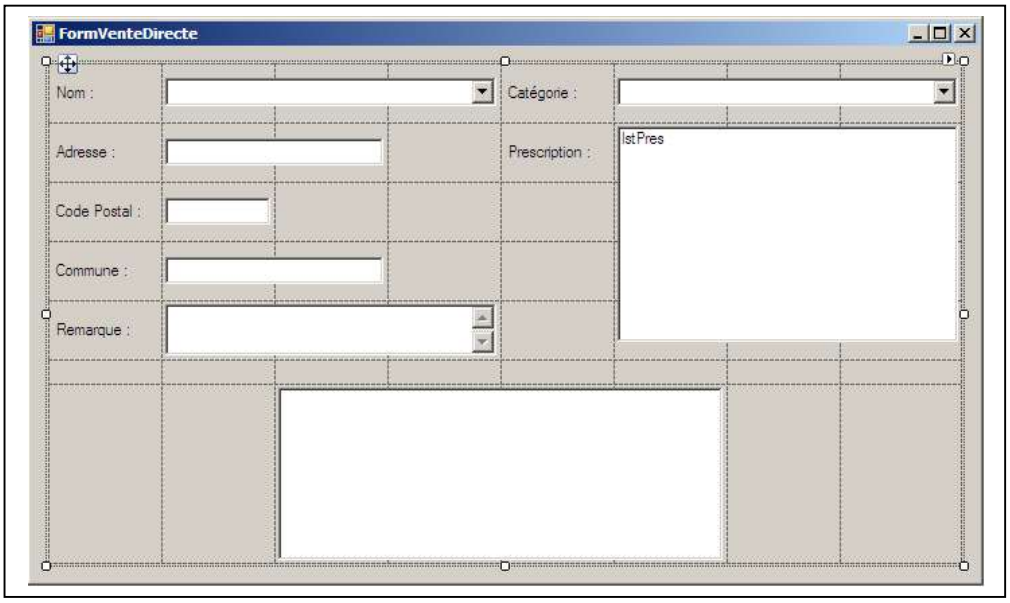
Idéalement, au moment de la création du contrôle, il faut spécifier un nombre de colonnes et de lignes suffisant : on créé une matrice composée de cases dans lesquelles on va ajouter les contrôles du formulaire. L'objectif est d'ajouter les contrôles dans la table en les ancrant sur les côté gauche et droit d'une case de la table. Cela afin de garantir que la longueur de contrôle varie correctement avec la taille du formulaire. Si on veut que la hauteur varie également, il faudra alors ancrer le contrôle aux bords supérieur et inférieur de la case.

Les contrôles ajoutés dans les cases de la matrice peuvent couvrir soit plusieurs lignes (propriété *rowspan*) et/ou plusieurs colonnes (propriété *columnSpan*).

On peut commencer par quelque chose comme ceci :



Résultat final :



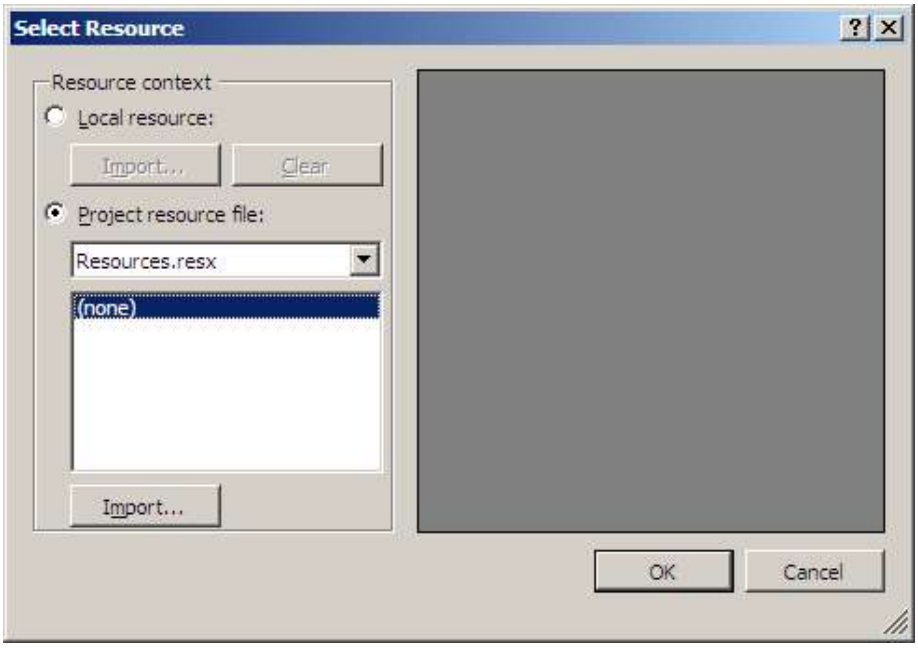
9.7.7. Les contrôles graphiques

9.7.7.A. Le contrôle PictureBox

Le contrôle *PictureBox* est utilisé pour afficher des images dans une application. Plusieurs formats de fichiers sont pris en charge par le contrôle :

- Les fichiers Bitmap (*.bmp),
- Les fichiers Icônes (*.ico),
- Les fichiers Gif (*.gif),
- Les métafichiers (*.wmf),
- Les fichiers jpeg (*.jpg).

Le contenu du contrôle est spécifié dans la propriété *Image*. Dans la fenêtre de propriétés, on peut rechercher le fichier à charger dans le contrôle grâce à la boîte de dialogue ci-après :



Le bouton *effacer* permet de réinitialiser la propriété pour éliminer un fichier déjà présent.

Pour modifier l'image affichée, par l'intermédiaire du code, on doit charger le contenu du fichier et l'affecter à la propriété *Image* qui attend une instance de la classe *Image*. Pour créer cette instance, on utilise la méthode statique *FromFile* de la classe *Image* qui prend comme paramètre le nom du fichier et qui renvoie l'instance de la classe *Image* créée. L'usage du nom complet peut s'avérer utile car le nom *Image* peut être défini dans plus d'un *namespace*. Attention, la chaîne de caractères utilisée pour le nom de fichier ne peut pas être vide.

```
pictPictureBox.Image = System.Drawing.Image.FromFile("file.jpg");
```

Pour effacer une image par le code, on peut utiliser le code suivant :

```
pictPictureBox.Image = null;
```

Par l'intermédiaire de la propriété *SizeMode*, on peut choisir comment le contrôle et l'image vont adapter leur taille respective :

- *Normal* : les deux éléments conservent leurs dimensions. L'image est placée dans le coin supérieur gauche du contrôle.
- *StretchImage* : l'image est agrandie pour occuper toute la surface du contrôle. Dans le cas d'images non vectorielles, le résultat est parfois décevant.
- *AutoSize* : le contrôle s'adapte à la taille de l'image.
- *CenterImage* : les deux éléments conservent leurs dimensions mais l'image est centrée par rapport au contrôle.
- *Zoom* : l'image est agrandie ou rétrécie pour occuper toute la largeur ou la hauteur du contrôle mais le même ratio est utilisé verticalement et horizontalement.

9.7.7.B. Le contrôle ImageList

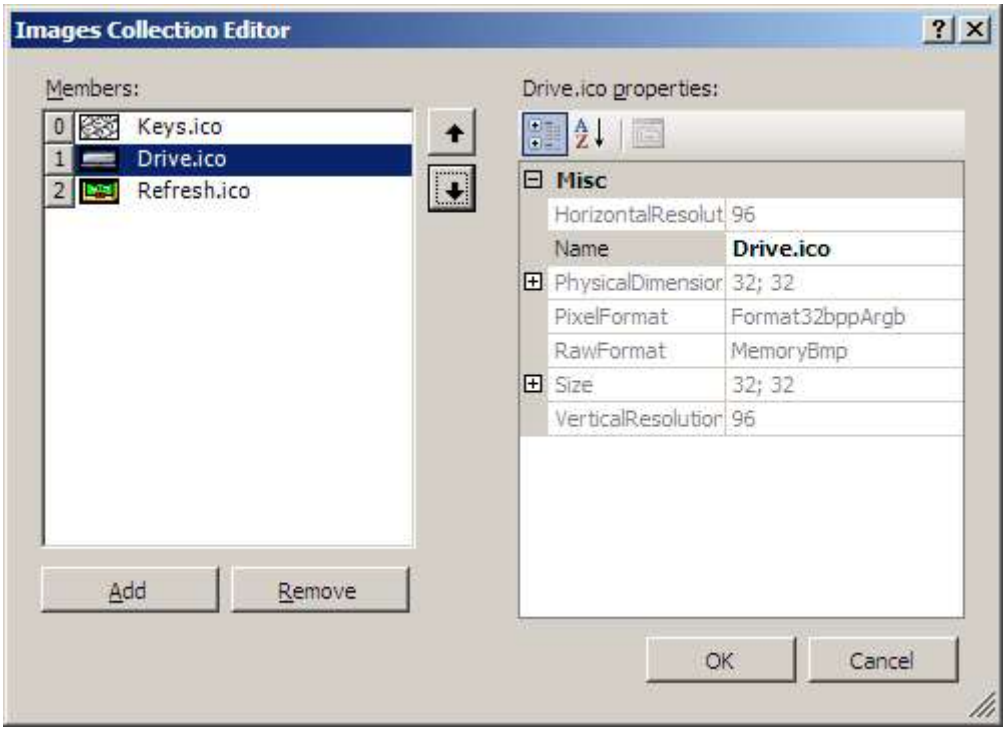
Le contrôle *ImageList* est utilisé dans une application pour servir de *réservoir d'images*. Il ne fournit aucune fonctionnalité pour l'affichage des images, mais permet simplement leur stockage dans l'application. De nombreux types de fichiers peuvent être utilisés dans ce contrôle (bmp, ico, gif, jpg,…). Toutefois, les images stockées dans ce contrôle ne sont pas forcément du même type. On peut, par exemple, mélanger des fichiers *bmp* et des fichiers *ico*. Une petite restriction : les images provenant de metafile (wmf) ne sont pas prises en compte.

Ce contrôle n'a pas d'interface visible au moment de l'exécution. Il ne sera donc pas placé sur la feuille mais dans une zone réservée à ce type de contrôles.

Ce contrôle sera utilisé en association avec tout type de contrôle possédant une propriété *ImageList, SmallImageList* ou *LargeImageList*. Pour ces contrôles, une autre propriété, en général la propriété *ImageIndex*, indique l'image stockée dans le contrôle *ImageList* que l'on veut lui associer. De cette manière, on peut rapidement modifier l'image affichée sur un bouton en modifiant la propriété *ImageIndex* du bouton, afin de sélectionner l'une des images disponibles dans le contrôle *ImageList*.

La propriété *Images* est la propriété principale du contrôle *ImageList*. C'est une collection dans laquelle les images sont stockées. Chaque image est accessible par l'intermédiaire de son index dans la collection.

Cette propriété peut être modifiée au moment de la conception de l'application, par l'intermédiaire de la fenêtre de l'éditeur de collection d'images affichée lorsque l'on modifie la propriété *Images* du contrôle.



Par l'intermédiaire de cet éditeur, on peut notamment ajouter des images, en supprimer ou modifier leur classement dans la collection. Cependant, il n'y a pas de possibilité de modifier l'image une fois insérée dans la collection. Il faut dans ce cas, la supprimer et en insérer une nouvelle avec le contenu du fichier correspondant.

L'insertion d'une image peut également s'effectuer par l'intermédiaire du code, en utilisant la méthode *Add* disponible dans la collection *Images* du contrôle :

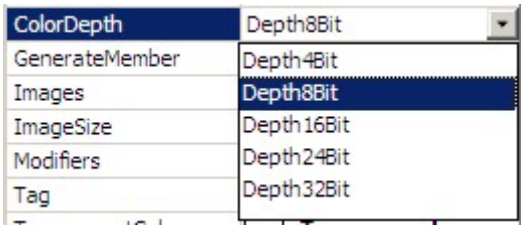
```
imageList1.Images.Add(Image.FromFile("file.bmp"));
```

De la même façon, on peut supprimer, par le code, une image déjà présente dans la liste en utilisant la méthode *RemoveAt* et en indiquant l'index de l'élément à supprimer.

```
imageList1.Images.RemoveAt(2);
```

La méthode *Clear* permet de vider complètement la collection *Images* supprimant ainsi toutes les images du contrôle *ImageList*.

Pour toutes les images de la liste, on peut également indiquer par l'intermédiaire de la propriété *ColorDepth* le nombre de couleurs gérées par le contrôle *ImageList*.



La propriété *ImageSize* indique la taille des images. Cette taille sera identique pour toutes les images placées dans le contrôle. Les images insérées seront éventuellement agrandies ou diminuées pour s'adapter à cette taille.

9.7.8. Les contrôles de gestion du temps

Deux contrôles sont disponibles pour la saisie de dates. Le contrôle *DateTimePicker* permet la saisie d'une date alors que le contrôle *MonthCalendar* autorise la saisie d'une plage de dates.

9.7.8.A. Le contrôle DateTimePicker

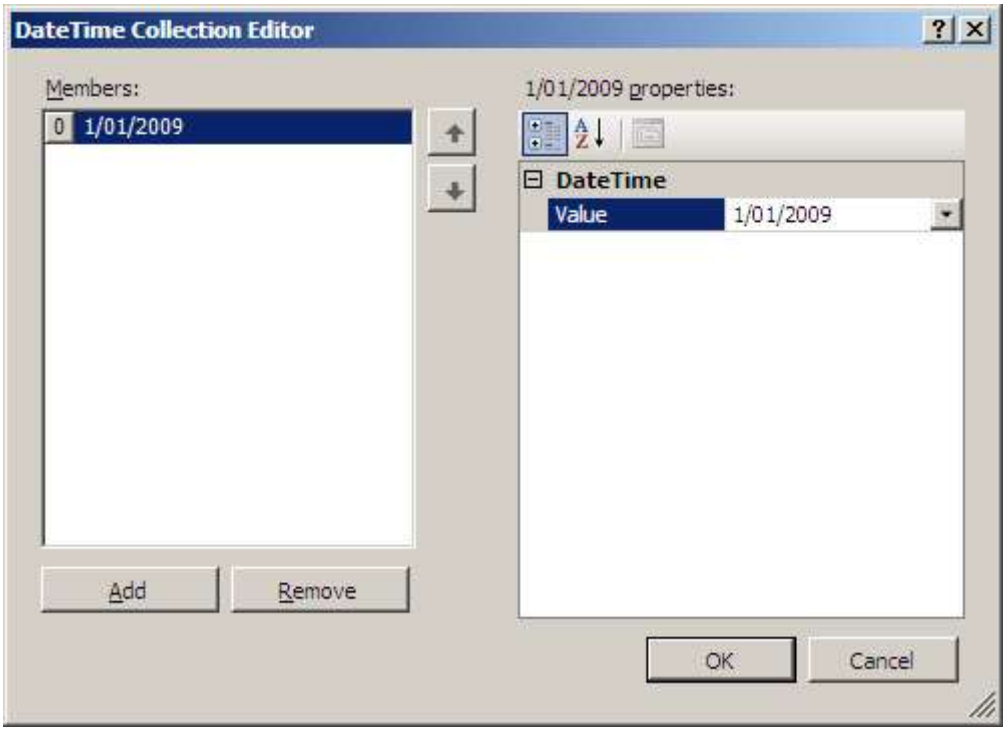
Ce contrôle associe une zone de texte et un calendrier. La date sera soit saisie directement dans la zone de texte (avec une validation des données introduites), soit beaucoup plus facilement en affichant le calendrier en utilisant le bouton situé à côté de la zone de texte (comme pour ComboBox) :



La propriété *Value* permet de récupérer la date saisie ou au contraire de spécifier la date sélectionnée sur le calendrier à son affichage. Par défaut, le calendrier est initialisé avec la date du jour.

9.7.8.B. Le contrôle MonthCalendar

Le contrôle *MonthCalendar* correspond, en fait, au calendrier associé au contrôle *DateTimePicker* avec bien sûr des fonctionnalités supplémentaires. On peut par exemple, indiquer une liste de dates qui apparaîtront en gras sur le calendrier (par exemple, les jours fériés). La propriété *AnnuallyBoldedDates* contient la liste de tous les jours *spéciaux*.



La propriété *MonthlyBoldedDates* permet d'indiquer les jours qui apparaîtront en gras chaque mois.

L'intérêt principal de ce contrôle est qu'il permet la sélection d'une plage de dates. La propriété *MaxSelectionCount* indique le nombre de jours maximal de la sélection (par défaut, 7). Il reste cependant un problème car il n'y a pas de solution simple pour choisir une plage de dates *à cheval* sur plusieurs mois. Les calendriers des mois compris dans la sélection doivent être tous visibles, pour permettre la sélection d'une plage sur plusieurs mois.

La solution consiste donc à configurer le contrôle pour qu'il affiche plusieurs mois simultanément. La propriété *CalendarDimensions* fixe le format du calendrier.



On peut, ensuite, obtenir l'intervalle sélectionné en utilisant la propriété *SelectonRange* qui contient elle-même une propriété *Start* et une propriété *End*. L'événement *DateSelected* se déclenche lorsque l'une des bornes de la sélection est modifiée. On peut donc l'utiliser pour mettre à jour une zone de texte, comme par exemple :

!!! problèmes avec les strings

9.7.8.C. Le contrôle Timer

Le contrôle *Timer* va permettre de déclencher des événements à intervalles réguliers dans l'application. Ce contrôle est très simple d'utilisation puisqu'il ne possède que deux propriétés.

- *Interval* : indique le délai entre deux événements. Ce délai est exprimé en millisecondes.
- *Enabled* : indique si le contrôle génère des événements ou s'il est inactif.

À chaque expiration du délai, un événement *Tick* est déclenché.

Il faut cependant être prudent lors de l'utilisation de ce contrôle en tenant compte des remarques suivantes :

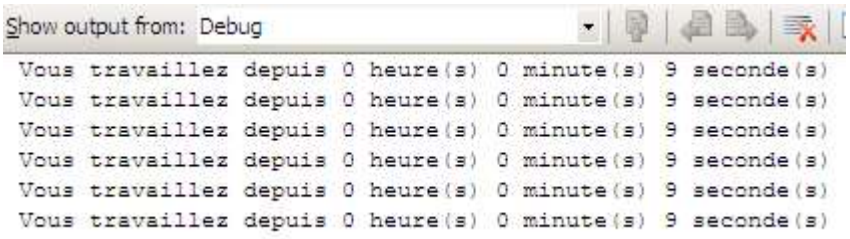
- Si le système est très chargé, il se peut que les événements *Tick* ne soient pas déclenchés régulièrement.

- La précision du contrôle *Timer* n'est pas digne d'une montre Suisse. Il ne faut pas, par exemple, incrémenter une variable sur l'événement *Tick* du contrôle *Timer* pour mesurer une durée, mais plutôt se servir de l'horloge système comme dans l'exemple ci-dessous.

```
private DateTime m_oDuree;

timer1.Enabled = true;
m_oDuree = DateTime.Now;

private void timer1_Tick(object sender, EventArgs e)
{
      TimeSpan oDuree;
      oDuree = DateTime.Now.Subtract(m_oDuree);
      this.Text = "Vous travaillez depuis " + new
                  DateTime(oDuree.Ticks).Hour+
                  " heure(s) " + new DateTime(oDuree.Ticks).Minute +
                  " minute(s) " + new DateTime(oDuree.Ticks).Second +
                  " seconde(s)";
      Console.WriteLine(this.Text);
}
```



9.7.8.D. Le composant BackGroundWorker

Il arrive parfois de devoir effectuer, dans une application, des opérations relativement longues comme par exemple des chargements d'images ou des recherches sur un disque dur. Ces opérations entraînent un blocage de l'application pendant leur exécution.

L'utilisateur a l'impression que l'application ne répond plus et il est parfois tenté de mettre fin à l'application brutalement.

Une solution envisageable est de fournir à l'utilisateur une information sur l'avancement du traitement avec par exemple l'affichage d'une barre de progression. Même avec cette solution, l'utilisateur sera obligé d'attendre la fin du traitement pour pouvoir continuer à utiliser l'application.

Le composant *BackGroundWorker* permet de résoudre ce problème en exécutant ces opérations de façon asynchrone en arrière plan. Le code est exécuté sur un thread différent du thread principal de l'application qui lui, continuera à gérer l'interface de l'application.

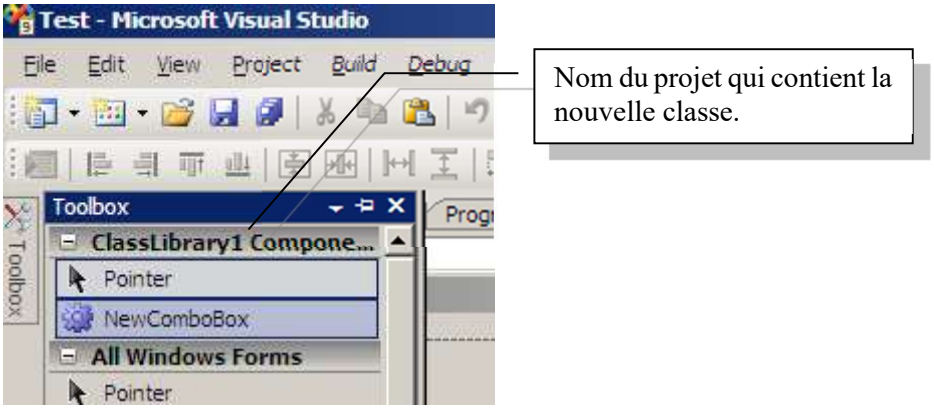
On doit juste indiquer au *BackGroundWorker* le code qu'il doit exécuter puis appeler sa méthode *RunWorkerAsync*. À la fin de l'exécution, celui-ci préviendra en déclenchant l'événement *RunWorkerCompleted*.

9.7.9. Comment ajouter un control à la ToolBox

Si on souhaite ajouter des fonctionnalités à un contrôle existant, il faudra créer une autre classe qui va hériter de la classe que l'on souhaite enrichir. Cela peut se faire très facilement par la déclaration suivante :

```
public class NewComboBox : ComboBox
    {
    }
```

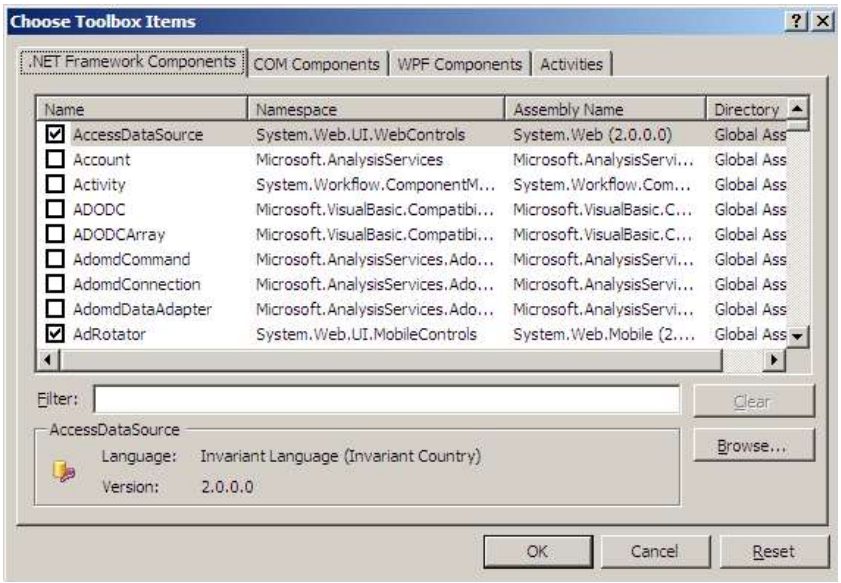
Tout objet déclaré dans la classe *NewComboBox* va hériter de tous les membres de la classe de base *ComboBox*. Si cette classe fait partie de la solution, alors ce nouveau contrôle sera visible et utilisable dans la ToolBox :



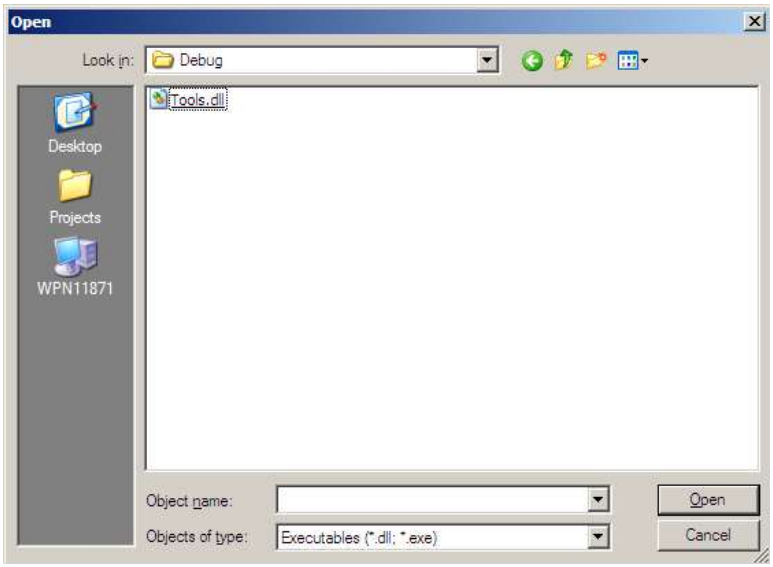
D'une manière générale, on peut créer son propre contrôle à condition de le dériver de la classe *Control*.

Si la classe fait partie d'une autre solution alors pour faire apparaître le nouveau contrôle dans la ToolBox, il faut procéder comme suit :

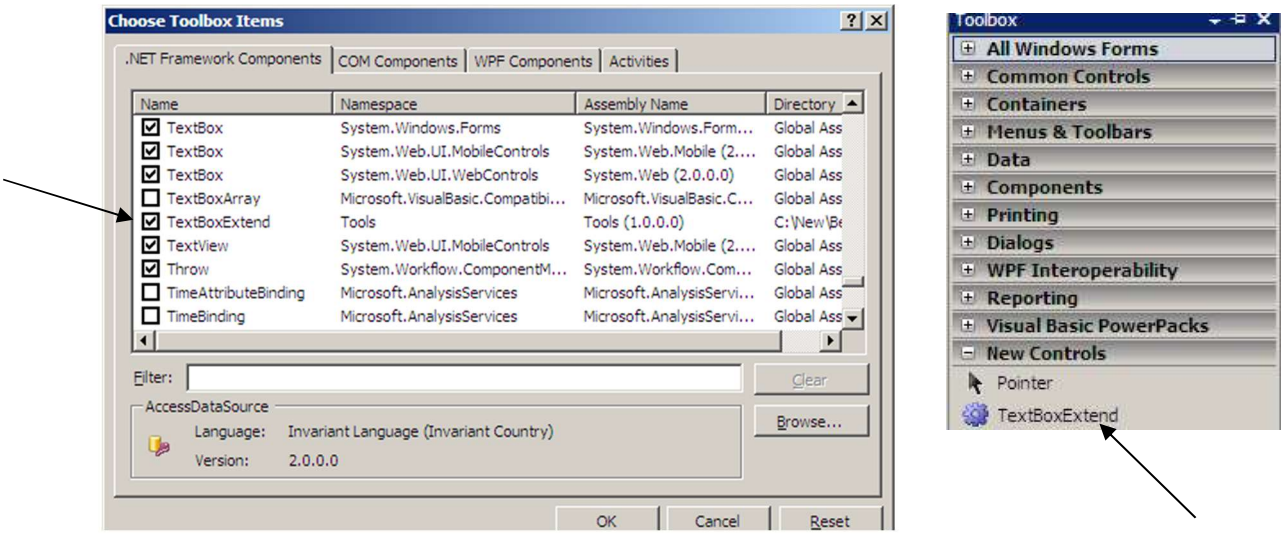
Dans la ToolBox, il faut sélectionner l'option *Choose Items* en cliquant sur le bouton droit dans le menu :



Cliquer sur le bouton *Browse* et sélectionner le fichier .exe ou .dll contenant l'assembly dans lequel se trouve la classe décrivant le nouveau contrôle :



En résultat, le nouveau contrôle est maintenant visible dans le *ToolBox* :



9.8. Autres classes

9.8.1. Directory ou DirectoryInfo

Ces deux classes permettent de manipuler des *directoriesi* et leur contenu. Ainsi par exemple, pour obtenir la liste des fichiers contenus dans une directory :

```
DirectoryInfo dirAnim = new DirectoryInfo(strPath);
FileInfo[] tFileName = dirAnim.GetFiles();
```

10. Gestion des Impressions

Voici tout d'abord la liste des objets et/ou variables nécssaires pour gérer une impression ou une prévisualisation d'une impression.

Type	Description
Font	Cet objet permet de renseigner la police de caractères à utiliser durant l'impression : `new Font(strFont, fSize, oStyle)`
PrintPreviewDialog	Dialog Box permettant de gérer la prévisualisation d'une impression.
PrintDocument	Contrôle a ajouter sur le formulaire. Ce contrôle n'a évidemment pas d'aspect visible mais il va permettre de gérer différents événements liés à l'impression. L'impression d'un document va toujours se déclencher par l'exécution de la fonction *print* sur cet objet. La commande d'impression va déclencher au minimum deux événements: ➢ Le premier est l'événement *BeginPrint* qui se déclenche au début de l'impression. ➢ Le deuxième est l'événement *PrintPage* qui se déclenche à chaque nouvelle page à imprimer.

La taille des marges de gauche et de dessus de page se calculent de la manière suivante :

```
float fLeftMargin = ev.MarginBounds.Left;
float fTopMargin = ev.MarginBounds.Top;
```

Le calcul du nombre de lignes par page se realize de la manière suivante :

```
fLinesPerPage = ev.MarginBounds.Height / m_fntStd.GetHeight(ev.Graphics);
```

avec *m_fntStd* un objet de la classe *Font* qui contient le font utilisé pour l'impression.

L'impression effective d'une ligne se réalise au moyen du code suivant :

```
fYPos = fTopMargin + (dCount * m_fntStd.GetHeight(ev.Graphics));
ev.Graphics.DrawString(strLine, m_fntStd, Brushes.Black, fLeftMargin, fYPos,
new StringFormat());
```

avec :

- *dCount* : variable contenant le nombre de lignes déjà imprimées.
- *strLine* : la ligne à imprimer

à la fin de la fonction liée à l'événement *PrintPage*, on indique si oui ou non il ya encore une page à imprimer par l'instruction suivante :

```
ev.HasMorePages = false;
```

Une impression au départ d'un fichier se réalise très facilement. Pour cela, il faut accéder au fichier de la manière suivante :

1. Il faut saisir le nom complet du fichier a accéder.

2. Il faut utiliser un objet de la classe *StreamReader* :

```
m_srFile = new StreamReader(m_strFileName);
```

Exemple extrait du MSDN :

```
public class PrintingExample
{
    private Font printFont;
    private StreamReader streamToPrint;
    static string filePath;


    public PrintingExample()
    {
        Printing();
    }

    // The PrintPage event is raised for each page to be printed.
    private void pd_PrintPage(object sender, PrintPageEventArgs ev)
    {
        float linesPerPage = 0;
        float yPos =  0;
        int count = 0;
        float leftMargin = ev.MarginBounds.Left;
        float topMargin = ev.MarginBounds.Top;
        String line=null;

        // Calculate the number of lines per page.
        linesPerPage = ev.MarginBounds.Height  /
           printFont.GetHeight(ev.Graphics) ;

        // Iterate over the file, printing each line.
        while (count < linesPerPage &&
           ((line=streamToPrint.ReadLine()) != null))
        {
           yPos = topMargin + (count * printFont.GetHeight(ev.Graphics));
           ev.Graphics.DrawString (line, printFont, Brushes.Black,
              leftMargin, yPos, new StringFormat());
           count++;
        }

        // If more lines exist, print another page.
        if (line != null)
           ev.HasMorePages = true;
        else
           ev.HasMorePages = false;
    }

    // Print the file.
    public void Printing()
    {
        try
        {
           streamToPrint = new StreamReader (filePath);
           try
           {
              printFont = new Font("Arial", 10);
              PrintDocument pd = new PrintDocument();
              pd.PrintPage += new PrintPageEventHandler(pd_PrintPage);
              // Print the document.
              pd.Print();
           }
           finally
           {
              streamToPrint.Close() ;
```

```
            }
        }
        catch(Exception ex)
        {
            MessageBox.Show(ex.Message);
        }
    }

    // This is the main entry point for the application.
    public static void Main(string[] args)
    {
        string sampleName = Environment.GetCommandLineArgs()[0];
        if(args.Length != 1)
        {
            Console.WriteLine("Usage: " + sampleName +" <file path>");
            return;
        }
        filePath = args[0];
        new PrintingExample();
    }
}
```

11. L'accès aux bases de données

11.1. Présentation d'ADO.NET

ADO.NET est un ensemble de classes, d'interfaces, de structures et d'énumérations permettant la manipulation des données. Les différents composants d'ADO.NET permettent de séparer l'accès aux données de la manipulation des données.

ADO.NET facilite également l'utilisation du langage XML, en permettant la conversion de données relationnelles au format XML ou l'importation de données au format XML dans un modèle relationnel.

Deux modes de fonctionnement sont disponibles dans ADO.NET :

- Le mode connecté,
- Le mode non connecté.

11.1.1. Le mode connecté

Dans un environnement connecté, l'application ou l'utilisateur est en permanence connecté à la source de données. Depuis les débuts de l'informatique, c'était le seul mode disponible. Ce mode présente certains avantages dans son fonctionnement :

- Il est facile à gérer : la connexion est réalisée au début de l'application puis est coupée lors de sa fermeture.
- L'accès concurrentiel est plus facile à contrôler : comme tous les utilisateurs sont connectés en permanence, il est plus facile de contrôler lequel travaille sur les données;
- Les données sont à jour : toujours grâce à la connexion permanente aux données, il est facilement envisageable de prévenir toutes les applications utilisant les données que des modifications viennent d'y être apportées.

Par contre, certains inconvénients viennent un peu noircir le tableau :

- La connexion réseau doit être constamment maintenue : en cas d'utilisation de l'application sur un ordinateur portable, l'accès au réseau risque de ne pas être disponible en permanence.
- Il y a un risque de gaspillage de ressources sur le serveur : au moment de l'établissement d'une connexion entre une application cliente et un serveur, des ressources sont allouées sur le serveur pour la gestion de cette connexion. Ces ressources restent monopolisées par la connexion, même si aucune information ne transite par cette connexion.

Cependant, dans certaines situations, l'utilisation d'un mode connecté est incontournable. C'est le cas, par exemple, des applications effectuant des traitements en temps réel;

11.1.2. Le mode non connecté

Un mode non connecté signifie qu'une application ou un utilisateur n'est pas constamment connecté à une source de données. Les applications internet utilisent souvent ce mode de fonctionnement. La connexion aux données est ouverte, les données sont extraites puis la connexion est coupée.

L'utilisateur travaille avec les données, à partir de son navigateur et la connexion est à nouveau ouverte pour la mise à jour de la source de données ou l'obtention d'autres données. Les utilisateurs, travaillant sur des ordinateurs portables, sont également les principaux utilisateurs d'environnements déconnectés. Un médecin, peut par exemple le matin, charger les dossiers médicaux des patients qu'il va visiter dans la journée, puis le soir, fusionner les modifications dans la base de données.

Les avantages du mode déconnecté sont les suivants :

- Les connexions sont utilisées pendant la plus courte durée possible. De cette manière, un petit nombre de connexions disponibles sur un serveur suffisent pour de nombreux utilisateurs.
- Un environnement déconnecté améliore l'évolutivité et les performances d'une application, en optimisant la disponibilité des connexions.

L'environnement déconnecté comporte cependant quelques inconvénients :

- Les données disponibles dans l'application ne sont pas toujours à jour. Par exemple, dans le cas du médecin, si sa secrétaire ajoute des résultats d'analyse après qu'il ait récupéré les dossiers médicaux de ces patients, il ne pourra pas disposer immédiatement des informations.
- Des conflits peuvent parfois survenir lors de la mise à jour des informations dans la base. Ce type de problèmes doit être pris en charge lors de la conception de l'application. Différentes approches sont disponibles pour la gestion de ces conflits :
    - Autoriser la prédominance des mises à jour, les plus récentes, en écrasant les données déjà présentes dans la base.
    - Autoriser la prédominance des mises à jour, les plus anciennes, en abandonnant les nouvelles mises à jour.
    - Prévoir du code permettant à l'utilisateur de choisir ce qu'il souhaite faire en cas de conflit lors d'une mise à jour.

11.1.3. Quelques définitions

Avant d'explorer plus en détail les manières de se connecter à une base de données, il est utile de donner quelques définitions au préalable :

Objet	Description
Connection	Ouvre une connexion vers une source de données spécifique
Command	Exécute une commande sur une source de données
DataReader	Lit un flux de données à partir d'une source de données en mode **connecté**. Le mode d'accès est en lecture seule avec un curseur en avant seulement.
DataSet	Représente un ensemble de données en mode **déconnecté**. Il peut être constitué de plusieurs tables ainsi que des relations et contraintes existant entre elles.
DataAdapter	Remplit un **DataSet** et répercute les mises à jour dans la source de données.

11.2. Utilisation du mode connecté

Dans ce chapitre, on va aborder les opérations pouvant être exécutées sur une base de données, en utilisant le mode connecté. Certaines notions abordées dans ce chapitre seront également utiles pour le fonctionnement en mode déconnecté.

11.2.1. Connexion à une base de données

Pour pouvoir travailler avec un serveur de base de données, une application doit établir une connexion réseau avec le serveur. La classe *SqlConnection* est capable de gérer une connexion vers un serveur SQL Server version 7.0 ou ultérieure. Comme pour tout objet, on doit en premier lieu déclarer une variable.

```
using System.Data.SqlClient;
using System.Data.Odbc;
```

```
SqlConnection oConnect;
OdbcConnection oConnectMS;
```

Ensuite, on doit créer l'instance de la classe et l'initialiser en appelant un constructeur. L'initialisation va consister essentiellement à indiquer les paramètres utilisés pour établir la connexion avec le serveur. Ces paramètres sont définis sous forme d'une chaîne de caractères. Ils peuvent être indiqués lors de l'appel du constructeur ou modifiés par la suite par la propriété *ConnectString*.

Voici la liste des espaces de noms disponibles en ADO.net :

Espace de nom	**Description**
System.Data.SqlClient	Fournisseur de données spécifiques pour SQLServer V7 ou supérieure
System.Data.OleDb	Propose la gestion de sources de données accédées via un driver OleDb
System.Data.Odbc	Propose la gestion de sources de données accédées via un driver Odbc
System.Data.OracleClient	Propose un accès à des sources de données Oracle (v8.1.7 ou supérieure)
System.Data.XML	Propose des classes permettant d'accéder à la fonctionnalité XML sous SQL Server 2000 ou supérieur
System.Data.SQLTypes	Propose des classes pour des types de données spécifiques à Microsoft SQL Server

11.2.1.A. La chaîne de connexion

Le format standard d'une chaîne de connexion est constitué d'une série de couples *mot clé/valeur* séparés par des points virgules. Le signe = est utilisé pour l'affectation d'une valeur à un mot clé. L'analyse de la chaîne est effectuée lors de l'affectation de la chaîne à la propriété *ConnectionString*. Les valeurs associées aux mots clés sont alors extraites et affectées aux différentes propriétés de la connexion.

Si une erreur de syntaxe est trouvée alors une exception est générée automatiquement et aucune propriété n'est modifiée. Par contre, la valeur de certaines propriétés ne pourra être contrôlée que lors de l'ouverture de la connexion. C'est à ce moment là qu'une exception sera générée si la chaîne de connexion contient des erreurs. La chaîne de connexion ne peut être modifiée que si la connexion est fermée. Les mots clés suivants sont disponibles pour une chaîne de connexion :

MOT CLE	SIGNIFICATION
Connect Timeout	Durée en secondes pendant laquelle l'application attendra une réponse du serveur à sa demande de connexion. Passé ce délai, une exception sera déclenchée.
Data Source	Nom ou adresse réseau du serveur vers lequel est établie la connexion. Le numéro du port peut être spécifié à la suite du nom ou de l'adresse réseau. S'il n'est pas indiqué, le numéro du port est égal à 1433.
Initial Catalog	Nom de la base sur laquelle doit s'effectuer la connexion.
Integrated Security	Si cette valeur est positionnée sur *false* alors un nom d'utilisateur et un mot de passe doivent être fournis dans la chaîne de connexion. Sinon, le compte Windows de l'utilisateur est utilisé pour l'authentification.
Persist Security Info	Si cette valeur est positionnée sur *true*, alors le nom de l'utilisateur et son mot de passe sont accessibles par la connexion. Pour des raisons de sécurité, cette valeur doit être positionnée sur *false*. C'est d'ailleurs le cas si on n'indique rien dans la chaîne de connexion.
Pwd	Mot de passe associé au compte SQL Server utilisé pour la connexion. S'il n'y a pas de mot de passe associé à un compte, cette information peut être omise dans la chaîne de connexion.

User ID	Nom du compte SQL Server utilisé pour la connexion
Connection LifeTime	Indique la durée de vie d'une connexion dans un pool de connexions. Une valeur égale à zéro indique une durée de vie infinie.
Connection Reset	Indique si la connexion est réinitialisée lors de sa remise dans le pool.
Max Pool Size	Nombre maximum de connexion dans le pool.
Min Pool Size	Nombre minimum de connexions dans le pool.
Pooling	Indique si la connexion peut être extraite d'un pool de connexion.

Table 21: Liste des mots clés d'une chaîne de connexion

Une chaîne de connexion prend donc la forme minimale suivante :

```
SqlConnection oConnect;
oConnect = new SqlConnection();
oConnect.ConnectionString = "Data Source = localhost;Initial Catalog =
                             Veterinaire;Integrated Security = true";

OdbcConnection oConnectMS;
oConnectMS = new OdbcConnection();
oConnectMS.ConnectionString = "DSN=veterinaire";
oConnectMS.Open();
```

An ODBC connection string has the following syntax:

```
connection-string ::= empty-string[;] | attribute[;] | attribute; connection-string
empty-string ::=
attribute ::= attribute-keyword=attribute-value | DRIVER=[{]attribute-value[}]
attribute-keyword ::= DSN | UID | PWD
 | driver-defined-attribute-keyword
attribute-value ::= character-string
driver-defined-attribute-keyword ::= identifier
```

11.2.1.B. Le Pool de connexion

Les pools de connexion permettent d'améliorer les performances d'une application, en évitant la création de connexions supplémentaires. Lorsqu'une connexion est ouverte, un pool de connexion est créé en se basant sur un algorithme basé, lui-même sur la chaîne de connexion. Chaque pool de connexion est donc associé à une chaîne de connexion particulière.

Si une nouvelle connexion est ouverte et qu'il n'existe pas de pool correspondant exactement à sa chaîne de connexion, alors un nouveau pool est créé. Les pools de connexion ainsi créés existeront jusqu'à la fin de l'application. Lors de la création du pool, d'autres connexions existeront peuvent être créées automatiquement pour satisfaire la valeur *Min Pool Size* indiquée dans la chaîne de connexion. D'autres connexions pourront, par la suite, être ajoutées au pool jusqu'à atteindre la valeur *Max Pool Size* de la chaîne de connexion.

Lorsqu'une connexion est requise, elle peut être obtenue à partir d'un pool de connexion (s'il en existe un correspondant exactement aux caractéristiques de la connexion demandée). Il faut bien sûr que le pool en contienne une disponible et active.

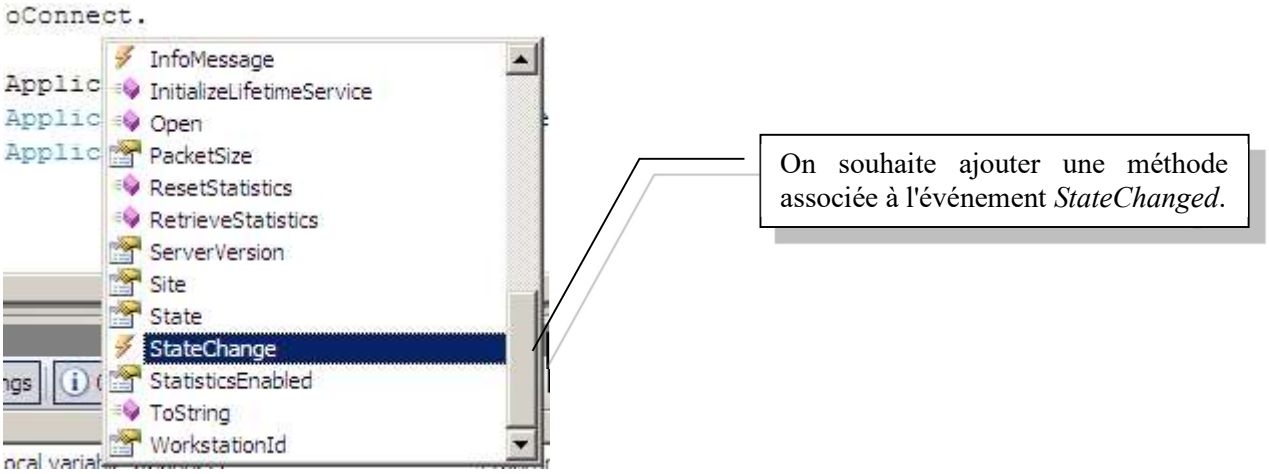
Si le nombre maximum de connexion dans le pool est atteint, la demande est mise en file d'attente jusqu'à ce qu'une connexion soit à nouveau disponible. Une connexion est remise à la disposition du pool, lors de sa fermeture ou lors de l'appel de la méthode *Dispose* sur la connexion. Pour cette raison, il est recommandé de fermer explicitement les connexions lorsqu'elles ne sont plus utilisées dans l'application.

Les connexions sont retirées du pool lorsque celui-ci détecte que la connexion n'a pas été utilisée depuis un certain temps, indiqué par la valeur *ConnectionLifeTime* de la chaîne de connexion. Elles sont également retirées du pool, s'il détecte que la connexion avec le serveur a été interrompue.

11.2.1.C. Événements de connexion

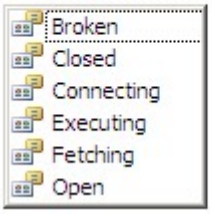
La classe *SQLConnection* propose deux événements permettant d'être prévenu lorsque l'état de la connexion change ou qu'un message d'information est envoyé par le serveur :

- L'événement *StateChanged* est déclenché lors d'un changement d'état de la connexion.le gestionnaire de cet événement reçoit un paramètre de type *StateChangeEventArg* permettant d'obtenir, avec la propriété *CurrentState*, l'état actuel de la connexion et avec la propriété *OriginalState*, l'état de la connexion avant le déclenchement de l'événement. Pour tester la valeur de ces deux propriétés, on peut utiliser l'énumération *ConnectionState*.




```
oConnect.StateChange += new
           System.Data.StateChangeEventHandler(etatConnection);

public void etatConnection (Object sender, StateChangeEventArgs e)
{
     if (e.CurrentState == ConnectionState.
```




```
oConnect.StateChange += new StateChangeEventHandler(etatConnection);
```

- L'événement *InfoMessage* est déclenché lorsque le serveur informe d'une situation anormale, mais qui ne justifie pas le déclenchement d'une exception (sévérité du message inférieure à 10). Le gestionnaire d'événements associé reçoit un paramètre de type *InfoMessageEventArgs*. Par la propriété *Errors* de ce paramètre, on a accès à des objets *SqlErrors* correspondant aux informations envoyées par le serveur. Le code suivant affiche sur la console, les messages d'information en provenance du serveur :

```
public static void InfoMessage(object sender, SqlInfoMessageEventArgs e)
{
     foreach (SqlError oInfo in e.Errors)
     {
```

```
            Console.WriteLine(oInfo.Message);
        }
    }

    oConnect.InfoMessage += new SqlInfoMessageEventHandler(InfoMessage);
```

11.2.2. Exécution d'une commande

Après avoir établi une connexion vers un serveur de base de données, on peut lui transmettre des instructions SQL. La classe *SqlCommand* est utilisée pour demander au serveur l'exécution d'une commande SQL. Cette classe contient plusieurs méthodes permettant l'exécution de différents types de requêtes SQL. La classe *SqlCommand* peut être instanciée de manière classique, en utilisant un de ses constructeurs ou une instance peut être obtenue par la méthode *CreateCommand* de la connexion.

11.2.2.A. Création d'une commande

La première possibilité de créer une *SqlCommand* est d'utiliser un des constructeurs de la classe. L'utilisation du constructeur par défaut oblige par la suite à utiliser différentes propriétés, pour fournir les informations nécessaires à l'exécution de l'instruction SQL.

La propriété *CommandText* contient le texte de l'instruction SQL à exécuter. La propriété *Connection* doit faire référence à une connexion valide vers le serveur de base de données. Le code suivant résume ces différentes opérations :

```
SqlCommand oCmd;
oCmd = new SqlCommand();

oCmd.Connection = oConnect;
oCmd.CommandText = "select * from Animal";
```

La deuxième solution est d'utiliser un constructeur surcharge, acceptant comme paramètres, l'instruction SQL sous forme d'une chaîne de caractères et la connexion utilisée par cette SqlCommand. Le code précédent peut donc se résumer comme suit :

```
oCmd = new SqlCommand("select * from Animal", oConnect);
```

La troisième solution est d'utiliser la méthode *CreateCommand* de la connexion. Dans ce cas, seule l'instruction SQL a besoin d'être spécifiée par la suite :

```
SqlCommand oCmd;
oCmd = oConnect.CreateCommand();
oCmd.CommandText = "select * from Animal";
```

11.2.2.B. Lecture d'informations

Fréquemment, l'instruction SQL d'une *SqlCommand* sélectionne un ensemble d'enregistrements dans la base de données, ou éventuellement une valeur unique étant le résultat d'un calcul effectué sur des valeurs continues dans la base. Une instruction SQL, renvoyant un ensemble d'enregistrements, doit être exécutée par la méthode *ExecuteReader*.

Cette méthode retourne un objet *DataReader* qui va permettre, par la suite, la lecture des informations en provenance de la base de données. Si l'instruction SQL ne renvoie qu'une valeur unique, la méthode *ExecuteScalar* se charge de l'exécution et retourne elle-même la valeur en provenance de la base de données.

Le code suivant permet la récupération du nombre d'animaux possédés par un propriétaire :

```
SqlConnection oConnect;
oConnect = new SqlConnection();
oConnect.ConnectionString = "Data Source = localhost;Initial Catalog =
     Veterinaire;Integrated Security = true";
oConnect.Open();

SqlCommand oCmd;
oCmd = new SqlCommand();

oCmd.Connection = oConnect;
oCmd.CommandText = "select count(AnimID) from Animal where PropID = 8";

Console.WriteLine("Le propriétéaire 8 possède {0} animaux",
                  oCmd.ExecuteScalar());
```

Le cas d'instructions renvoyant plusieurs enregistrements est un peu plus complexe. Après avoir exécuté l'instruction par la méthode *ExecuteReader* et récupéré l'objet *DataReader*, on peut utiliser ce dernier pour parcourir les résultats renvoyés.

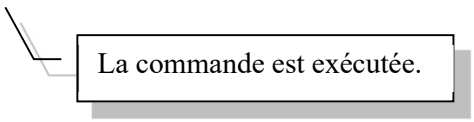
La méthode *Read* de la classe *DataReader* permet le déplacement dans l'ensemble des enregistrements renvoyés. Cette méthode retourne un booléan indiquant s'il reste un enregistrement suivant; le déplacement n'est possible que du premier au dernier enregistrement. Ce type de déplacement est appelé *Forward Only*. Les informations contenues dans l'enregistrement courant sont accessibles par une des méthodes *Get…* de la classe *DataReader*.

Ces méthodes permettent d'extraire les données de l'enregistrement et de les convertir dans un type de données .NET. Il en existe une version pour chaque type de données du framework .NET. Il faut bien sûr que les informations présentes dans l'enregistrement, puissent être converties dans le type correspondant. Si la conversion est impossible, il y a déclenchement d'une exception.

Les méthodes *Get* attendent, comme paramètre, le numéro du champ à partir duquel elles récupèrent l'information. On peut aussi utiliser la propriété, par défaut, du *DataReader* en indiquant le nom du champ concerné. Il n'y a pas dans ce cas, de conversion et la valeur renvoyée est de type *Object*.

Le code suivant affiche la liste de tous les noms des animaux du propriétaire :

```
OdbcDataReader oLecteur;
oCmdMS.CommandText = "select * from Animal where PropID = 8";
oLecteur = oCmdMS.ExecuteReader();
```




```
while (oLecteur.Read())
{
         Console.WriteLine("Nom de l'animal : {0} \tDate Naissance : {1}",
             oLecteur["Nom"], oLecteur["DateNaiss"]);
}
oLecteur.Close();
oConnectMS.Close();
```

Remarques :

- Les opérateurs *cast* ne fonctionnent pas sur un objet qui ne possède pas de valeur. Afin de savoir si l'objet référencé contient ou non une valeur, on peut utiliser le test suivant :

```
m_oRecordSetAnim["Image"] == DBNull.Value
```

11.2.2.C. Modification des informations

La modification des informations dans une base de données s'effectue principalement par les instructions *SQL Insert, Update* et *Delete*. Ces instructions ne retournent pas d'enregistrements en provenance de la base de données. Pour utiliser ces instructions, on doit créer une *SqlCommand ou OdbcCommand*, puis demander l'exécution de cette commande par la méthode *ExecuteNonQuery*. Cette méthode retourne le nombre d'enregistrements affectés par l'exécution de l'instruction SQL contenue dans la *SqlCommand ou OdbcCommand*;

Si la propriété *CommandText* contient plusieurs instructions SQL, alors la valeur renvoyée par la méthode *ExecuteNonQuery* correspond au nombre total de lignes affectées par toutes les instructions SQL de la *SqlCommand ou OdbcCommand*.

Le code suivant ajoute un nouvel *animal* dans la table *Animal* :

```
oConnectMS.Open();
oCmdMS.CommandText = "Insert into Animal (Nom, DateNaiss, sexe, PropID)" +
                     "values ('NewAnimal', '25/12/2008', 'Mâle', '8')";

Console.WriteLine("{0} ligne(s) ajoutée(s) dans la table",
                     oCmdMS.ExecuteNonQuery());
oConnectMS.Close();
```

11.2.2.D. Utilisation de paramètres

La manipulation d'instructions SQL peut être facilitée par la création de paramètres. Ils permettent de construire des instructions SQL génériques, pouvant facilement être réutilisées. Le principe de fonctionnement est semblable aux procédures et fonctions de Visual C#. une alternative à l'utilisation de paramètres pourrait être la construction dynamique d'instructions SQLL par concaténation de chaînes de caractères.

Ci-dessous, un exemple utilisant cette technique et permettant la recherche d'un animal par son nom : on verra ensuite comment améliorer ce code en utilisant des paramètres.

```
Console.Write("Saisir le nom de l'animal recherché : ");
string strNomANim;

strNomANim = Console.ReadLine();
oCmdMS.CommandText = " Select * from Animal where Nom = '" + strNomANim +
                                "'";
oLecteur = oCmdMS.ExecuteReader();

while (oLecteur.Read())
{
     Console.WriteLine("Robe de l'animal : {0}", oLecteur["Robe"]);
}
```

La partie importante de ce code se situe lors de l'affectation d'une valeur à la propriété *CommandText*. Une instruction SQL correcte doit être construite par concaténation de chaînes de caractères. Dans l'exemple ci-dessus, c'est relativement simple puisqu'il n'y a qu'une valeur variable dans l'instruction SQL, mais si plusieurs informations doivent varier, il y a une multitude de concaténations à réaliser.

Les erreurs classiques dans ces concaténations sont :

- L'oubli d'un espace,
- L'oubli des caractères '' pour encadrer une valeur de type chaîne de caractères,
- Un nombre impair de '.

Toutes ces erreurs ont, pour même effet, la création d'une instruction SQL invalide qui sera rejetée à son exécution par le serveur.

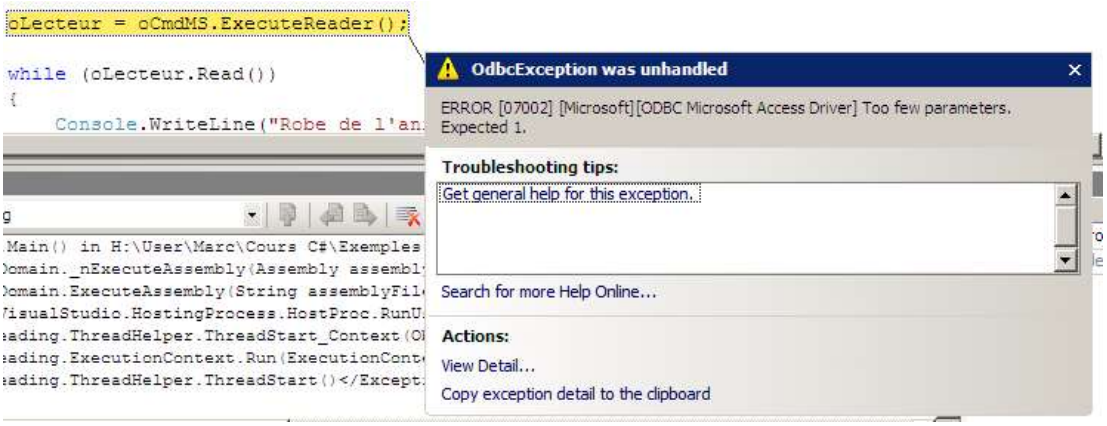
L'utilisation de paramètres simplifie considérablement l'écriture de ce type de requête. Les paramètres sont utilisés pour marquer un emplacement dans une requête où sera placé, au moment de l'exécution, une chaîne de caractères ou chaîne numérique. Les paramètres peuvent être nommés ou anonymes. Un paramètre anonyme est introduit dans une requête par le caractère '?'. Les paramètres nommés sont spécifiés par le caractère @ suivi du nom du paramètre.

La requête de l'exemple précédent peut prendre les formes suivantes :

```
oCmdMS.CommandText = " Select * from Animal where Nom = ?";

oCmdMS.CommandText = " Select * from Animal where Nom = @Nom";
```

L'exécution de la *SqlCommand* échoue maintenant si aucune information n'est fournie pour le ou les paramètres.



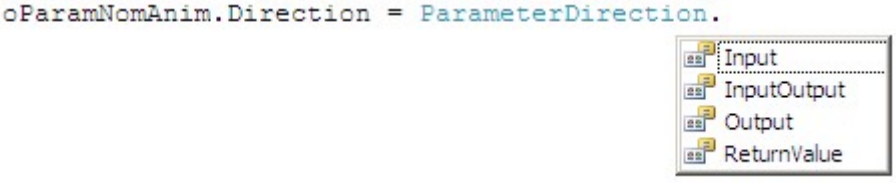
La *SqlCommand* doit avoir une liste de valeurs utilisées pour le remplacement des paramètres, au moment de l'exécution. Cette liste est stockée dans la collection *Parameters* de la *SqlCommand.* Avant l'exécution de la *SqlCommand*, il faut donc créer les objets *SqlParameter* et les ajouter à la collection. Pour chaque *SqlParameter*, il faut fournir :

- Le nom du paramètre,
- La valeur du paramètre,
- La direction d'utilisation du paramètre.

Les deux premières informations sont renseignées lors de la construction de l'objet :

```
OdbcParameter oParamNomAnim;
oParamNomAnim = new OdbcParameter("@Nom", strNomANim);
```

La direction d'utilisation indique si l'information contenue dans le paramètre est passée au code SQL pour son exécution (Input) ou si c'est l'exécution du code SQL qui va modifier la valeur du paramètre (Output) ou les deux (InputOutput). La propriété *Direction* de la classe *SqlParameter* indique le mode d'utilisation du paramètre.



Le paramètre est maintenant prêt à être ajouté à la collection *Parameters*. Il convient d'être vigilent à ce niveau si la requête utilise les paramètres anonymes. Les paramètres doivent obligatoirement être ajoutés à la collection, dans

l'ordre de leur apparition dans la requête. Si les paramètres nommés sont utilisés, il n'est pas obligatoire de respecter cette règle, mais il est prudent de s'y conformer, si un jour le code SQL est modifié et n'utilise plus les paramètres nommés. Ceci pourrait être le cas si on doit changer de fournisseur de données et que le nouveau n'accepte pas les paramètres nommés dans une instruction SQL (c'est le cas de MS Access, par exemple).

```
oConnectMS = new OdbcConnection();
oConnectMS.ConnectionString = "DSN=veterinaire";
oConnectMS.Open();

OdbcCommand oCmdMS;

oCmdMS = new OdbcCommand();
oCmdMS.Connection = oConnectMS;

OdbcDataReader oLecteur;
string strNomAnim;

strNomAnim = "MINET";

oCmdMS.CommandText = "Select * from Animal where Nom = ?";

OdbcParameter oParamNomAnim;

oParamNomAnim = new OdbcParameter();
oParamNomAnim.Value = "MINET";
oParamNomAnim.Direction = ParameterDirection.Input;
oCmdMS.Parameters.Add(oParamNomAnim);

oLecteur = oCmdMS.ExecuteReader();

while (oLecteur.Read())
{
     Console.WriteLine("Robe de l'animal : {0}", oLecteur["Robe"]);
}
oLecteur.Close();
```

11.2.3. Exécution de procédures stockées

Les procédures stockées (stored procedures) sont des éléments d'une base de données correspondant à un ensemble d'instructions SQL, pouvant être exécutées par simple appel de leur nom. Ce sont de véritables programmes SQL pouvant recevoir des paramètres et renvoyer des valeurs. De plus, les procédures stockées sont enregistrées dans la mémoire cache du serveur, sous forme compilée lors de leur première exécution, ce qui accroît les performances pour les exécutions suivantes.

Un autre avantage des procédures stockées est de centraliser sur le serveur de base de données tous les codes SQL d'une application. Si des modifications doivent être apportées dans les instructions SQL, il n'y aura que des modifications à effectuer sur le serveur sans avoir à reprendre le code de l'application, donc sans avoir à regénérer et redéployer l'application.

L'appel à une procédure stockée, à partir de Visual C# , est pratiquement similaire à l'exécution d'une instruction SQL. La propriété *CommandText* contient le nom de la procédure stockée. On peut également modifier la propriété *CommandType* avec la valeur *CommandType.StoredProcedure* pour indiquer que la propriété *CommandText* contient le nom d'une procédure stockée.

Comme pour une instruction SQL, une procédure stockée peut utiliser des paramètres en entrée ou en sortie. Il y a un troisième type de paramètre disponible pour les procédures stockées : le type *ReturnValue*. Ce type de paramètre sert à récupérer la valeur renvoyée par l'instruction *return* de la procédure stockée (même principe qu'une fonction Visual C#).

11.3. Utilisation du mode non connecté

Dans un mode non connecté, la liaison avec le serveur de base de données n'est pas permanente. Il faut donc conserver localement les données sur lesquelles on souhaite travailler. L'idée est de recréer, à l'aide de différentes classes, une organisation similaire à celle d'une base de données. Les principales classes sont représentées sur le schéma suivant :

Classe	Signification
Dataset	C'est le conteneur de plus haut niveau, il joue le même rôle que la base de données.
DataTable	Comme son nom l'indique, c'est l'équivalent d'une table de base de données.
DataRow	Cette classe joue le rôle d'un enregistrement (ligne)
DataColumn	Cette classe remplace un champ (colonne) d'une table
UniqueConstraint	C'est l'équivalent de la clé primaire d'une table
ForeignKeyConstraint	C'est l'équivalent de la clé étrangère
DataRelation	Représente un lien parent/enfant

Table 22: Liste des classes utilisables dans le mode non connecté

12. Déploiement d'applications et de composants

Lorsque l'application est terminée, testée et déboguée, il est temps de penser à la mettre à disposition des utilisateurs. Deux solutions sont disponibles :

- La technologie de déploiement Windows Installer. L'application est empactée dans un ou plusieurs fichiers qui sont ensuite distribués aux utilisateurs. Ceux-ci exécutent le fichier *setup.exe* pour installer l'application.
- Le déploiement *ClickOnce*. Avec cette solution, la publication des fichiers de l'application se fait à un emplacement centralisé et l'utilisateur installe ou exécute l'application à partir de cet emplacement.

12.1. Déploiement avec Windows Installer

Microsoft Windows Installer est un service d'installation et de configuration d'application disponible sur tous les systèmes d'exploitation Microsoft. Le principe de base du fonctionnement de Windows Installer repose sur le regroupement dans un seul élément de toutes les données et instructions nécessaires pour le déploiement d'une application. C'est une évolution importante par rapport aux procédures d'installation classiques qui consistaient essentiellement à fournir l'ensemble des fichiers nécessaires pour le bon fonctionnement de l'application et un script chargé de la recopie de ces fichiers sur le disque dur de la machine.

Avec Windows Installer, le système conserve une trace de toutes opérations effectuées pendant l'installation : directories créées, fichiers copiés, entrées de la base de registre modifiées,… Ces informations sont par la suite utilisées lors de la désinstallation de l'application. Windows Installer effectue alors les opérations inverses pendant la désinstallation de l'application. Un contrôle est cependant réalisé pour s'assurer qu'aucune autre application ne nécessite un fichier, une clé de registre ou un composant qui s'apprête à être supprimé. Cette vérification permet de s'assurer que la suppression d'une application n'entraîne pas de problèmes de fonctionnement sur une autre application.

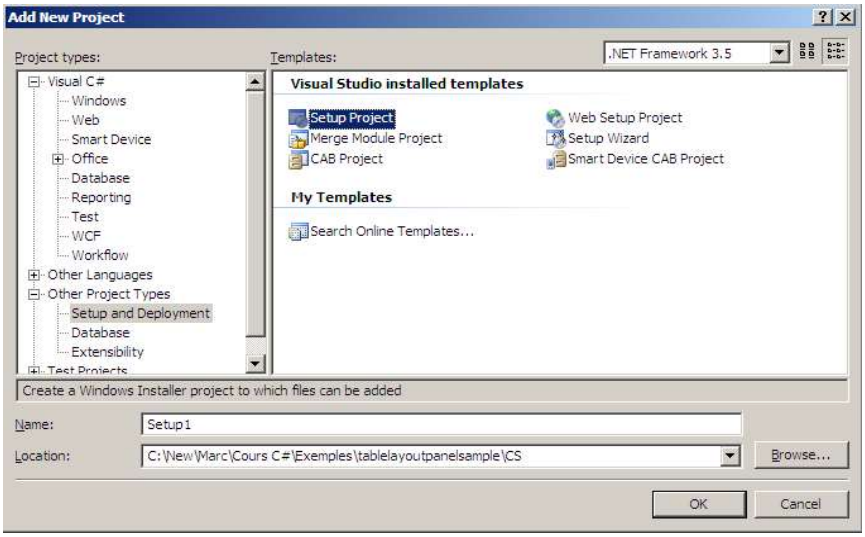
Windows Installer gère également la réparation d'une application en réinstallant automatiquement les fichiers manquants qui ont pu être supprimés, par mégarde, par l'utilisateur.

La procédure d'instllation est effectuée à l'intérieur d'une transaction, garantissant que l'application sera installée complètement ou que, en cas d'échec au cours de l'installation, le système retrouvera son état initial.

Les procédures dinstallation sont en fait de véritables applications, elles sont d'ailleurs gérées par Visual Studio comme des projets à part entière.

12.1.1. Création d'un projet d'installation

La méthode de création d'un projet est identique à celle utilisée pour n'importe que autre type de projet de Visual Studio. Dans le solution explorer, il faut cliquer droit sur le nom de la solution et il faut sélectionner *Ajouter*, puis *Nouveau Projet*. Dans la boîte de dialogue d'*Ajoute de projet*, sélectionner ensuite *Autres types de projets* et ensuite *Projet d'installation et de déploiement*. Enfin il faut choisir parmi les modèles proposés, celui qui correspond aux besoins. Le projet est alors ajouté à la solution actuelle et l'éditeur du système de fichiers est ouvert automatiquement.



On peut maintenant configurer les propriétés du programme d'installation qui vont déterminer son comportement;

Voici ci-dessous, les principales propriétés d'un programme d'installation (attention, ces propriétés s'obtiennent par la vue *Properties* et non avec un clic droit sur le nom du projet) :

- *AddRemoveProgramsIcon* : indique l'icône utilisée pour représenter l'application dans la boîte de dialogue *Ajoute/Suppression de programmes*.

- *Author* : spécifie le nom de l'auteur de l'application. Par défaut, cette propriété correspond au nom du détenteur de la licence de Visual Studio utilisé pour développer l'application.

- *DetectNewerInstalledVersion* : indique si le programme d'installation vérifie l'existence d'une version plus récente de l'application sur le poste client. Si c'est le cas, la procédure d'installation est annulée.

True	The installer will check for a newer version and abort deployment if found.
False	The installer 'will not check for newer versions.

- *ManufacturerUrl* : adrersse du site Web du fabricant de l'application. Elle est affichée dans la boîte de dialogue *Informations de support technique* accessible à partir de la boîte de dialogue *Ajoute/Suppression de programmes*.

- *ProductCode* : Windows Installer utilise cette propriété pour identifier une application lors de chaque installation ou mise à jour ; deux applications différentes ne peuvent pas comporter le même code *ProductCode*.

- *ProductName* : contient le nom décrivant l'application qui est installée sur un ordinateur cible. Par défaut, ce nom est identique à celui du projet de déploiement. Cette propriété *ProductName* est affichée dans la description de l'application sur la boîte de dialogue *Ajoute/Suppression de programmes*. Elle est également utilisée dans la création du chemin d'installation par défaut utilisé lors de l'installation.

- *RemovePreviousVersions* : spécifie si le programme d'installation doit supprimer, lors de l'installation, toute version antérieure d'une application. Si cette propriété à la valeur *True* et qu'un numéro de version antérieure est détectée lors de l'installation, la fonction de désinstallation de cette version est appelée.

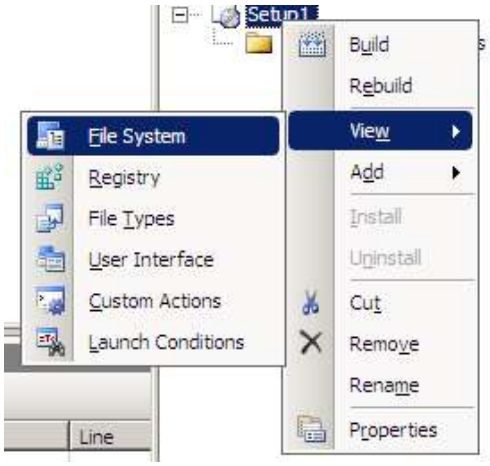
- *TargetPlatform* : spécifie la plate-forme matérielle pour laquelle l'application a été conçue. Au moment de l'installation, cette propriété est érifiée par rapport à l'ordinateur cible pour déterminer si l'installation peut continuer.

- *Version* : spécifie le numéro de version du programme d'installation. Cette propriété doit être modifiée pour chaque version finale du programme d'installation. Si on modifie cette propriété, on doit également mettre à jour la propriété *ProductCode*.

Un petit détail doit sembler bizarre car parmi toutes ces propriétés, aucune d'entre elles ne permet d'indiquer quelle application doit être installée. Cette information doit en fait être fournie lors de la configuration du programme d'installation.

12.1.2. Configuration du programme d'installation

La configuration du fonctionnement du programme d'installation se fait par l'intermédiaire d'un éditeur spécifique à chacune des fonctionnalités. Ces éditeurs sont disponibles par le menu contextuel affiché en effectuant un clic droit sur le nom du projet de déploiement dans l'explorateur de solutions.

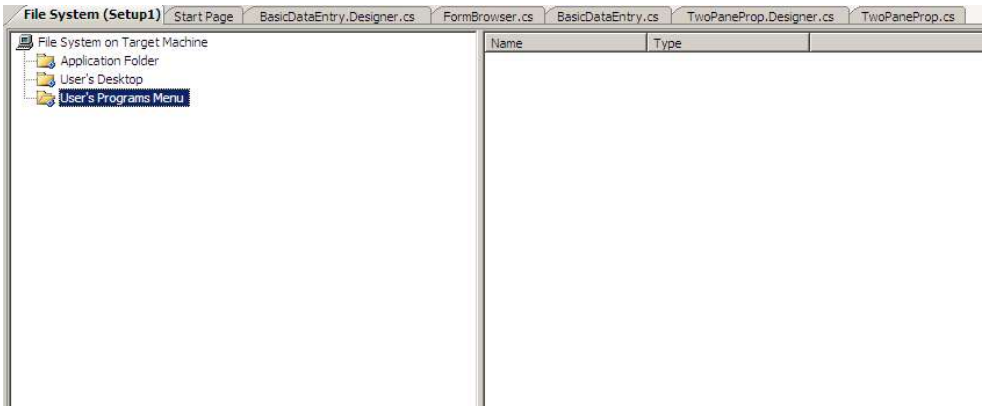


Ces éditeurs vont permettre de configurer le fonctionnement du programme d'installation.

- L'éditeur du système de fichiers va permettre d'ajouter à un projet de déploiement les fichiers constituant l'application à déployer.
- L'éditeur de registre configure les clés et les valeurs de registre à ajouter au Registre de l'ordinateur cible.
- L'éditeur des types de fichier spécifie les associations entre les types de fichiers et les actions autorisées pour chaque type de fichier.
- L'éditeur de l'interface utilisateur permet de spécifier les actions supplémentaires à exécuter pendant l'installation.
- L'éditeur des conditions de lancement spécifie les conditions requises pour l'installation.

12.1.2.A. L'éditeur du système de fichiers

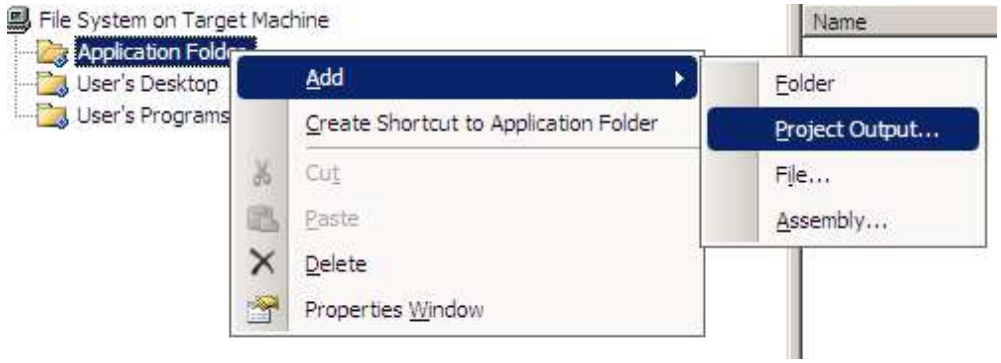
C'est certainement l'éditeur le plus important de la configuration d'un projet de déploiement. Par son intermédiaire, on va indiquer les modifications à apporter au système de fichiers de l'ordinateur sur lequel l'application va être déployée.

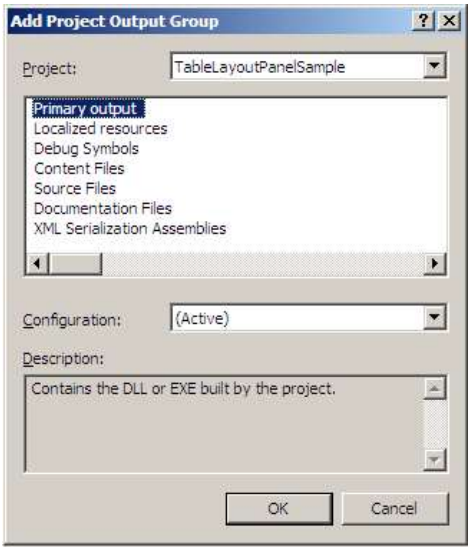


Il est composé de deux parties : un volet de navigation à gauche et un volet d'informations à droite. Le volet de navigation contient la liste hiérarchique des dossiers correspondant au système de fichiers de l'ordinateur d'installation. Les noms des dossiers correspondent aux dossiers Windows standard.

Par exemple, la rubrique dossier d'application désigne le sous-dossier du dossier *Program Files* où l'application sera installée. Le menu contextuel de chacun des dossiers permet d'y ajouter différents éléments. Dans le dossier d'application, on peut par exemple ajouter un sous-dossier, un fichier, un assembly ou, option la plus fréquemment utilisée, une sorte de projet (les fichiers créés par la compilation d'un projet).

C'est donc grâce à cet éditeur que l'on va enfin pouvoir indiquer quel projet on souhaite déployer. La boîte de dialogue suivante permet de choisir le projet à déployer et lesquels de ces éléments on souhaite déployer sur les postes clients.





Pour que l'application puisse s'éxécuter sur l'ordinateur client, il faut au minimum sélectionner l'option *Primary Output*. Par contre, l'option *Source File* est plus rarement utilisée. On doit indiquer si on souhaite déployer la version *debug* ou la version *release* de l'application.

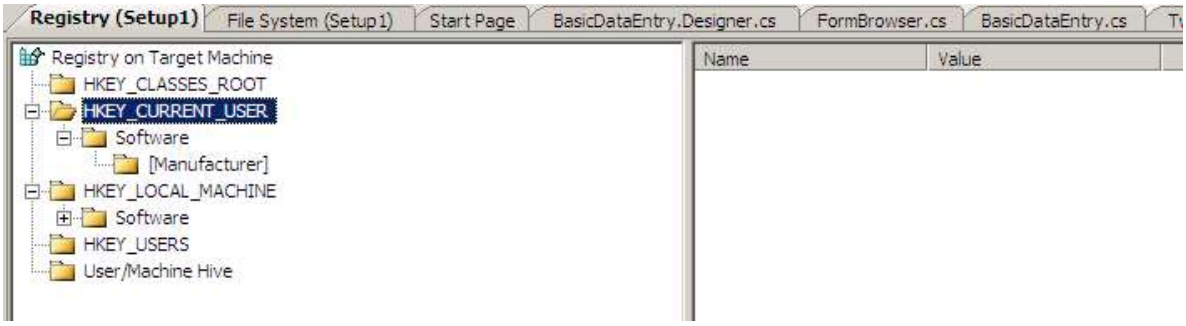
Les dossiers *User's Desktop* et *User's Program Menu* reçoivent généralement un simple raccourci vers l'application. Ce raccourci doit au préalable être créé par le menu contextuel de la sortie principale de l'application.

Il doit ensuite être déplacé vers le dossier correspondant (*User's Desktop* ou *User's Program Menu*).

Pour que chacun de ces dossiers soient créés même s'il ne contient aucun élément, il faut modifier la propriété *Always Create* sur *true*.

12.1.2.B. L'éditeur de registre

L'éditeur de registre permet d'ajouter des clés de registre à un projet de déploiement. Si une clé n'existe pas dans le registre de la machine lors du déploiement, elle est ajoutée pendant l'installation. Il est possible d'ajouter des clés sous n'importe quelle clé de niveau supérieur dans l'éditeur de registre.



Pour ajouter une clé dans le registre, on doit au préalable sélectionner un nœud de niveau supérieur, ou une sous-clé puis à l'aide du menu contextuel utiliser l'option *New Key*. La clé doit ensuite être renommée et ses propriétés modifiées en fonction des besoins. La principale modification consiste à configurer la propriété *DeleteAtUninstall* pour que cette clé de registre soit supprimée lors de la désinstallation de l'application.

La suppression d'une clé est réalisée tout aussi simplement avec l'option *Supprimer* du menu contextuel. Il faut cependant être prudent car la suppression d'une clé entraîne la suppression de toutes ses sous-clés et valeurs contenues dans celle-ci. Un message d'avertissement est affiché pour prévenir de cette situation dangereuse et demande de confirmer le choix.

L'éditeur du registre est également utilisé pour spécifier les valeurs des nouvelles clés ou modifier les valeurs existantes. On peut ajouter des valeurs de type chaîne, binaire et DWORD. Pendant l'installation, les valeurs sont écrites dans le registre ; les valeurs existantes sont remplacées par les valeurs spécifiées dans le programme d'installation.

Il est possible d'ajouter des clés et des valeurs de registre à un projet de déploiement en important un fichier de registre (.reg) dans l'éditeur du registre. Cela permettra de copier une section complète d'un registre existant en une seule fois pour gagner du temps. Les fichiers de registre peuvent être créés à l'aide d'outils tels que l'éditeur du registre de Windows (regedit.exe). Cette solution est très pratique pour transférer sur le poste des utilisateurs une portion de registre récupérée sur le poste utilisé pour le développement de l'application.

Le menu contextuel disponible sur l'élément registre de l'ordinateur cible propose l'option *Importer* permettant de réaliser cette opération. On doit simplement choisir le fichier (.reg) contenant les informations à importer.

12.1.2.C. L'éditeur des types de fichiers

L'éditeur des types de fichiers est utilisé pour indiquer les types de documents et les extensions de fichier associés à l'application lors de son installation sur un ordinateur. Une fois l'association installée, l'extension et la description du type de fichier figurent dans la liste des types de fichiers du système.

L'association d'un type de fichier avec une application nécessite trois étapes :

- L'ajout d'un type de document,
- L'association d'une extension de fichier,
- L'association d'un fichier exécutable.

Pour ajouter un type de fichier, on doit utiliser le menu contextuel de l'éditeur des types de fichiers et choisir l'option *AJouterun type de fichier*. Il faut ensuite modifier le nom de l'élément qui vient d'être ajouté. L'étape suivante

consiste à indiquer l'extension ou les extensions associées à ce type de fichier. Elles doivent être saisies dans la fenêtre de propriétés, dans la rubrique *Extensions*, sans être précédées d'un point. Si plusieurs extensions sont disponibles, elles dooivent être séparées par un point virgule lors de la saisie dans la fenêtre de propriétés.

La dernière étape est maintenant d'associer une application (un fichier exécutable) à ce type de fichier. La propriété *Command* est uilisée à cet effet. Un éditeur particulier permet de rechercher parmi les dossiers du projet l'exécutable qui sera utilisé pour manipuler ce type de document.

C'est cet exécutable qui sera lancé lorsque l'utilisateur réalisera un double-clic sur un fichier de ce type dans l'Explorateur Windows. Il est possible d'ajouter d'autres actions qui seront disponibles par l'intermédiaire du menu contextuel de l'Explorateur Windows. Ces éléments peuvent être ajoutés en utilisant le menu contextuel disponible sur un type de fichier et en choisissant l'option *Ajouter une action*. L'action doit ensuite être configurée en indiquant les paramètres suivants :

- Nom de l'action : utilisé pour représenter l'action dans le menu contextuel de l'explorateur Windows. Pour inclure une touche d'accès rapide dans le nom de l'action, faites précéder du signe & la lettre utilisée pour accéder à la commande.
- Arguments : représente les paramètres passés sur la ligne de comande permettant la réalisation de l'action. Par exemple, la chaîne suivante peut être associée à l'action d'impression d'un document :

  /p "%1"

  Dans cette chaîne de caractères, la partie *%1* représente le nom du fichier sur lequel le menu contextuel est activé. Les caractères ″ sont obligatoires pour éviter les problèmes si le nom de fichier comprte un espace. Il faut bien sûr que l'application ait été conçue pour traiter ces paramètres lors de son démarrage.

- Verb : indique le verbe utilisé pour demander l'exécution de l'action.

12.1.2.D. L'éditeur de l'interface utilisateur

Avec l'Éditeur de l'interface utilisateur, on peut spécifier et définir les propriétés des boîtes de dialogue prédéfinies qui seront affichées pendant l'installation sur l'ordinateur.

Cet éditeur contient deux sections : *Install* et *Administrative Install*. La section *Install* contient les boîtes de dialogue qui seront affichées quand l'utilisateur final exécutera le programme d'installation. La section *Administrative Install* contient des boîtes de dialogue qui seront affichées quand un administrateur système téléchargera le programme d'installation vers un emplacement réseau.

Un ensemble de boîtes de dialogue prédéfinies est affiché dans l'éditeur ; on peut les réorganiser ou les supprimer.

Les boîtes de dialogue prédéfinies se répartissent en trois catégories :

- Les boîtes de dialogue *Début* s'affichent avant que l'installation ne commence. Elles servent généralement à récupérer des informations sur l'utilisateur ou permettre de changer de répertoire d'installation.
- Une boîte de dialogue *Progression* donne des informations sur l'avancement d'une installation.
- Les boîtes de dialogue *Fin* s'affichent lorsque l'installation a réussi. Elles servent généralement à signaler à l'utilisateur que l'installation est terminée ou à lui permettre de lancer l'application.

On peut déplacer des boîtes de dialogue entre les nœuds des catégories par un glisser-déplacer ou à l'aide des commandes *Couper* et *Coller* du menu *Edition*.

Pour que l'installation se fasse sans boîtes de dialogue, on doit supprimer toutes les boîtes de dialogue dans l'Éditeur de l'interface utilisateur.

Chaque boîte de dialogue peut être personalisée en fonction des besoins. Pour chacune d'entre elles, les propriétés disponibles modifient l'aspect de la boîte de dialogue. Sur toutes ces boîtes de dialogue, la propriété *BannerBitmap* représente le logo affiché sur la boîte de dialogue. Des propriétés spécifiques sont accessibles sur chacune des boîtes de dialogue.

Boîte de dialogue bienvenue

- La propriété *WelcomeText* contient le message de présentation de l'application.
- La propriété *CopyrightWarning* indique les informations relatives au copyright de l'application.

Boîte de dialogue Dossier d'installation

- La propriété *InstallAllUsersVisible* détermine si les boutons permettant de choisir le type d'installation (pour un seul utilisateur ou pour tous les utilisateurs) sont visibles.

Boîte de dialogue de progression

- L'affichage d'une barre de progression au cours de l'installation est déterminé par la propriété *ShowProgressBar*.

Boîte de dialogue terminé

Les informations concernant les mises à jour disponibles sont affichées par la propriété *UpdateText*.

12.1.2.E. L'éditeur des actions personalisées

Cet éditeur permet de spécifier des actions supplémentaires sur l'ordinateur pendant l'installation. Par exemple, il peut être utile d'exécuter un programme créant une base de données sur un serveur.

Avant de pouvoir être ajoutées à un projet de déploiement, les actions personnalisées doivent être compilées sous forme de fichier .dll ou .exe ou ajotées à un projet en tant que script ou assembly. Elles ne peuvent être exécutées qu'à la fin de l'installation.

L'éditeur contient quatre dossiers qui correspondent chacun à une phase de l'installation :

Installer

Les actions personnalisées placées sous ce nœud seront exécutées à la fin de la phase d'installation, une fois tous les fichiers installés.

Valider

Les actions personnalisées placées sous ce nœud seront exécutées à la fin de la phase de validation de l'installation, qui a lieu une fois la phase d'installation terminée sans erreur.

Restaurer

Les actions personnalisées placées sous ce nœud seront exécutées à la fin de la phase de restauration de l'installation, qui est déclenchée lorsqu'une erreur d'installation se produit.

Désinstaller

Les actions personnalisées placées sous ce nœud seont exécutées à la fin de la phase de désinstallation de l'installation, qui a lieu lorsqu'une application est désinstallée.

Pour chacune des ces rubriques, on peut ajouter une action personnalisée par l'option *Add Custom Action* du menu contextuel. On doit ensuite sélectionner parmi les éléments disponibles dans le projet, celui chargé de réaliser l'action personnalisée (.exe, .dll, .vbs, …).

Les actions personnalisées seront exécutées dans leur ordre d'affichage dans l'éditeur. On peut modifier cet ordre par un glisser-déplacer ou à l'aide des commandes *Couper* et *Coller* du menu *Edition*.

12.1.2.F. L'éditeur des conditions de lancement

Cet éditeur doit être utilisé pour spécifier les conditions requises pour l'installation de l'application. Les conditions peuvent concerner la recherche d'un fichier, d'une entrée du registre ou d'un composant. Le résultat de la recherche est retourné sous forme d'une propriété qui est ensuite évaluée durant l'installation. L'installation échoue si une condition n'est pas remplie.

L'éditeur des conditions de lancement est formé d'une arborescence composée de deux sections :

- La section *Recherche de l'ordinateur cible pour configurer les recherches*
- La section *Conditions de lancement pour établir les conditions basées sur le résultat des recherches*

Dans un premier temps, on doit ajouter par le menu contextuel de la rubrique *recherche de l'ordinateur cible*, les rechcerches à effectuer avant l'installation.

Recherche sur un fichier

Pour cette catégorie de recherche, les propriétés suivantes sont disponibles :

- *Depth* : spécifie le nombre de niveaux de sous-dossiers dans lesquels rechercher un fichier. La valeur par défaut est zéro indiquant que seul le dossier indiqué sera parcouru.

- *FileName* : spécifie le nom du fichier à rechercher. Les caractères génériques (* ?) ne sont pas autorisés.

- *Folder* : spécifie le dossier dans lequel la recherche va débuter. Les noms des dossiers spéciaux reconnus par Windows Installer doivent être saisis entre les caractères [et]. La valeur par défaut est [SystemFolder].

- *MaxDate* : le fichier recherché ne doit pas être plus récent que cette date.

- *MaxSize* : spécifie la taille maximale (en bytes) du fichier recherché. Elle ne doit pas être supérieure à cette valeur.

- *MaxVersion* : spécifie le numéro de version maximal d'un fichier.

- *MinDate* : le fichier recherché doit être plus récent que la date indiquée.

- *MinSize* : spécifie la taille minimale (en octets) d'un fichier lors d'une recherche de fichiers. Elle doit être supérieure à cette valeur.

- *MinVersion* : spécifie le numéro de version minimal d'un fichier. Ce numéro doit être saisi avec le format n.n.n.n où *n* est un entier.

- *Name* : spécifie le nim utilisé dans l'Éditeur des conditions d elancement pour identifier cette recherche de fichiers.

- *Property* : spécifie le nom de la propriété utilisée au moment de l'installation pour tester le résultat de la recherche.

Recherche dans le registre

Les propriétés suivantes permettent de configurer ce type de recherche :

- *Name* : spécifie le nom utilisé dans l'Éditeur des conditions de lancement pour identifier cette recherche dans le registre sélectionné.

- *Property* : spécifie le nom de la propriété utilisée au moment de l'installation pour tester le résultat de la recherche.

- *RegKey* : spécifie une clé du registre à rechercher. Cette valeur doit représenter le chemin complet de la clé du registre. Elle doit correspondre exactement au nom et au chemin d'accès qui figurent dans le registre, espaces éventuels compris, sinon la recherche de la clé dans le registre échoue.

- *Root* : spécifie le point de départ de la recherche dans le registre. Les valeurs possibles sont :

  HKLM : débute la recherche dans HKEY_LOCAL_MACHINE.
  HKCU : débute la recherche dans HKEY_CURRENT_USER.
  HKCR : débute la recherche dans HKEY_CLASSES_ROOT.
  HKU : débute la recherche dans HKEY_USERS.

- *Value* : spécifie la valeur à rechercher dans la base de registre. Cette propriété doit être une chaîne correspondant à la valeur affichée dans la colonne *Données* de l'éditeur du registre Windows.

Recherche de composants

Cette recherche permet de vérifier qu'un composant requis par l'application est disponible sur la machine d'installation. Les propriétés de recherche sont les suivantes :

- *ComponentId* : spécifie le GUID du composant à rechercher. Cette valeur doit être fournie sous la forme d'une chaîne mise en forme comme un GUID, à l'aide du format {XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX} où *X* est un chiffre hexadécimal. Les accolades sont obligatoires.

- *Name* : spécifie le nom utilisé dans l'Éditeur des conditions de lancement pour identifier cette recherche dans le registre sélectionné.

- *Property* : spécifie le nom de la propriété utilisée au moment de l'installation pour tester le résultat de la recherche.

Le résultat de ces recherches peut ensuite être utilisé pour ajouter des conditions de lancement de l'installation par le menu contextuel de la rubrique *Conditions de lancement*.

Pour chaque condition de lancement, les propriétés suivantes sont disponibles :

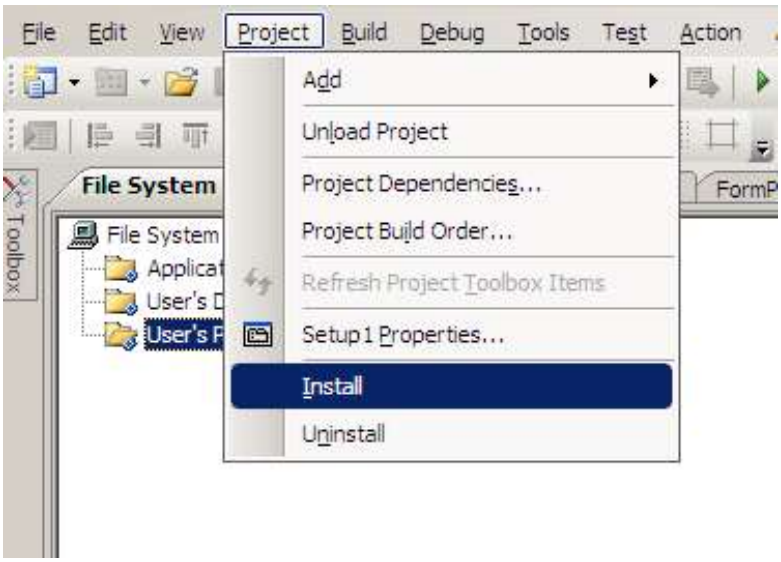
- *InstallUrl* : indique l'emplacement où les fichiers manquant peuvent être téléchargés si une condition de lancement associée à la présence d'un fichier échoue.

- *Message* : spécifie le message à afficher lorsqu'une condition a la valeur *false* au moment de l'installation.

- *Name* : spécifie le nom utilisé dans l'Éditeur des conditions de lancement pour identifier cette conition de lancement.

- *Condition* : spécifie une condition qui doit être remplie au moment de l'installation pour que l'installation puisse continuer.

La dernière étape consiste à lancer la génération du projet et à tester son bon fonctionnement par l'intermédiaire de l'option *Install* du menu *Project*.



12.2. Déploiement avec ClickOnce

ClickOnce est une technologie de déploiement permettant de créer des applications Windows dont la mise à jour peut être effectuée automatiquement. L'installation de ce type d'application est effectuée avec un minimum d'intervention de la part de l'utilisateur. Cette technologie simplifie l'étape de déploiement qui parfois se transforme en véritable casse tête. Les problèmes suivants sont fréquemment rencontrés lors du déploiement d'une application.

Mise à jour de l'application

Avec une méthode de déploiement classique, lorsqu'une nouvelle version de l'application est disponible, l'utilisateur doit en général réinstaller l'application pour bénéficier des mises à jour. La technologie ClickOnce est capable de fournir les mises à jour automatiquement. Dans ce cas, seules les parties de l'application qui ont été changées sont téléchargées, puis, l'application complète mise à jour est réinstallée automatiquement à partir du noveau dossier.

Composants partagés

Les applicaitions dépendent souvent de composants partagés, d'où l'existence d'un risque potentiel de conflit de versions. Dans le cas d'un déploiement avec ClickOnce, chaque application est autonome et ne peut pas interférer avec les autres applications.

Autorisations de sécurité

Généralement l'installation d'une application avec une méthode classique exige des autorisations d'administration sur le poste de travail où est effectuée l'installation. Le déploiement avec ClickOnce autorise les utilisateurs n'ayant pas de privilèges d'administarteur a réaliser l'installation de l'application.

Toutes ces contraintes ont parfois conduit les développeurs à choisir une technologie Web au lieu d'applications Windows classiques simplement pour bénéficier des facilités de déploiement de ce type d'application. La contre partie de ce choix se retrouve au niveau de la moins bonne réactivité de l'application et d'une interface utilisateur moins élaborée. La technologie ClickOnce rend le déploiement d'applications Windows aussi simple que le déploiement d'applications Web. N'importe quelle application console ou Windows Forms peut être publiée avec ClickOnce. Trois techniques de publication sont disponibles :

- À partir d'une page Web,

- À partir d'un partage de fichiers réseau,
- À partir d'un support tel qu'un CD ou DVD.

L'exécution de l'application dispose également de deux variantes. L'application peut être installée sur l'ordinateur d'un utilisateur et exécutée même si l'ordinateur est hors connexion. Elle peut également être exécutée uniquement en mode ligne sans installer aucun élement de façon permanente sur l'ordinateur. Les applications ClickOnce sont isolées les unes des autres et l'installation ou l'exécution d'une application ne peut pas interrompre des applications exitantes. Par défaut, les applications ClickOnce s'exécutent dans les zones de sécurité Internet ou Intranet. En fonction des besoins, l'application peut demander des autorisations de sécurité plus élevées.

La mise à jour de l'application peut également avoir plusieurs modes de fonctionnement. Elles peuvent être automatiques et dans ce cas, l'application vérifieà chaque démarrage si des mises à jour sont disponibles, puis elle les installe automatiquement. L'utilisateur peut manuellement vérifier l'existence d'une mise à jour et décider ou non de son installation. L'administrateur peut aussi rendre obligatoire l'installation d'une mise à jour.

12.2.1. Principe de fonctionnement de ClickOnce

Le mécanisme de déploiement ClickOnce repose sur deux fichiers XML appelés manifestes :

- Un manifeste d'application,
- Un manifeste de déploiement.

Le manifeste d'application décrit l'application elle-même, les assemblies et les fichiers qui la composent, les dépendances, les autorisations requises pour l'exécution et l'emplacement dans lequel les mises à jour seront disponibles.

Le manifeste de déploiement décrit comment l'application est déployée, y compris l'emplacement du manifeste d'application et la version de l'application que les clients doivent exécuter. Ces deux fichiers sont générés par Visual Studio.

Le manifeste de déploiement est copié vers l'emplacement de déploiement. Cet emplacement peut être un serveur Web, un répertoire partagé sur le réseau ou des supports amovibles tels qu'un CD. Le manifeste d'application et tous les fichiers de l'application sont également copiés vers l'emplacement de déploiement spécifié dans le manifeste de déploiement. Ces fichiers peuvent être copiés vers le même emplacement ou dans deux emplacements distincts. Visual Studio prend également en charge les copies de ces fichiers.

Après le déploiement de l'application dans l'emplacement de déploiement, les utilisateurs peuvent télécharger et installer l'application en cliquant sur l'icône représentant le fichier manifeste de déploiement disponible sur une page Web ou dans un dossier. L'utilisateur voit simplement s'afficher une simple boîte de dialogue lui demandant de confirmer l'installation. Après validation, l'installation conitnue et l'application est lancée sans autre intervention. Si l'application nécessite des autorisations d'exécution plus élevées, la boîte de dialogue demande à l'utilisateur d'accorder les autorisations pour que l'installation puisse se poursuivre.

L'application est ajoutée au menu *Démarrer* de l'utilisateur et à la rubrique *Ajout/Suppression de Programmes* du *Panneau de configuration*. Contrairement à d'autres technologies de déploiement, rien n'est ajouté au dossier *Program Files*, dans la base de registre ou sur le bureau. De pllus, aucun droit d'administration n'est nécessaire pour l'installation.

Lorsque l'on créé une version mise à jour de l'application, on dooit également générer un nouveau manifeste d'application et copier les fichiers vers un emplacement de déploiement, généralement un dossier frère du dossier de déploiement d'origine. La manifeste doit aussi être mis à jour pour qu'il pointe vers l'emplacement de la nouvelle version de l'application.

12.2.2. Les différentes méthodes de déploiement

Pour déployer une application ClickOnce, trois stratégies sont possibles. La stratégie que l'on choisira dépend principalement du type d'application que l'on déploit. Les trois stratégies de déploiement sonr les suivantes :

- Installation à partir du Web ou d'un partage réseau,
- Installation à partir d'un CD,
- Démarrage de l'application à partir du Web ou d'un partage réseau.

12.2.2.A. Installation à partir du Web ou d'un partage réseau

Cette stratégie permet de déployer l'application sur un serveur Web ou d'un partage de fichiers réseau. Lllorsqu'un utilisateur final souhaite installer l'application, il clique sur une icône d'une page Web ou double clique sur une icône de partage de fichiers. L'application est ensuite téléchargée, installée et démarrée sur l'ordinateur de l'utilisateur.

Étant donné que cette stratégie dépend de la connexion réseau, elle fonctionne de manière optimale pour les applications qui seront déployées pour les utilisateurs qui ont accès à un réseau local ou une connexion internet rapide.

12.2.2.B. Installation à partir d'un CD

Cette stratégie permet de déployer l'application sur un support amovible tel qu'un CD ou un DVD. Comme pour l'option précédente, lorsque l'utilisateur choisit d'installer l'application, cette dernière est installée, lancée et des éléments sont ajoutés au menu *Démarrer* et au group *Ajout/Suppression de Programmes* dans le *Panneau de configuration*.

Cette stratégie fonctionne meiux dans le cas d'applications déployées sur les ordinateur d'utilisateurs qui ne possèdent pas une connexion réseau ou qui ont des connexions à faible bande passante. L'application étant installée à partir d'un support amovible, aucune connexion réseau n'est nécessaire pour l'installation ; la connectivité réseau est néanmoins nécessaire pour les mises à jour de l'application.

12.2.2.C. Démarrage de l'application à partir du Web ou d'un partage réseau

Cette stratégir est similaire à la première, sauf que l'application se comporte comme une application Web. Lorsque l'utilisateur clique sur un lien d'une page Web, l'application est lancée. Lorsque les utilisateurs ferment l'application, cette dernière n'est plus disponible sur leur ordinateur local. Aucun élément n'est ajouté au menu *Démarrer* ou au groupe *Ajout/Suppression de Programmes* dans le *Panneau de Configuration*.

Techniquement, l'application est téléchargée et installée dans un cache d'application de l'ordinateur local, de la même façon qu'une application Web et téléchargée vers le cache Web. Comme pour le cache Web, les fichiers sont netoyés du cache d'application en fin d'utilisation. Toutefois, l'utilisateur a l'impression que l'application est exécutée à partir du Web ou du partage de fichiers.

Cette stratégie est à priviliégier pour les applications rarement utilisées.

12.2.3. Les mises à jour de l'application

ClickOnce peut fournir automatiquement les mises à jour de l'application. Une application ClickOnce lit périodiquement son fichier manifeste de déploiement pour vérifier si les mises à jour de l'application sont disponibles. Si elle est disponible, la nouvelle version de l'application est téléchargée et exécutée. Pour des raisons d'efficacité, seuls les fichiers modifiés sont téléchargés.

Trois stratégies de base sont disponibles pour les mises à jour :

- La vérification des mises à jour au démarrage de l'application,
- La vérification des mises à joour après le démarrage de l'application (exécutée dans un thread d'arrière-plan),
- La présentation d'une interface destinée aux mises à jouor.

On peut également déterminer la fréquence de vérification des mises à jour effectuée par l'application ou configurer une mise à jour obligatoire. Les mises à jour d'application exigent une connexion auréseau. En l'absence d'une connexion réseau, l'application s'exécute sans vérifier les mises à jour, quelle que soit la stratégie de mise à jour choisie.

Vérification des mises à jour après le démarrage

Par défaut, l'application tente de localiser et de lire le fichier manifeste de déploiement en arrière-plan pendant son exécution. Si une mise à jour est disponible, lors de la prochaine exécution, l'utilisateur sera invité à télécharger et à installer la mise à joour.

Cette stratégie est tout particulièrement adaptée aux connexions réseau à bande passante restreinte ou aux applications volumineuses, pouvant nécessiter de longs téléchargements.

Vérification des mises à jour au démarrage

Avec cette stratégie, l'application tente de localiser et de lire le fichier manifeste de déploiement à chaque lancement. Si une mise à joour est disponible, elle sera téléchargée et exécutée. Sinon, la version existante sera exécutée.

Cette stratégie est bien adaptée aux connexions réseau à large bande passante ; le délai nécessaire au lancement de l'application peut être inacceptable sur des connexions à bande passante plus restreinte.

Mises à jour obligatoire

Il est parfois souhaitable d'obliger les utilisateurs à exécuter une version mise à jour de l'application si, par exemple, on a modifié une ressource qui risque de perturber le fonctionnement de l'ancienne version de l'application. On peut marquer dans ce cas la mise à jouor comme étant obligatoire et de ce fait, empêcher l'exécution d'une version plus ancienne de l'application. Cette stratégie doit être associée avec la vérification des mises à jour de démarrage.

Intervalles de mise à jour

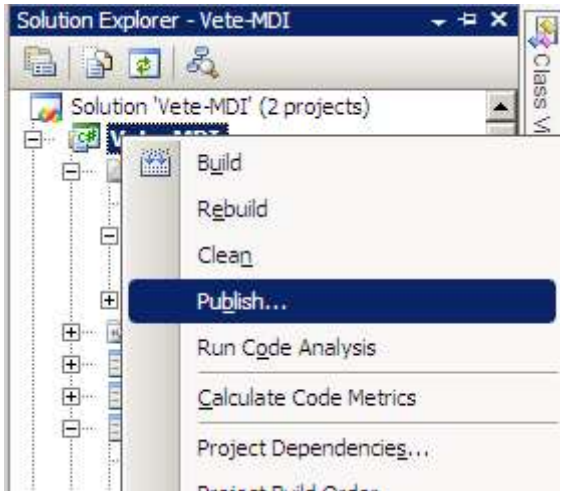
Dans le cadre des mises à jour automatiques, on peut égaelement spécifier la fréquence de vérification des mises à joour. Par exemple, on peut souhaiter une vérification à chaque exécution de l'application, une fois par semaine ou une fois par mois. Si aucune connexion réseau n'est disponible au moment spécifié pour la vérification, celle-ci est effectuée à la prochaine exécution de l'application.

Blocage des mises à jour

Il est également possible de faire en sorte que l'application ne vérifie jamais les mises à jour. Par exemple, on peut souhaiter une vérification à chaque exécution de l'application, une fois par semaine ou une fois par mois. Si aucune connexion réseau n'est disponible au moment spécifié pour la vérification, celle-ci est effectuée à la prochaine exécution de l'application.

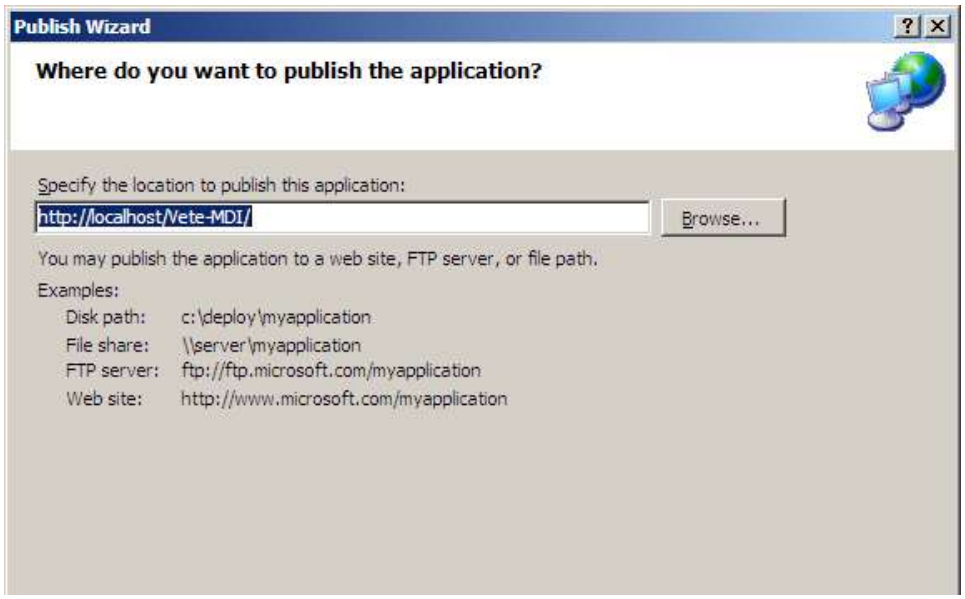
12.2.4. Mise en œuvre de la publication ClickOnce

La publication d'une application avec la technologie ClickOnce est grandement facilitée par un assistant permettant de recueillir la majorité des informations nécessaires au déploiement. Cet assistant est disponible en choisissant l'option *Publier* du menu contextuel accessible sur le projet à déployer dans l'explorateur de solution.



Certaines options de déploiement ne sont cependant pas gérées par cet assistant et doivent être configurées manuellement via la boîte de dialogue de propriétés du projet.

La première étape consiste à configurer l'emplacement où doit se faire la publication.



Cet emplacement peut être :

- Un répertoire sur la machine,
- Un répertoire partagé sur une autre machine en indiquant un chemin UNC de la forme suivante : \\NomMachine\Nom du répertoire. on doita avoir l'autorisation d'écrire sur le partage (share) pour que la publication puisse être réalisée.
- Le serveur IIS de la machine sur lequel on doit avoir au préalable ajouté un répertoire virtuel pour accueillir les fichiers.
- Un serveur FTP pour lequel on doit communiquer les informations de connexion :

    - L'adresse IP ou le nom du serveur FTP,
    - Le numéro du port utilisé pour contacter le serveur (en général 21),
    - Le répertooire du serveur dans lequel sera effectuée la copie des fichiers.
    - Si on est situ derrière un paref-eu en activant l'otpion *Mode passif*.

- Si on se connecte de manière anonyme ou non. Dans ce dernier cas, il faudra fournir le nim de l'utilisateur et son mot de passe.

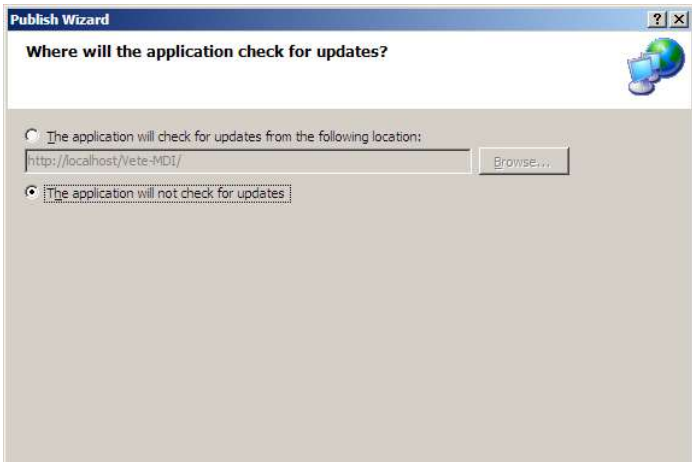
La deuxième étape détermine comment les utilisateurs vont installer l'application :



Les options possibles sont :

- À partir d'un site Web dont on indique l'URL.
- À partir d'un partage réseau dont on spécifie l'UNC. Les utilisateurs devront bien sûr avoir le droit de lecture sur ce partage (share).
- À partir d'un CR ou DVD que l'on fournira. La création de ce support n'est pas réalisée par l'assistant et doit être effectué par une application de gravure externe.

L'assistant propose ensuite de configurer la stratégie de mise à jour :

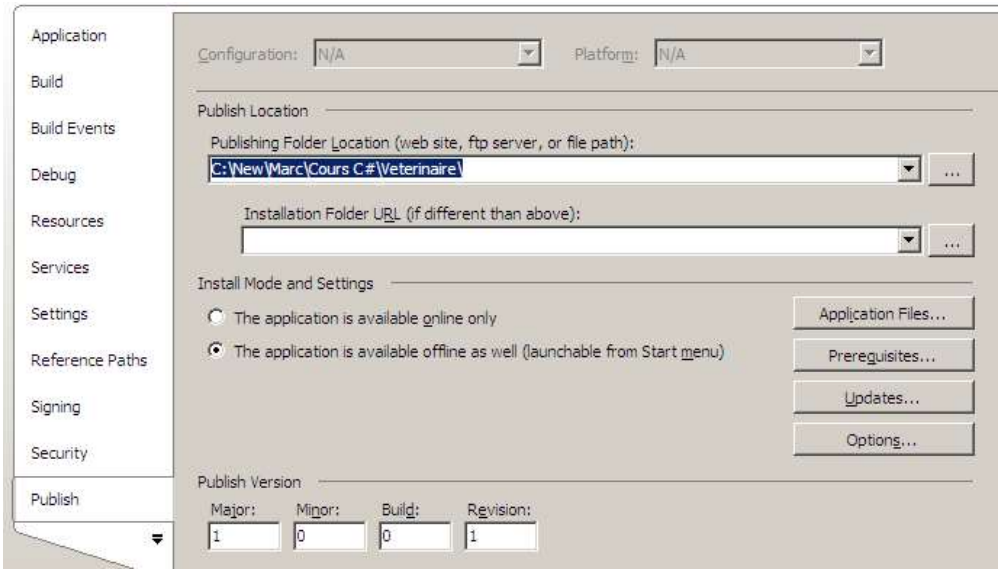


Si on souhaite activer les mises à jour, il faut indiquer l'emplacement à partir duquel elles seront obtenues. Cet emplacement n'est pas forcément le même que celui utilisé pour l'installation.

L'ultime étape affiche un résumé des informations sélectionnées et permet de lancer la publication avec le bouton *Finish*.

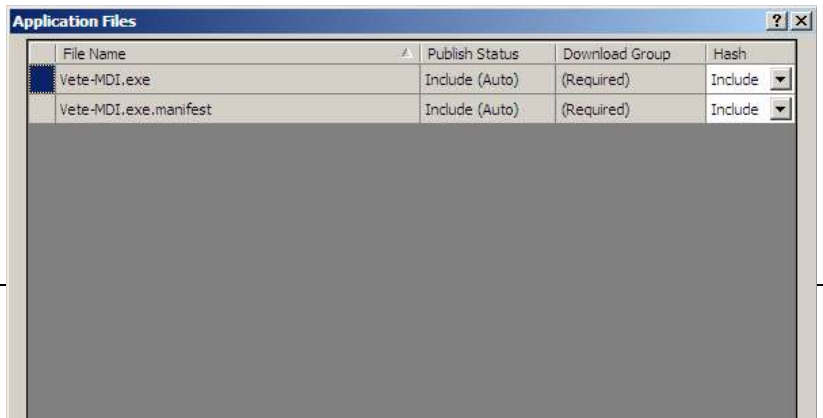


Les options de déploiement plus spécifiques doivent être configurées via la rubrique *Publier* disponible dans les propriétés du projet. Cette boîte de dialogue reprend les propriétés configurées par l'assistant de publication :



Les boutons *Application Files, Prerequisites, Updates* et *Options* permettent de peaufiner les réglages.

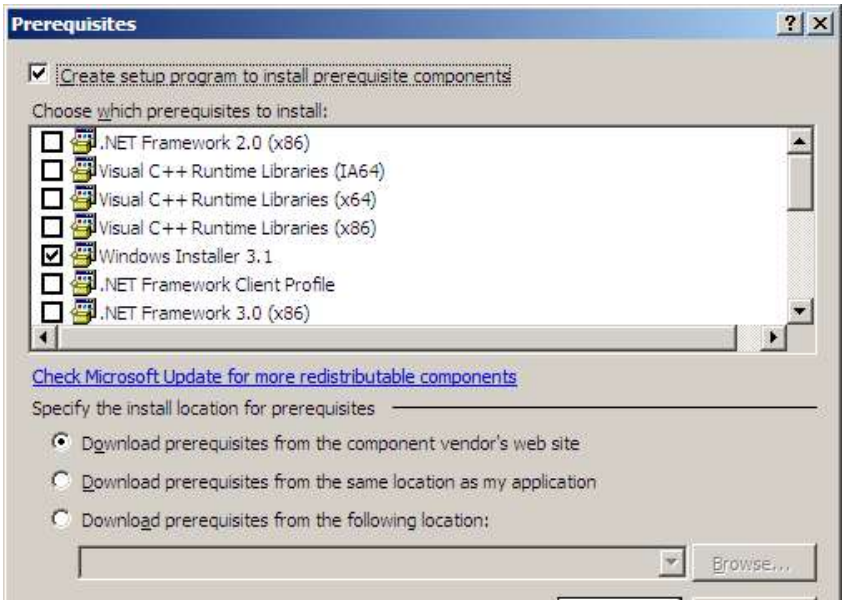
Le bouton *Fichiers d'application* affiche la boîte de dialogue suivante concernant les fichiers constituants l'application :



L'état de la publication de chaque fichier peut être configuré avec trois valeurs différentes :

- *Inclure* : le fichier sera disponible pour les utilisateurs sur le support de déploiement.
- *Exclure* : le fichier n'est pas recopié sur le support de déploiement.
- *Fichier de données* : le fichier contient des données nécessaires au bon fonctionnement de l'application et sera inclus dans la publication.

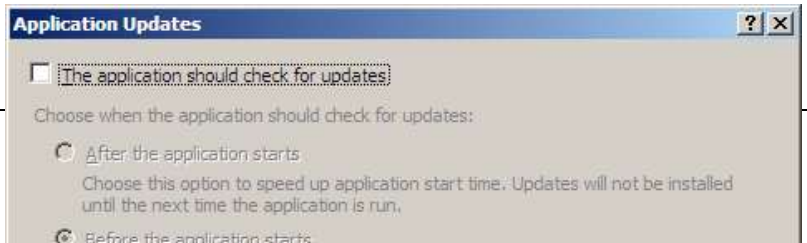
Le bouton *Composants requis* est utilisé pour configurer les éléments nécessaires pour le fonctionnement de l'application.



On peut choisir de créer un programme d'installation pour les composants requis pour le fonctionnement de l'application en cochant la case *Créer un programme d'installation des composants requis*. Les composants concernés sont à choisir dans la liste présentée. On doit également indiquer à partir de quel emplacement ces composants seront installés. Trois options sont possibles :

- À partir du site Web du fournisseur du composant.
- À partir du même emplacement que celui utilisé pour installer l'application.
- À partir de l'emplacement indiqué.

La configuration des mises à jour prévue lors de l'utilisation de l'assistant peut être modifiée par le bouton *Mise à jour*.

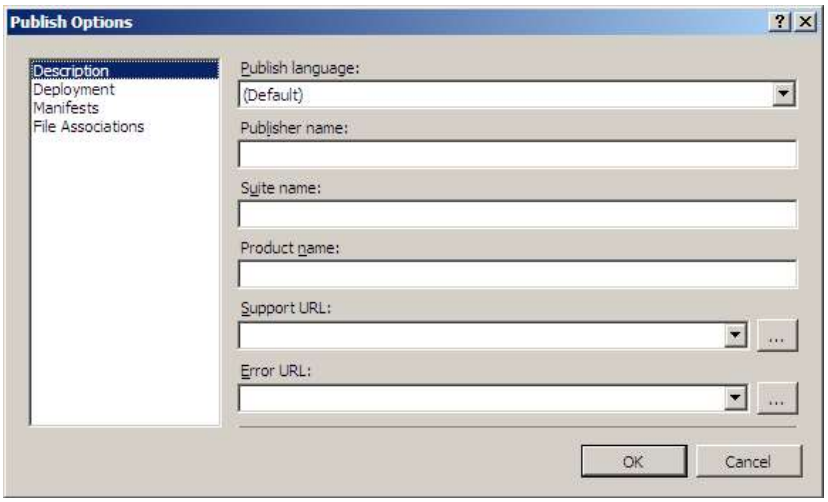


La case à cocher *L'application doit vérifier la disponibilité de mise à jour* spécifie que l'application doit vérifier la disponibilité des mises à jour dés son installation. Si on sélectionne cette option, les autres options deviennent disponibles. Elles permettent de choisir le moment où aura lieu la vérification de la disponibilité d'une mise à jour.

L'option *Avant le démarrage de l'application* indique que l'application doit vérifier la disponibilité des mises à jour avant le démarrage. Cela garantit que les utilisateurs qui sont connectés au réseau disposent toujours de la version la plus récente de l'application. Cette option peut ralentir le démarrage de l'application dans le cas où des mises à jouor sont disponibles.

L'option *Après le démarrage de l'application* planifie l'exécution de la mise à jour lors du prochain redémarrage de l'application. La fréquence de vérification des mises à joour peut également être indiquée par un nombre d'heures, de jours ou de semaines ou bien être exécutée à chaque démarrage de l'application. On doit aussi indiquer l'emplacement à partir duquel les mises à jour sont disponibles si celui-ci est différent de l'emplacement d'installation.

Le dernier bouton va servir à configurer diverses options de déploiement.



Les options suivantes sont disponibles :

- **Langue de publication :** Spécifie la langue et les paramètres régionaux dans laquelle l'application doit être publiée.

- **Nom de l'éditeur** : spécifie le nom de l'éditeur de l'application. Si cette zone est vide, la valeur de la propriété *RegisteredOrganization* de l'ordinateur sera utilisée. Si cette valeur est nulle, le nom du projet est utilisé.

- **Nom du produit** : spécifie le nom de produit de l'application. Si le nom de produit est vide, le nom de l'assembly est utilisé.

- **URL du support technique** : spécifie un site Web qui contient des informations d'assistance pour l'application. La spécification de cette URL est facultative. Si elle est utilisée, cette URL apparaît dans l'entrée *AJouot/Suppression de programmes* pour l'application dans le *Panneau de Configuration*.

- **Page Web de déploiement** : spécifie un nom pour la page Web de déploiement. Le nom de fichier par défaut est *Publish.htm*.

- **Générer automatiquement la page Web de déploiement après la publication** : si cette option est sélectionnée, le processus de publication génère une page Web de déploiement à chaque publication. Cette option n'est disponible que si une page Web de déploiement est spécifiée.

- **Ouvrir la page Web après la publication** : si cette option est sélectionnée, la page Web de déploiement générée automatiquement s'ouvre après la publication.

- **Bloquer l'activation de l'application via une URL** : si cette option est désactivée, l'application s'exécute automatiquement après l'installation. Si elle est activée, l'utilisateur devra démarrer l'application à partir du raccourci de programme dans le menu *Démarrer*.

- **Utiliser l'extension de fichier "**.deploy" : si cette otpion est sélectionnée, le fichier de déploiement utilise l'extension *.deploy*. certains serveurs Web sont configurés pour bloquer, par raison de sécurité, les fichiers qui ne sont pas habituellement présents dans un contenu Web. Par exemple, les fichiers portant les extensions suivantes peuvent être bloqués : *.dll, .config, .mdf*. Les applications Windows contiennent généralement des fichiers portant certaines de ces extensions. Si un utilisateur essaie d'exécuter une application ClickOnce qui accède à un fichier bloqué sur un serveur Web, une erreur se produit. Plutôt que de débloquer toutes les extensions de fichier, chaque fichier d'application est publié par défaut avec une extension de fichier *.deplay*. si cette option est utilisée, le serveur Web ne doit être configuré que pour débloquer les trois extensions suivantes : *.application, .manifest* et *.deploy.*

- **Autoriser le transfert des paramètres d'URL vers l'application** : par défaut cette option est désactivée. Si cette option est activée, l'application sera capable d'accéder et de traiter les informations de paramètres de l'URL.

- **Pour les installations depuis un CD, démarrer automatiquement l'installation dés l'insertion du CD** : si cette option est sélectionnée, elle ajoute un fichier *Autorun.inf* à la racine du support pour les applications ClickOnce qui sont installées via CD ou DVD.

- **Vérifier les fichiers transférés sur un serveur Web** : si cette option est activée, le processus de publication télécharge chaque fichier pour vérifier qu'ils peuvent bien être téléchargés. On est informé des fichiers qui ne peuvent pas être téléchargés.

- **Utiliser le manifeste d'application pour les informations d'approbation** : lorsque cette option est sélectionnée, on peut signer à noouveau le manifeste de l'application à l'aide d'un certificat contenant les coordonnées du programmeur.

13. Annexes

13.1. Explication de quelques fonctions

Fonction	Description

TryParse	Converts the string representation of a number to its 32-bit signed integer equivalent. A return value indicates whether the conversion succeeded.
	
	`public static bool TryParse(`
	`  string s,`
	`  out int result`
	`)`
	
	The TryParse method is like the Parse method, except the TryParse method does not throw an exception if the conversion fails. It eliminates the need to use exception handling to test for a FormatException in the event that *s* is invalid and cannot be successfully parsed.
	The *s* parameter contains a number of the form:
	[ws][sign]digits[ws]
	Items in square brackets ([and]) are optional.

Table 23: Explication de certaines fonctions

14. Bibliographie

- Développez avec Visual Studio 2008 par Thierry Groussard. L'essentiel de ce livre fait partie de ce syllabus.